1 Introduction

In this module we will explore *Industrial Data Science*, a term that is not widely used, defined or standardized (yet).

Generally, it refers to the application of Data Science techniques within industrial and business domains. This includes optimizing processes, predictive maintenance, quality control, supply chain management and automation by leveraging large volumes of sensor and process data.

The term *Industrial Artificial Intelligence* (Industrial AI) is more common in literature and appears to be used interchangeably with the fuzzy definition of Industrial Data Science. According to Peres et al. (2020), Industrial AI distinguishes itself within the field of AI in five dimensions:

- **Algorithms**: Combines physical, digital, and heuristic models, requiring complex management and deployment.
- **Data**: Often involves large volume diverse datasets from multiple sources.
- **Decision-making**: Demands high reliability, low error tolerance, and efficient handling of uncertainty.
- **Infrastructure**: Large emphasis on real-time processing capabilities.
- **Objectives**: Targets value creation, such as reducing waste, improving quality, and boosting operational efficiency.

Unlike typical courses that use clean, polished datasets, in this lecture we will get our hands dirty and work with (synthetic) raw machine data with the ultimate goal of setting up a data science product on a simulated machine using live data.

In Chapter 1 to Chapter 3, we will start with an introduction to Industrial Data Science, its significance, its history, and its applications.

In Chapter 4 to Chapter 11, we will prepare ourselves by exploring time series data, including common applications and methods.

Chapter 13 to Chapter 16 provide an overview of curated datasets of machine sensors.

In Chapter 17 to Chapter 23, we will work directly with simple simulated machines.

Finally, Chapter 24 to Chapter 28 will give a brief outlook on selected topics in Industrial Data Science.

Please note that Industrial Data Science is a vast and rapidly evolving field, encompassing a wide range of industries, technologies, and specialized methods. Each industrial context presents unique challenges and data architectures. Covering more of the relevant tools, algorithms, and real-world scenarios in depth would require far more time and resources than a single module allows. The focus of this material, therefore, is to introduce key concepts and practical skills, laying a foundation for further exploration.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

2 History

2.1 Industry 1.0 – Mechanisation (late 18th century)

The First Industrial Revolution, beginning at the end of the 18th century, introduced mechanized production systems powered by water and steam.

This shift significantly increased productivity, particularly in industries like textiles, mining, transportation, and agriculture. Steam engines replaced muscle power in factories, leading to rapid production and reduced costs. Innovations such as steam locomotives and steamships accelerated the delivery of commodities.

Overall, this revolution marked a transition from an agricultural economy to one dominated by machine-driven production, setting the stage for modern advances in various sectors. However, it also precipitated sociocultural shifts, replacing manual labor with machines, leading to job insecurity, and exposing workers to harsh conditions due to inadequate regulations.

2.2 Industry 2.0 – Electrification (late 19th century)

During the late 19th century, the Second Industrial Revolution brought mass production enabled by electricity.

This era also saw the use of electricity as a key source of energy, which was more efficient and less costly than earlier steam and water-powered systems. The introduction of the integrated assembly line further streamlined production.

Despite the efficiency gains and increased output, this industrial shift also resulted in significant job losses due to automation. Consequently, while productivity increased drastically and goods became cheaper, standard labor methods used in factories were replaced with new machines and again reduced jobs.

2.3 Industry 3.0 – Automation (mid to late 20th century)

The Digital Revolution, which began in the 1970s, marked the transition to extensive use of digital technologies such as computers, microprocessors, and the Internet.

This era led to significant automation across various industries, enhancing speed, accuracy, and reducing human labor through inventions like the *Programmable Logic Controller* (PLC). The integration of electronic hardware necessitated complementary advancements in software, boosting the software development industry and enabling improved management systems in enterprises, including resource planning and logistics. As a result, industries increasingly automated and globalized production, which contributed to the development of Supply Chain Management.

However, this period also created significant environmental challenges, including substantial electronic waste and high energy consumption.

2.4 Industry 4.0 – Digitalisation (late 20th century, 21st century)

Industry 4.0 represents a significant transformation in manufacturing and production, embracing digital technologies such as the *Internet of Things* (IoT), cloud computing, and smart devices.

This evolution builds on the automation introduced in the Third Industrial Revolution but pushes further towards a fully integrated digital industrial environment, aiming for seamlessly integrated solutions. Key features of Industry 4.0 include the interconnection of machines, devices, and sensors, enabling real-time data exchange and intelligent decision-making, allowing for self-monitoring, predictive maintenance, and adaptive manufacturing processes. Advanced analytics and artificial intelligence are leveraged to optimize production, improve quality, and reduce downtime. Industry 4.0 not only increases efficiency and productivity but also enables new business models, such as mass customization and service-oriented manufacturing.

Challenges include potential job losses, particularly for blue-collar workers, due to increased automation and customization. The digital nature of this industrial phase also raises cybersecurity risks. Transitioning to these new technologies demands significant capital investment and skilled workers familiar with AI/ML, adding economic and educational challenges.

2.5 Industry 5.0 – Personalisation (21st century onward)

Industry 5.0 highlights a paradigm shift focused on the synergy between humans and machines, emphasizing personal focus, resilience, and sustainability as core principles.

This human-centric approach supports social well-being by considering workers as valuable investments and designing technology to cater to their diverse needs and rights, such as independence, dignity, and privacy. The shift encourages continuous skill development for better job opportunities and work-life balance, and it prioritizes sustainability through resource efficiency and recycling practices. In terms of technology, Industry 5.0 integrates advances like the Internet of Everything (IoE), 4D imaging technologies, smart sensors, and humanoid robots, mainly enhancing healthcare services by improving diagnostics and patient care while ensuring safety and efficiency.

However, challenges include the need for a highly skilled workforce, increased risks of cyberattacks due to digitalization, substantial initial investments, and the complexity of regulating automated, personalized production processes.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

3 Application areas

> **Note**
>
> The following is a non-exhaustive list of common application areas of Industrial Data Science.

3.1 Process optimization

- Energy consumption prediction and optimization
- Improving production efficiency
- Improving production quality
- Supply chain optimization by predicting market demands

3.2 Quality control

- Automated visual inspection
- Defect prediction to mitigate multistage propagation
- Online quality prediction

3.3 Predictive maintenance

- Detecting possible problems before they occur
- Reducing unnecessary inspection and maintenance operations

3.4 Human-Robot collaboration and ergonomics

- Workforce training
- Task support
- Collaborative robotics
- Ergonomics

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

4 Nature of Time Series

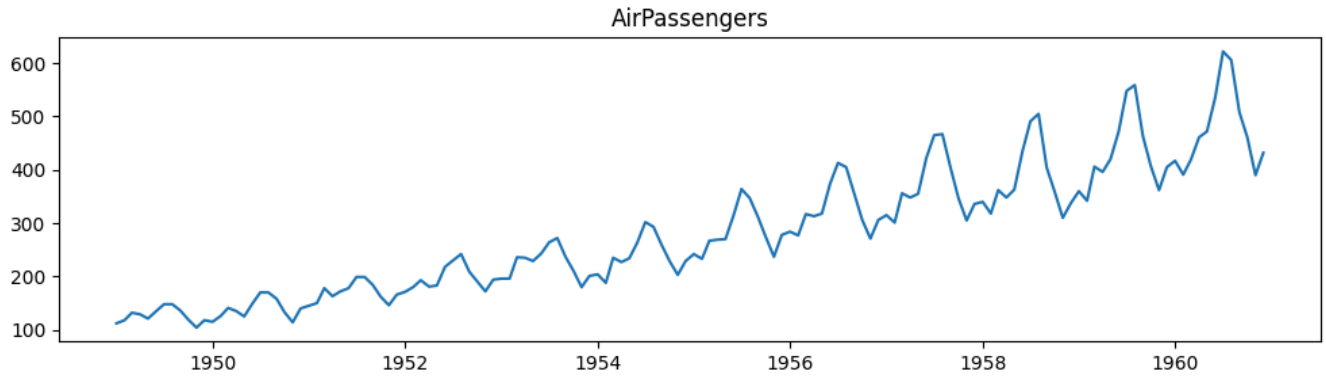
Time series data is a sequence of data points collected or recorded at successive points in time, typically at uniform intervals. It is characterized by its temporal ordering, meaning that the order of the data points matters and can reveal patterns, trends, and seasonal variations over time. Typically, time series data exhibits sequential dependencies, indicating that the value at a given time point may be influenced by previous values in the series.

Time series are not only used in industrial contexts, but also in various other fields such as finance, economics, environmental science, healthcare, and social sciences.

4.1 Examples of Time Series Data

4.1.1 Airline passenger numbers

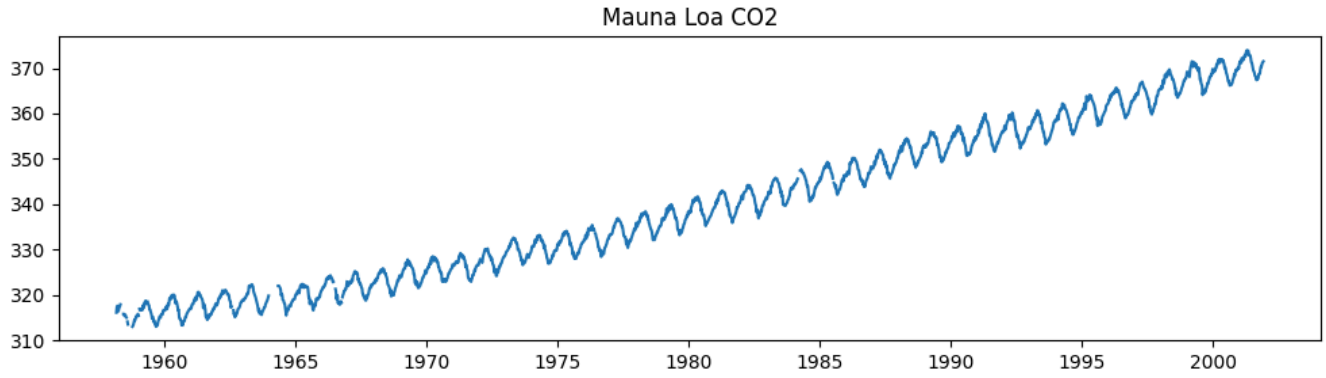
This time series shows the monthly totals of international airline passengers from 1949 to 1960. Source: `RDatasets` (accessed via `statsmodels`).



Airline passenger numbers

4.1.2 Atmospheric CO2 concentrations

This time series shows the monthly average atmospheric CO2 concentrations measured at the Mauna Loa Observatory in Hawaii. Source: `statsmodels.datasets`.

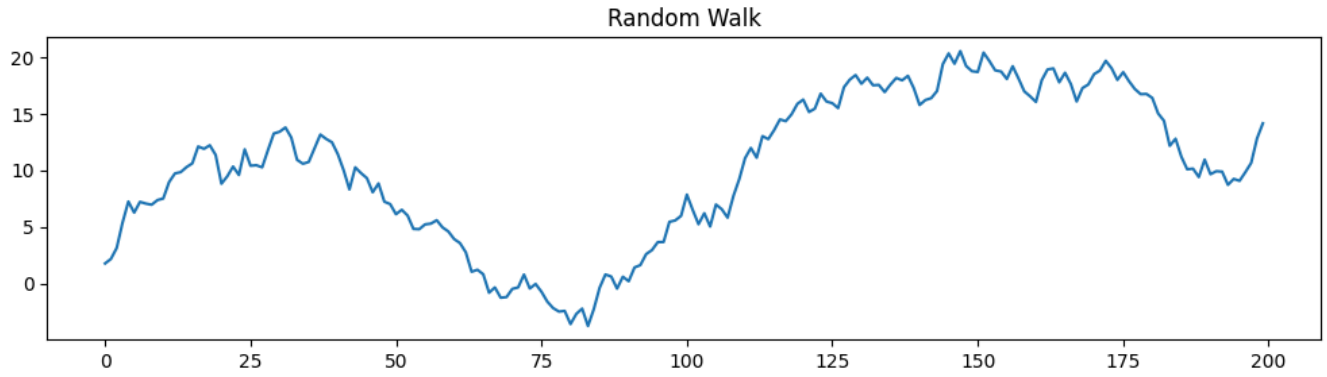


Atmospheric CO2 concentrations

4.1.3 Simulated Random Walk

Just a simulated random walk time series. Code:

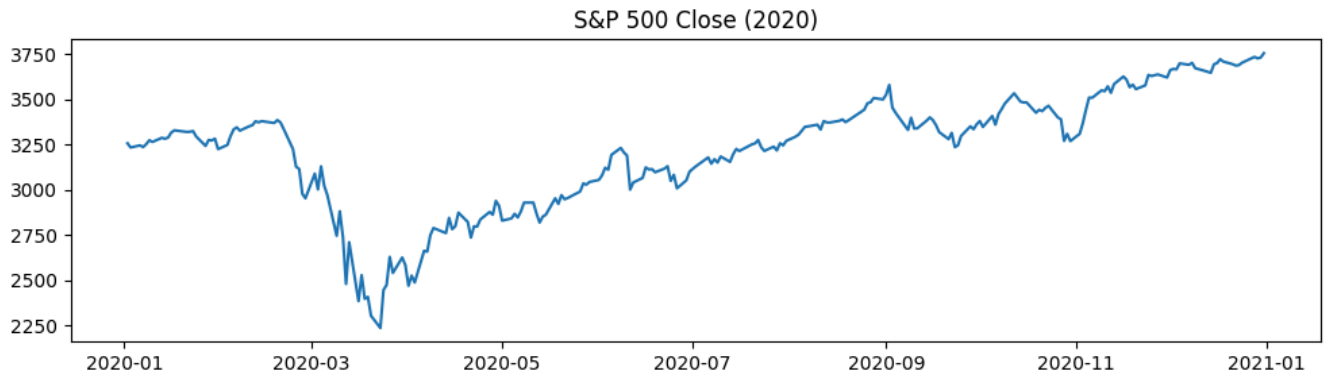
```
import numpy as np

random_walk = np.cumsum(np.random.randn(200))
```



Simulated Random Walk

4.1.4 S&P 500 index

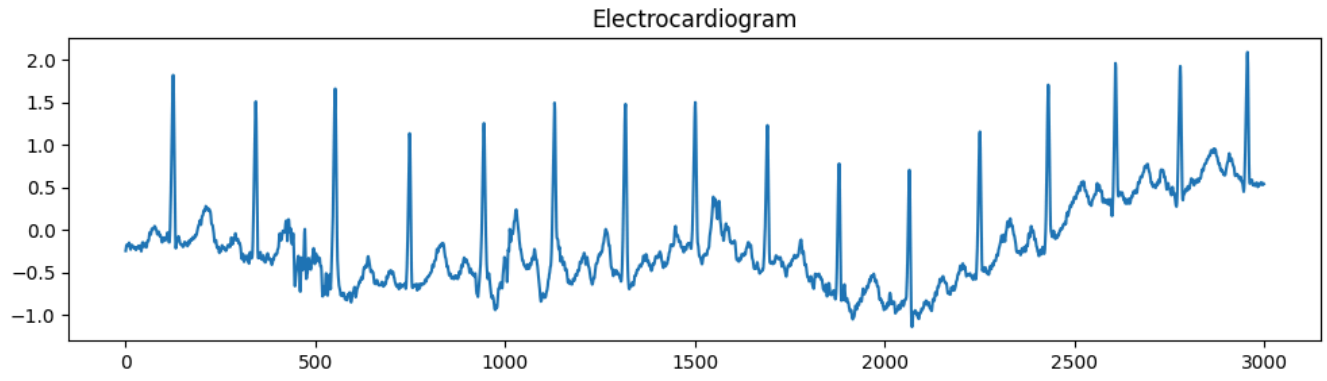
This time series shows the daily closing prices of the S&P 500 in 2020. Source: Yahoo Finance (downloaded via the `yfinance` package).



S&P 500 index

4.1.5 Electrocardiagram (ECG) Signal

This time series represents an electrocardiogram (ECG) signal, which measures the electrical activity of the heart over time. Source: `scipy.misc.electrocardiogram`.



Electrocardiogram (ECG) Signal

Industrial Data Science - 2026 - CC BY-NC-SA 4.0

Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

5 Data imputation

Real datasets often contain missing values or gaps due to various reasons such as sensor malfunctions, data corruption, or transmission errors. Handling these missing values is crucial for accurate analysis and modeling.

In this section we will explore common techniques for imputing missing data in time series.

5.1 Examples for time series imputation

In this section, we will use the the dataset *AI4I 2020 Predictive Maintenance Dataset* (2020), which can be downloaded from this link.

This dataset contains sensor readings from a simulated manufacturing process, including air temperature, process temperature, rotational speed, torque, and tool wear, along with labels for machine failures. We will have a closer look at this dataset in Chapter 13.

Here, we will solely use the data to demonstrate different imputation techniques and since the data comes with no missing values, we will introduce gaps in the following.

The collapsed code cells include the imports, helper functions, and data loading.

5.1.1 Imports

▶ Code

5.1.2 Data loading

▶ Code

```
df.head()
```

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	0	0	0	0	0
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	0	0	0	0	0
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	0	0	0	0	0
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	0	0	0	0	0
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	0	0	0	0	0

Extract process temperature and add gaps of random lengths at random positions.

```
dftemp = df[["UDI", "Process temperature [K]"]].rename(
    columns={"Process temperature [K]": "process_temperature_K"}
)[
    :1000
]  # Only take first 1000 samples, since this is enough for demonstration purposes
```

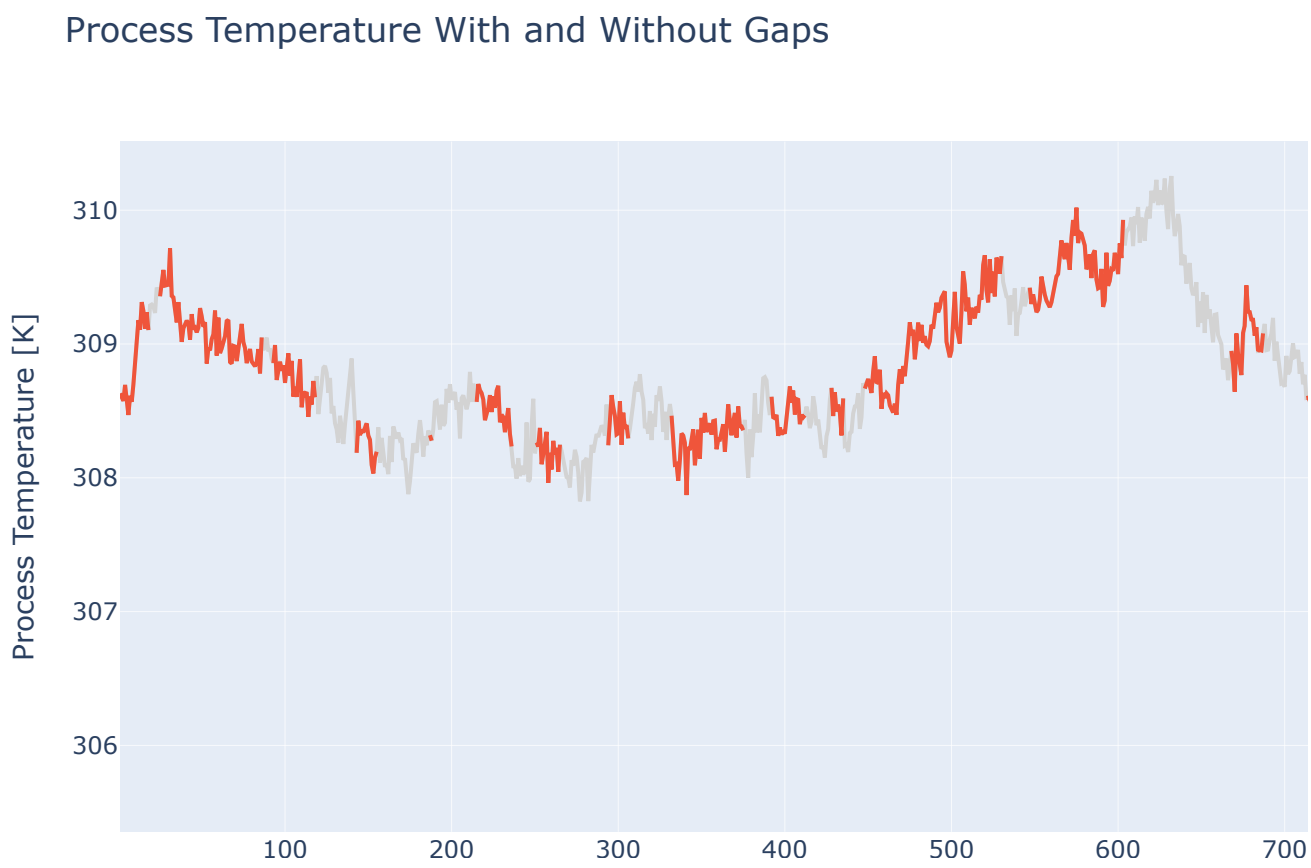
```
dftemp["process_temperature_K"] += rng.normal(
    loc=0,
    scale=0.1,
    size=len(dftemp),
)  # Add a little bit of noise to the process temperature (mean 0, std 0.5)
dftemp.loc[:, "process_temperature_K_gaps"] = dftemp["process_temperature_K"]

# Randomly select start indices and gap lengths
num_gaps = 50  # Number of gaps to introduce
gap_min = 3  # Minimum gap length
gap_max = 20  # Maximum gap length

# Introduce gaps
n = len(dftemp)
starts = rng.choice(n - gap_max, size=num_gaps, replace=False)
lengths = rng.integers(gap_min, gap_max + 1, size=num_gaps)

for start, length in zip(starts, lengths):
    dftemp.loc[start : (start + length), "process_temperature_K_gaps"] = None
```

*Helper function `plot_temperature_data`, which visualizes the original temperature data with and without gaps. If imputated column **int_column** is provided, it will be plotted as well alongside a difference plot.*

▶ Code

Visualization of the temperature data with and without gaps

```
plot_temperature_data(dftemp)
```



UDI

5.1.3 Imputation methods

5.1.3.1 Forward fill imputation

Forward fill imputation (also known as *last observation carried forward*) is a simple and commonly used method for handling missing data in time series. It involves replacing missing values with the most recent non-missing value prior to the gap.

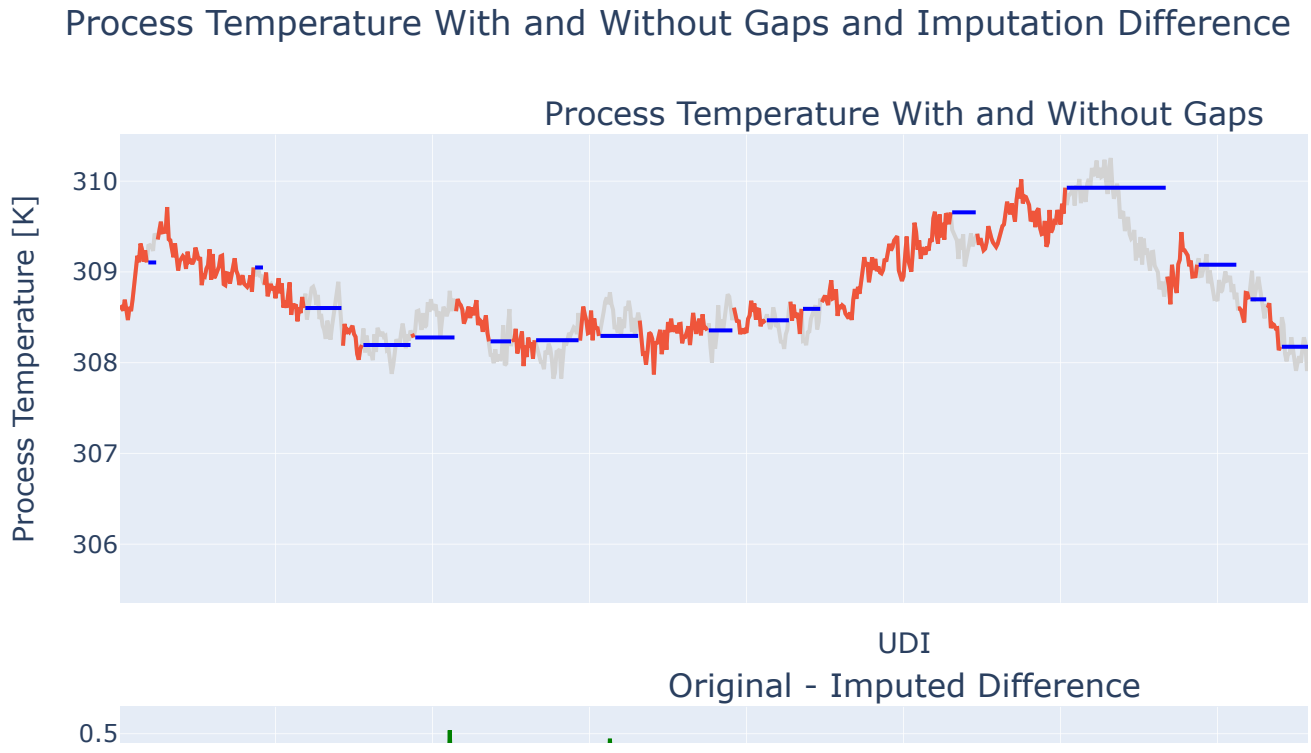
Advantages:

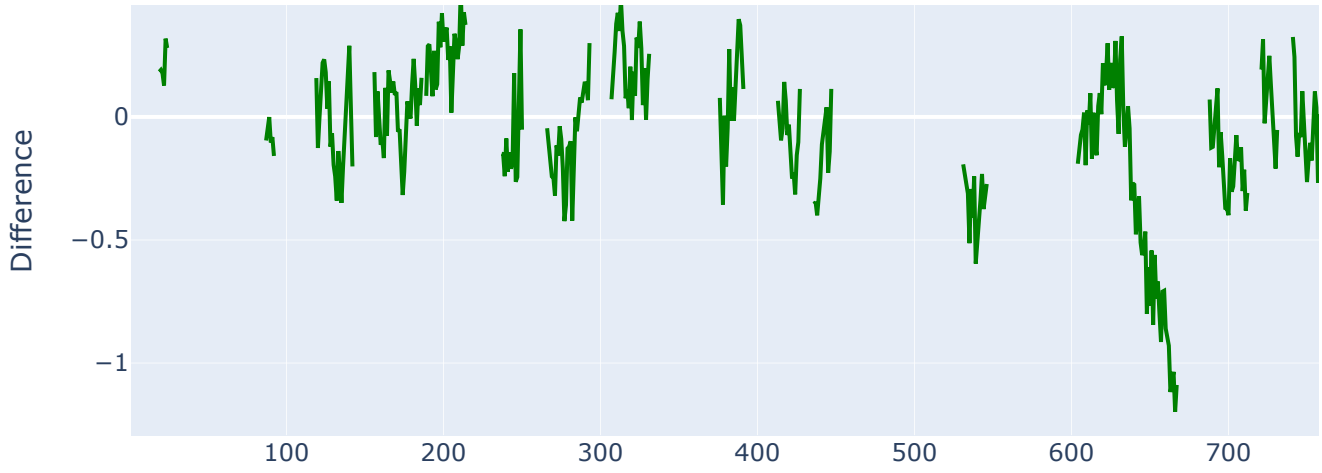
- Simple to implement and computationally efficient.
- Preserves the last known state, which can be useful in certain contexts.
- Works well for short gaps where the last observation is a reasonable estimate for the missing values.

Disadvantages:

- Can introduce bias if the last observation is not representative of the missing values.
- May not capture trends or patterns in the data, especially for long gaps.
- Can lead to unrealistic flat segments in the time series.
- Does not account for seasonality or cyclic patterns in the data.

```
dftemp.loc[:, "temp_ff"] = dftemp[
    "process_temperature_K_gaps"
].ffill()  # Pandas provides forward fill imputation out of the box.

plot_temperature_data(dftemp, int_column="temp_ff")
```





5.1.3.2 Backward fill imputation

Backward fill imputation (also known as *next observation carried backward*) is another simple method for handling missing data in time series. It involves replacing missing values with the next non-missing value that follows the gap.

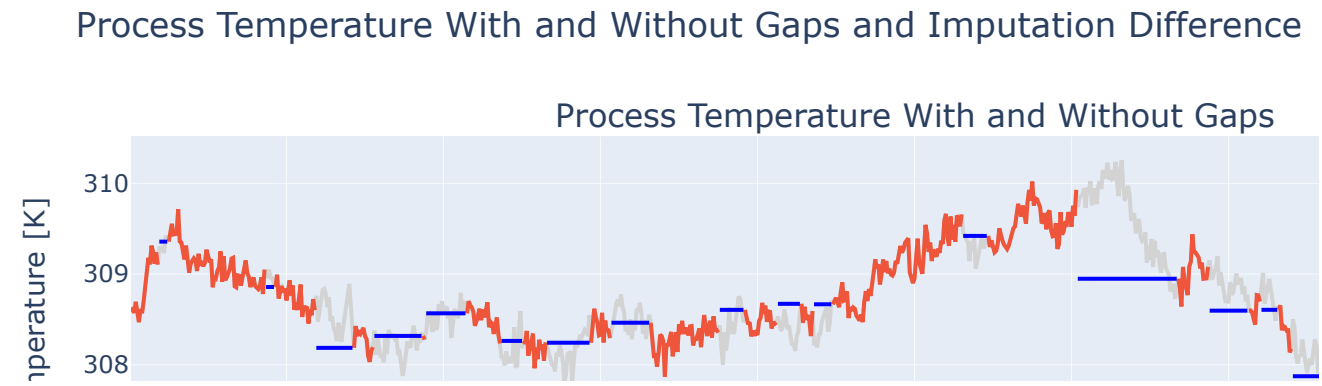
Advantages:

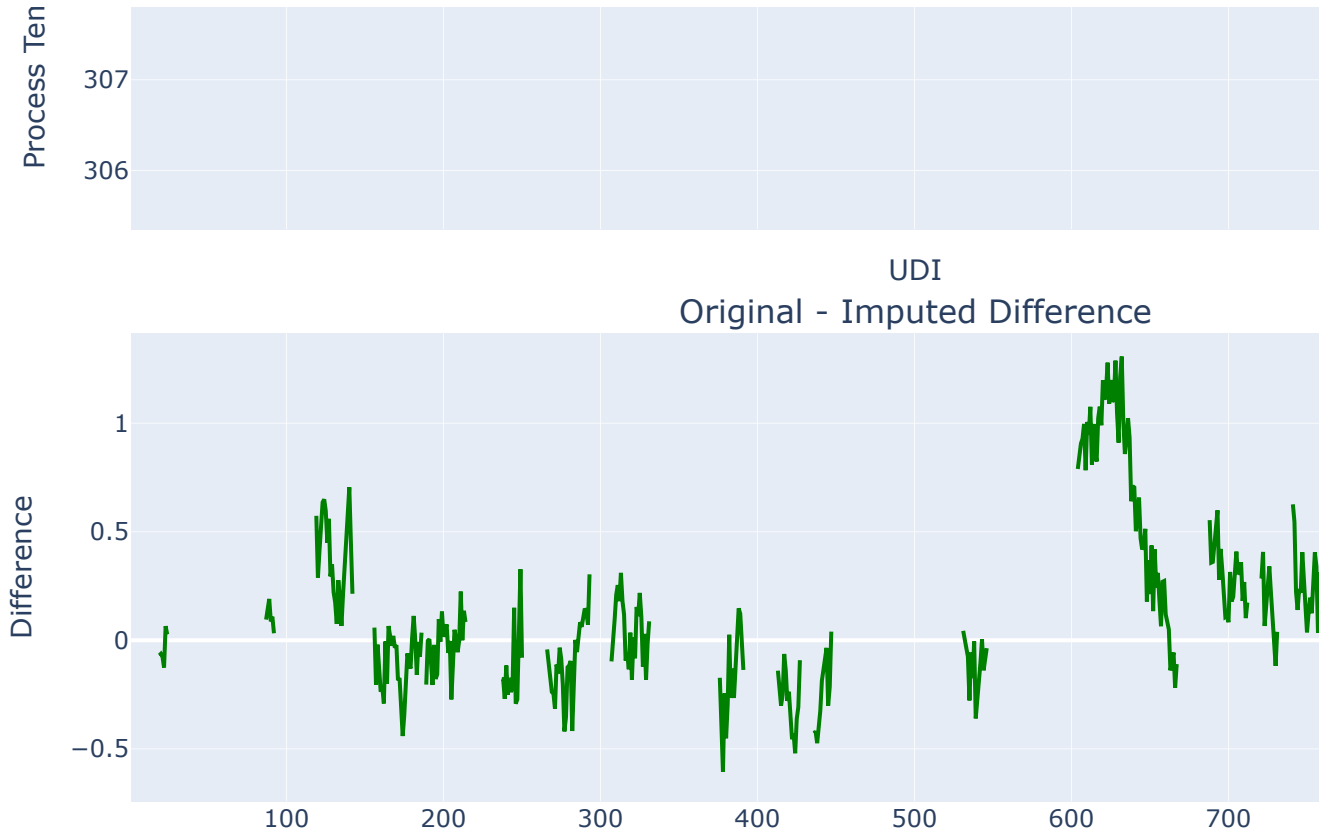
- Simple to implement and computationally efficient.
- Preserves the next known state, which can be useful in certain contexts.
- Works well for short gaps where the next observation is a reasonable estimate for the missing values.

Disadvantages:

- Can introduce bias if the next observation is not representative of the missing values.
- May not capture trends or patterns in the data, especially for long gaps.
- Can lead to unrealistic flat segments in the time series.
- Does not account for seasonality or cyclic patterns in the data.

```
dftemp.loc[:, "temp_bf"] = dftemp[
    "process_temperature_K_gaps"
].bfill()  # Pandas provides backward fill imputation out of the box.

plot_temperature_data(dftemp, int_column="temp_bf")
```





5.1.3.3 Linear Interpolation

Linear interpolation is a method used to estimate missing values in a time series by connecting two known data points with a straight line and using that line to fill in the gaps.

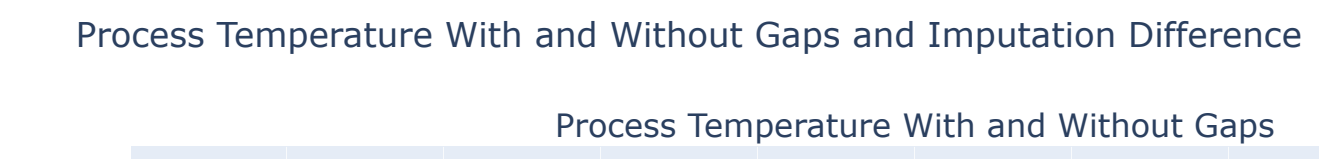
Advantages:

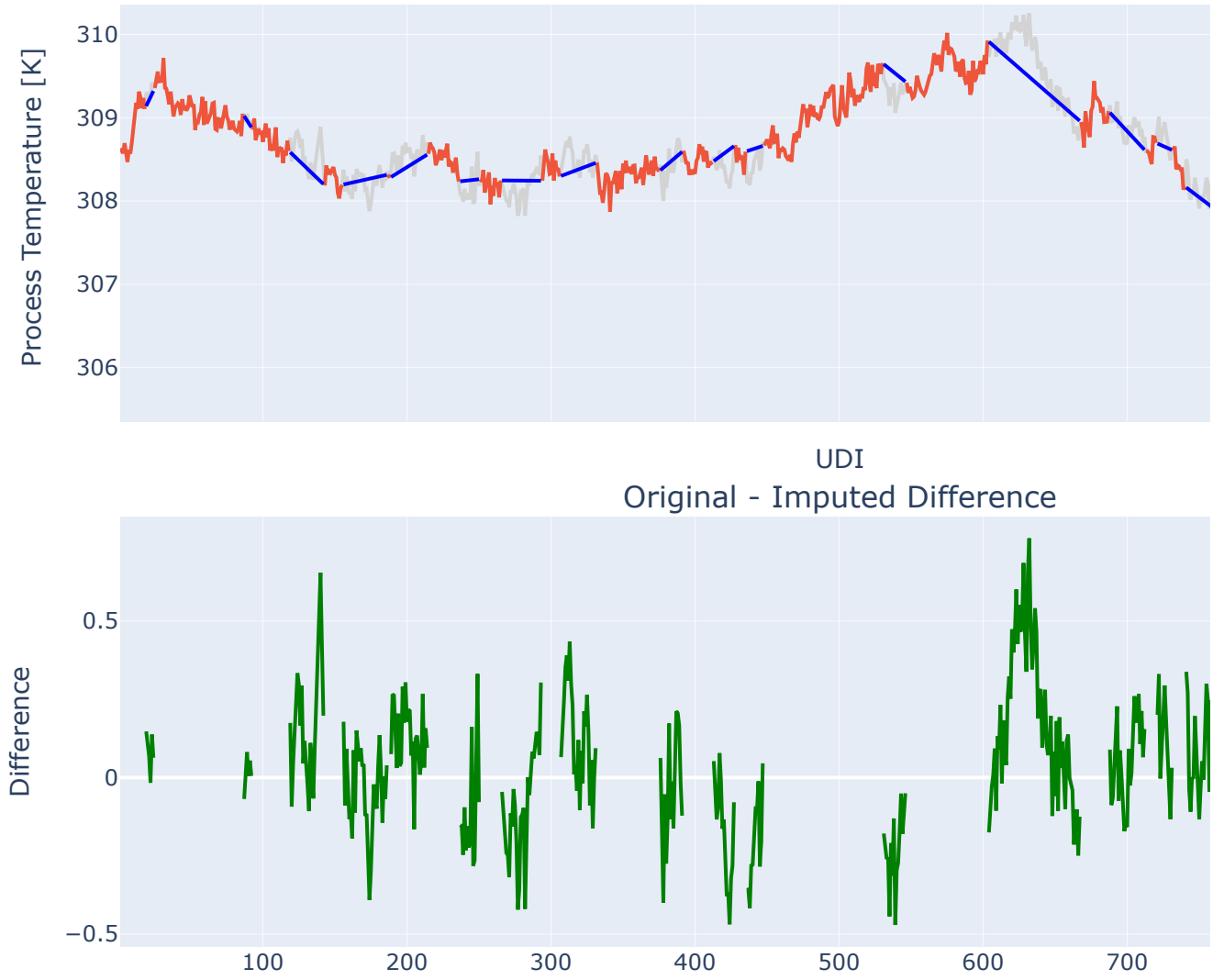
- Can provide more accurate estimates than forward or backward fill, especially for short gaps.
- Preserves trends and patterns in the data better than simple imputation methods.

Disadvantages:

- Assumes a linear relationship between data points, which may not always be valid.
- Can introduce bias if the underlying data has non-linear trends.
- May not perform well for long gaps or highly volatile data.

```
dftemp.loc[:, "temp_linear"] = dftemp["process_temperature_K_gaps"].interpolate(
    "linear",
)

plot_temperature_data(dftemp, int_column="temp_linear")
```





5.1.3.4 Polynomial interpolation

Polynomial interpolation is a method used to estimate missing values in a time series by fitting a polynomial function of a given order to the known data points and using that function to fill in the gaps.

Advantages:

- Can provide more accurate estimates than linear interpolation, especially for non-linear trends.

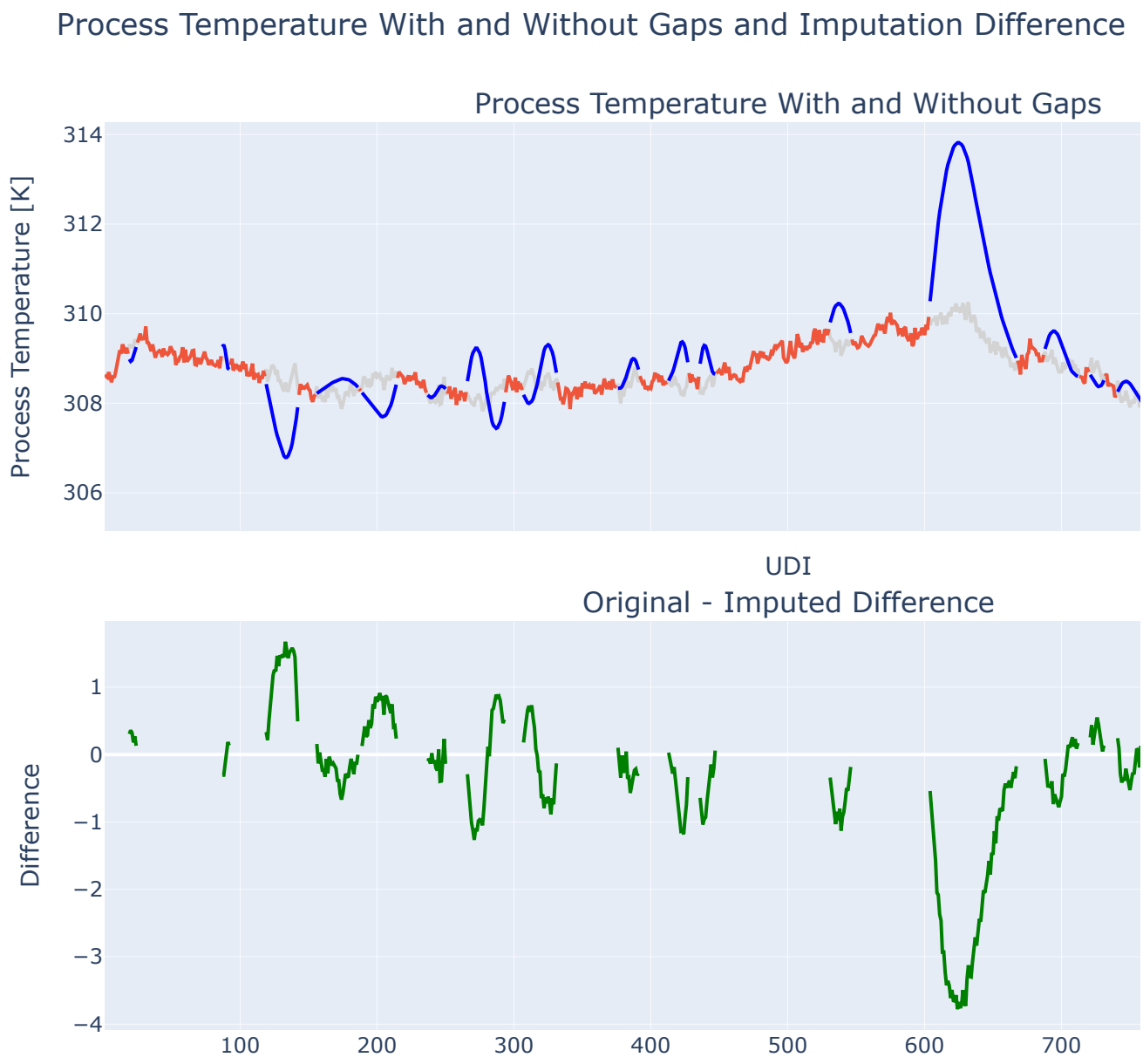
Disadvantages:

- More computationally expensive than other methods.
- Tends to overshoot or oscillate between known data points, leading to unrealistic estimates.
- May not perform well for short gaps or sparse data.
- Sensitive to noise in the data.

Demonstrate polynomial interpolation with a polynom of order 2

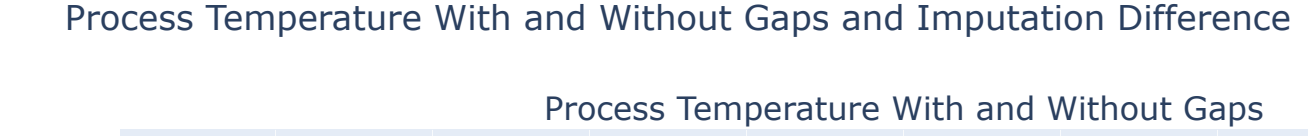
```
dftemp.loc[:, "temp_poly2"] = dftemp["process_temperature_K_gaps"].interpolate(
    "polynomial",
    order=2,
```

```
)

plot_temperature_data(dftemp, int_column="temp_poly2")
```



Filling gaps with a polynomial of order 5

```
dftemp.loc[:, "temp_poly5"] = dftemp["process_temperature_K_gaps"].interpolate(
    "polynomial", order=5
)

plot_temperature_data(dftemp, int_column="temp_poly5")
```



5.1.3.5 Spline interpolation

Spline interpolation is a method used to estimate missing values in a time series by fitting piecewise polynomial functions (splines) to the known data points and using those functions to fill in the gaps.

The specific type *Piecewise Cubic Hermite Interpolating Polynomial* (PCHIP) in particular preserves the monotonicity of the data and avoids overshooting, making it suitable for many real-world applications.

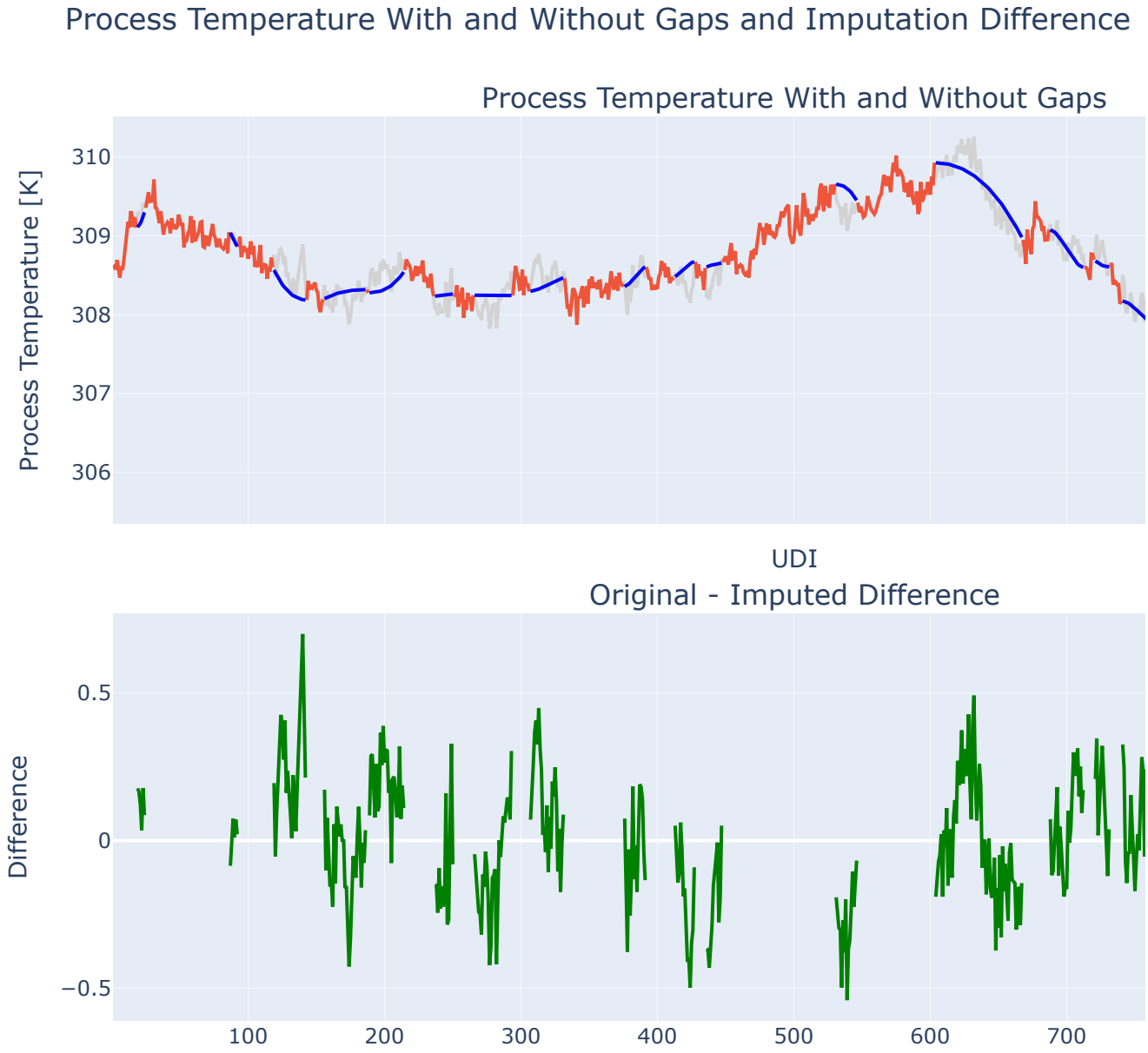
Advantages:

- Can provide smooth and accurate estimates, especially for non-linear trends.
- Can capture complex patterns in the data better than simpler methods.
- Less prone to overfitting compared to high-degree polynomial interpolation.

Disadvantages:

- More computationally expensive than simpler methods.
- Requires careful selection of spline parameters (e.g., degree, knots).
- May not perform well for very short gaps or sparse data.

```
dftemp.loc[:, "temp_spline"] = dftemp["process_temperature_K_gaps"].interpolate("pchip")

plot_temperature_data(dftemp, int_column="temp_spline")
```



5.1.4 Data smoothing

Especially for noisy data and higher-order methods like splines or polynomials, interpolation methods can produce unrealistic results, such as overshooting or oscillations between known data points. Smoothing the time series prior to interpolation can help to mitigate this issue by reducing noise and capturing the underlying trend of the data.

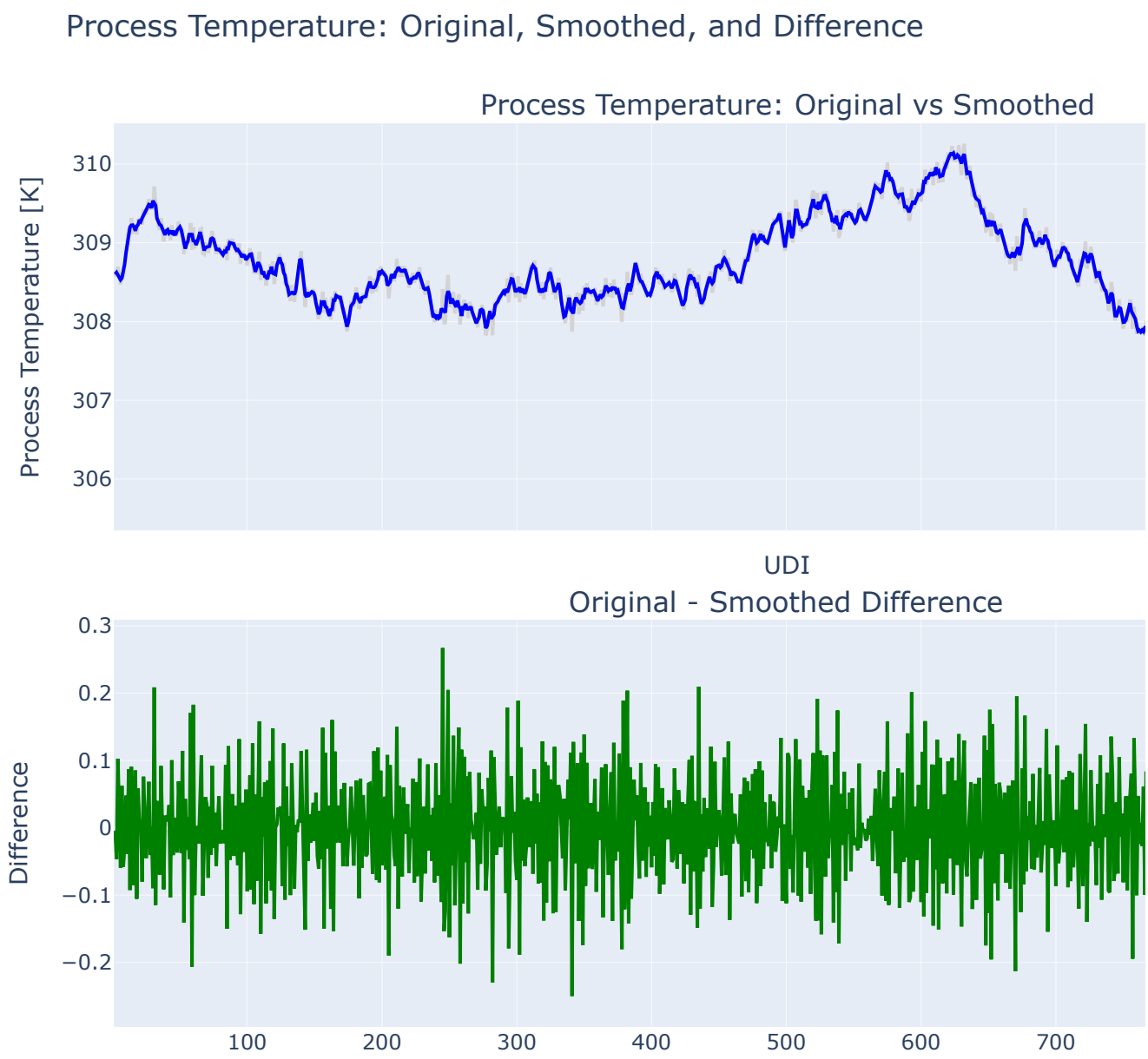
Smoothing techniques, such as moving averages or rolling medians, can be applied to the time series before performing interpolation. Rolling median, by design, is more robust to outliers and preserves edges better than rolling mean. Rolling mean tends to smooth out the data more uniformly but is more sensitive to outliers.

When applying smoothing, it is important to choose an appropriate window size. The window-size determines the number of consecutive data points used to calculate the smoothed value at each point in the time series. The larger the window size, the smoother the resulting time series will be, but it may also lead to a loss of detail and important features in the data.

5.1.4.1 Rolling mean smoothing

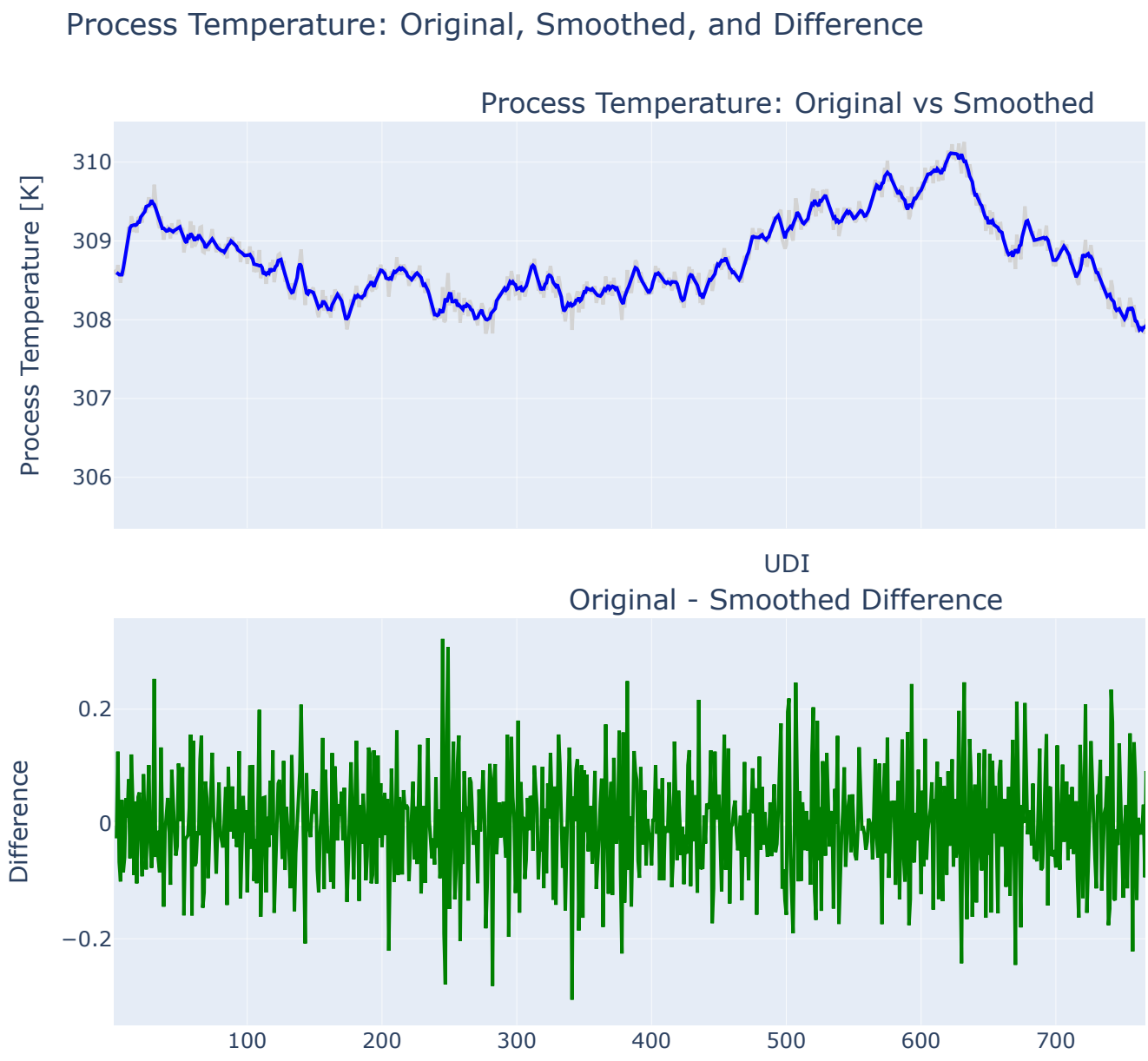
Rolling mean with a window size of 3:

```
dftemp["temp_mean_rolling3"] = (
    dftemp["process_temperature_K"].rolling(window=3, center=True).mean()
)
plot_smoothed_temperature_data(dftemp, smoothed_column="temp_mean_rolling3")
```



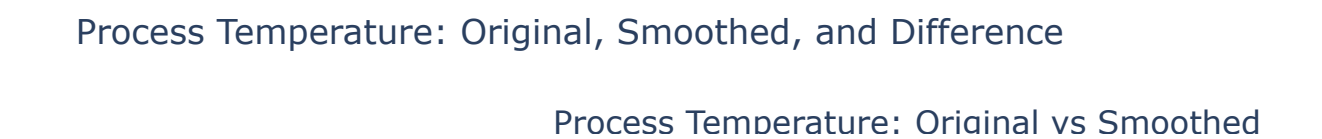
Rolling mean with a window size of 5

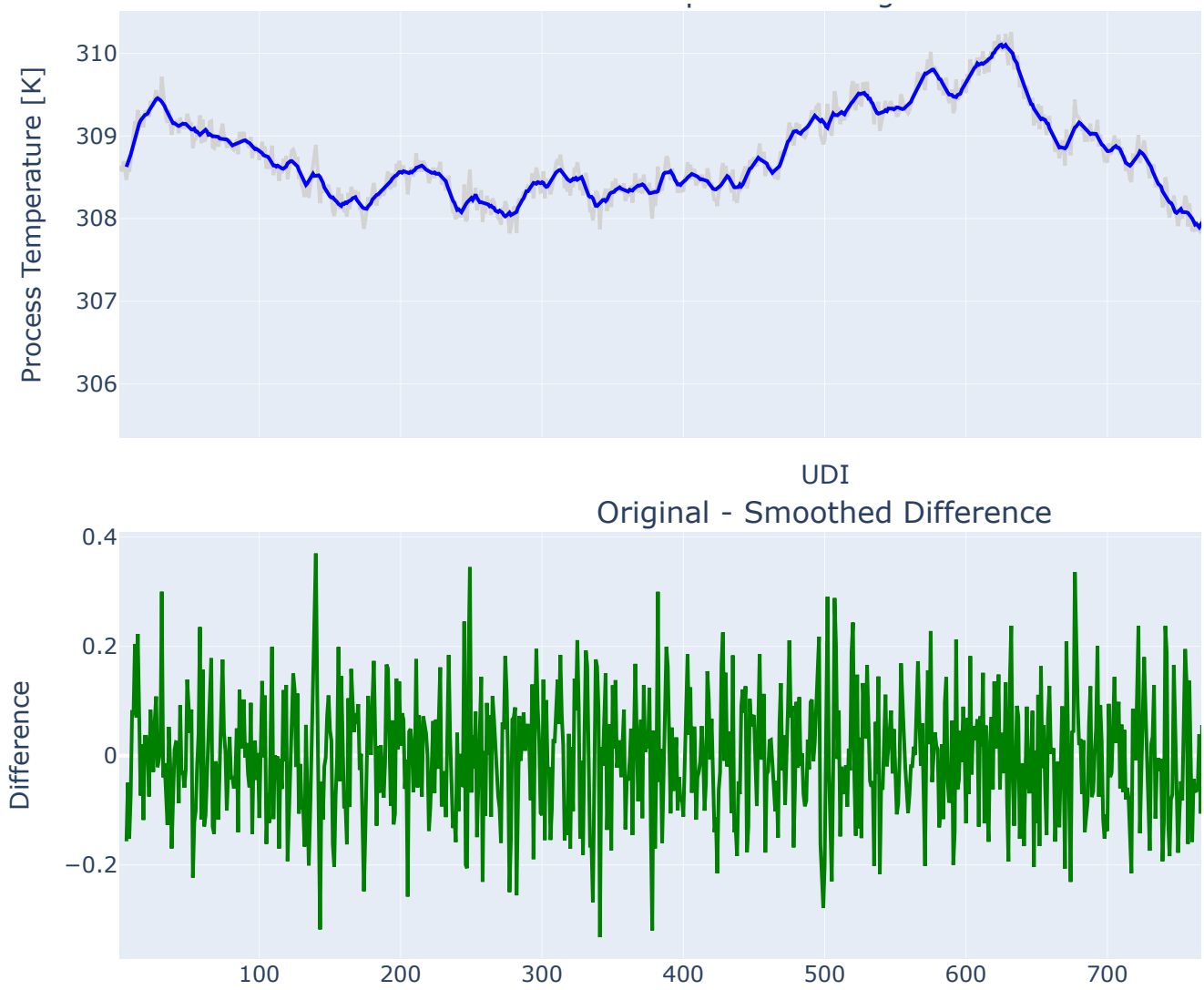
```
dftemp["temp_mean_rolling5"] = (
    dftemp["process_temperature_K"].rolling(window=5, center=True).mean()
)
plot_smoothed_temperature_data(dftemp, smoothed_column="temp_mean_rolling5")
```



Rolling mean with a window size of 10

```
dftemp["temp_mean_rolling10"] = (
    dftemp["process_temperature_K"].rolling(window=10, center=True).mean()
)
plot_smoothed_temperature_data(dftemp, smoothed_column="temp_mean_rolling10")
```

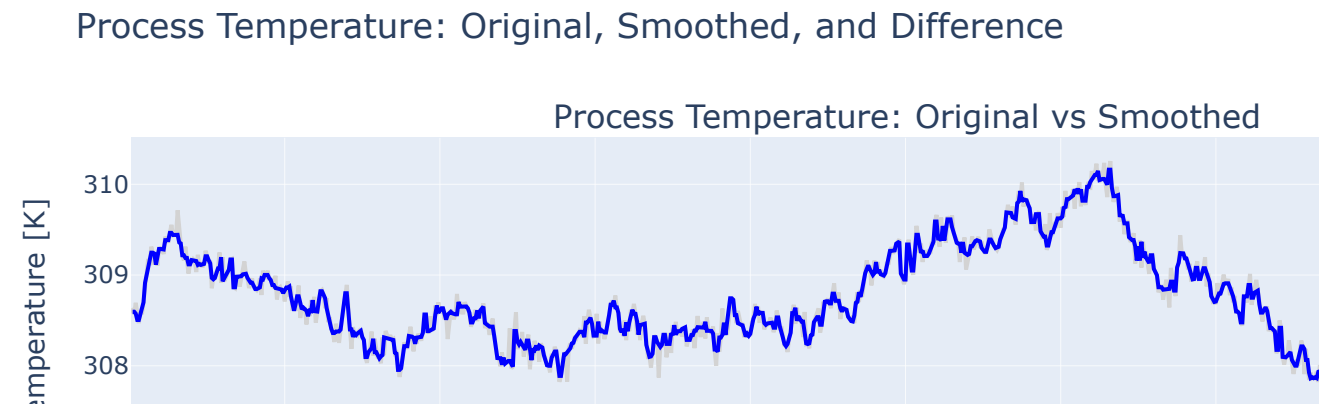


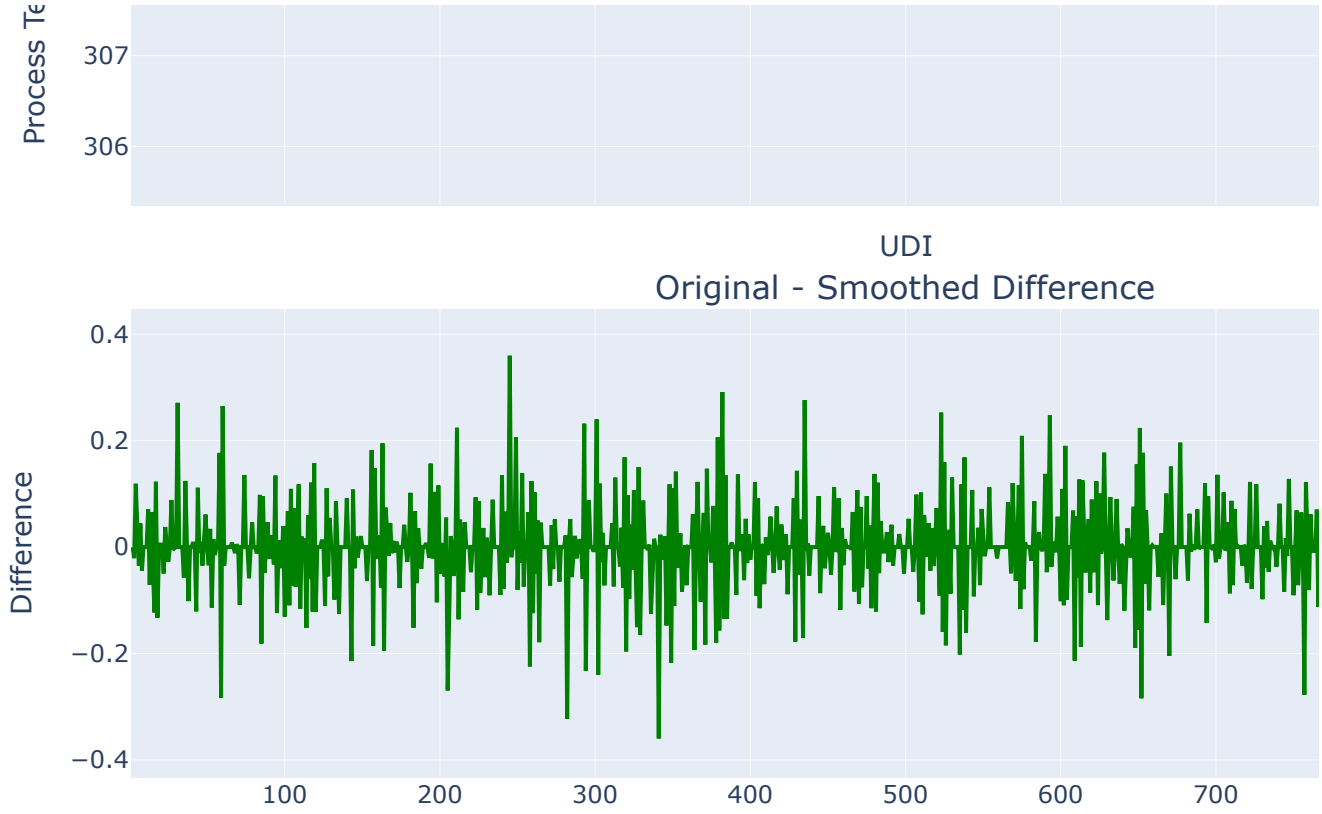


5.1.4.2 Rolling median smoothing

Rolling median with a window size of 3

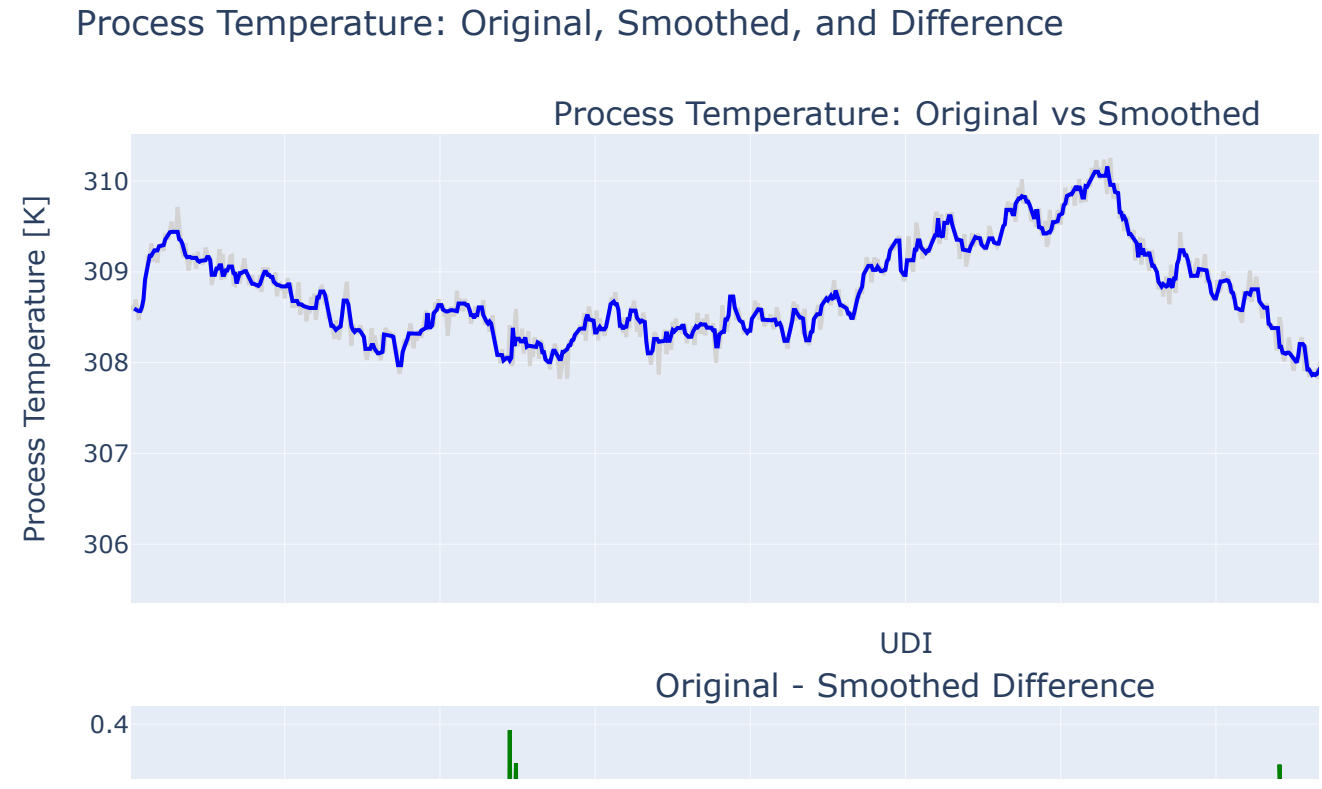
```
dftemp["temp_median_rolling3"] = (
    dftemp["process_temperature_K"].rolling(window=3, center=True).median()
)
plot_smoothed_temperature_data(dftemp, smoothed_column="temp_median_rolling3")
```

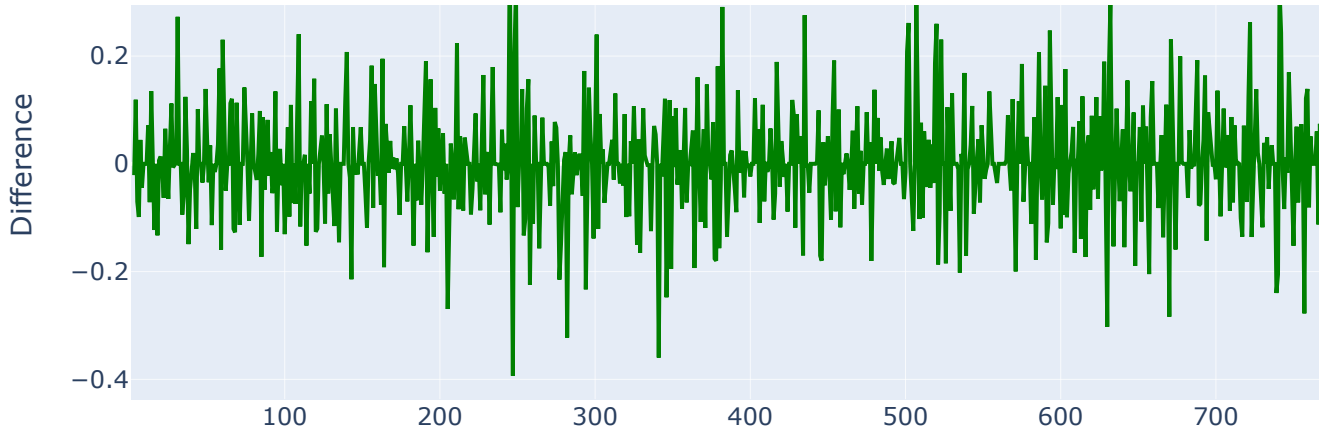




Rolling median with a window size of 5

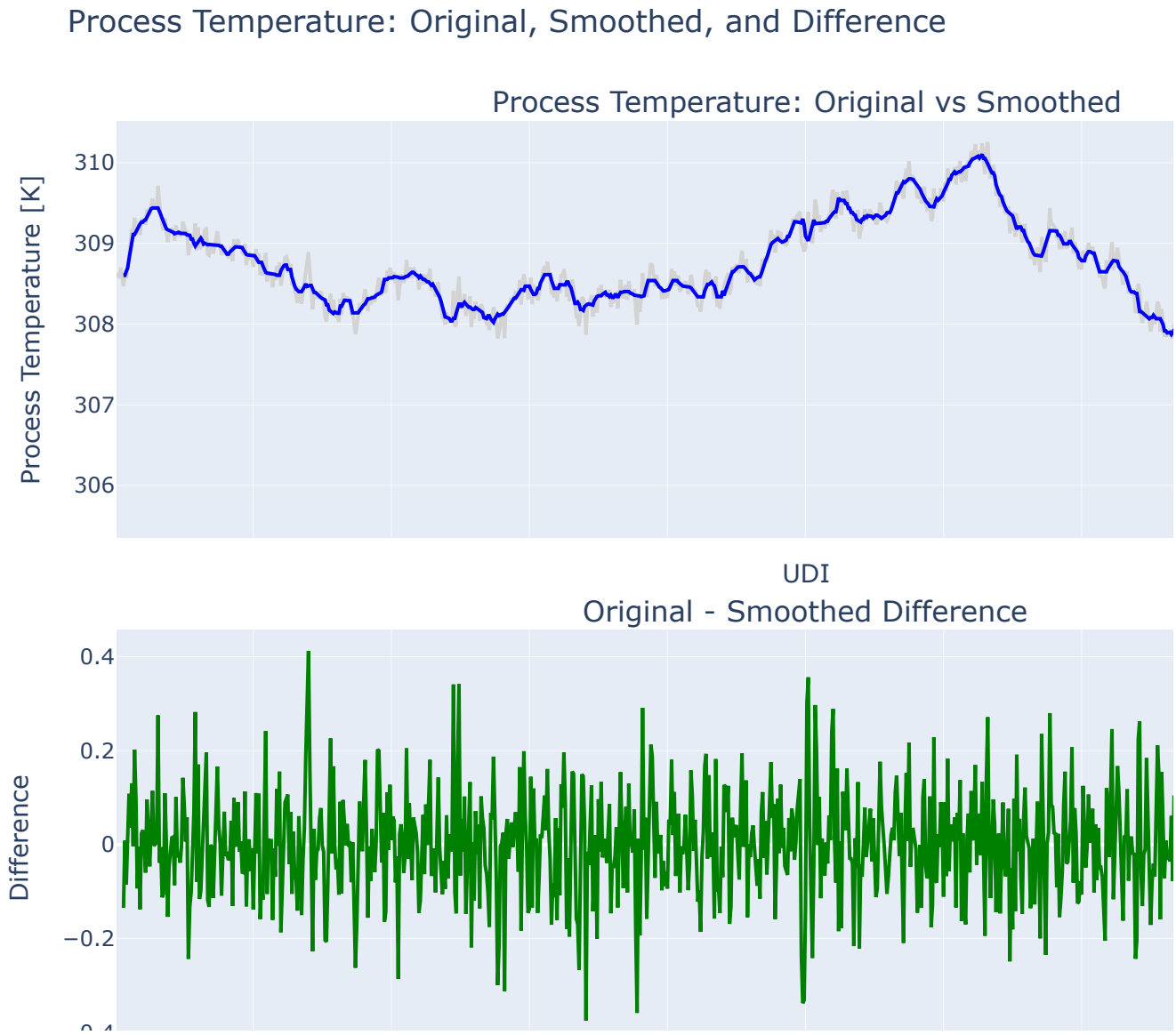
```
dftemp["temp_median_rolling5"] = (
    dftemp["process_temperature_K"].rolling(window=5, center=True).median()
)
plot_smoothed_temperature_data(dftemp, smoothed_column="temp_median_rolling5")
```





Rolling median with a window size of 10

```
dftemp["temp_median_rolling10"] = (
    dftemp["process_temperature_K"].rolling(window=10, center=True).median()
)
plot_smoothed_temperature_data(dftemp, smoothed_column="temp_median_rolling10")
```



5.1.5 Applying smoothing prior to interpolation

Applying smoothing prior to interpolation can help to reduce noise and improve the accuracy of the imputed values, especially for noisy data. Note that smoothing is applied on the gapped data, not on the original data.

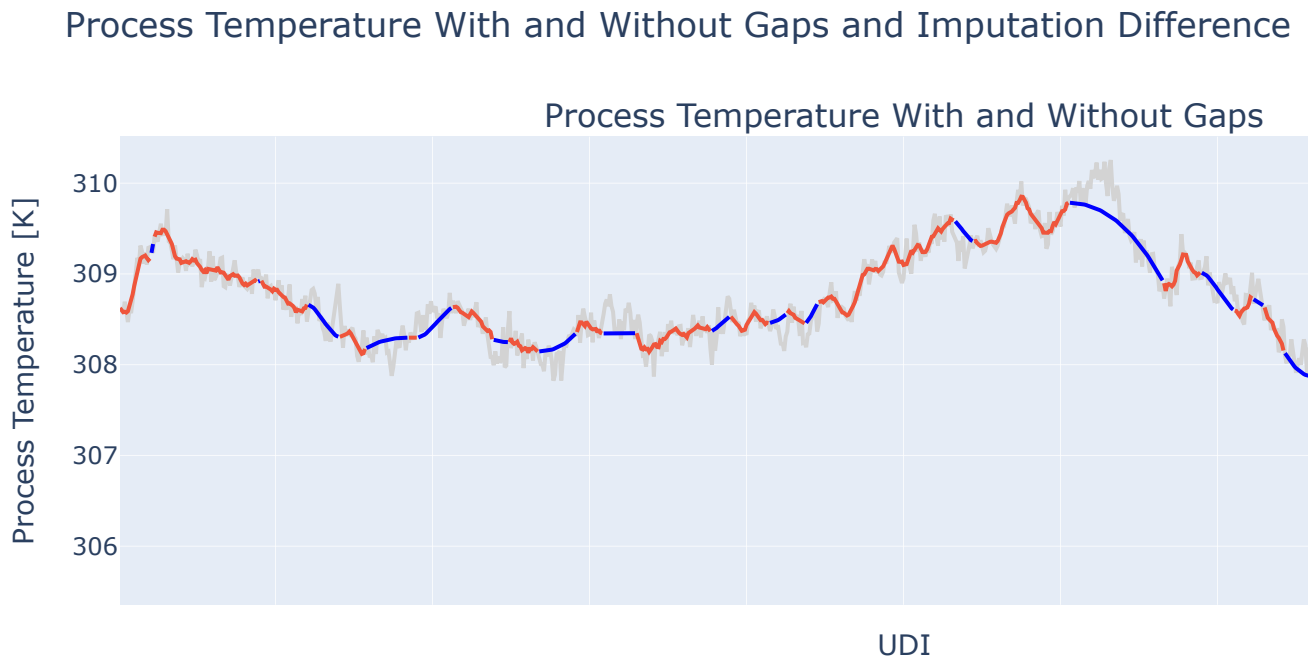
The argument `min_periods=2` in the rolling mean function ensures that at least two non-NaN values are required to compute the mean, which helps to not further erode the gapped series. PCHIP interpolation is then applied on the smoothed series to fill the gaps.

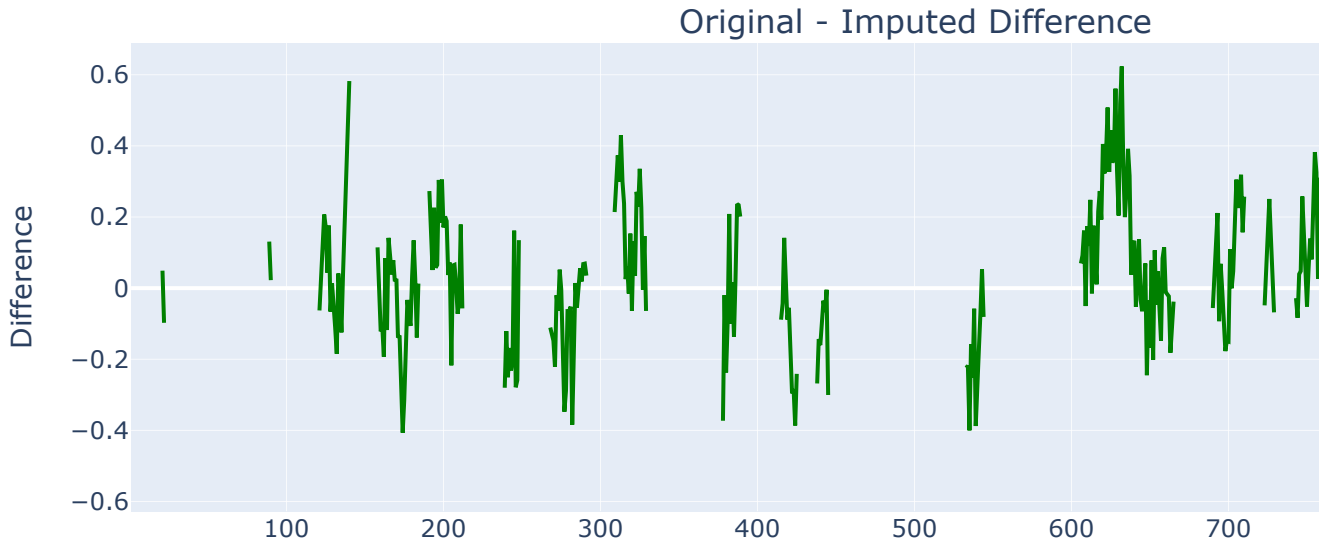
```
dftemp.loc[:, "process_temperature_K_gaps_smoothed"] = (
    dftemp["process_temperature_K_gaps"]
    .rolling(window=7, center=True, min_periods=2)
    .mean()
)
dftemp.loc[:, "temp_smoothed_pchip"] = dftemp[
    "process_temperature_K_gaps_smoothed"
].interpolate("pchip")
```

Prepare dataframe for plotting

▶ Code

```
plot_temperature_data(
    dftemp,
    gap_column="process_temperature_K_gaps_smoothed",
    int_column="temp_smoothed_pchip",
)
```





Industrial Data Science - 2026 - CC BY-NC-SA 4.0

Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

6 Time Series Decomposition

Time series data often exhibits patterns such as trends, seasonality, and cycles. Decomposing a time series int its components can help to better understand the underlying processes.

Time series are usually decomposed into three components:

- **Trend**: T_t, long-term progression of the series (e.g., increasing or decreasing).
- **Seasonality**: S_t, regular patterns that repeat over a fixed period (e.g., daily, weekly, yearly).
- **Residual**: I_t, random noise or irregular component.

Additive models assume that the components add together to form the time series:

$$Y_t = T_t + S_t + I_t$$

Multiplicative models assume that the components multiply together to form the time series:

$$Y_t = T_t \times S_t \times I_t$$

> **Note**
>
> Sometimes, also a fourth component, **Cyclic** (C_t), is considered, which represents long-term oscillations that are not of fixed period.

6.1 Examples

This example uses the package `statsmodels` for time series decomposition.

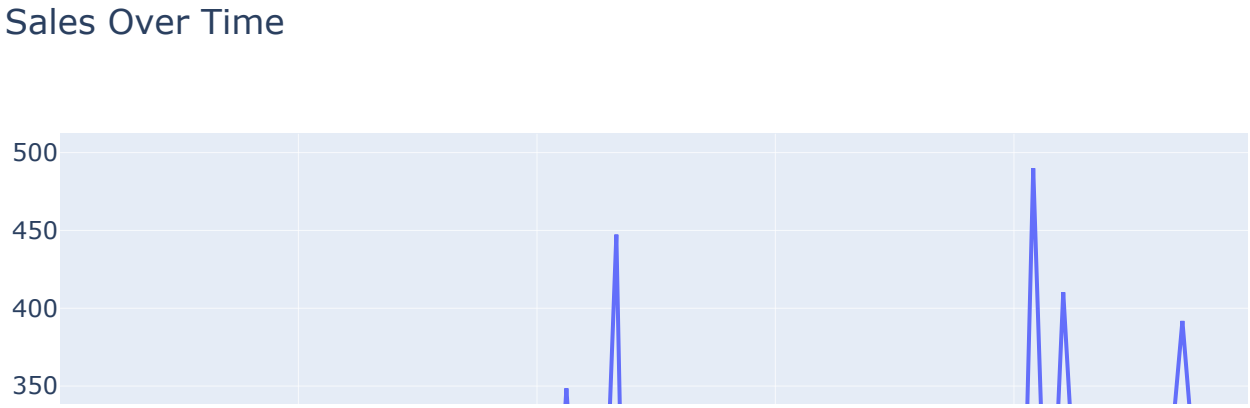
6.1.1 Imports

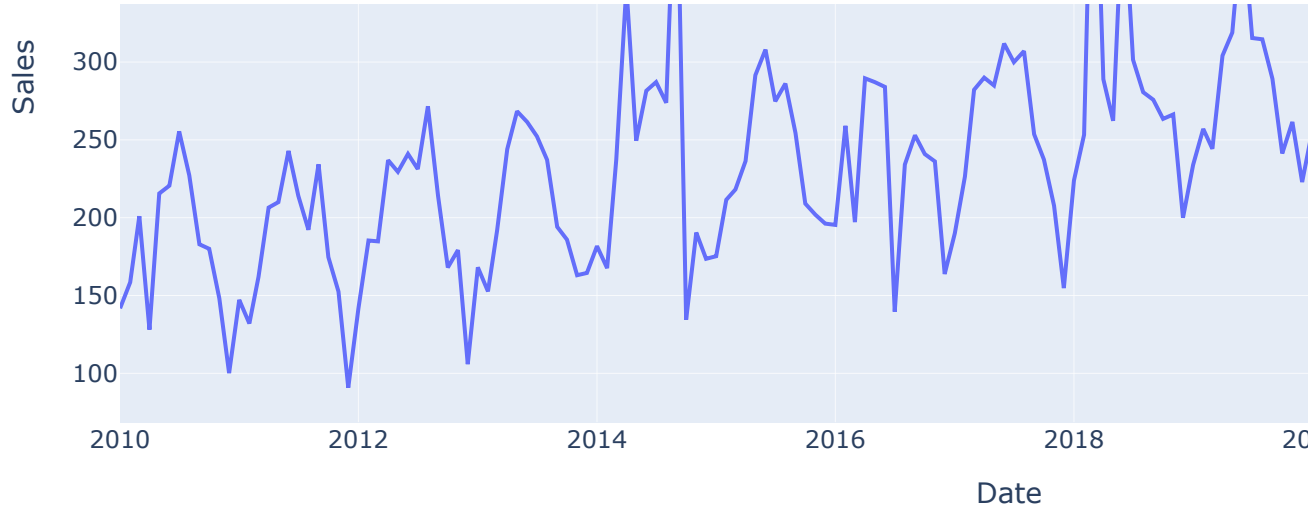
▶ Code

6.1.2 Generating Example Data

▶ Code

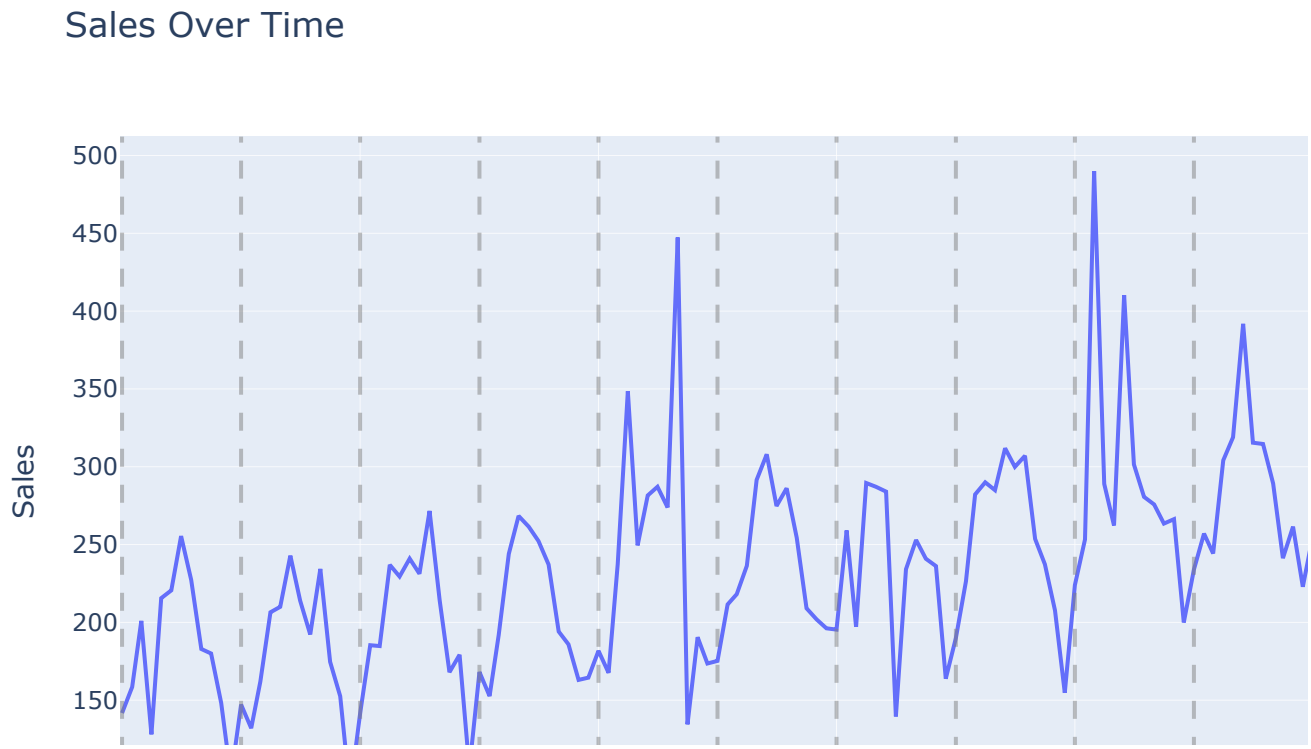
```
fig = px.line(df, y="Sales", title="Sales Over Time")
fig.show()
```

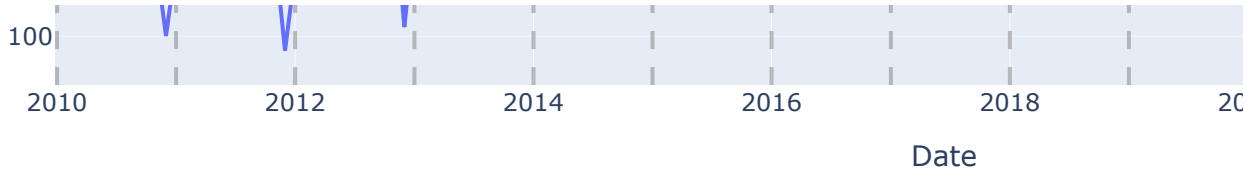




In the above plot, we can see a clear upward trend, but identifying seasonality is more difficult. To support this visual analysis, we add vertical lines at the beginning of each year.

```
fig = px.line(df, y="Sales", title="Sales Over Time")

for year in range(
    df.index.to_series().dt.year.min(), df.index.to_series().dt.year.max() + 1
):  # type: ignore[attr-defined]
    fig.add_vline(
        x=pd.Timestamp(f"{year}-01-01"),
        line_dash="dash",
        line_color="gray",
        opacity=0.5,
    )

fig.show()
```





Now it is apparent that there is a yearly seasonal pattern, with peaks around mid-year.

6.1.3 Algorithmic Decomposition

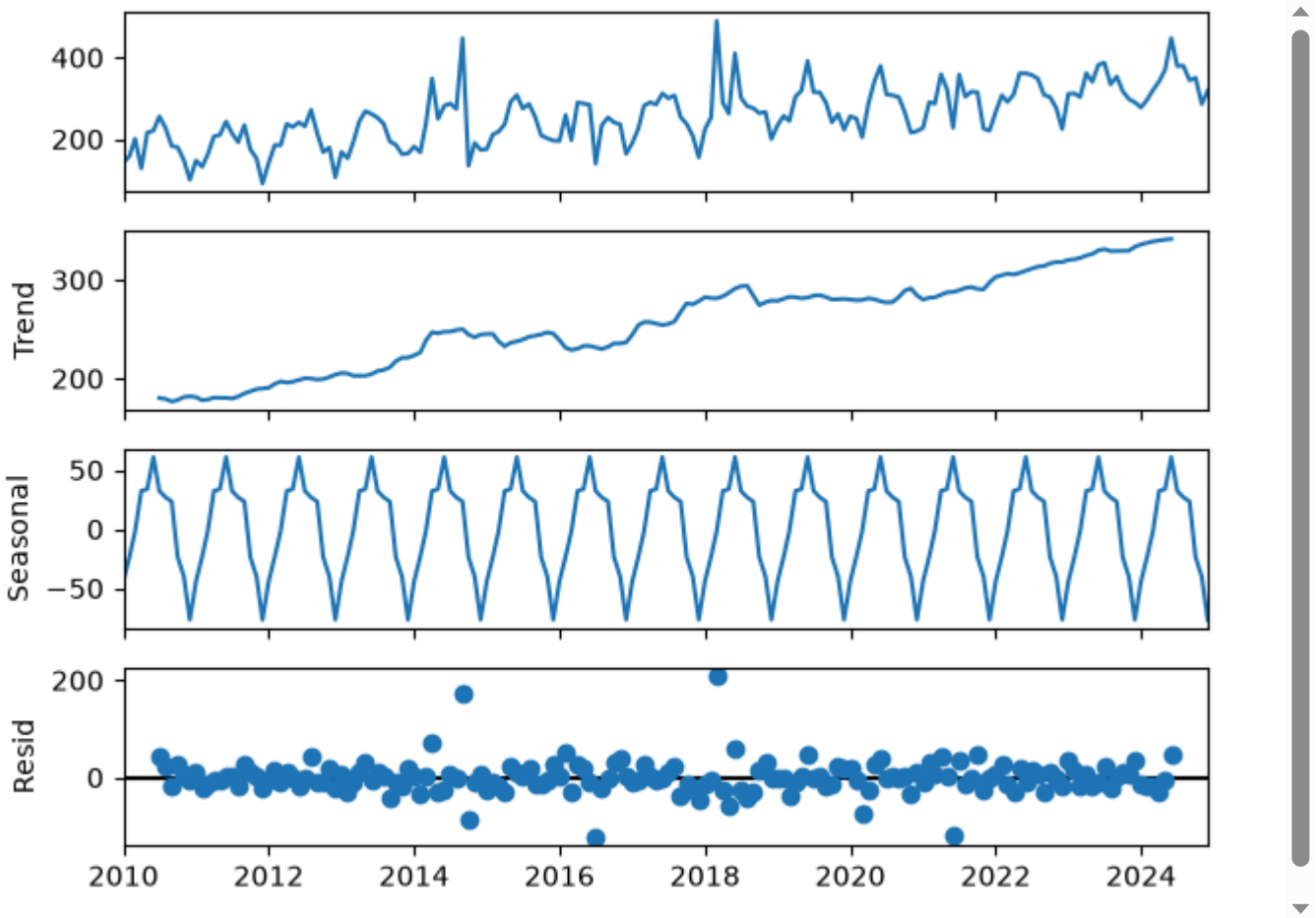
Algorithmic decomposition of a time series into its components is not a trivial task. Many algorithms exist to achieve this, each with its own advantages and disadvantages.

In the following, we use a very simple additive model based on moving averages to decompose our time series. First, the trend is estimated using a moving average with a smaller window size. Then, the detrended series is calculated by subtracting the trend from the original series. The seasonal component is estimated by averaging the values for each season (e.g., each month, year, ..) in the detrended time series. Finally, the residual component is calculated by subtracting both the trend and seasonal components from the original series.

Note that this is a very naive approach and according to the documentation of `seasonal_decompose` a more sophisticated method should be preferred. However, it is easy to understand and serves well for demonstration purposes.

```
decomposed_ts = seasonal_decompose(df, model="additive")
```

```
decomposed_ts.plot();  # the object has a built-in plot method (based on matplotlib)
```

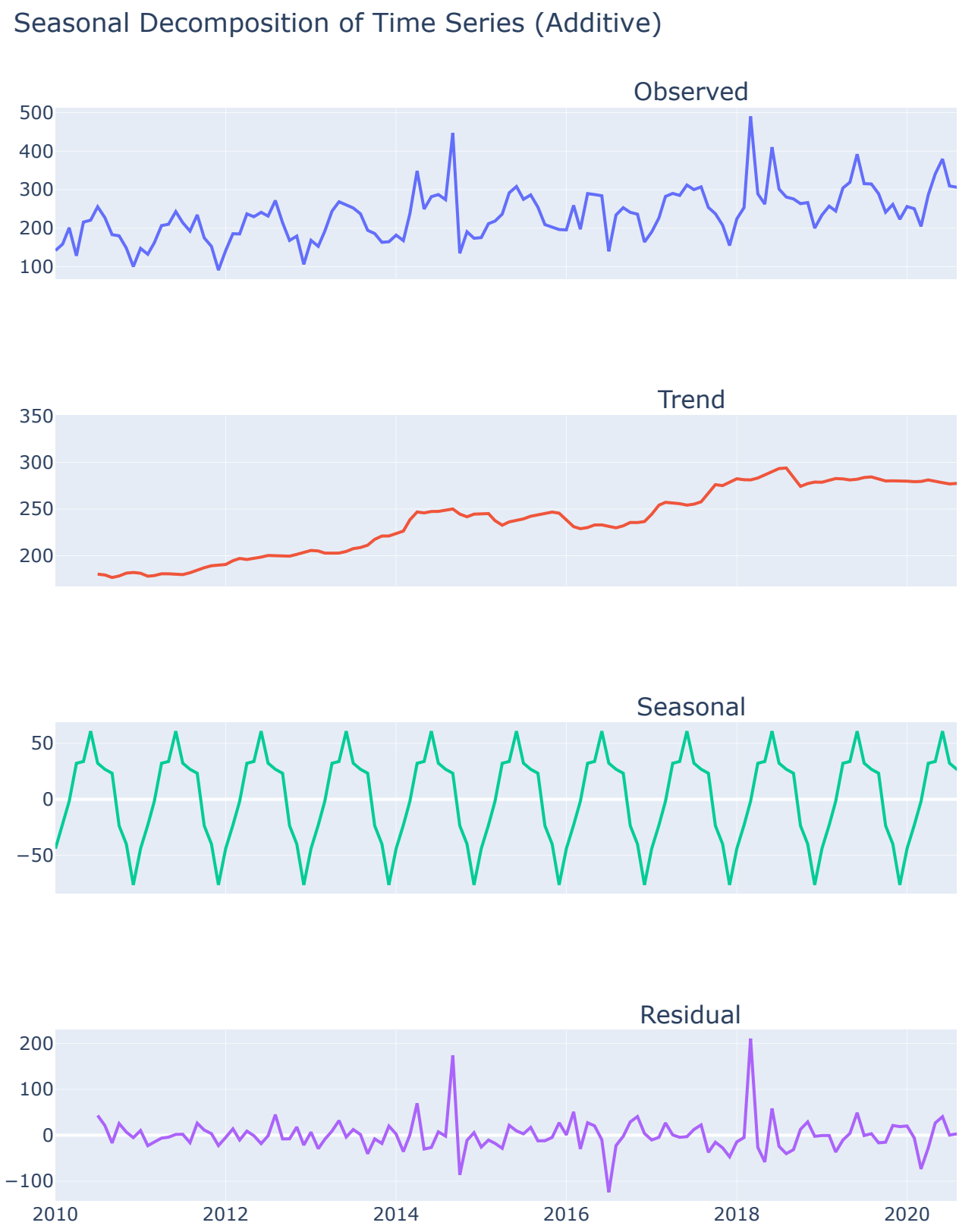


Helper function to create a plotly figure with four plots

▶ Code

Please note the different y-axes scales in the following plot.

```
seasonal_decompose_plotly(
    decomposed_ts, title="Seasonal Decomposition of Time Series (Additive)"
)
```

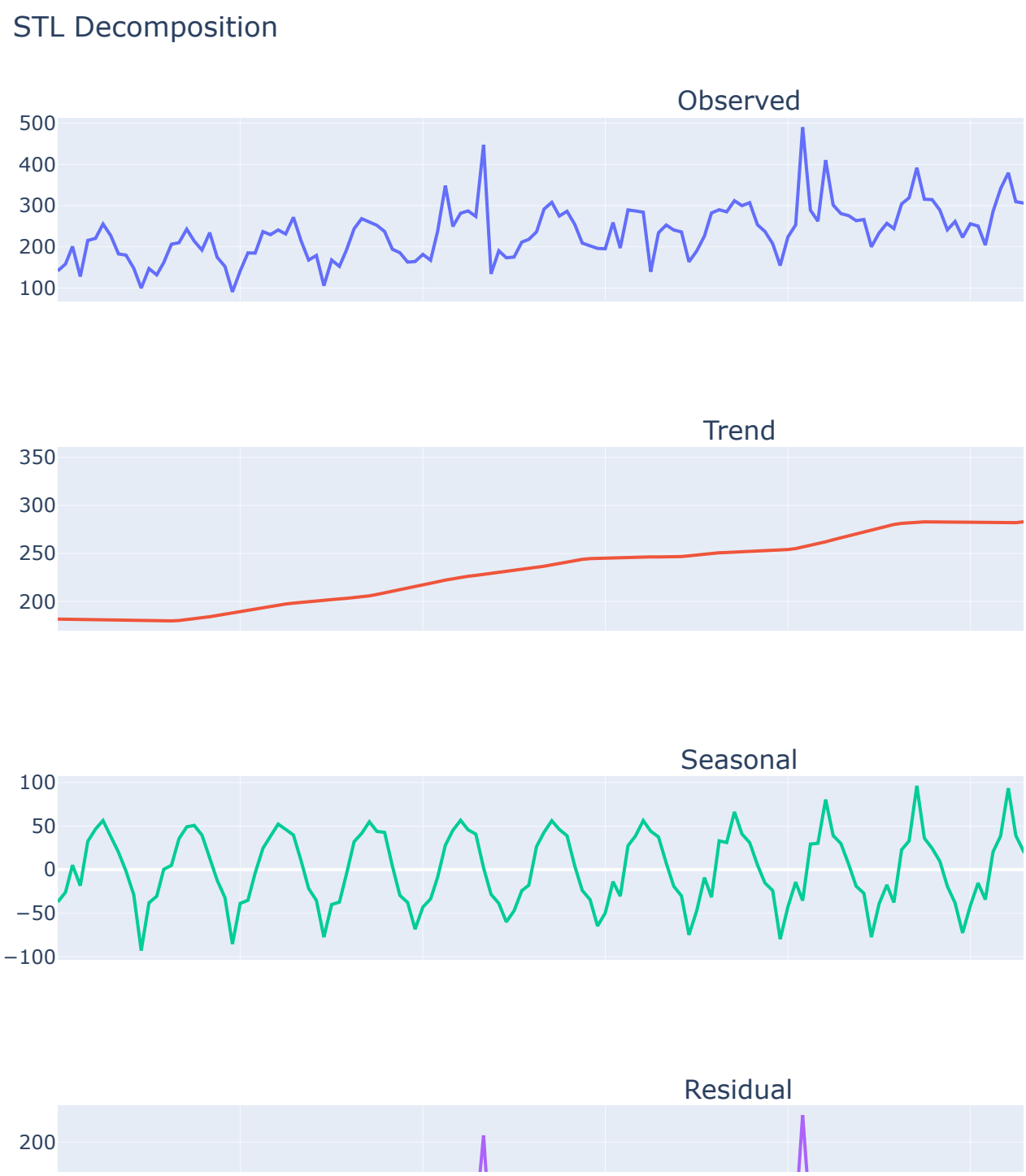


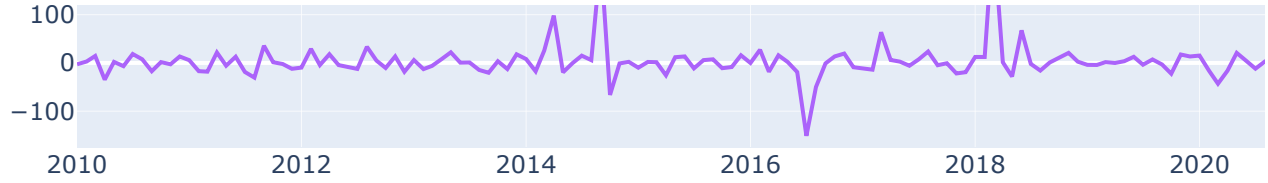
A more robust method for time series decomposition is STL (*Seasonal and Trend decomposition using Loess*).

It not only allows for more flexibility in the decomposition process (e.g. look at seasonal changes over time), but is also more robust to outliers.

```
decomp_stl = STL(
    df, robust=True
).fit()  # robust=True to be less sensitive to expected outliers
```

```
seasonal_decompose_plotly(decomp_stl, title="STL Decomposition")
```





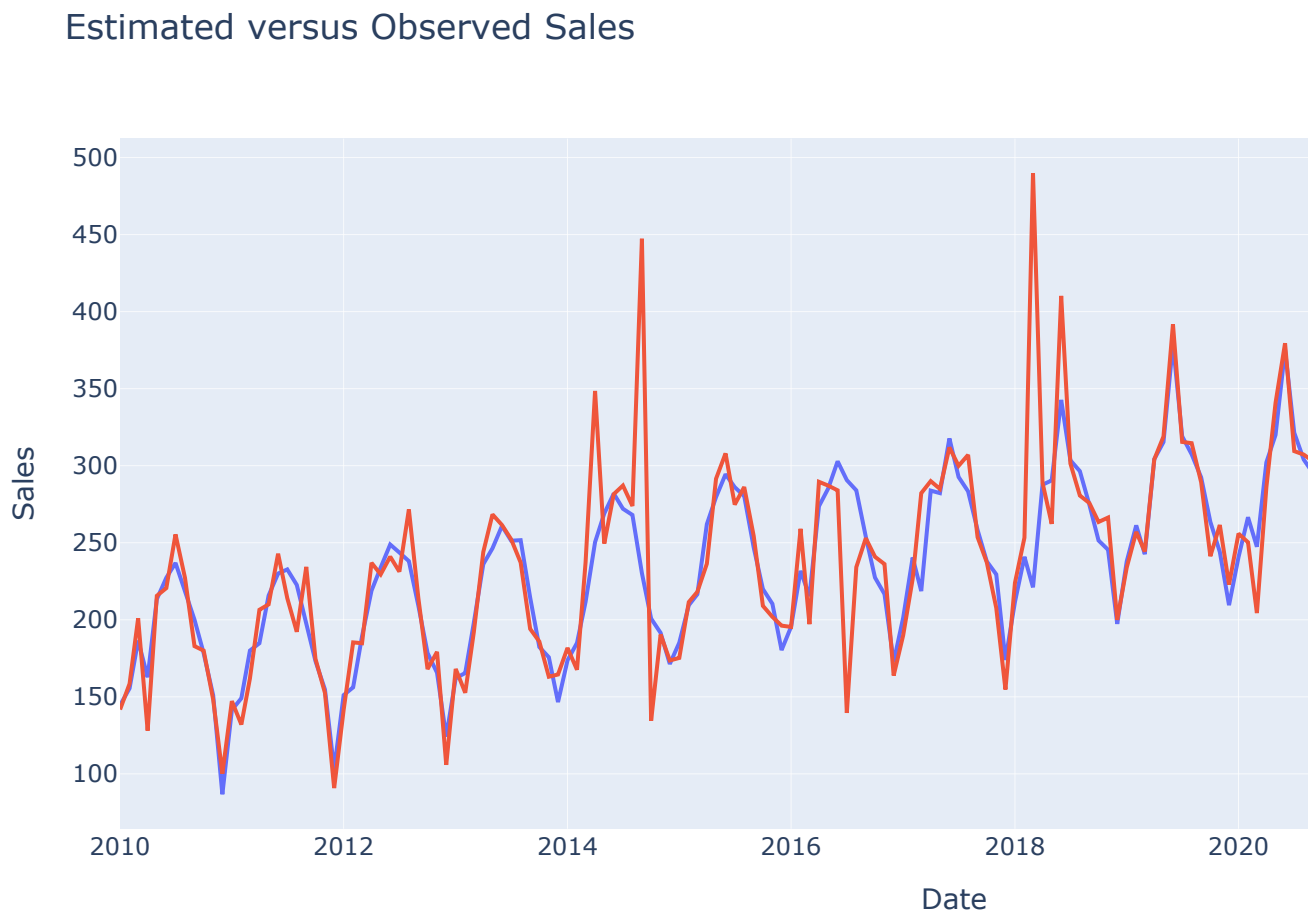
Having the decomposed components at hand, opens a variety of applications.

6.1.4 Denoising

Using the trend and seasonal components from the STL decomposition, we can create a denoised version of the original time series.

```
decomp_stl_estimated = decomp_stl.seasonal + decomp_stl.trend
```

```
fig = px.line(
    decomp_stl_estimated,
    title="Estimated versus Observed Sales",
    labels={"value": "Sales", "index": "Date"},
)
fig.update_traces(name="Estimated", showlegend=True)

fig.add_trace(go.Scatter(x=df.index, y=df["Sales"], mode="lines", name="Observed"))
fig.show()
```



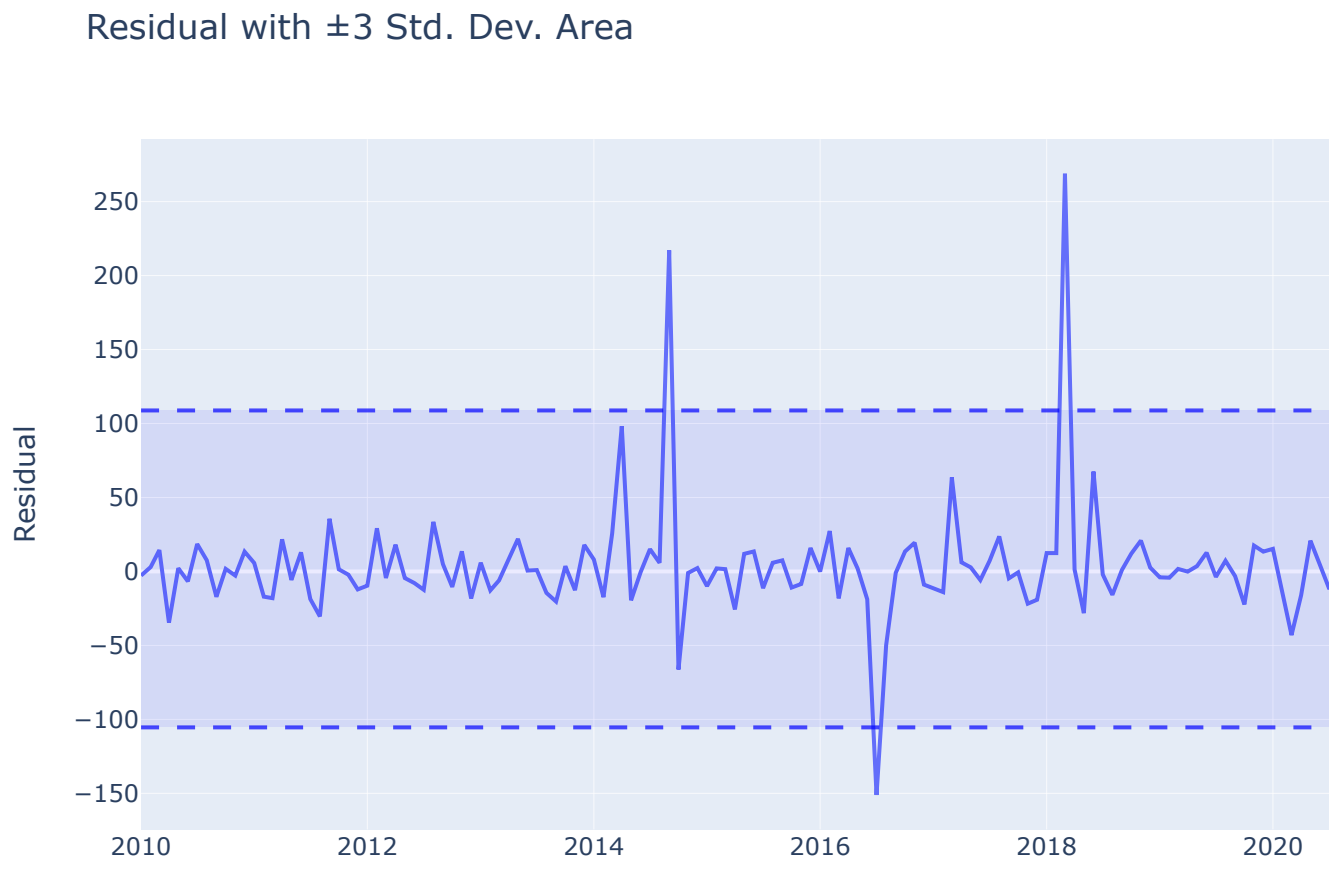
In previous plot, we can see that the estimated values follow the observed values quite closely, indicating that the STL decomposition has effectively captured the underlying patterns in the data. But we can also see some points with larger deviations, which are likely outliers in the original data.

6.1.5 Anomaly Detection

One way to quantify outliers is to apply a threshold to the residuals (e.g., 3 times the standard deviation of the residuals, also known as the 3-sigma rule).

```
resid = decomp_stl.resid
std_resid = resid.std()

fig = go.Figure()
fig.add_trace(go.Scatter(x=resid.index, y=resid, mode="lines", name="Residual"))
# Add horizontal lines for ±3 std. dev.
fig.add_hline(
    y=resid.mean() + 3 * std_resid,
    line_dash="dash",
    line_color="blue",
    opacity=0.7,
    name="+3 Std. Dev.",
)
fig.add_hline(
    y=resid.mean() - 3 * std_resid,
    line_dash="dash",
    line_color="blue",
    opacity=0.7,
    name="-3 Std. Dev.",
)

# Fill area between the lines
fig.add_traces(
    [
        go.Scatter(
            x=resid.index,
            y=[resid.mean() + 3 * std_resid] * len(resid),
            mode="lines",
            line={"width": 0},
            showlegend=False,
            hoverinfo="skip",
        ),
        go.Scatter(
            x=resid.index,
            y=[resid.mean() - 3 * std_resid] * len(resid),
            mode="lines",
            fill="tonexty",
            fillcolor="rgba(0,0,255,0.08)",
            line={"width": 0},
            showlegend=False,
            hoverinfo="skip",
        ),
```

```
    ]
)

fig.update_layout(title="Residual with ±3 Std. Dev. Area", yaxis_title="Residual")
fig.show()
```



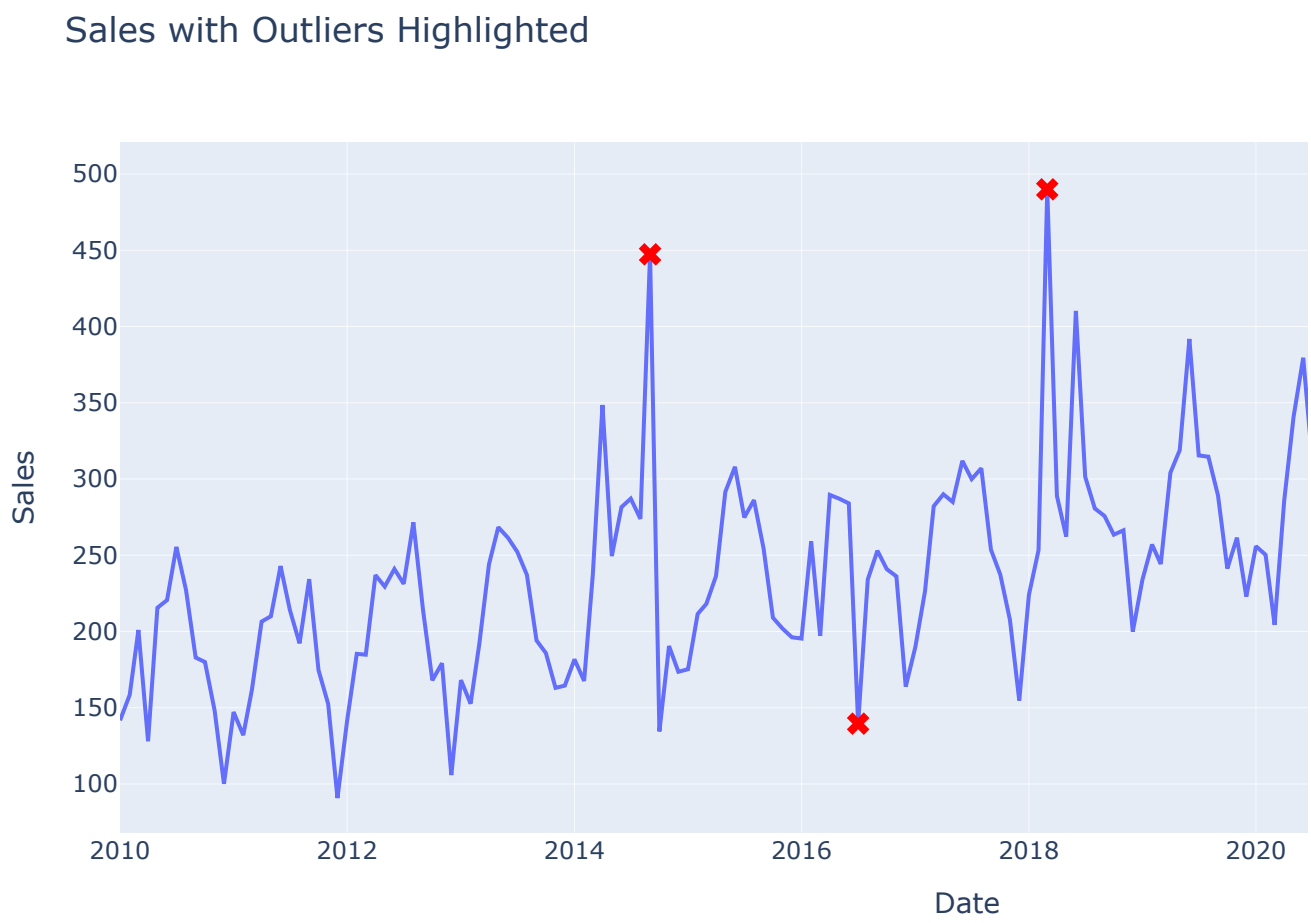
So the outliers are:

```
outliers = df[
    (resid > resid.mean() + 3 * std_resid) | (resid < resid.mean() - 3 * std_resid)
]
outliers
```

	Sales
Date	
2014-09-01	447.316646
2016-07-01	139.561235
2018-03-01	489.942096
2021-06-01	227.748919

```
fig = px.line(
    df,
```

```
    y="Sales",
    title="Sales with Outliers Highlighted",
    labels={"value": "Sales", "index": "Date"},
)
fig.add_scatter(
    x=outliers.index,
    y=outliers["Sales"],
    mode="markers",
    marker={"color": "red", "size": 10, "symbol": "x"},
    name="Outliers",
)
fig.show()
```

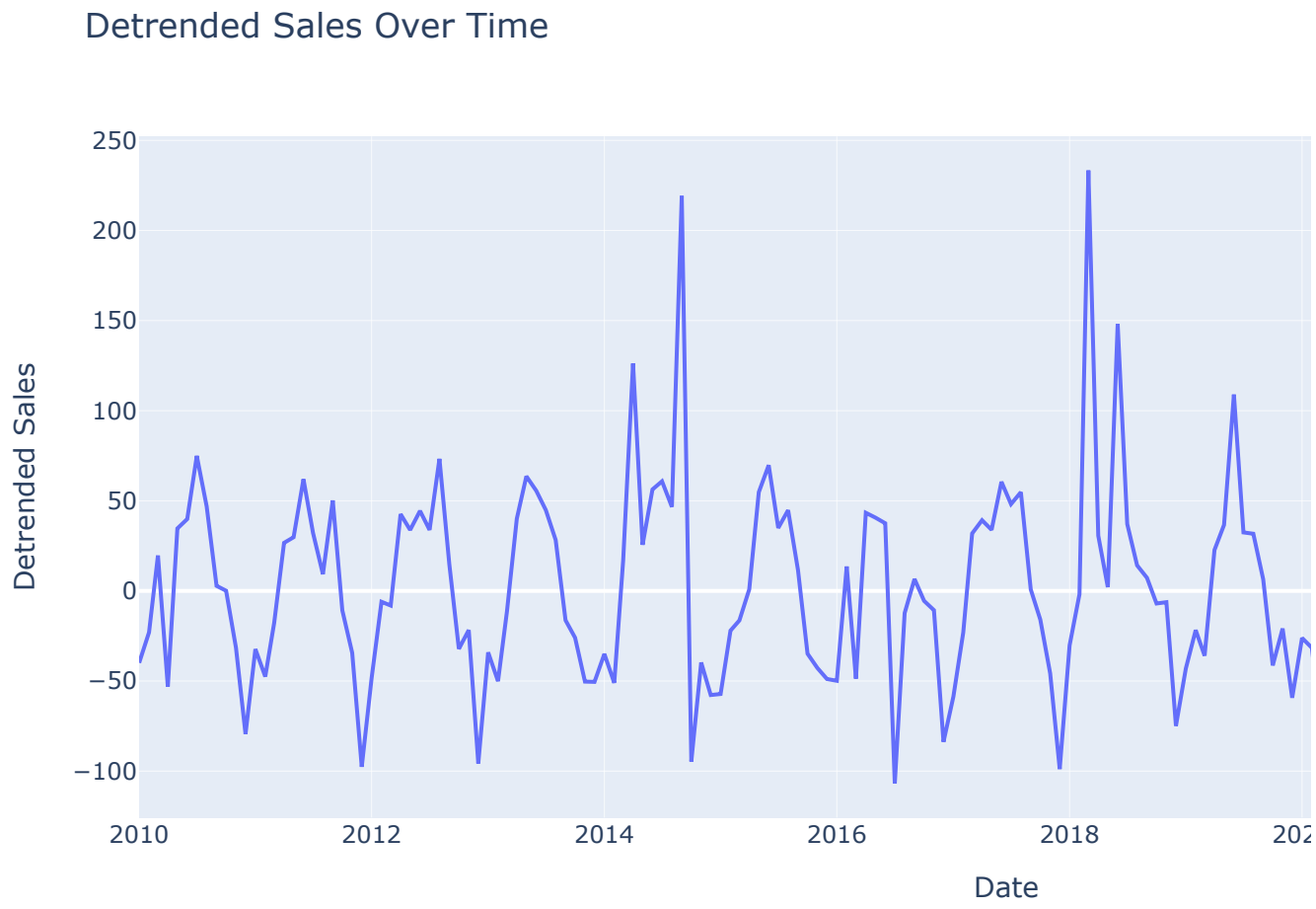


6.1.6 Detrending

Using the decomposed components also allows for detrending the time series, which can be useful for further analysis.

```
# Detrend the observed series by subtracting the trend component
detrended = decomp_stl.observed["Sales"] - decomp_stl.trend

fig = px.line(
    detrended,
    title="Detrended Sales Over Time",
    labels={"value": "Detrended Sales", "index": "Date"},
```

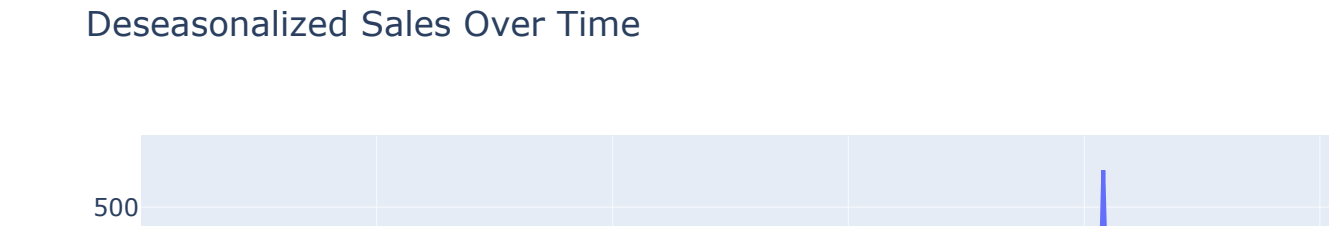
```
)
fig.show()
```

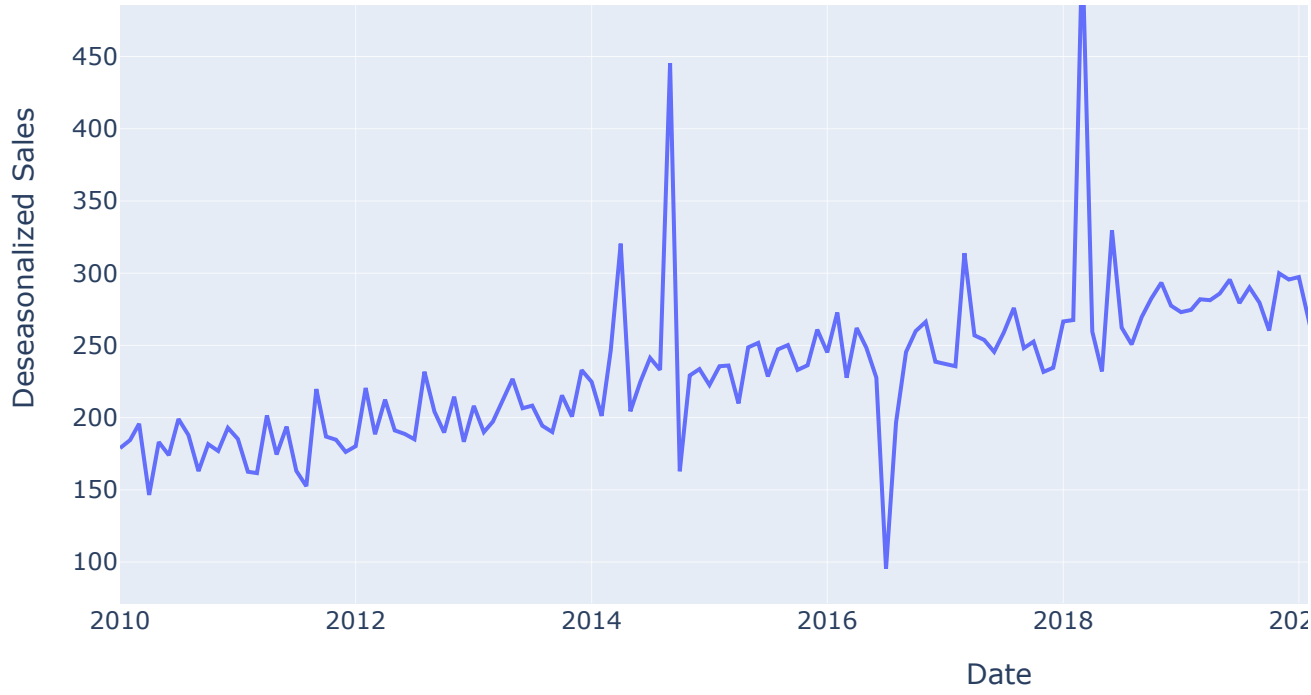


6.1.7 Deseasonalizing

Another important application is to deseasonalize the time series, which is often a prerequisite for forecasting models.

```
# Deseasonalize the observed series by subtracting the seasonal component
deseasonalized = decomp_stl.observed["Sales"] - decomp_stl.seasonal

fig = px.line(
    deseasonalized,
    title="Deseasonalized Sales Over Time",
    labels={"value": "Deseasonalized Sales", "index": "Date"},
)
fig.show()
```





Industrial Data Science - 2026 - CC BY-NC-SA 4.0

Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

7 Time Series Clustering

The goal of clustering is to group similar time series together based on their characteristics or patterns.

But what does **similar** mean in the context of time series data? Defining **similarity** is not straight forward due to the inherent subjectivity involved.

Difficulties are:

- **Context-Dependent**: Similarity can vary based on the specific application or domain.
- **Human Bias**: Metrics are often chosen based on human intuition, which can introduce bias.
- **Inconsistent Interpretations**: Different experts may have different perspectives on what constitutes similarity.
- **Parameter Sensitivity**: Many similarity measures come with tunable parameters that can significantly affect the results.

7.1 Clustering algorithms

Time series clustering algorithms can be broadly categorized into four main types:

- **Distance-Based**: These algorithms rely on a distance measure to quantify the similarity between time series. Examples include k-means and k-median.
- **Distribution-Based**: These methods model the distribution (in a very general sense) of the time series data and cluster based on statistical properties or descriptive features that represent the characteristics of the time series. Examples include *Gaussian Mixture Models* (GMM) and DBSCAN.
- **Subsequence-Based**: These algorithms focus on clustering representative subsequences of time series data rather than the entire series. Examples include methods based on shapelets and sliding windows.
- **Representation-Learning-Based**: These methods use techniques like autoencoders or recurrent neural networks to learn a lower-dimensional representation of the time series data, which is then clustered using traditional clustering algorithms.

Paparrizos et al. (2024) gives a good overview of different time series clustering methods.

In the following, we will briefly introduce distance-based clustering using two common similarity measures: Euclidean distance and *Dynamic Time Warping* (DTW).

7.2 Similarity Measures

7.2.1 Imports

▶ Code

7.2.2 Helper functions for plotting

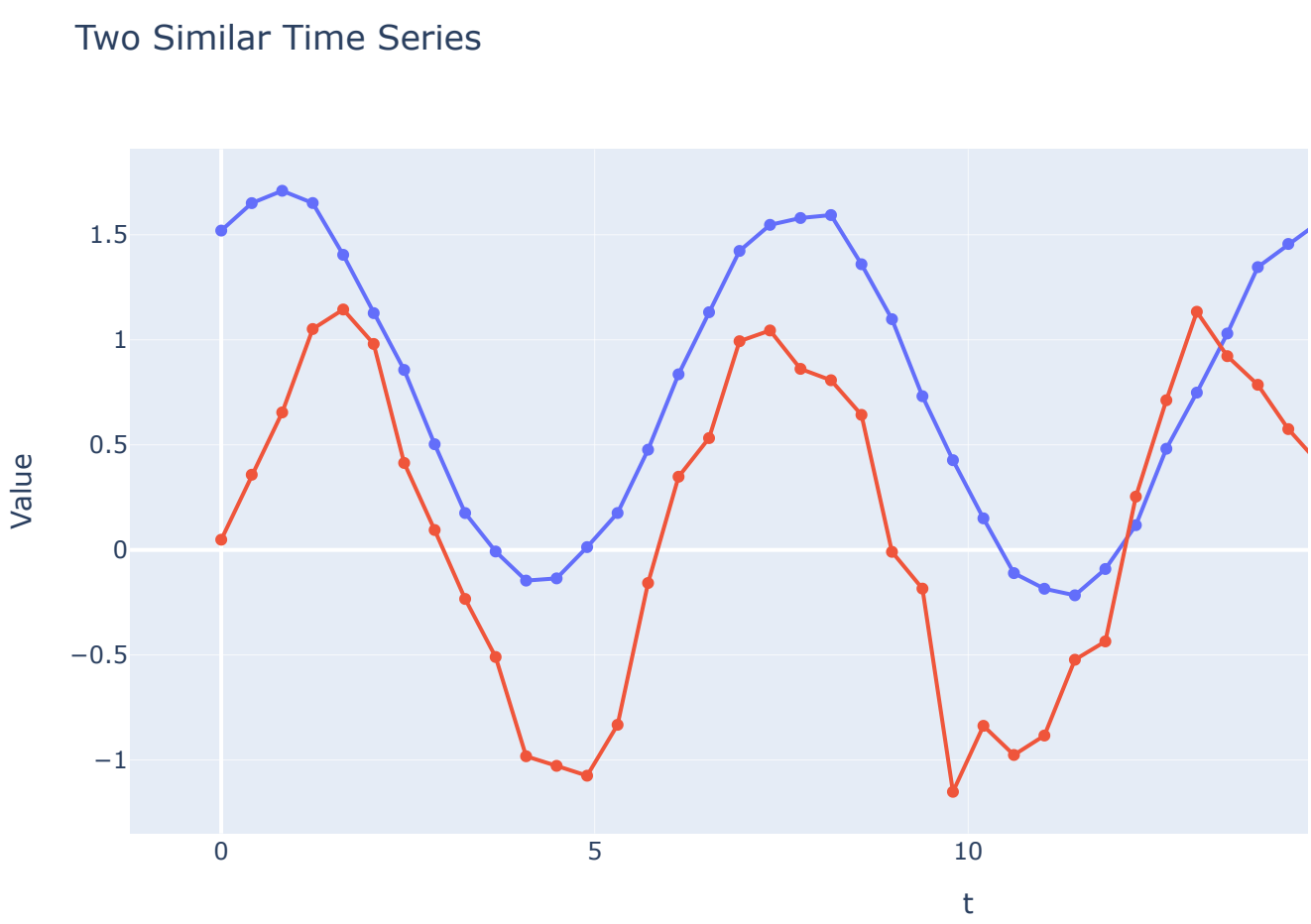
▶ Code

7.2.3 Data Generation

Creating two similar synthetic time series.

```
nr_points = 50
interval_start = 0
interval_end = 20

t = np.linspace(interval_start, interval_end, nr_points)

series1 = (
    0.8
    + 0.9 * np.sin(0.9 + 0.9 * t)
    + 0.03 * np.random.randn(nr_points)
    - 0.2 * (t / (interval_end - interval_start))
)

series2 = np.sin(1.05 * t) + 0.15 * np.random.randn(nr_points)
```

```
plot_time_series_plotly(t, series1, series2, "Two Similar Time Series")
```



7.2.4 Measures

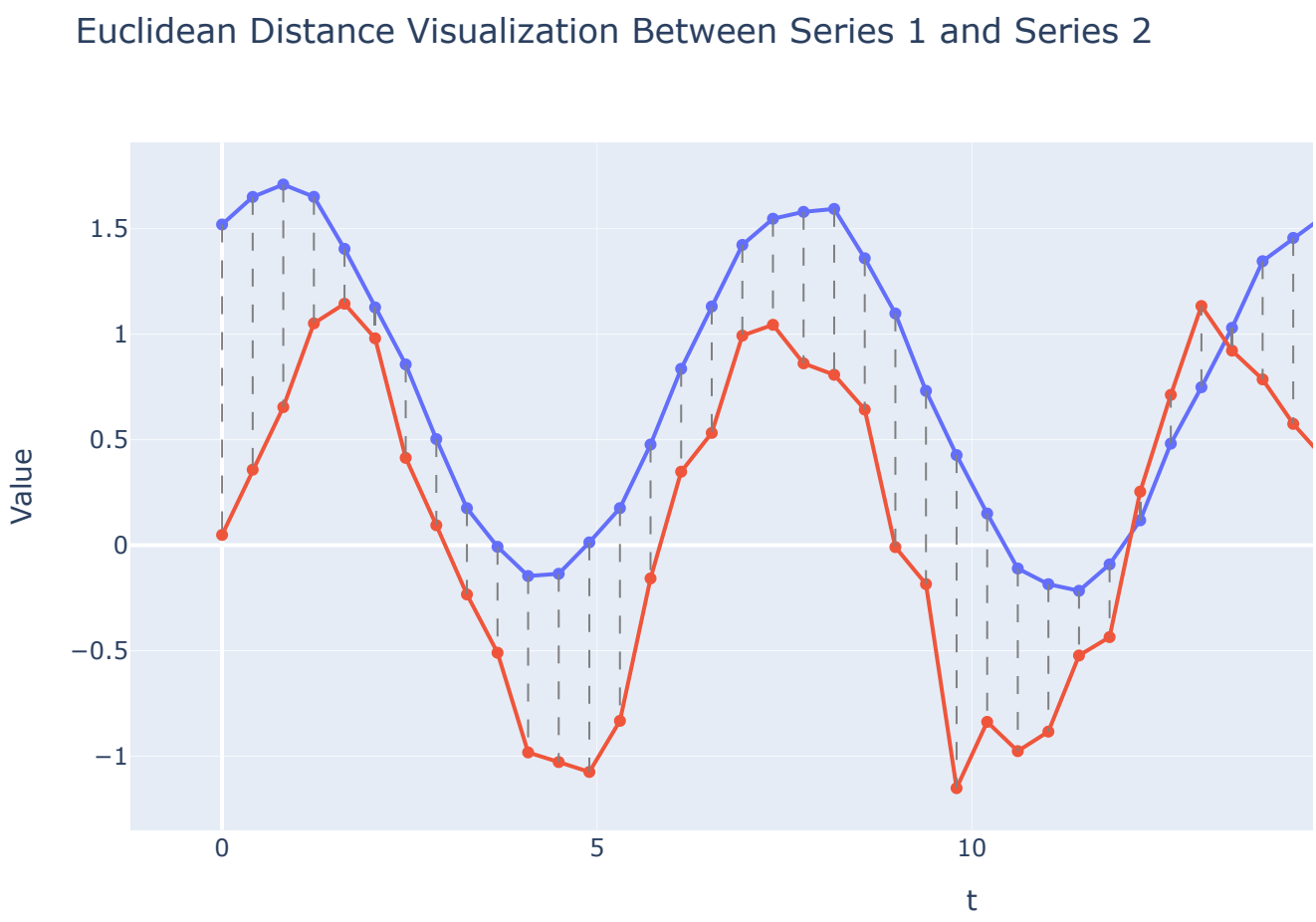
7.2.4.1 Euclidean Distance

The *Euclidean distance* is the most straightforward similarity measure. It calculates the straight-line distance between two points. For time series data, this means comparing the values at each point of time directly.

Having two time series $X = (x_1, x_2, \ldots, x_n)$ and $Y = (y_1, y_2, \ldots, y_n)$ of equal length n, the Euclidean distance d between them is calculated as:

$$d(X, Y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

```
plot_euclidean_distance(series1, series2, t)
```




```
def euclidean_distance(series1: np.ndarray, series2: np.ndarray) -> float:
    """Compute Euclidean distance between two series.

    Args:
        series1: First series.
        series2: Second series.

    Returns:
        Euclidean distance.
    """
    return np.sqrt(np.sum((series1 - series2) ** 2))
```

```
d_euclidean = euclidean_distance(series1, series2)
print(f"Euclidean Distance: {d_euclidean:.2f}")
```

```
Euclidean Distance: 6.33
```

7.2.4.2 Dynamic Time Warping

Dynamic Time Warping (DTW) is a more advanced similarity measure that accounts for shifts and distortions in the time axis. It finds the optimal alignment between two time series by warping the time dimension, allowing for comparisons even when the series are out of phase or have different lengths.

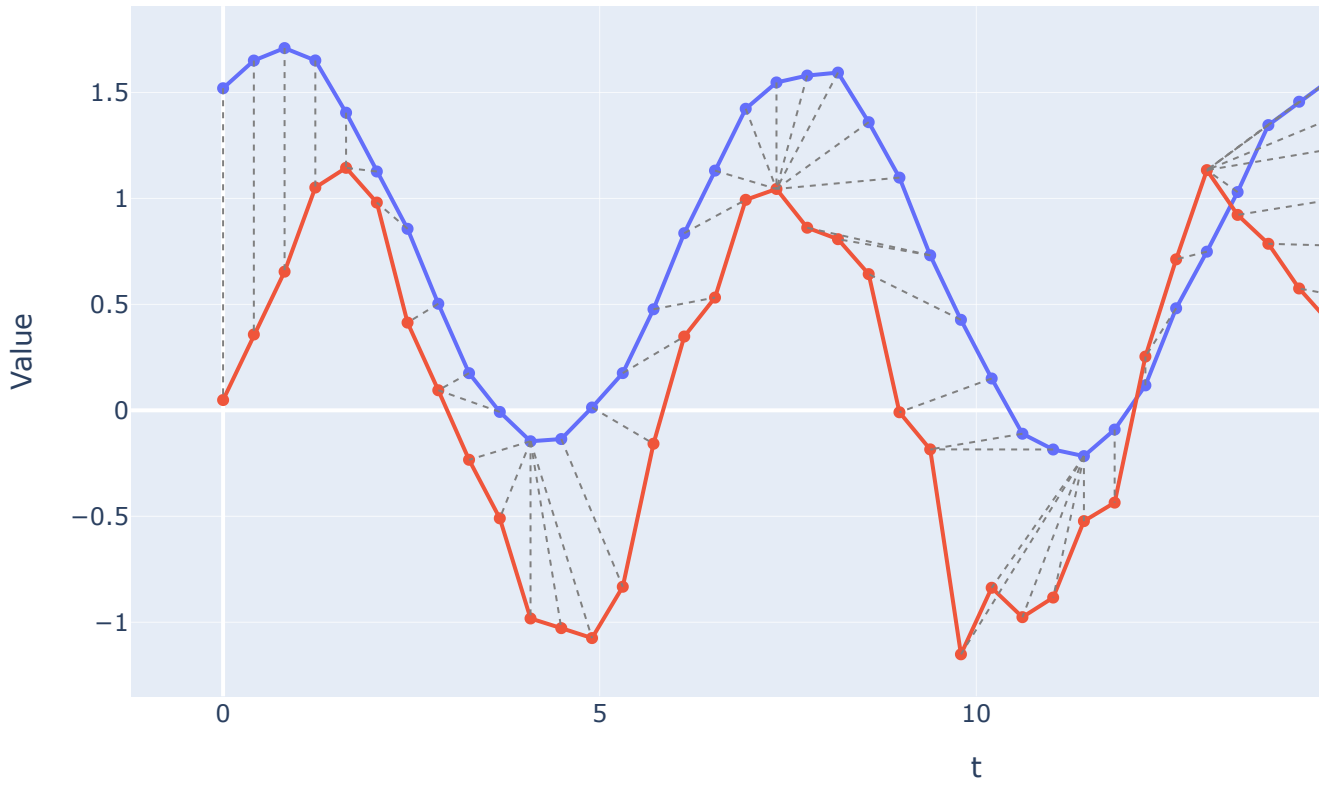
```
def dtw_distance(s1: np.ndarray, s2: np.ndarray) -> tuple[float, list[tuple[int, int]]]:
    """Compute Dynamic Time Warping distance between two series.

    Args:
        s1: First series.
        s2: Second series.

    Returns:
        Tuple of (DTW distance, alignment path).
    """
    n, m = len(s1), len(s2)

    # initializing cost matrix
    cost = np.full((n + 1, m + 1), np.inf)
    cost[0, 0] = 0

    # filling cost matrix
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            dist = abs(s1[i - 1] - s2[j - 1])
            cost[i, j] = dist + min(
                cost[i - 1, j],  # insertion
                cost[i, j - 1],  # deletion
                cost[i - 1, j - 1],
            )  # match

    # backtracking path (just for visualization)
    path = []

    i, j = n, m

    while i > 0 and j > 0:
        path.append((i - 1, j - 1))
        steps = [(i - 1, j), (i, j - 1), (i - 1, j - 1)]
        costs = [cost[s] if s[0] >= 0 and s[1] >= 0 else np.inf for s in steps]
        min_step = steps[np.argmin(costs)]
        i, j = min_step

    path = path[::-1]

    return cost[n, m], path
```

```
d_dtw, path = dtw_distance(series1, series2)

print(f"Dynamic Time Warping Distance: {d_dtw:.2f}")
```

```
Dynamic Time Warping Distance: 22.92
```

Note that this DTW implementation is using a very naive approach computing the full cost matrix. Many different variants exist that are more efficient and/or add constraints to the warping path. Lahreche and Boucheham (2021) provides a good overview.

```
def plot_dtw_alignment(
    s1: np.ndarray, s2: np.ndarray, t: np.ndarray, path: list[tuple[int, int]]
) -> go.Figure:
    """Plot two time series with DTW alignment path.

    Args:
        s1: First series.
        s2: Second series.
        t: Time axis.
        path: DTW alignment path as list of (i, j) index pairs.

    Returns:
        Plotly figure.
    """
    fig = go.Figure()
    fig.add_trace(go.Scatter(x=t, y=s1, mode="lines+markers", name="Series 1"))
    fig.add_trace(go.Scatter(x=t, y=s2, mode="lines+markers", name="Series 2"))

    # Draw alignment lines
    for i, j in path:
        fig.add_trace(
            go.Scatter(
                x=[t[i], t[j]],
                y=[s1[i], s2[j]],
                mode="lines",
                line={"color": "gray", "width": 1, "dash": "dot"},
                showlegend=False,
            )
        )

    fig.update_layout(
        title="Dynamic Time Warping Alignment Between Series 1 and Series 2",
        xaxis_title="t",
        yaxis_title="Value",
    )
    return fig


plot_dtw_alignment(series1, series2, t, path)
```



7.2.5 Importance of Preprocessing

In the previous plots we observe that noise, outliers and positioning of the time series can have a significant impact on similarity measures. To mitigate these effects, we can apply various preprocessing techniques such as:

- **Smoothing**: Applying filters (e.g., moving average, Gaussian) to reduce noise.
- **Normalization**: Scaling the time series to a common range to ensure that differences in amplitude do not dominate the similarity measure.
- **Detrending**: Removing trends to focus on the fluctuations around a mean level.

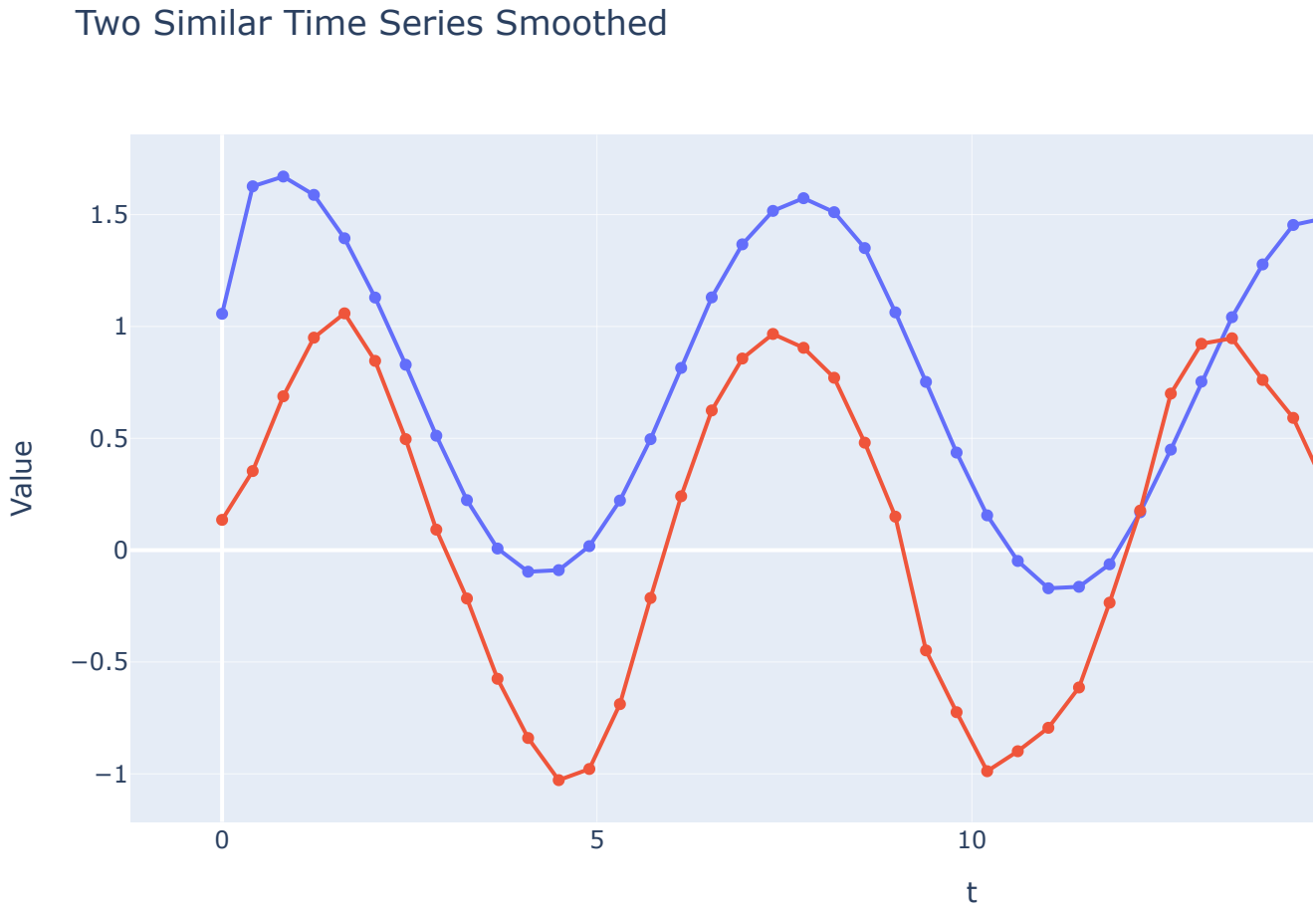
Note that the applied preprocessing techniques should be chosen based on the specific characteristics of the data and the analysis goals.

Here, we first smooth the time series using a simple moving average filter. This can easily be realized by convoluting with a specific kernel (a vector consisting of equal weights summing to 1, where the length of the vector is equal to the desired window size).

```
def moving_average(series: np.ndarray, window_size: int = 3) -> np.ndarray:
    """Compute moving average of a series.

    Args:
        series: Input series.
        window_size: Size of the moving average window.

    Returns:
        Smoothed series.
```

```
    """
    kernel = np.ones(window_size) / window_size
    return np.convolve(series, kernel, mode="same")


series1_smooth = moving_average(series1)
series2_smooth = moving_average(series2)
```

```
plot_time_series_plotly(
    t, series1_smooth, series2_smooth, "Two Similar Time Series Smoothed"
)
```



Then we normalize the smoothed time series to the range [0, 1] using min-max normalization:

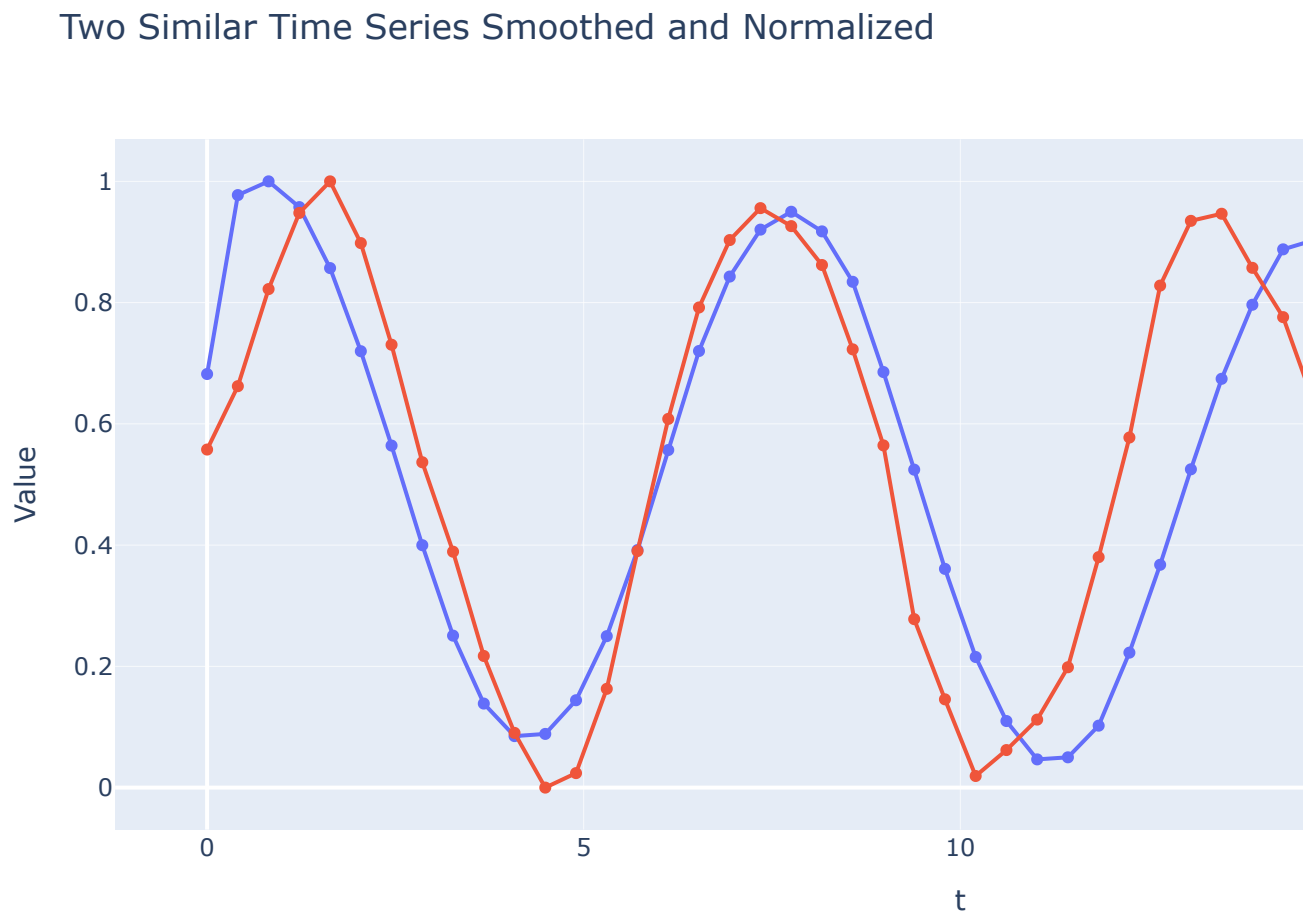
$$X_{norm} := \frac{X - X_{min}}{X_{max} - X_{min}}$$

where X_{min} and X_{max} are the minimum and maximum values of the time series X, respectively.

```
def normalize(series: np.ndarray) -> np.ndarray:
    """Min-max normalize a series to [0, 1].

    Args:
        series: Input series.

    Returns:
```

```
        Normalized series.
    """
    return (series - np.min(series)) / (np.max(series) - np.min(series))


series1_smooth_norm = normalize(series1_smooth)
series2_smooth_norm = normalize(series2_smooth)
```

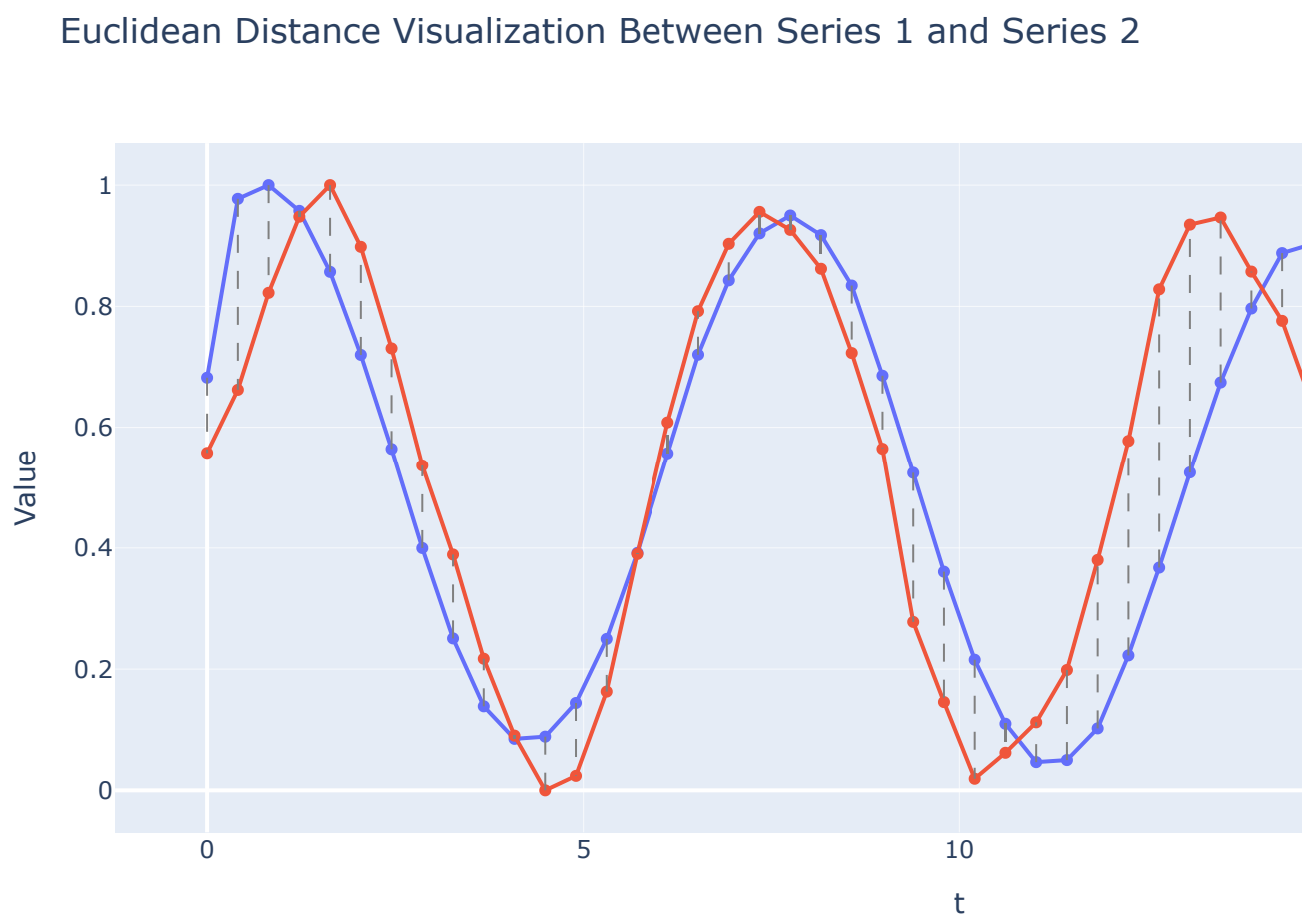
```
plot_time_series_plotly(
    t,
    series1_smooth_norm,
    series2_smooth_norm,
    "Two Similar Time Series Smoothed and Normalized",
)
```



The euclidean distance between the two smoothed and normalized time series can then be calculated as before.

```
d_euclidean_sn = euclidean_distance(series1_smooth_norm, series2_smooth_norm)
print(f"Euclidean Distance: {d_euclidean_sn:.2f}")

plot_euclidean_distance(series1_smooth_norm, series2_smooth_norm, t)
```

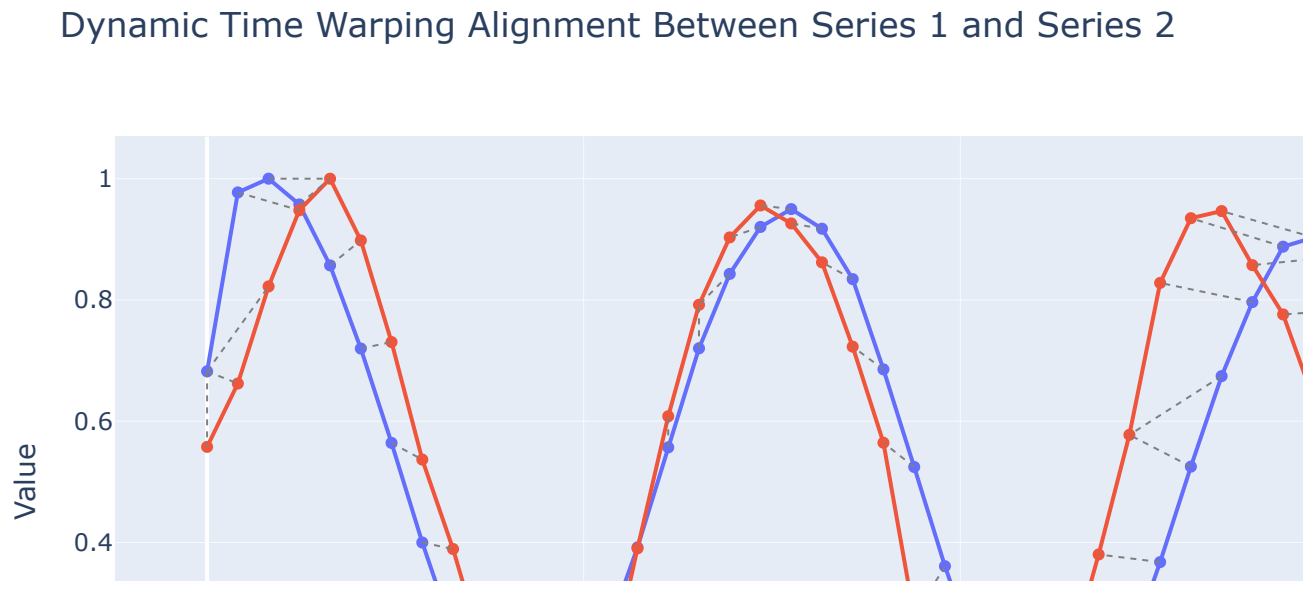
```
Euclidean Distance: 2.18
```

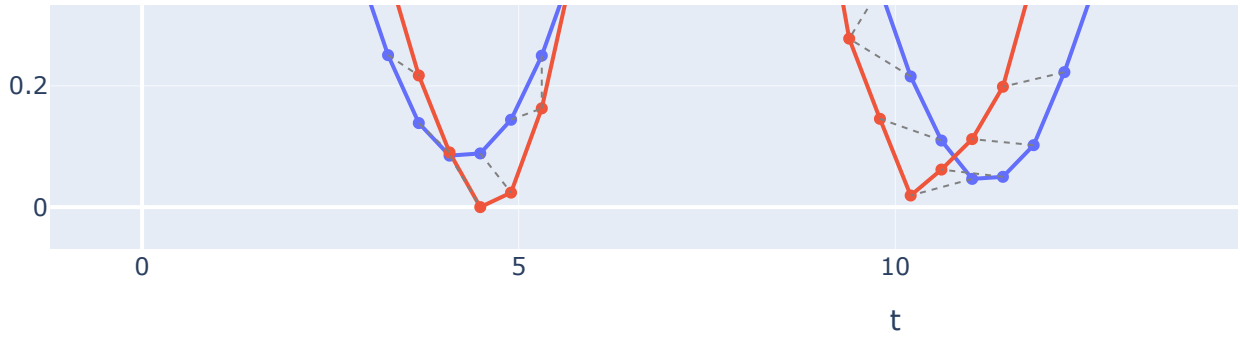


And also dynamic time warping can of course be applied similarly on the preprocessed time series.

```
d_dtw_sn, path_sn = dtw_distance(series1_smooth_norm, series2_smooth_norm)

print(f"Dynamic Time Warping Distance: {d_dtw_sn:.2f}")

plot_dtw_alignment(series1_smooth_norm, series2_smooth_norm, t, path_sn)
```

```
Dynamic Time Warping Distance: 4.88
```





7.3 k-Means Clustering

Just like the k-means algorithm for tabular data, the k-means algorithm for time series data aims to partition a set of time series into k clusters, where each time series belongs to the cluster with the nearest mean (centroid) time series.

The algorithm works as follows:

1. **Initialization**: Randomly select k initial centroids from the time series dataset.
2. **Assignment Step**: Assign each time series to the cluster whose centroid is closest, based on a chosen similarity measure (e.g., Euclidean distance, Dynamic Time Warping).
3. **Update Step**: Recalculate the centroids of the clusters by taking the mean of all time series assigned to each cluster.
4. **Repeat**: Repeat the assignment and update steps until convergence (i.e., when assignments no longer change or a maximum number of iterations is reached).

In practice, calculating the centroid of a cluster needs to be compatible with the chosen similarity measure (at least it is very beneficial if it is compatible). Especially for non-Euclidean measures with time series of variable length like Dynamic Time Warping, specialized methods may be used to compute the centroid.

Here we will have a look at an example using the `sktime` library. The dataset we will use is a subset of the `trace` dataset, which is a synthetic dataset introduced by Roverso (2002) for the purpose of plant diagnostics. The subset includes only 4 classes of univariate time series.

7.3.1 Imports

▶ Code

7.3.2 Loading the dataset

```
X_train, y_train, X_test, y_test = CachedDatasets().load_dataset("Trace")

# Combine train and test sets since clustering does not require a train-test split
X = np.concatenate((X_train, X_test))
y = np.concatenate((y_train, y_test))
```

```
X.shape
```

```
(200, 275, 1)
```

```
y.shape
```

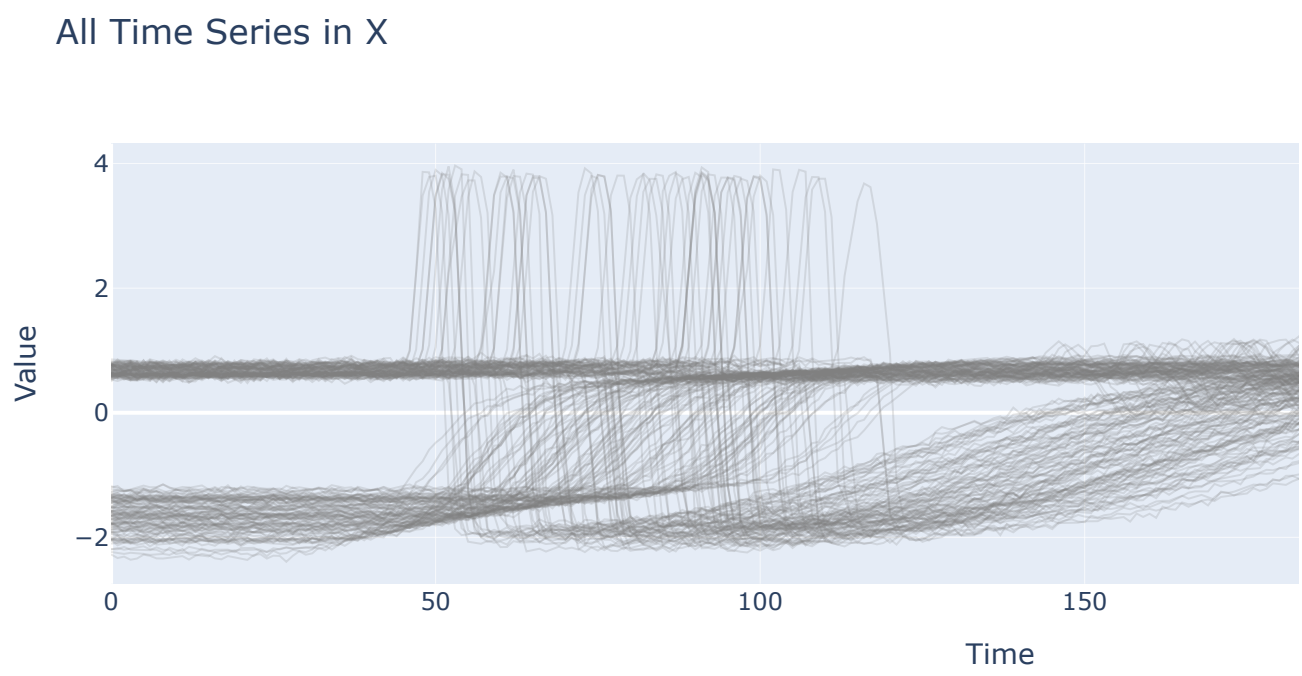
```
(200,)
```

We have 200 time series in total, each of length 275.

Note that we also have class labels, which is not the case in real clustering problems. We will solely use them in the end to evaluate the clustering performance.

Let us plot all the time series to get an idea of what they look like.

```
fig = go.Figure()
for i in range(X.shape[0]):
    fig.add_trace(
        go.Scatter(
            y=X[i, :, 0],
            mode="lines",
            line={"width": 1, "color": "grey"},
            opacity=0.2,
            showlegend=False,
        )
    )

fig.update_layout(
    title="All Time Series in X", xaxis_title="Time", yaxis_title="Value", height=400
)
fig.show()
```



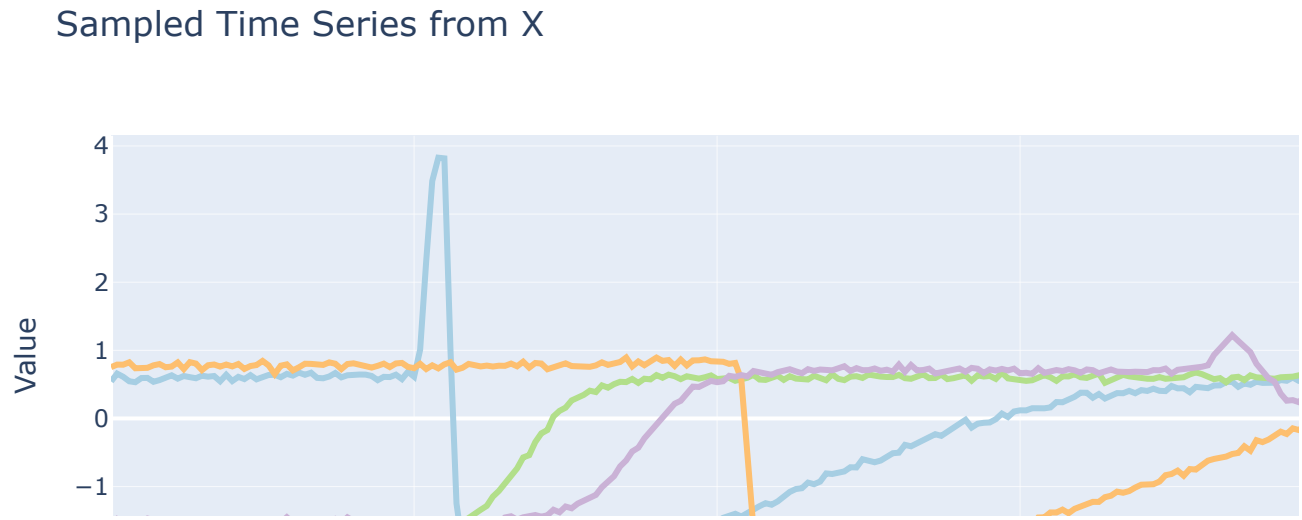
Without class labels it is hard to count the number of classes in the data, but we can see that there are some patterns in the data.

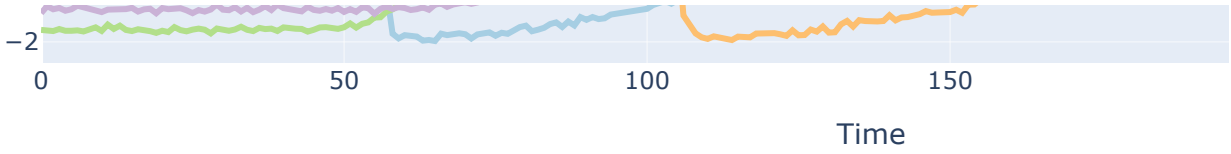
Since normalization and scaling is important for distance-based methods, we will use the `StandardScaler` from `sklearn` to standardize the data to have zero mean and unit variance.

```
X_scaled = StandardScaler().fit_transform(
    X[:, :, 0]
)  # In this case, the dataset was already scaled beforehand, but we do it here explicitely fo
```

Let us also just visualize a few time series from the dataset to get a better idea of the data.

```
COLORS = ["#A6CEE3", "#B2DF8A", "#FDBF6F", "#CAB2D6"]  # pastel, colorblind-friendly

sampled_ids = [0, 10, 25, 81]

fig = go.Figure()

for idx, i in enumerate(sampled_ids):
    fig.add_trace(
        go.Scatter(
            y=X[i, :, 0],
            mode="lines",
            line={"width": 3, "color": COLORS[idx]},
            name=f"Sample {i}",
        )
    )

fig.update_layout(
    title="Sampled Time Series from X",
    xaxis_title="Time",
    yaxis_title="Value",
    height=400,
)
fig.show()
```





7.3.3 Clustering based on Euclidean Distance

In this example we know that there are 4 classes in the data, so we will set $k = 4$.

Luckily, `sktime` already supports a variety of clustering algorithms out of the box, including the k-means algorithm.

```
k = 4  # number of clusters

clusterer = TimeSeriesKMeansTslearn(n_clusters=4, metric="euclidean", random_state=42)

start_time = time.time()
y_predicted = clusterer.fit_predict(X)
duration = time.time() - start_time

print(f"Clustering took {duration:.2f} seconds.")
```

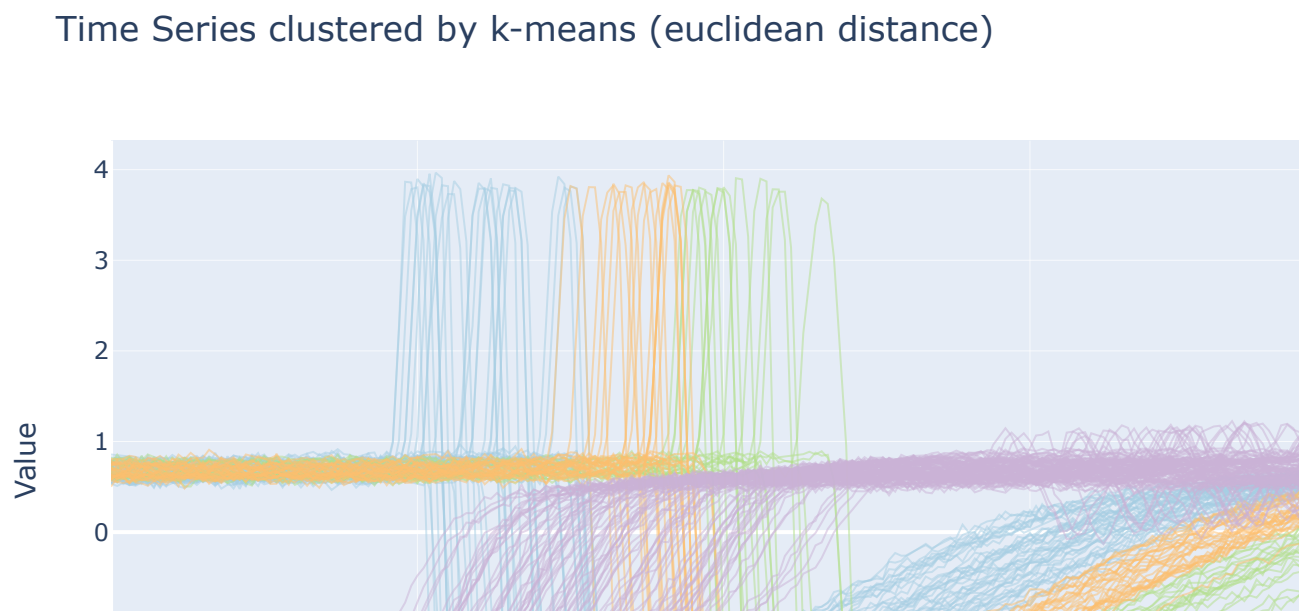
```
Clustering took 0.31 seconds.
```

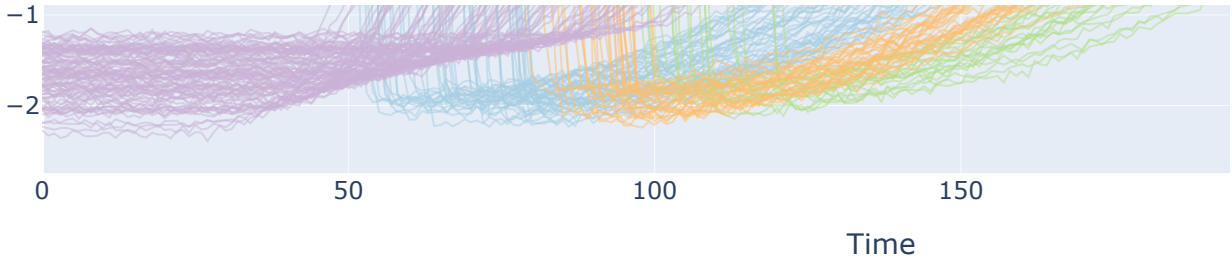
Helper function for plotting clusters and cluster centers

▶ Code

Next, we plot the k-means clusters. Remember that the legend is clickable.

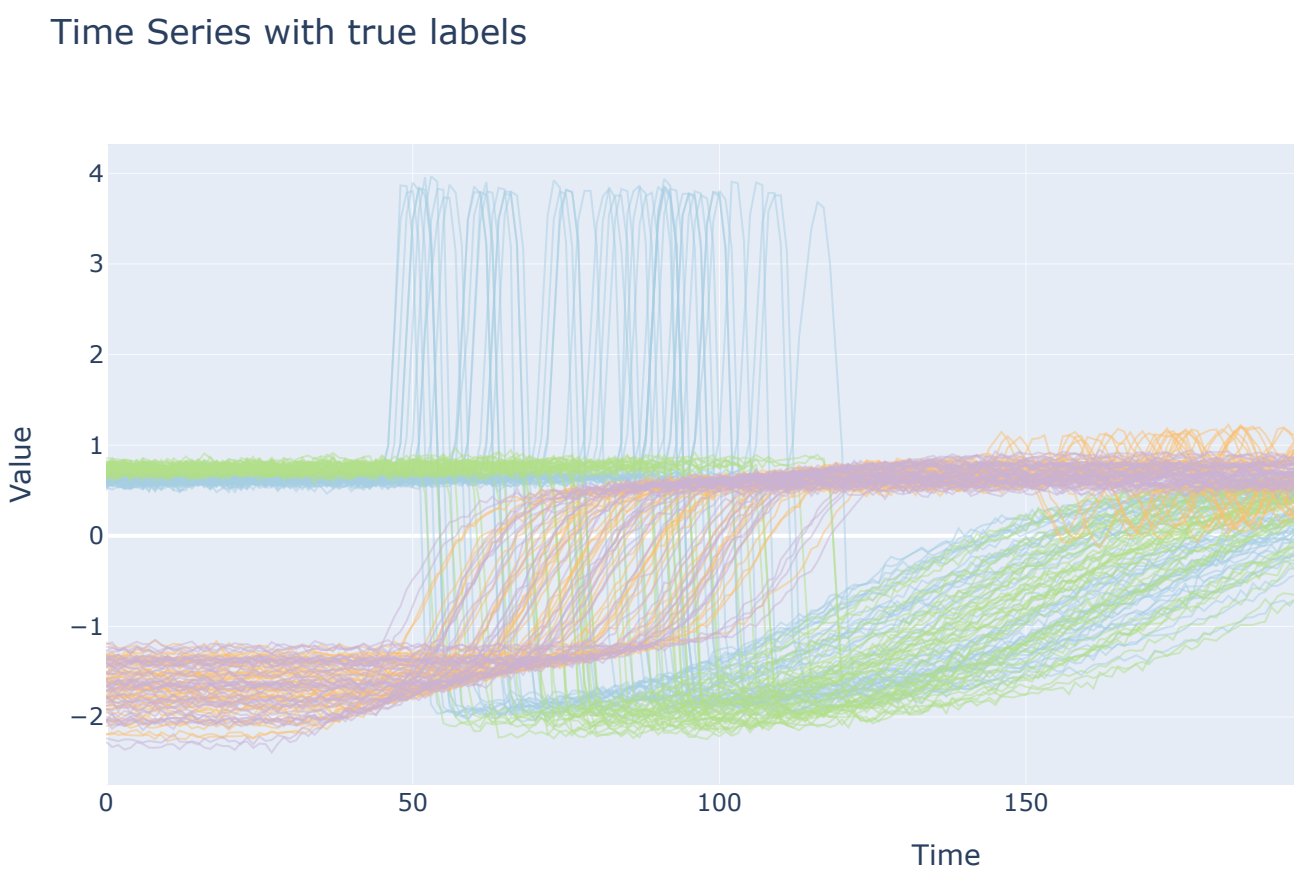
```
plot_clusters(
    X, y_predicted, "Time Series clustered by k-means (euclidean distance)", False
)
```





In comparison, here are the true classes according to the labels in the dataset.

```
plot_clusters(X, y, "Time Series with true labels", True)
```



The four classes from the original dataset can be described as follows:

- Cluster 1: Time series that start high, rise to a huge peak around the middle, fall back to low and then gradually rise to high again.
- Cluster 2: Time series that start high, drop to a low point around the middle, and then rise back up.
- Cluster 3: Time series that start low, quickly rise to high around the middle, and then show some oscillations on high plateau
- Cluster 4: Similiar to Cluster 3, but without oscillations.

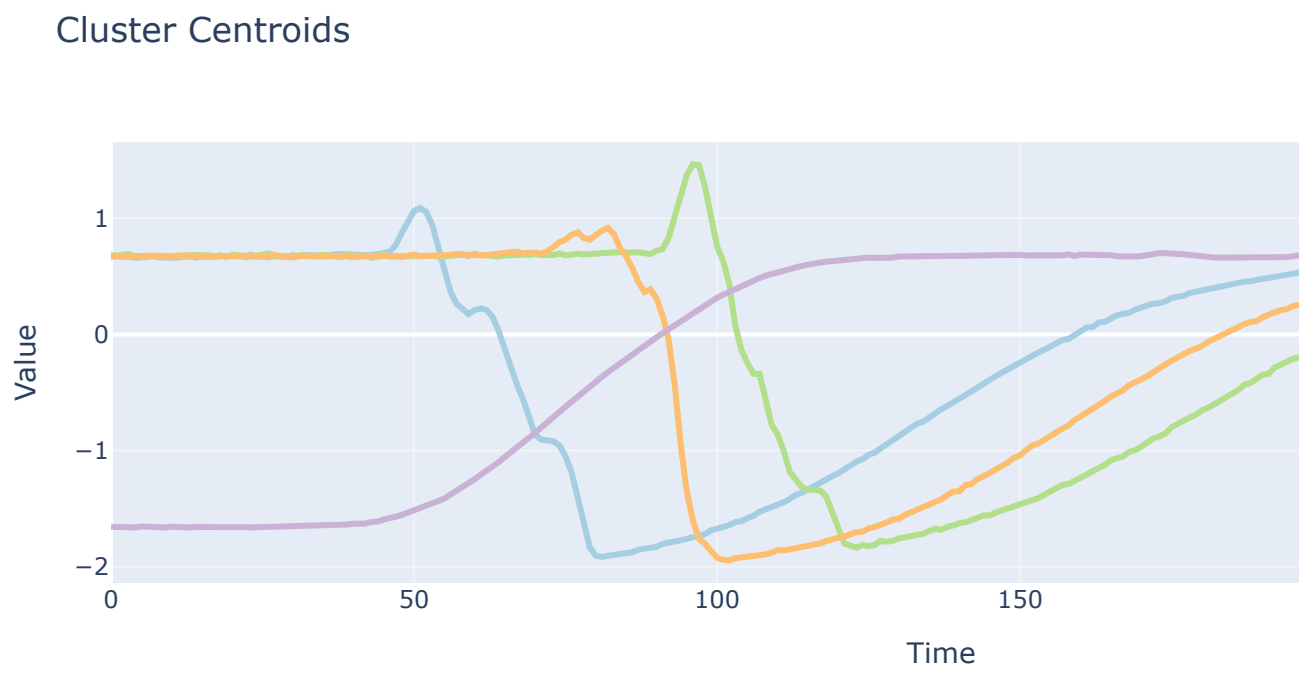
The clusters found by the k-means algorithm do not correspond well to these classes:

- Cluster 1 and 2 are mixed up in Cluster B.
- Cluster 3 and 4 are mixed up in Cluster A, C, and D.

This is due to the fact that the k-means algorithm is based on minimizing the within-cluster distances based on the euclidean distance, which does not necessarily correspond to the true classes in the data.

This also reflects when we visualize the cluster centers.

```
plot_centroids(clusterer)
```



Next, let us check how the clustering performs with dynamic time warping as distance measure.

7.3.4 Clustering based on Dynamic Time Warping

While euclidean distance uses the mean for computing the centroid, dynamic time warping uses a more complex method called soft-DTW barycenter averaging, which is compatible with the DTW distance measure.

Soft-DTW, a differentiable version of DTW, enables the use of gradient-based optimization techniques for computing the barycenter. Its differentiable nature also facilitates its integration as loss function into machine learning algorithms, in particular neural networks.

More information on soft-DTW can be found in the related paper Cuturi and Blondel (2017).

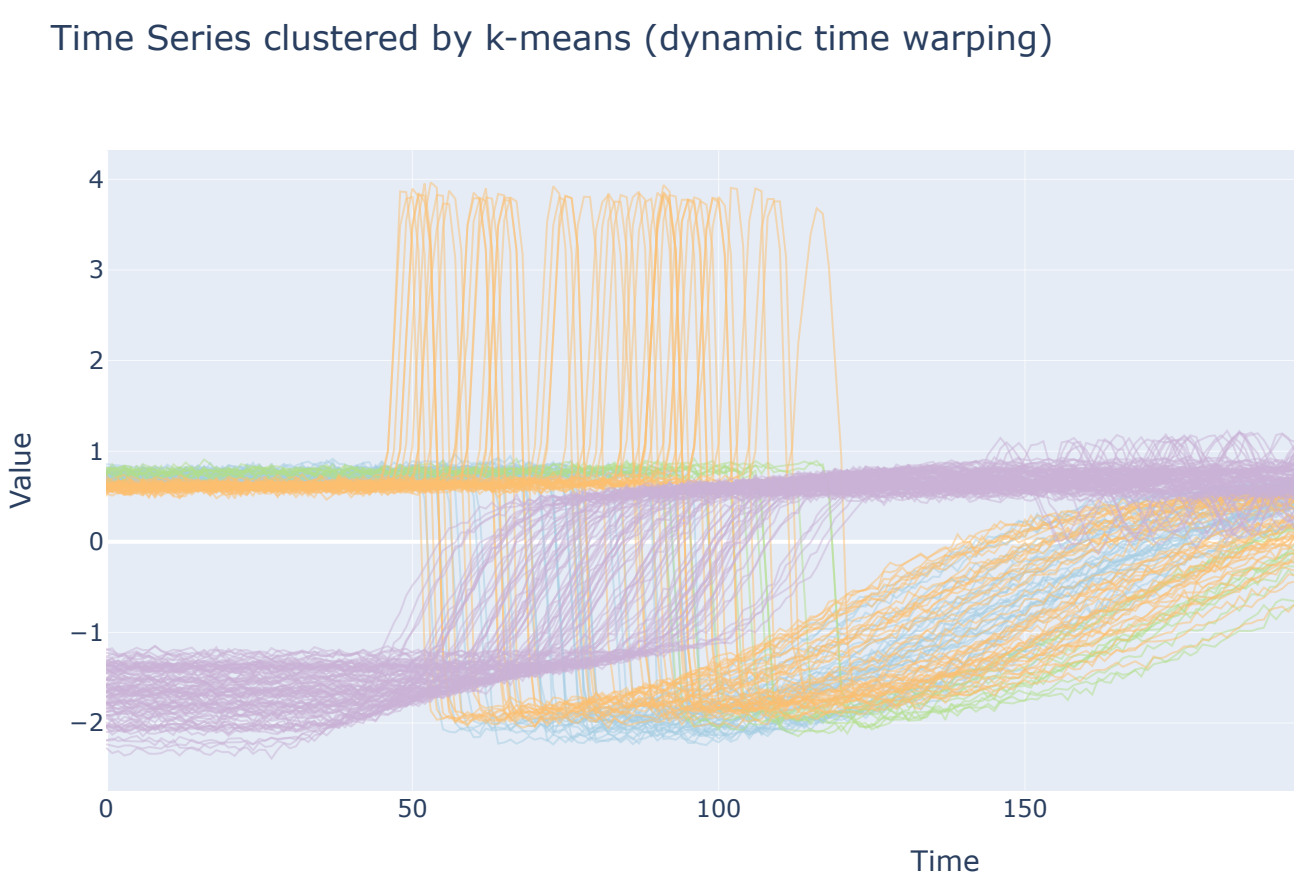
```
k = 4  # number of clusters

clusterer_dwt = TimeSeriesKMeansTslearn(
    n_clusters=4, metric="softdtw", n_jobs=-1, random_state=42
)  # n_jobs=-1 uses all available CPU cores


start_time = time.time()
y_predicted_dwt = clusterer_dwt.fit_predict(X)
duration = time.time() - start_time

print(f"Clustering took {duration:.2f} seconds.")
```

```
Clustering took 877.81 seconds.
```

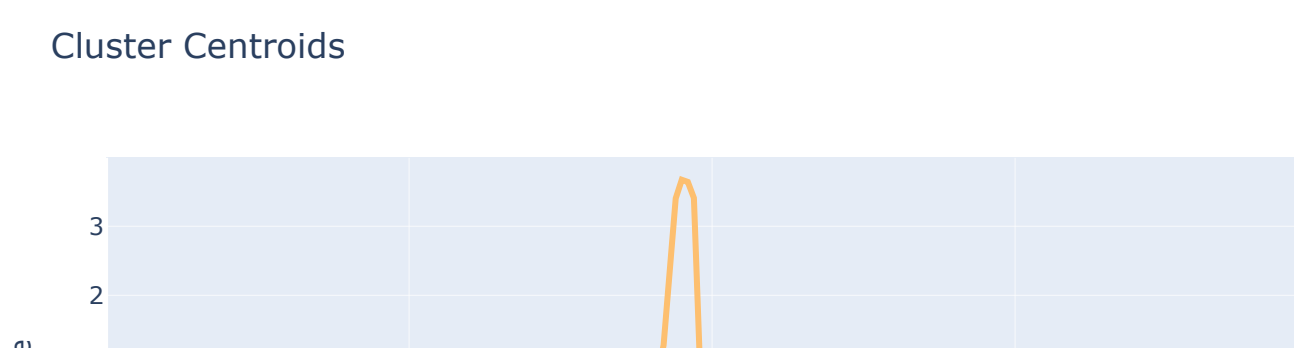
Have a look at the execution times of both algorithms. While DTW is executed in parallel (`n_jobs=-1`), it is still significantly slower than the euclidean distance version. The reason for this is that the DTW algorithm got a time complexity of $O(NM)$, where N and M are the lengths of the two time series to be compared. More sophisticated algorithms are able to slightly reduce this time complexity, but so far, the runtime complexity stays quadratic.

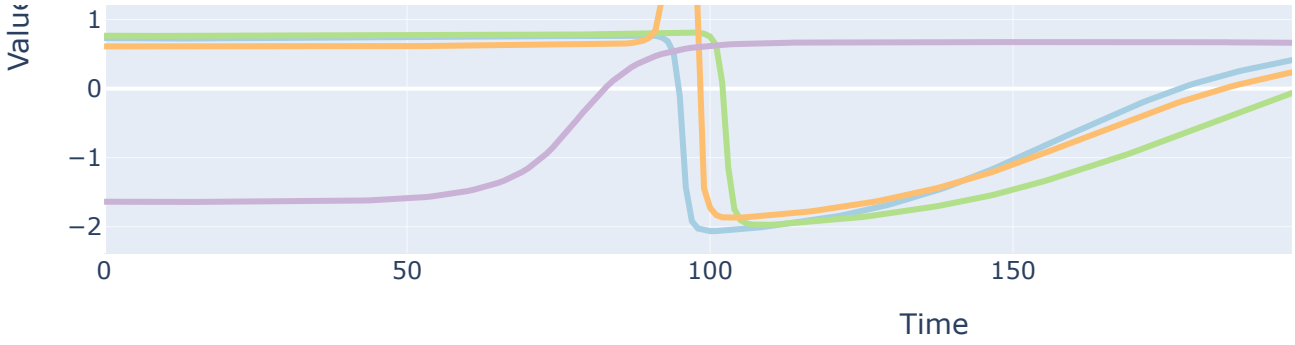
However, the clustering results look better now.

```
plot_clusters(
    X, y_predicted_dwt, "Time Series clustered by k-means (dynamic time warping)", False
)
```




```
plot_centroids(clusterer_dwt)
```





Cluster 1 matches class C very well, cluster 2 matches class D. However, cluster 3 and 4 still contain a mix of class A and B.

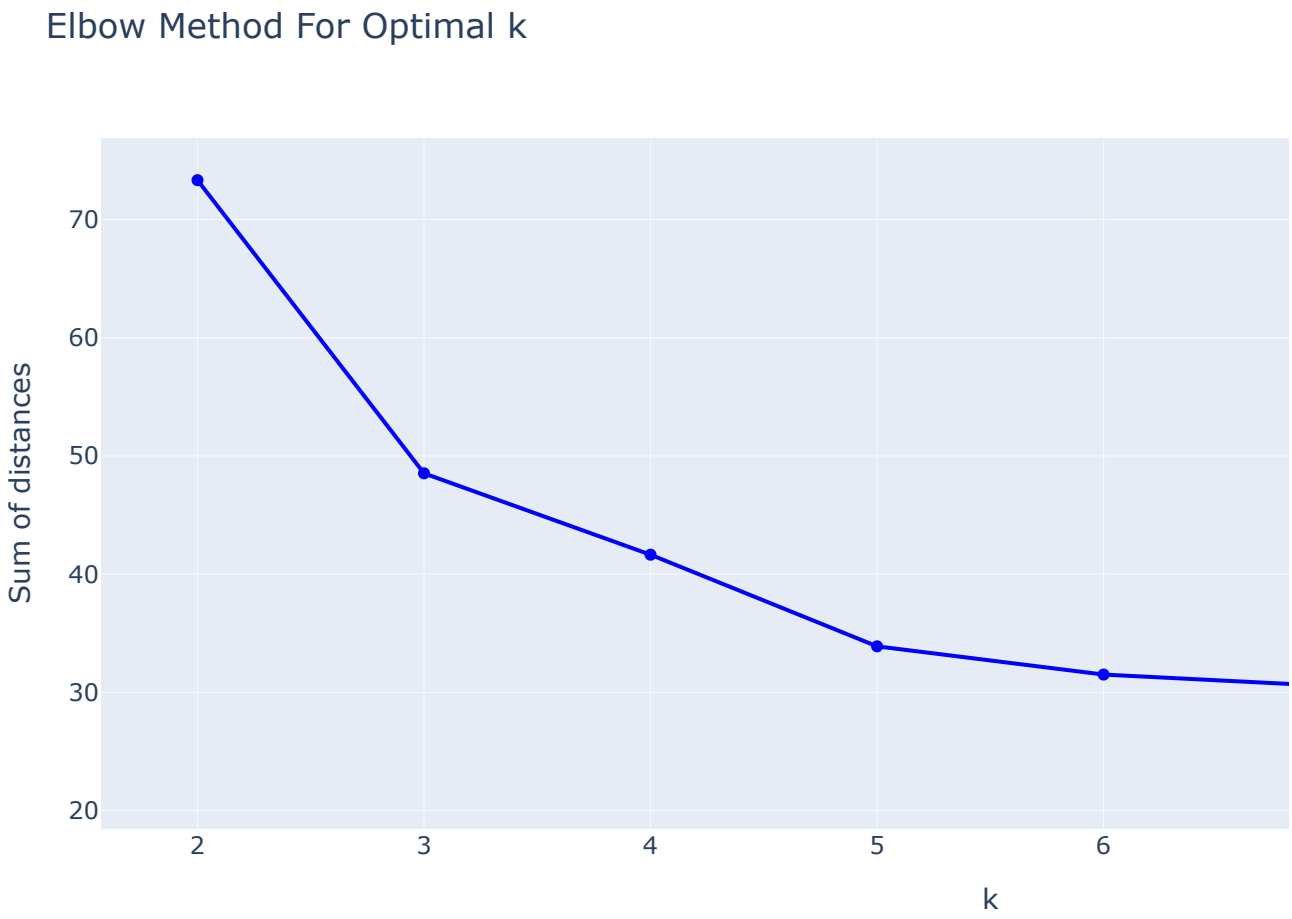
7.4 Elbow method for determining the number of clusters

One open question is how to determine the number of clusters k. A common method is the elbow method, which plots the sum of distances to the nearest cluster center for different values of k. The idea is to choose the value of k at which the rate of decrease sharply shifts, forming an elbow shape in the plot.

```
sum_of_distances = []
K = range(2, 10)

for k in K:
    km = TimeSeriesKMeansTslearn(
        n_clusters=k,
        metric="euclidean",
        random_state=42,
        n_jobs=-1,
    )

    km = km.fit(X)
    sum_of_distances.append(km.inertia_)
```

```
fig = go.Figure()

fig.add_trace(
    go.Scatter(
        x=list(K),
        y=sum_of_distances,
        mode="lines+markers",
        marker={"color": "blue"},
        line={"color": "blue"},
        name="Sum of distances",
    )
)

fig.update_layout(
    title="Elbow Method For Optimal k", xaxis_title="k", yaxis_title="Sum of distances"
)
```

```
fig.show()
```



The elbow method suggests that five or six clusters are a good choice for k, which is close to the true number of classes in the data.

Often it makes sense to start with a larger number of clusters and then merge similar clusters later on. This allows to capture more subtle patterns in the data.

7.5 Additional readings

tslearn dtw documentation (accessed: 16 09 2026) briefly introduces dynamic time warping, barycenters and soft-DTW. To learn more about different methods used to calculate barycenters, have a look at the tslearn barycenter documentation (accessed: 16 09 2026).

To learn more about dynamic time warping, the wikipedia article on dynamic time warping (accessed: 16 09 2026) is a good starting point.

k-means clustering in time series is still a very active research area. A recent preprint by Holder et al. (2024) gives a nice overview over the variants of k-means for time series and discusses their pros and cons.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

8 Time Series Classification

Time series classification belongs to the class of supervised learning and is defined as the task of assigning a label to a time series.

Again, this is a very active research area and many different methods have been proposed. For a good overview, check e.g. Faouzi (2024), which divides time series classification methods into the following categories:

- **Metric**: Distance-based methods, e.g. nearest neighbor classifiers with different distance measures.
- **Kernels**: Methods that use kernel functions to measure similarity between time series.
- **Shapelets**: Methods that learn discriminative subsequences (shapelets).
- **Tree-based**: Methods that use decision trees or random forests.
- **Bag-of-Words**: Methods that represent time series as a "bag of words".
- **Image**: Methods that treat time series as images.
- **Deep Learning**: Methods that use deep learning architectures.
- **Random Convolutions**: Methods that use random convolutional filters.
- **Ensemble**: Methods that combine multiple classifiers for improved performance.

We can see that many of these categories already appeared in Section 7.1.

A simple metric-based (distance-based) baseline method for time series classification is to use a nearest neighbor classifier with a suitable distance measure for time series, e.g. dynamic time warping (DTW). This is implemented in `sktime` as `KNeighborsTimeSeriesClassifier` or `KNeighborsTimeSeriesClassifierTslearn` (the latter is just a wrapper for the implementation in `tslearn`). These classifiers work similar to the `KNeighborsClassifier` for tabular data from `sklearn`, but use a distance measure suitable for time series. We have learned about `euclidean` distance and `DTW` in Section 7.2.

In this section we will have a brief look at time series forests (tree-based) in the context of time series classification.

8.1 Time Series Forests

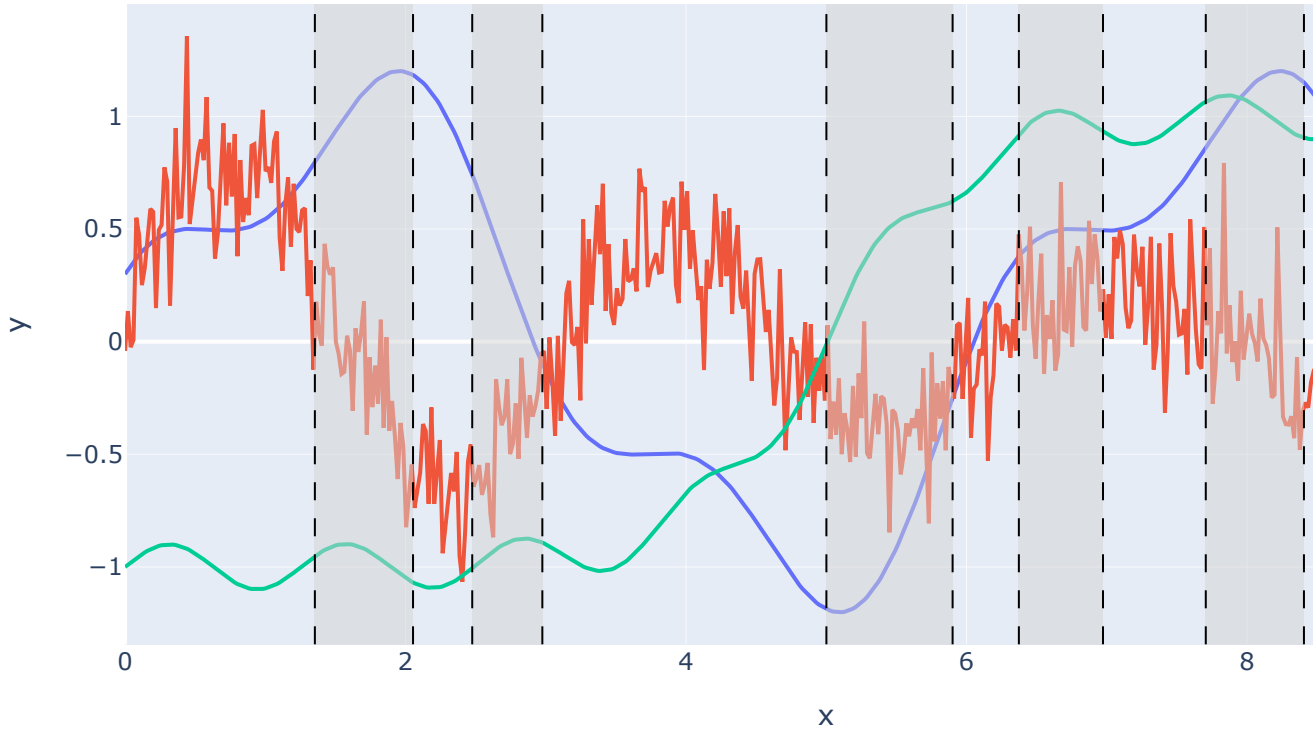
A time series forest is an ensemble of time series decision trees. Each decision tree is built using a random set of intervals from the time series.
For each interval, summary statistics (mean, standard deviation, slope) are computed and used as features for the decision tree.
The final prediction is made by aggregating the predictions of all trees in ensemble (e.g. by majority vote for classification).

Here is a simple example showing the feature extraction of three time series:

▶ Code

▶ Code

Non-linear Curves with Decision Tree Intervals



Interval	Curve	Mean	Std	Slope
Interval 1	Curve 1	1.07	0.13	0.61
	Curve 2	-0.14	0.30	-1.17
	Curve 3	-0.95	0.05	-0.22
Interval 2	Curve 1	0.31	0.25	-1.72
	Curve 2	-0.41	0.21	1.02
	Curve 3	-0.91	0.04	0.25
Interval 3	Curve 1	-0.89	0.30	1.11
	Curve 2	-0.35	0.19	-0.04
	Curve 3	0.43	0.18	0.65
Interval 4	Curve 1	0.48	0.03	0.15
	Curve 2	0.20	0.21	0.28
	Curve 3	0.99	0.03	-0.01
Interval 5	Curve 1	1.10	0.10	0.46
	Curve 2	-0.04	0.26	-0.67
	Curve 3	1.03	0.06	-0.30

`TimeSeriesForestClassifier` as implemented in `sktime` in particular uses 200 trees (`n_estimators`) by default and samples `sqrt(m)` intervals per tree, where `m` is the length of the time series.
More configurable tree based ensembles are provided with `ComposableTimeSeriesForestClassifier`.

8.1.1 Example

We will use the same dataset as in the clustering example from Section 7.3, the `Trace` dataset.

8.1.1.1 Helper function for plotting

▶ Code

8.1.1.2 Loading the dataset

```
X_train, y_train, X_test, y_test = CachedDatasets().load_dataset("Trace")

# Fix shape for TimeSeriesForestClassifier
X_train = X_train[:, :, 0]
X_test = X_test[:, :, 0]
```

8.1.2 Visualization

```
plot_clusters(X_train, y_train, "Training Set Clusters")
```

Note: Here, the dataset already comes split into training and test set. In practice, you would want to do a proper train-test split on your own dataset.

8.1.2.1 Classifier

We set up the classifier and train on the training set (X_{train}, y_{train}) as follows.

```
clf = TimeSeriesForestClassifier(random_state=42)
clf.fit(X_train, y_train)
```

▾ TimeSeriesForestClassifier

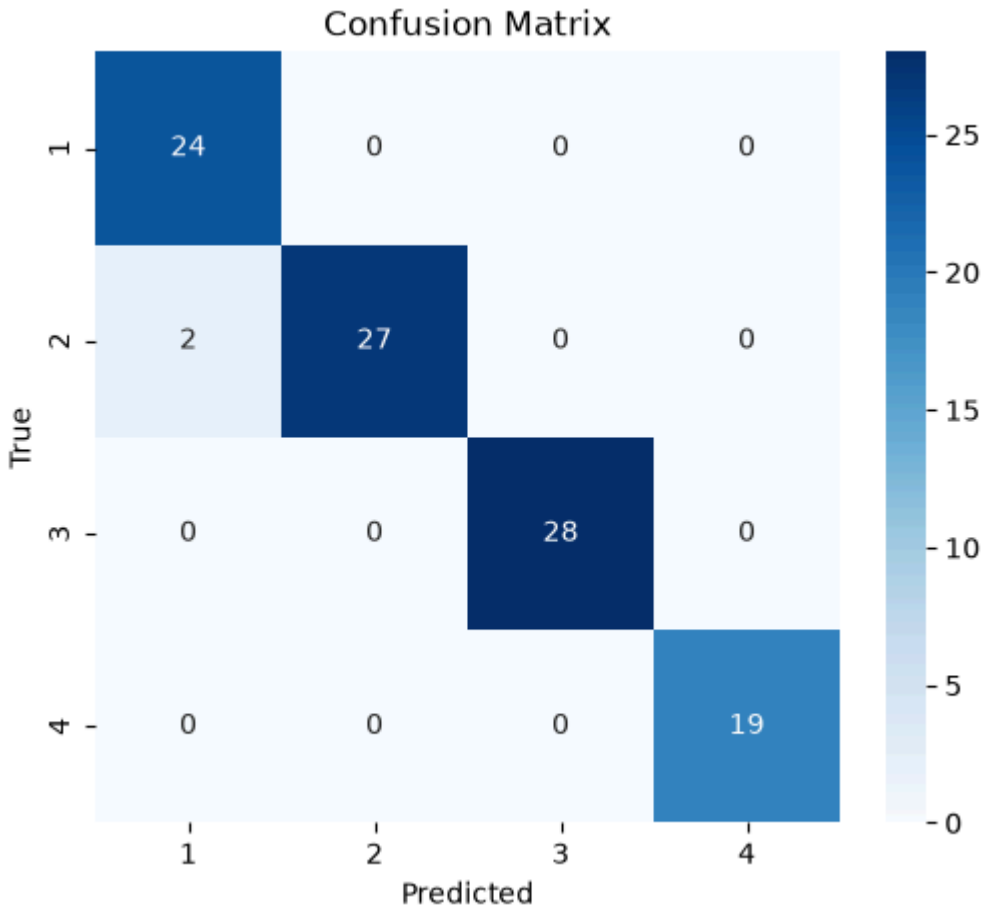
▶ Parameters

Having a fitted classifier model, we then predict the labels of the previously unseen test set X_{test}.

```
y_pred = clf.predict(X_test)
```

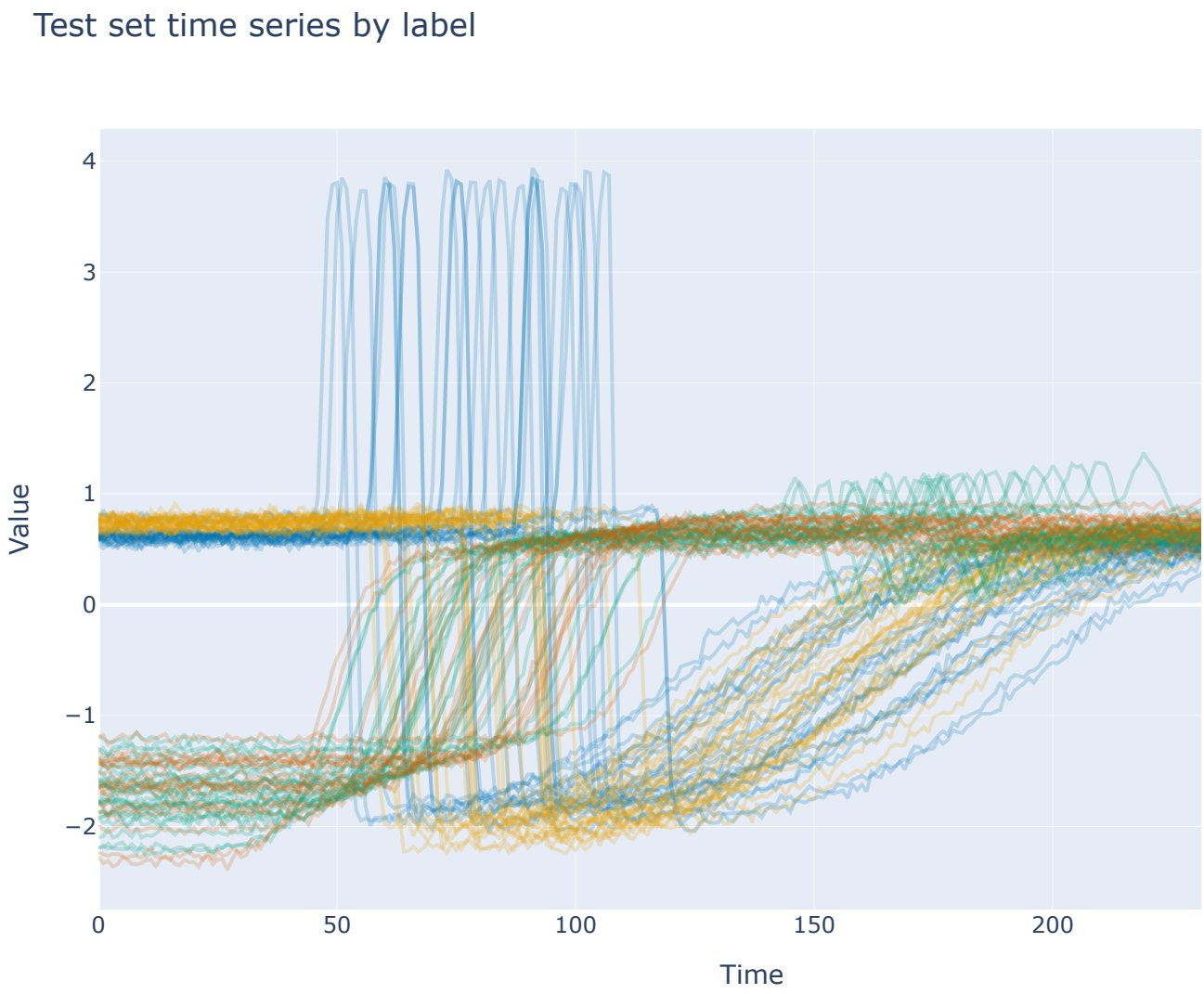
The following confusion matrix shows that the classifier performs extraordinarily well on this dataset, achieving an accuracy of almost 100% on the test set.

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
labels = sorted(set(y_test))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues", xticklabels=labels, yticklabels=labels
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()
```



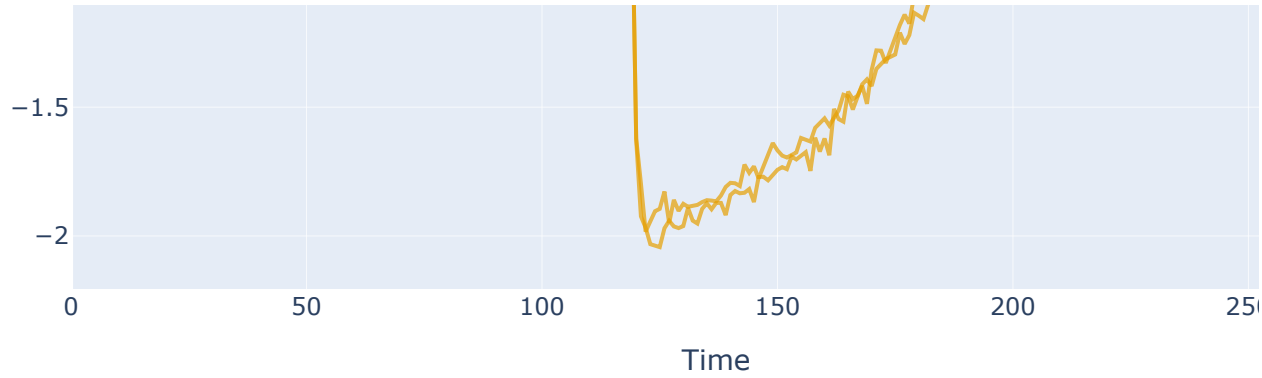
The following code plots the time series from the test set colored by their predicted label.

▶ Code



Let us also visualize the two misclassified time series:

▶ Code




```
Indices of misclassified test samples:
[52 75]
```

The two misclassified time series actually belong to class 2 but were predicted class 1.

Exercise 8.1 (Discussing results)

Why might these two time series have been misclassified?

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

9 Time Series Regression

Time series regression belongs to the class of supervised learning and is defined as the task of predicting a continuous target variable based on time series input data.

The github repository of sktime (accessed: 16 09 26) categorizes its implemented time series regression methods into the following categories:

- **Distance-based**: Methods that use distance measures to compare time series (e.g., k-nearest neighbors).
- **Interval-based**: Methods that extract features from intervals of the time series (e.g., time series forest regression).
- **Deep learning-based**: Methods that use deep learning architectures.
- **Kernel-based**: Methods that use kernel functions to measure similarity between time series (e.g. time series support vector regression).
- **Composed methods**: Methods that combine multiple models.

Again, we observe that these categories can directly be mapped to those in Section 7.1 and Chapter 8.

An easy baseline method for time series regression is to use a k-nearest neighbors regressor with a suitable distance measure for time series, e.g. dynamic time warping. In contrast to nearest neighbor in classification, where the predicted label is determined by e.g. a majority vote of the nearest neighbors, in nearest neighbor regression the predicted value is typically computed as the average (or weighted average) of the target values of the nearest neighbors. This is implemented in `sktime` as `KNeighborsTimeSeriesRegressor`.

Also tree-based methods like random forests can be adapted for time series regression by simply taking e.g. the mean value of the target variable of the samples in a leaf node as the predicted value. This is implemented in `sktime` as `TimeSeriesForestRegressor`. Note that this is (as of `v1.1.0`) not a very parametrizable implementation; the only tunable parameter is the number of estimators (trees) and the minimum width of the intervals.

In this section we will have a brief look at a deep-learning based method in the context of time series regression, namely a convolutional neural network (CNN) based regressor.

9.1 Convolutional Neural Network Regressor

Convolutional Neural Networks (CNN), by design, are able to capture local patterns in time series data through the use of convolutional layers. These layers apply filters that slide over the input data, allowing the model to learn local temporal patterns that are important. At the same time, CNNs can also capture global patterns by stacking multiple convolutional layers and using pooling operations. This hierarchical structure enables the model to learn both local and global features of the time series data, making CNNs versatile for various time series tasks.

Zhao et al. (2017) propose a CNN architecture specifically designed for time series classification. The architecture consists of a series of 1D convolutional layers followed by a pooling layer. The convolutional layers are responsible for extracting local features from the time series data, while the pooling layer helps to reduce the dimensionality and capture more global patterns. After the convolutional and pooling layers, the model includes a fully connected layer (the authors refer to it as a *feature layer*) that maps the extracted features to n

output nodes, where n is the number of classes. The node with the highest value determines the predicted class.

For regression, `sktime` provides `CNNRegressor`, which is based on the architecture described in Zhao et al. (2017). To adapt the architecture for regression tasks, the final layer is modified to have a single output node with a linear activation function that predicts a continuous value instead of class probabilities.

In this example, we will use the IEEEPPG dataset Tan et al. (2020), which focuses on heart rate monitoring during physical exercise using wrist-type *photoplethysmographic* (PPG) signals. The dataset consists of two PPG signals and three-axis acceleration signals. The goal is to predict ECG values from the PPG and acceleration signals. All signals were sampled at 125 Hz.

9.1.1 Imports

▶ Code

9.1.2 Loading the dataset

▶ Code

First we look at the raw training data. Each row corresponds to one sample in the dataset. The data comes in five columns, where each column represents a different signal source (PPG or acceleration signal). Note that each entry in the DataFrame is itself a time series.

```
X_train_raw
```

	dim_0	dim_1	dim_2	dim_3	dim_4
0	0 -23.0 1 -24.0 2 -26.5 3 -27....	0 4.0 1 6.0 2 3.0 3 3....	0 -0.0702 1 -0.0702 2 -0.0546 3 ...	0 0.3432 1 0.3588 2 0.3666 3 ...	0 0.9594 1 0.9438 2 0.9360 3 ...
1	0 -20.5 1 -19.0 2 -20.0 3 -19....	0 -37.0 1 -38.5 2 -40.5 3 -41....	0 0.4056 1 0.4212 2 0.4368 3 ...	0 0.6084 1 0.6162 2 0.6084 3 ...	0 0.5148 1 0.4992 2 0.4758 3 ...
2	0 22.0 1 23.5 2 22.5 3 23....	0 12.5 1 13.5 2 11.5 3 13....	0 0.4758 1 0.4758 2 0.4758 3 ...	0 0.6864 1 0.6864 2 0.6864 3 ...	0 0.3900 1 0.3900 2 0.3978 3 ...
3	0 -5.5 1 -5.5 2 -6.5 3 -6....	0 -1.0 1 -0.5 2 -1.5 3 -1....	0 0.4602 1 0.4602 2 0.4602 3 ...	0 0.6708 1 0.6786 2 0.6708 3 ...	0 0.3978 1 0.4134 2 0.4056 3 ...
4	0 30.5 1 27.0 2 26.0 3 24....	0 74.5 1 70.0 2 67.0 3 62....	0 0.6162 1 0.6162 2 0.6240 3 ...	0 0.7098 1 0.6942 2 0.6786 3 ...	0 0.3978 1 0.3900 2 0.3744 3 ...
...	...	...	...	...	...
1763	0 130.0 1 142.5 2 148.0 3 ...	0 68.0 1 86.0 2 100.5 3 ...	0 -0.0858 1 -0.0936 2 -0.1092 3 ...	0 0.6084 1 0.6240 2 0.6162 3 ...	0 0.2340 1 0.2418 2 0.2418 3 ...
1764	0 -33.5 1 -36.0 2 -38.0 3 -38....	0 32.0 1 26.0 2 21.0 3 18....	0 -0.2340 1 -0.2418 2 -0.2262 3 ...	0 0.2496 1 0.2340 2 0.1872 3 ...	0 1.0998 1 1.1154 2 1.1076 3 ...
1765	0 -38.0 1 -41.5 2 -43.5 3 -44....	0 21.5 1 15.5 2 11.0 3 7....	0 -0.1716 1 -0.1638 2 -0.1560 3 ...	0 0.0156 1 0.0312 2 0.0546 3 ...	0 0.8034 1 0.8034 2 0.8034 3 ...
1766	0 111.5 1 100.5 2 85.5 3 ...	0 155.5 1 147.0 2 134.0 3 ...	0 -0.0780 1 -0.1092 2 -0.1326 3 ...	0 0.1794 1 0.1170 2 0.0624 3 ...	0 0.9594 1 0.9672 2 0.9438 3 ...

	dim_0	dim_1	dim_2	dim_3	dim_4
1767	0 66.5 1 59.5 2 51.5 3 44....	0 95.5 1 90.0 2 84.0 3 77....	0 0.1638 1 0.1716 2 0.1794 3 ...	0 0.3978 1 0.4134 2 0.4290 3 ...	0 0.7956 1 0.7800 2 0.7566 3 ...

1768 rows × 5 columns

```
y_train_raw
```

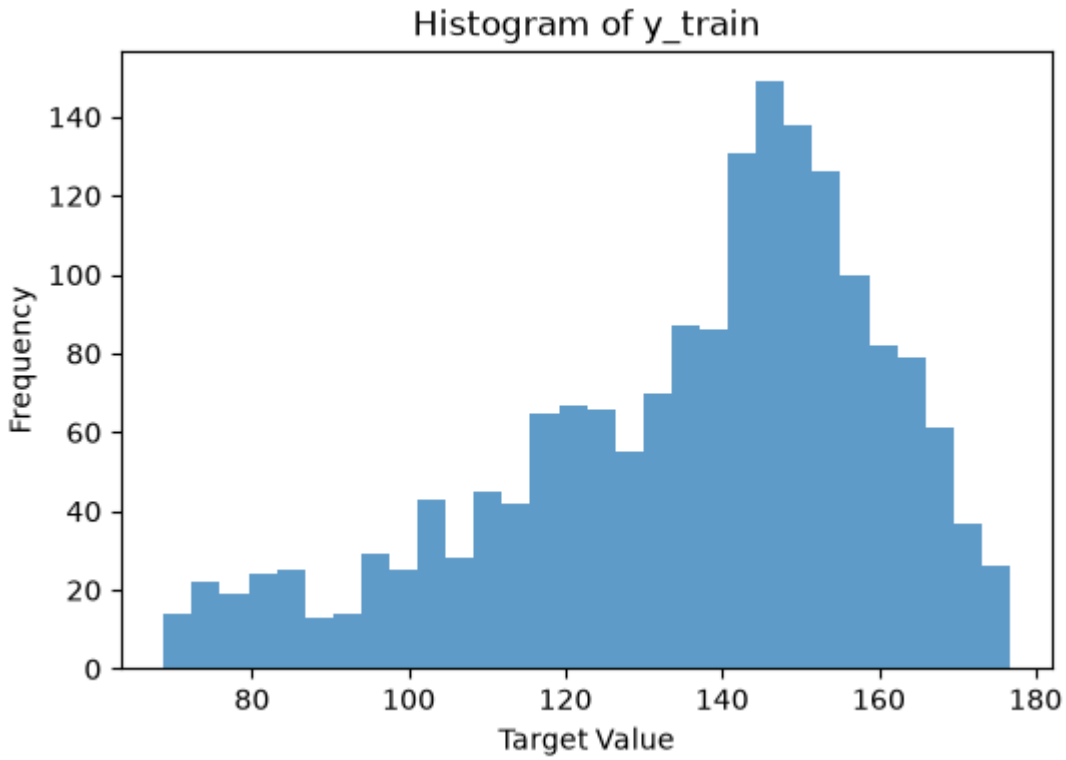
```
array([ 74.33920705,  76.35746606,  77.14285714, ..., 156.25      ,
       155.4404    , 154.0041    ], shape=(1768,))
```

To get a clearer picture, we can plot the signals of some of the training samples.

```
sampled = X_train_raw.sample(3, random_state=1337).sort_index()

fig, axes = plt.subplots(
    nrows=sampled.shape[1], ncols=3, figsize=(16, 3 * sampled.shape[1]), sharex=True
)

for row_idx, col in enumerate(sampled.columns):
    for col_idx, (i, row) in enumerate(sampled.iterrows()):
        axes[row_idx, col_idx].plot(row[col].values, label=f"Sample {i}", linewidth=0.8)
        axes[row_idx, col_idx].set_title(f"{col} - Heart rate {y_train_raw[i]:.0f}")
        axes[row_idx, col_idx].set_xlabel("Time")
    axes[row_idx, 0].set_ylabel("Value")

plt.tight_layout()
plt.show()
```

The following plot shows the distribution of the target values in the training set.

```
plt.figure(figsize=(6, 4))
plt.hist(y_train_raw, bins=30, alpha=0.7)
plt.xlabel("Target Value")
plt.ylabel("Frequency")
plt.title("Histogram of y_train")
plt.show()
```



To feed the data into the CNN, we first need to convert the dataframe into a so-called tensor (in this case a 3D array).

```
def convert_df_to_tensor(df: pd.DataFrame) -> np.ndarray:
    """Convert a DataFrame of time series to a 3D numpy tensor.

    Args:
        df: DataFrame where each cell contains a time series.

    Returns:
        3D array of shape (n_samples, ts_length, n_features).
    """
    tensor = np.array(
        [[df.iloc[i, j].values for j in range(df.shape[1])] for i in range(len(df))]
    )
    tensor = np.transpose(tensor, (0, 2, 1))

    for i, j in zip(range(len(df)), range(len(df.columns))):
        assert np.array_equal(df.iloc[i, j].values, tensor[i, :, j])

    return tensor


X_train_tensor = convert_df_to_tensor(X_train_raw)
X_test_tensor = convert_df_to_tensor(X_test_raw)
```

Let us check the shapes orf the resulting tensors.

```
print(X_train_tensor.shape)
print(X_test_tensor.shape)
```

```
(1768, 1000, 5)
(1328, 1000, 5)
```

The training data set has 1768 samples, each consisting of 5 channels (PPG and acceleration signals) with a length of 1000 time steps (which are 8 seconds at 125 Hz).

Neural networks are generally trained on normalized data. We will use a leaky ReLU activation function in our CNN, which works best with input data within a [0, 1] range.

Unfortunately the `TimeSeriesScalerMinMax` from `tslearn` needs the data in the shape (n_samples, n_timestamps, n_channels), while the CNN expects (n_samples, n_channels, n_timestamps), so we need to swap the last two axes again after scaling.

```
scaler_x = TimeSeriesScalerMinMax()
X_train = scaler_x.fit_transform(X_train_tensor, per_feature=True)
X_test = scaler_x.transform(X_test_tensor, per_feature=True)

scaler_y = MinMaxScaler()
y_train = scaler_y.fit_transform(y_train_raw.reshape(-1, 1))
y_test = scaler_y.transform(y_test_raw.reshape(-1, 1))

X_train = np.transpose(X_train, (0, 2, 1))
X_test = np.transpose(X_test, (0, 2, 1))
```

```
print(X_train.shape)
print(X_test.shape)
```

```
(1768, 5, 1000)
(1328, 5, 1000)
```

9.1.3 Regressor

The `CNNRegressor` from `sktime` can now be used to create and train the convolutional neural network for time series regression.

```
regressor = CNNRegressor(
    activation="linear",
    activation_hidden="leaky_relu",
    batch_size=64,
    metrics=[],
    n_conv_layers=5,
    n_epochs=80,
    optimizer=Adam(),
    random_state=42,
    verbose=True,
)

regressor.fit(X_train, y_train)
```

Model: "functional_1"

Layer (type)	Output Shape	Param #

input_layer_1 (InputLayer)	(None, 1000, 5)	0
conv1d_5 (Conv1D)	(None, 994, 6)	216
average_pooling1d_5 (AveragePooling1D)	(None, 331, 6)	0
conv1d_6 (Conv1D)	(None, 325, 12)	516
average_pooling1d_6 (AveragePooling1D)	(None, 108, 12)	0
conv1d_7 (Conv1D)	(None, 102, 12)	1,020
average_pooling1d_7 (AveragePooling1D)	(None, 34, 12)	0
conv1d_8 (Conv1D)	(None, 28, 12)	1,020
average_pooling1d_8 (AveragePooling1D)	(None, 9, 12)	0
conv1d_9 (Conv1D)	(None, 3, 12)	1,020
average_pooling1d_9 (AveragePooling1D)	(None, 1, 12)	0
flatten_1 (Flatten)	(None, 12)	0
dense_1 (Dense)	(None, 1)	13

Total params: 3,805 (14.86 KB)

Trainable params: 3,805 (14.86 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - loss: 0.0801
Epoch 2/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - loss: 0.0457
Epoch 3/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 0.0435
Epoch 4/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 0.0415
Epoch 5/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0387
Epoch 6/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 0.0369
```

```
Epoch 7/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 0.0357
Epoch 8/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 0.0344
Epoch 9/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - loss: 0.0336
Epoch 10/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 0.0320
Epoch 11/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 0.0310
Epoch 12/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 0.0297
Epoch 13/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 0.0282
Epoch 14/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0265
Epoch 15/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0236
Epoch 16/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0222
Epoch 17/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 0.0205
Epoch 18/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0191
Epoch 19/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0179
Epoch 20/80
```

```
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0170

Epoch 21/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - loss: 0.0162

Epoch 22/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 29ms/step - loss: 0.0154

Epoch 23/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 0.0148

Epoch 24/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - loss: 0.0143

Epoch 25/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0138

Epoch 26/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 0.0132

Epoch 27/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0130

Epoch 28/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 0.0125

Epoch 29/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 0.0123

Epoch 30/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0120

Epoch 31/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 0.0118

Epoch 32/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 0.0115

Epoch 33/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 0.0112
```

```
Epoch 34/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0111
Epoch 35/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0108
Epoch 36/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0106
Epoch 37/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0104
Epoch 38/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0103
Epoch 39/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0101
Epoch 40/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0099
Epoch 41/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0097
Epoch 42/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 0.0096
Epoch 43/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 0.0095
Epoch 44/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0093
Epoch 45/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0092
Epoch 46/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 0.0091
Epoch 47/80
```

```
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0090

Epoch 48/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0088

Epoch 49/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0087

Epoch 50/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0086

Epoch 51/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0085

Epoch 52/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0084

Epoch 53/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0083

Epoch 54/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0082

Epoch 55/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0081

Epoch 56/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0080

Epoch 57/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0079

Epoch 58/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 0.0078

Epoch 59/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0077

Epoch 60/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0075
```

```
Epoch 61/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0074
Epoch 62/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0073
Epoch 63/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 0.0072
Epoch 64/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 0.0071
Epoch 65/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0070
Epoch 66/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 0.0070
Epoch 67/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0069
Epoch 68/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0069
Epoch 69/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0068
Epoch 70/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0068
Epoch 71/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0067
Epoch 72/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0066
Epoch 73/80
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 0.0066
Epoch 74/80
```

```
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0065

Epoch 75/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0065

Epoch 76/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0064

Epoch 77/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0064

Epoch 78/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0063

Epoch 79/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.0063

Epoch 80/80

28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0062
```

CNNRegressor

```
CNNRegressor(activation_hidden='leaky_relu', batch_size=64, metrics=[],
             n_conv_layers=5, n_epochs=80,
             optimizer=<keras.src.optimizers.adam.Adam object at 0x7bcda30b1d10>,
             random_state=42, verbose=True)
```

To verify the performance of the trained model, we now predict the target values for the test set and compute the *root mean squared error* (RMSE) between the predicted and true target values.

Keep in mind, that we have to inverse transform the predicted target values back to the original scale before computing the RMSE.

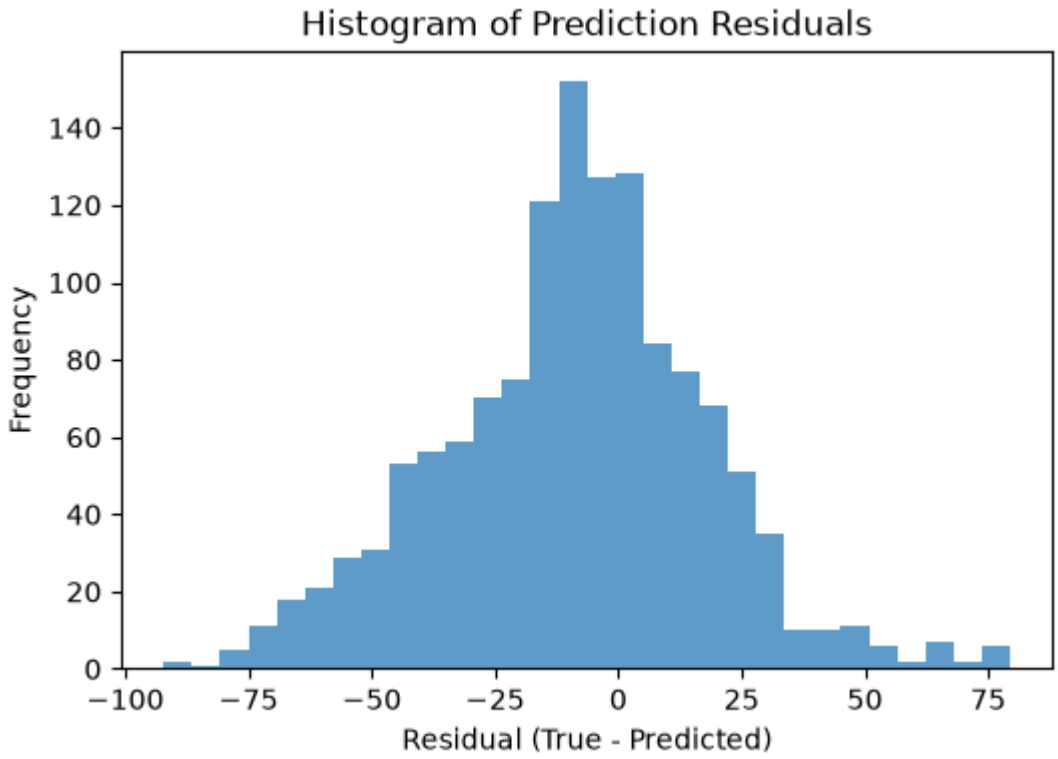
```
y_pred_scaled = regressor.predict(X_test)
y_pred = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1))

rmse = root_mean_squared_error(y_test_raw, y_pred)
print(f"RMSE: {rmse:.4f}")
```

```
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step

RMSE: 28.2592
```

The RMSE indicates that the model is not performing very well on this task.

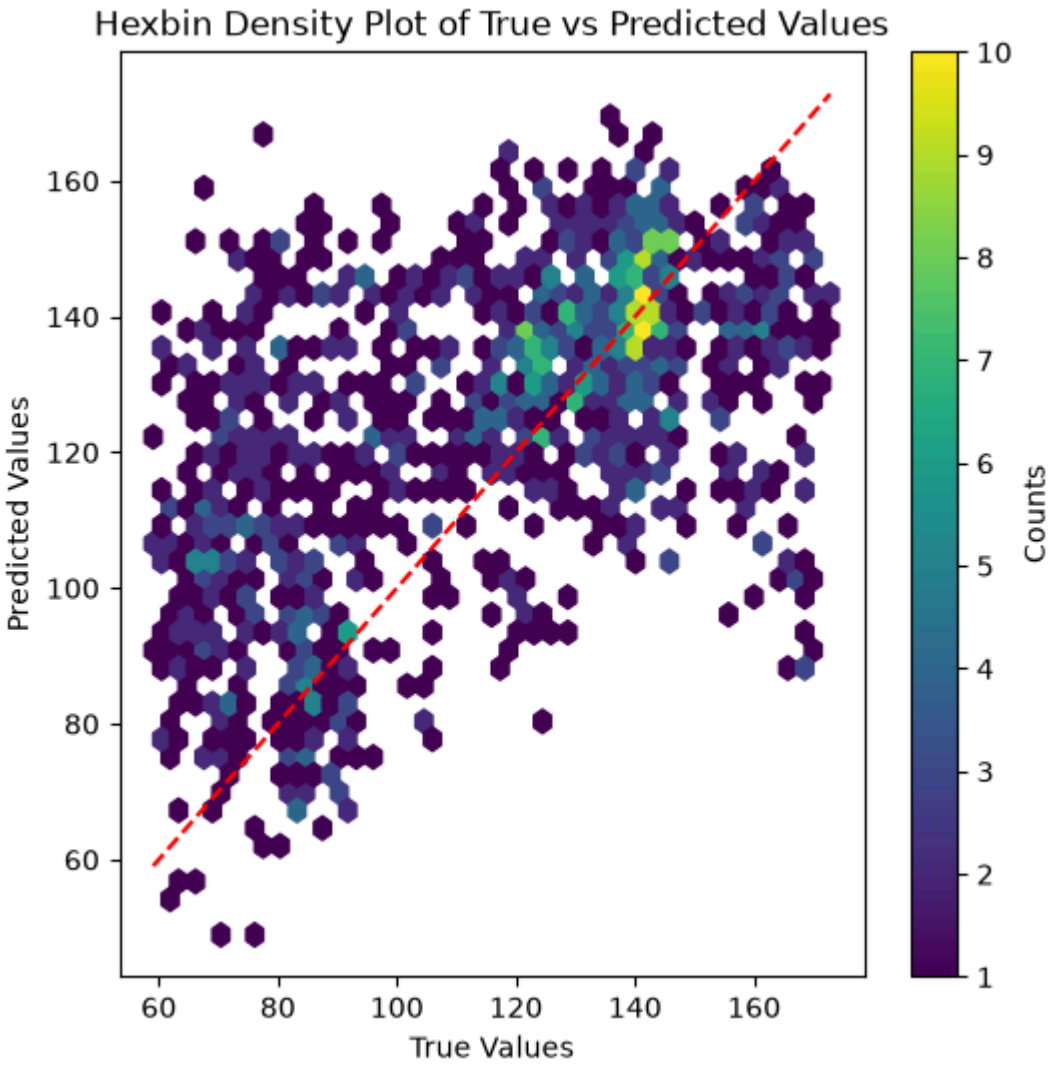
Let us visualize the residuals (the difference between the true and predicted target values) to get a better understanding of the model's performance.

```
plt.figure(figsize=(6, 4))
plt.hist(y_test_raw - y_pred.flatten(), bins=30, alpha=0.7)
plt.xlabel("Residual (True - Predicted)")
plt.ylabel("Frequency")
plt.title("Histogram of Prediction Residuals")
plt.show()
```



A scatter plot showing true against predicted values reveals that the model tends to overestimate smaller values and underestimate larger values.

```
plt.figure(figsize=(6, 6))
plt.hexbin(y_test_raw, y_pred.flatten(), gridsize=40, cmap="viridis", mincnt=1)
plt.xlabel("True Values")
plt.ylabel("Predicted Values")
plt.title("Hexbin Density Plot of True vs Predicted Values")
plt.plot(
    [y_test_raw.min(), y_test_raw.max()], [y_test_raw.min(), y_test_raw.max()], "r--"
)
plt.colorbar(label="Counts")
plt.show()
```



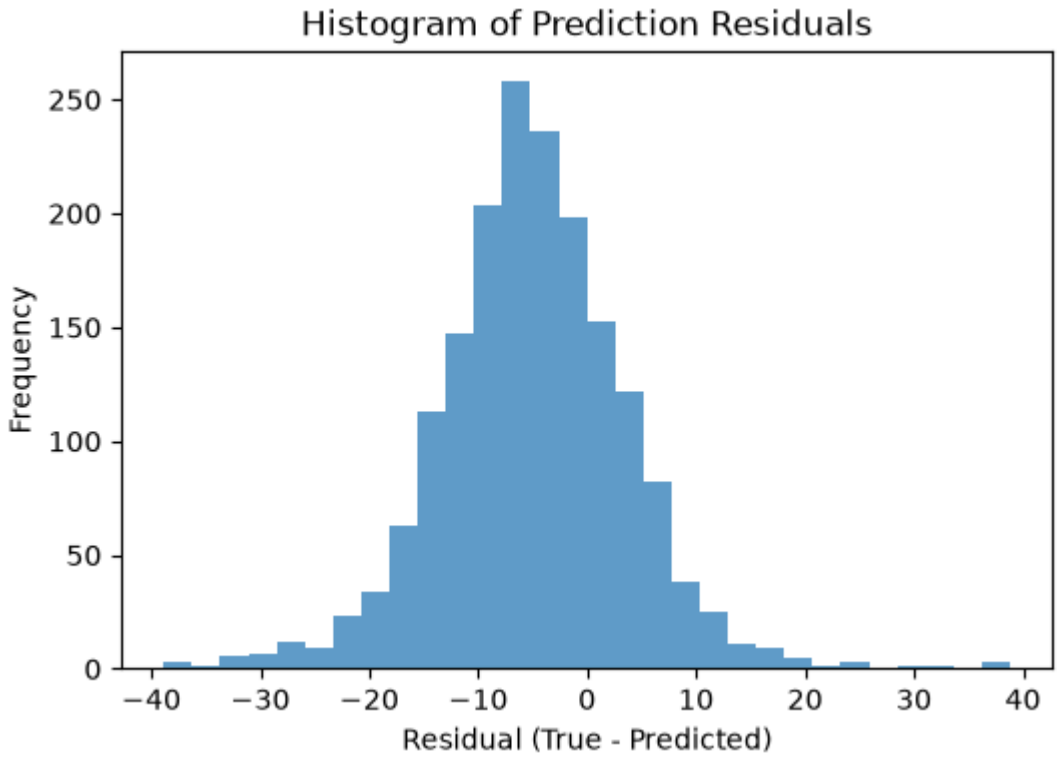
Especially when encountering a model that performs worse than expected, it is important to analyze potential reasons for this behavior.

Here, we predict the training set using the trained model. Usually, the training error should be significantly lower than the test error. If this is not the case, we can suspect issues with the code, model (architecture) or with the data itself.

Plotting the histogram of residuals on training set

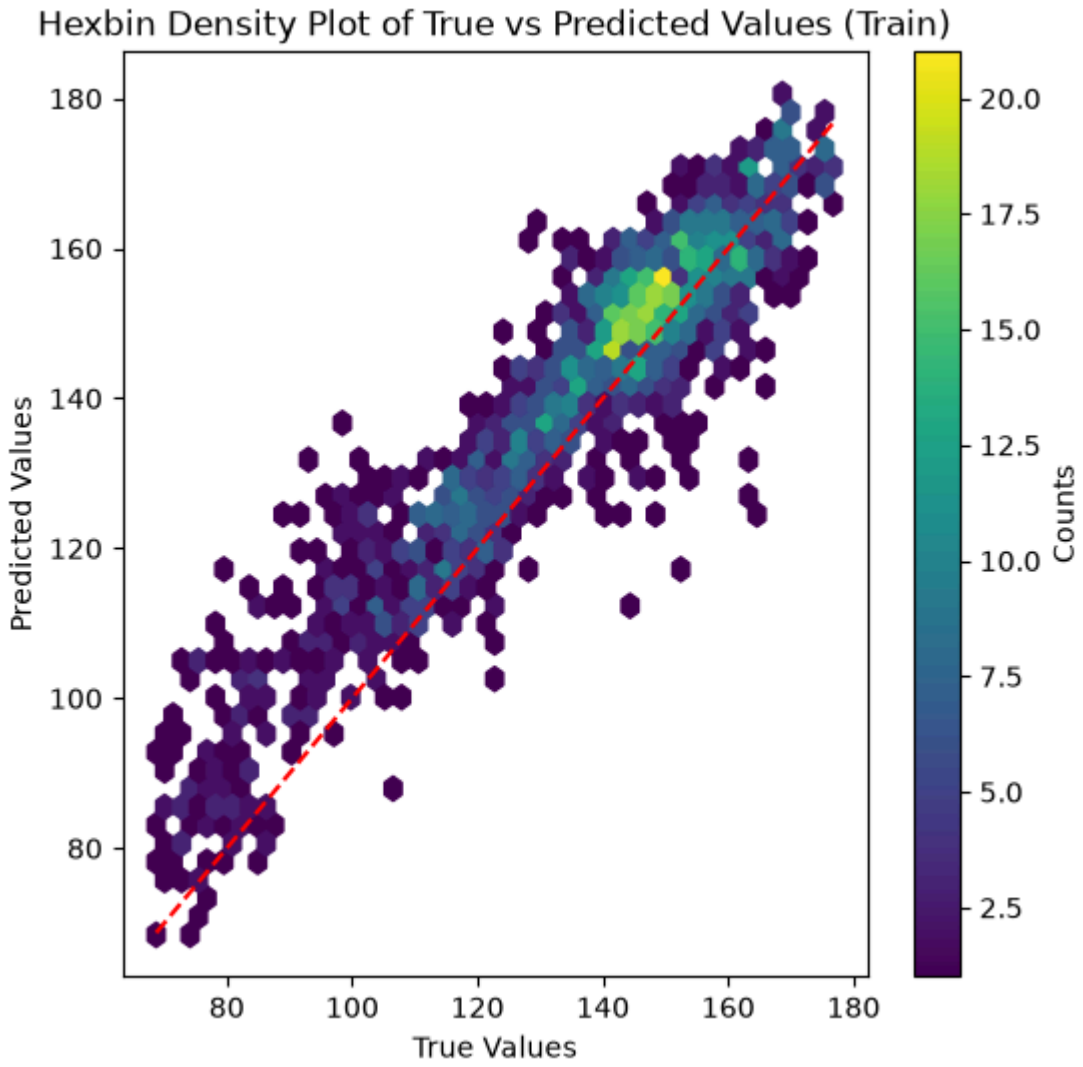
```
y_pred_train_scaled = regressor.predict(X_train)
y_pred_train = scaler_y.inverse_transform(y_pred_train_scaled.reshape(-1, 1))

plt.figure(figsize=(6, 4))
plt.hist(y_train_raw - y_pred_train.flatten(), bins=30, alpha=0.7)
plt.xlabel("Residual (True - Predicted)")
plt.ylabel("Frequency")
plt.title("Histogram of Prediction Residuals")
plt.show()
```

```
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
```



Plotting the scatter plot of true vs. predicted values on training set

```
plt.figure(figsize=(6, 6))
plt.hexbin(y_train_raw, y_pred_train.flatten(), gridsize=40, cmap="viridis", mincnt=1)
plt.xlabel("True Values")
plt.ylabel("Predicted Values")
plt.title("Hexbin Density Plot of True vs Predicted Values (Train)")
plt.plot(
    [y_train_raw.min(), y_train_raw.max()],
    [y_train_raw.min(), y_train_raw.max()],
    "r--",
)
plt.colorbar(label="Counts")
plt.show()
```



The model is able to fit the training data quite well, indicating that the code is implemented correctly and the model architecture is complex enough to capture the underlying patterns in the data. Still, the performance on the test set is significantly worse than on the training set. This suggests that the model is likely overfitting the data, which could be due to various factors such as e.g. insufficient training data, or lack of regularization or that the test data distribution differs significantly from the training data distribution.

While these results are not fully satisfying, the results align with those reported by Tan et al. (2021) for other simpler neural network architectures on this dataset.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

10 Self study - Session 1

This class contains self-study sessions to reinforce your understanding of the material and explore other aspects of the topic in a self-guided manner.

In this specific session, we will get hands-on with some essential tools for reporting: `uv`, `JupyterLab`, `seaborn`, and `Quarto`.

Each exercise focuses on a single topic.

Exercise 10.1 (Setting up an isolated environment with `uv`)

`uv` is a fast, modern package and environment manager. It installs and manages Python versions and packages in isolated environments (`venv`).

1. Install `uv` following the official instructions.
2. In the project directory, initialize a new project and create a virtual environment using `uv`.
3. Install the following packages into the environment: `numpy`, `matplotlib`.
4. Verify the environment works by creating a simple Python script that creates a simple plot of $f(t) = (16(\sin(t))^3, 13\cos(t) - 5\cos(2t) - 2\cos(3t) - \cos(4t))$ using the `numpy` and `matplotlib` package. (Hint: Use `numpy.linspace` to create a range of values for `t`, the function produces the x and y coordinates of the plot.)

After this exercise you should be able to 1. Create and manage isolated Python environments using `uv`. 1. Install and manage Python packages within these environments. 1. Additionally be able to explain what `uv` does and why it is useful.

Now that you have a working environment, you can use it to install additional packages as needed for the exercises below.

The data set we will rely on for this session is the AI4I predictive maintenance dataset, which is a synthetic dataset that tries to mirror real predictive maintenance data encountered in industry.

As we are only using the data to explore a couple of libraries and tools, we are not going into the details of the dataset itself in this session.

Exercise 10.2 (Working with JupyterLab)

JupyterLab is a web-based interactive development environment (IDE) for Jupyter Notebooks. It provides a flexible interface for interactive data exploration and reporting.

1. Launch JupyterLab from your `uv` environment by adding it to your environment first or alternatively, using VS Code, you can also use the Jupyter extension instead.
2. Create a new notebook called `ai4i_explore.ipynb`.
3. Load the AI4I dataset into a `pandas` DataFrame using a code cell. Keep only the columns `UDI` (unique identifier which can be used as x—axis for plotting), `Torque`, `Rotational speed` and `Air temperature`.
4. Display the first 10 rows and print basic statistics.
5. Create a line plot of `Torque` over `UDI` using `matplotlib`.
6. Experiment with different cell types: code, markdown, and raw cells.

7. Export the notebook as HTML and PDF.

After this exercise you should be able to:

1. Create and organize notebooks in JupyterLab.
2. Differentiate between code, markdown, and raw cells.
3. Export notebooks to other formats (HTML, PDF).
4. Structure a notebook with self-contained, readable code cells.

`matplotlib` is a widely used Python library for creating static visualizations. Now we will introduce `seaborn`, which is a statistical data visualization library built on top of `matplotlib`, providing a high-level interface for drawing attractive and informative statistical graphics.

Exercise 10.3 (Visualizing the AI4I data with seaborn)

In this exercise you will explore the AI4I dataset (columns `UDI`, `Torque`, `Rotational speed` and `Air temperature`) using `seaborn`.

Basic plots

1. Load the AI4I dataset (keeping only `Torque`, `Rotational speed` and `Air temperature`).
2. Create a **line plot** of `Torque` over `UDI`.
3. Create a **histogram** of each column to observe the distribution of values.
4. Create **box plots** of each column to identify outliers and understand the spread of the data.
5. Create **violin plots** of each column and discuss how they differ from box plots.

Relationships between variables

1. Create a **scatter plot** of `Torque` vs `Rotational speed` using `sns.scatterplot`. Add `Air temperature` as the `hue` parameter to see how the third variable influences the relationship.
2. Create a **pair plot** (`sns.pairplot`) of all three numeric columns to observe pairwise relationships and distributions on the diagonal.
3. Create a **correlation heatmap** using `sns.heatmap` on the correlation matrix of the three columns. Discuss which variables are most strongly correlated.

Advanced plots

1. Create a **joint plot** of `Torque` vs `Air temperature` with a regression line overlay.
2. Create a **strip plot** or **swarm plot** of `Torque` colored by a binned version of `Rotational speed` (e.g., low/medium/high using `pd.cut`).
3. Create a **count plot** showing the distribution of data points across bins of `Torque` (e.g., binned into 10 intervals).

Observations

1. Based on all the plots above, answer the following:
   - Are there any obvious outliers in the data?
   - Which two variables seem most related?
   - Are there any non-linear patterns visible?
   - How are the distributions shaped?

After this exercise you should be able to:

1. Create a wide range of static statistical plots with `seaborn`.
2. Use parameters to add additional dimensions to your plots.

3. Compare and discuss different plot types for different analytical questions.
4. Draw meaningful observations from statistical visualizations.

Congratulations, you are now able to create and manage isolated Python environments, work with JupyterLab, and visualize data using `seaborn`. The next step is to integrate your code and visualizations into a comprehensive report which you can hand over to stakeholders using Quarto.

Exercise 10.4 (Generating a report with Quarto)

Quarto is an open-source scientific and technical publishing system. It allows you to create dynamic documents, reports, and presentations from your code and narrative text. The material of this course itself is rendered using Quarto.

1. Create a Quarto Markdown (`.qmd`) file in your project.
2. Embed the code and plots from the previous exercise into the Quarto document.
3. Use code chunk options to control output (e.g., `echo=FALSE` to hide code, `fig-width` to set figure dimensions).
4. Render the document as HTML and PDF.
5. Explore collapsing code chunks in your document (see Quarto docs).
6. Quarto also allows to render Jupyter notebook files (`.ipynb`) directly to HTML and PDF (see Quarto docs):
    - Prepare the notebook you created in the previous exercise for rendering with Quarto by adding additional information that makes a nice presentation.
    - Render the notebook you created as HTML and PDF using Quarto.

After this exercise you should be able to:

1. Create a Quarto document that integrates code, output, and narrative.
2. Render Quarto documents to HTML and PDF formats.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

11 Self study - Session 2

In this session, we will revisit Chapter 5 and Chapter 6.

In Chapter 5, we have explored different methods to handle missing data in time series and have seen that the choice of imputation method can significantly affect the results.

The data set we will rely on in this session is the synthetic time series from the previous section which simulates weekly production data of a manufacturing plant over 20 years. Unfortunately, some data points are missing due to frequent outages in the data collection system and sometimes we observe huge outliers due to temporary errors in the counting process.

Your task is to impute the missing values and handle the outliers appropriately to prepare the data for a 20 years jubilee management report.

Again, you can download the data set from here.

Additionally, here are some useful helper functions:

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

def plot_production_data(
    df,
    orig_data_column,
    gap_column,
    imp_column=None,
):
    """
    Plots production data with and without gaps using seaborn.
    Parameters:
      df (DataFrame): DataFrame containing the production data.
      orig_data_column (str): Column name for the original data.
      gap_column (str): Column name for the data with gaps.
      imp_column (str): Column name for the imputed data.
    """
    fig, axes = plt.subplots(3 if imp_column else 2, 1, figsize=(12, 10))

    # Original data
    sns.lineplot(x=df.index, y=df[orig_data_column], ax=axes[0], color="grey", label="Original
    axes[0].set_title("Original Data")
    axes[0].set_ylabel("pieces")

    # With gaps
    sns.lineplot(x=df.index, y=df[gap_column], ax=axes[1], color="black", label="With Gaps")
    axes[1].set_title("Data with Gaps")
    axes[1].set_ylabel("pieces")

    if imp_column is not None:
```

```
        dff = df[[gap_column, imp_column]].copy()
        dff.loc[~df[gap_column].isna(), imp_column] = None
        sns.lineplot(x=dff.index, y=dff[imp_column], ax=axes[2], color="blue", label="Imputed"
        axes[2].set_title("Imputed Data")
        axes[2].set_ylabel("pieces")

    plt.tight_layout()
    plt.show()


def seasonal_decompose_seaborn(decomposed_ts, title):
    """
    Plots a decomposed time series using seaborn.
    Parameters:
      decomposed_ts (DecomposeResult): Result of STL or seasonal_decompose.
      title (str): Title for the plot.
    """
    fig, axes = plt.subplots(4, 1, figsize=(12, 10))
    sns.lineplot(x=decomposed_ts.observed.index, y=decomposed_ts.observed.values, ax=axes[0])
    axes[0].set_title("Observed")
    axes[0].set_ylabel("pieces")

    sns.lineplot(x=decomposed_ts.trend.index, y=decomposed_ts.trend.values, ax=axes[1])
    axes[1].set_title("Trend")
    axes[1].set_ylabel("pieces")

    sns.lineplot(x=decomposed_ts.seasonal.index, y=decomposed_ts.seasonal.values, ax=axes[2])
    axes[2].set_title("Seasonal")
    axes[2].set_ylabel("pieces")

    sns.lineplot(x=decomposed_ts.resid.index, y=decomposed_ts.resid.values, ax=axes[3])
    axes[3].set_title("Residual")
    axes[3].set_ylabel("pieces")

    plt.suptitle(title)
    plt.tight_layout()
    plt.show()
```

Exercise 11.1 (Load the data with gaps and get rid of outliers)

- Download and load the dataset. It has three columns: `Date`, `pieces` and `pieces_gaps`. The column `pieces` contains the full data set without gaps, while `pieces_gaps` contains the same data set with several missing values (NaNs). The period covered by the data set is from 2005-01-01 to 2024-12-31 in weekly frequency.
- Visualize both time series (`pieces` and `pieces_gaps`) using the provided `plot_production_data` function.
- Smooth the `pieces_gaps` time series using a rolling window with appropriate window size and decide whether it is better to use `mean` or `median` as aggregation function. Discuss your choice. Ideally the resulting smoothed time series should have the same gaps as the original `pieces_gaps` time series.
- Visualize the resulting smoothed time series using `seaborn`.

Let us try to impute the missing values in a second step.

Exercise 11.2 (Impute the missing values with known methods)

- Use the smoothed time series with gaps from the previous exercise as input.
- Experiment with different imputation methods discussed in Chapter 5.
- Visualize the results using the provided `plot_production_data` function.
- Discuss the results.

Of course, we have only seen the tip of the iceberg when it comes to imputation methods for time series. Especially for seasonal data, more advanced methods such as STL can be very effective. We have had a look at STL in Chapter 6, where we decomposed a time series into its components. Now we will use it for imputation to not lose the seasonal structure of the data while filling the gaps.

Exercise 11.3 (Impute the missing values with STL)

- Use the smoothed time series with gaps from the first exercise as input.
- The STL needs data that is complete, i.e., without missing values. Therefore, first impute the missing values using a simple method of your choice (e.g., linear interpolation).
- Then, apply the STL decomposition to the imputed time series.
- Visualize the decomposed components (trend, seasonal, residual) using the provided `seasonal_decompose_seaborn` function.
- The seasonal part of the decomposition can now be used to deseasonalize the smoothed time series with gaps.
- Now that we have a deseasonalized time series with missing values, we can again impute the missing values using a simple method of your choice.
- Finally, reseasonalize the imputed deseasonalized time series by adding back the seasonal component obtained from the STL decomposition.
- Visualize the final imputed time series using the provided `plot_production_data` function.
- Discuss the results.

Great job, you have provided the graphs in time for the management report and the board liked the results. In the meanwhile your colleagues have found the original data set without gaps on some old dusty server and would like to know the number of outliers in the data set.

Exercise 11.4 (Count the number of outliers in the original data set)

- Use the original data set without gaps (`pieces` column) as input.
- Use an appropriate method to identify the outliers in the data set.
- Visualize the outliers and the original data set using `seaborn` .
- Count the number of detected outliers.
- Discuss the results.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

12 Self study - Session 3

In this session, we will revisit the chapters Chapter 7, Chapter 8, and Chapter 9.

We have seen that the algorithms for all three tasks more or less can be categorized into the same categories:

- Based on distances or metrics: E.g. trees and forests
- Based on distributions, kernels: E.g. Gaussian Mixture Models, SVMs
- Based on subsequences, shapelets: E.g. bags of words, shapelet-based methods
- Based on deep learning: E.g. recurrent neural networks, or treating the time series as images using CNNs
- Based on random convolutions: E.g. *Random Convolutional Kernel Transform* (ROCKET)
- Based on ensembles of the above.

We have implemented nearest neighbors (distance-based) for clustering, time series forests (distance-based) for classification, and convolutional neural networks for regression (deep learning-based).

Let us have a look at one additional algorithm for each task.

> **Note**
>
> You can use `sktime`'s `load_UCR_UEA_dataset` function to load the data sets mentioned below, e.g.

```
from sktime.datasets import load_UCR_UEA_dataset

ds_name = "UWaveGestureLibrary"
ret_type = "numpy3d"

X_train, y_train = load_UCR_UEA_dataset(
    name=ds_name,
    split="train",
    return_type=ret_type,
)

X_test, y_test = load_UCR_UEA_dataset(
    name=ds_name,
    split="test",
    return_type=ret_type,
)

# For classification, convert y to categorical
y_train = pd.Series(y_train).astype("category")
y_test = pd.Series(y_test).astype("category")
```

`sktime` is not the sole source of time series based algorithms. Besides `sktime`, there are numerous others, e.g. `aeon`.

> **Note**
>
> `aeon` is quite strict about dependencies. In case you have problems installing it, consider to initialize a new project and first install `aeon` before adding further packages.

Exercise 12.1 (Clustering of univariate time series)

- Load the `GesturePebbleZ1` data set. Note that it is a variable-length time series dataset and you need to either use `ret_type="df-list"` or `ret_type="nested_univ"` to load it (using the example above).
- Bring the data into a suitable format and explore the data set visually (using plots).
- Use the Time Series k means algorithm to cluster the data. Do you need to reshape the data for this algorithm?
- Use the KASBA Holder and Bagnall (2026) algorithm to cluster the data (e.g. implemented in `aeon`).
- Include useful (!) visualizations.
- How do the results compare?

After the exercise, you should be able to answer the following questions: - How does KASBA differ from the commonly used k means algorithm? - In you will find MSM defined. You should be able to illustrate how MSM is calculated.

In Chapter 8 and Chapter 9, our input was univariate, meaning each sample consisted of exactly one time series. Now, we will work with multivariate time series, which are more common in real-world scenarios since typically multiple sensors are available.

Exercise 12.2 (Classification of multivariate time series)

- Load the `UWaveGestureLibrary` data set.
- Use the time series k nearest neighbour algorithm to classify the time series.
- Use the CNN classifier to classify the time series.
- How do the results compare?

Note that not all of the algorithms have implementations for multivariate time series, e.g. `sktime`'s `TimeSeriesForestClassifier` only works for univariate time series (and raises an exception when used with a multivariate series). Luckily, there are other implementations available, e.g. `aeon` offers a `TimeSeriesForestClassifier` that is compatible with multivariate time series.

Exercise 12.3 (Classification using aeon's Time Series Forest Classifier)

- Load the `UWaveGestureLibrary` data set.
- Use `aeon`'s `TimeSeriesForestClassifier` to classify the data set.
- Are you able to achieve better results than with the previous algorithms?

Note

You can use `aeon`'s `load_regression` function to load the data set mentioned below, e.g.

```
from aeon.datasets import load_regression

ds_name = "AppliancesEnergy"

X_train, y_train = load_regression(ds_name, split="train")
X_test, y_test = load_regression(ds_name, split="test")
```

Exercise 12.4 (Regression of multivariate time series)

- Load the `AppliancesEnergy` data set.
- Use the time series k nearest neighbour algorithm to predict the energy consumption of the appliances.
- Use the time series random forest tree algorithm to predict the energy consumption of the appliances.
- Use the CNN regressor to predict the energy consumption of the appliances.
- How do the results compare?

Industrial Data Science - 2026 - CC BY-NC-SA 4.0

Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

13 Predictive Maintenance via Exploratory Data Analysis

Still under construction

This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

In this section, we will explore the concept of predictive maintenance using a synthetic dataset that reflects real predictive maintenance data encountered in industry.

You can download the dataset *AI4I 2020 Predictive Maintenance Dataset* (2020) at this link.

Here is a summary of the variables present in the dataset:

Variable Name	Role	Type	Units	Description	Target
UDI	ID	Integer		Unique identifier	
Product ID	ID	Categorical		Consists of Type and a variant-specific serial number	
Type	Feature	Categorical		Product quality variant given by letter L (low), M (medium), or H (high) variants	
Air temperature	Feature	Continuous	K		
Process temperature	Feature	Continuous	K		
Rotational speed	Feature	Integer	rpm		
Torque	Feature	Continuous	Nm		
Tool wear	Feature	Integer	min		
Machine failure	Target	Integer		Equals 1 if at least one of the following failures occurs	x
TWF	Target	Integer		Tool wear failure	x
HDF	Target	Integer		Heat dissipation failure	x
PWF	Target	Integer		Power failure	x
OSF	Target	Integer		Overstrain failure	x

Variable Name	Role	Type	Units	Description	Target
RNF	Target	Integer		Random failure	x

> **Note**
>
> Unfortunately the documentation neither provides details on the type of machine that is simulated nor the product it is thought to produce. For the sake of imagination, we will think of a very simple milling machine that produces parts in three quality variants: low, medium, and high.

The table shows various error types and our task is to gain insights on when they occur.

> **Note**
>
> This is a warmup task with a polished, synthetic dataset and luckily we are provided with a description of the nature of the different failures.

13.1 Notebook

13.1.1 Loading packages

```
import warnings  # used to ignore warnings

import matplotlib.pyplot as plt  # used for plotting
import numpy as np  # used for calculations
import pandas as pd  # used for data handling
import seaborn as sns  # used for plotting
import sklearn.tree as skt  # used for decision tree modeling

np.random.seed(42)  # for reproducibility


warnings.filterwarnings(
    "ignore"
)  # ignore warnings for cleaner output (seeing warnings is important, but for sake of a clean
```

13.1.2 Loading the dataset

The dataset is available online in the form of a zipped `csv` file.

```
df = pd.read_csv(
    "https://archive.ics.uci.edu/static/public/601/ai4i+2020+predictive+maintenance+dataset.zi
)
```

13.1.3 Quick check data integrity

It is always good practice to check the integrity of the dataset before performing any analysis. This includes checking for missing values, duplicate entries, ensuring that the data types are appropriate for each feature,

and looking at simple statistics.

```
df
```

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	0	0	0	0	0
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	0	0	0	0	0
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	0	0	0	0	0
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	0	0	0	0	0
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9995	9996	M24855	M	298.8	308.4	1604	29.5	14	0	0	0	0	0	0
9996	9997	H39410	H	298.9	308.4	1632	31.8	17	0	0	0	0	0	0
9997	9998	M24857	M	299.0	308.6	1645	33.4	22	0	0	0	0	0	0
9998	9999	H39412	H	299.0	308.7	1408	48.5	25	0	0	0	0	0	0
9999	10000	M24859	M	299.0	308.7	1500	40.2	30	0	0	0	0	0	0

10000 rows × 14 columns

Just displaying a couple of rows is useful for quickly inspecting the data structure and see if the dataset was correctly ex- and imported.

A few remarks:

- `UDI` appears to be some sort of linear step counter.
- Apparently, `Product ID` also seems to be a unique identifier.
- Interestingly, `Rotational speed` is represented as an integer (rpm), which is a discrete value, while `Torque` and `Tool wear` are continuous values.
- The target variables are binary indicators of failure types, represented as integers (0 or 1).
- The dataset contains 10.000 rows and 14 columns.

```
df.isna().sum()
```

```
UDI                        0
Product ID                 0
Type                       0
Air temperature [K]        0
Process temperature [K]    0
Rotational speed [rpm]     0
Torque [Nm]                0
Tool wear [min]            0
Machine failure            0
TWF                        0
HDF                        0
PWF                        0
```

```
OSF                      0
RNF                      0
dtype: int64
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

`isna` detects missing values in the dataframe and `duplicated` checks for duplicate entries. The dataset appears to be clean, with no missing values or duplicates detected.

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
UDI	10000.0	5000.50000	2886.895680	1.0	2500.75	5000.5	7500.25	10000.0
Air temperature [K]	10000.0	300.00493	2.000259	295.3	298.30	300.1	301.50	304.5
Process temperature [K]	10000.0	310.00556	1.483734	305.7	308.80	310.1	311.10	313.8
Rotational speed [rpm]	10000.0	1538.77610	179.284096	1168.0	1423.00	1503.0	1612.00	2886.0
Torque [Nm]	10000.0	39.98691	9.968934	3.8	33.20	40.1	46.80	76.6
Tool wear [min]	10000.0	107.95100	63.654147	0.0	53.00	108.0	162.00	253.0
Machine failure	10000.0	0.03390	0.180981	0.0	0.00	0.0	0.00	1.0
TWF	10000.0	0.00460	0.067671	0.0	0.00	0.0	0.00	1.0
HDF	10000.0	0.01150	0.106625	0.0	0.00	0.0	0.00	1.0
PWF	10000.0	0.00950	0.097009	0.0	0.00	0.0	0.00	1.0
OSF	10000.0	0.00980	0.098514	0.0	0.00	0.0	0.00	1.0
RNF	10000.0	0.00190	0.043550	0.0	0.00	0.0	0.00	1.0

`describe` provides a summary of the numerical features in the dataset, including count, mean, standard deviation, minimum, maximum, and quartiles.

There are no obvious outliers or anomalies in the data based on this summary.

```
df.dtypes
```

```
UDI                        int64
Product ID                object
Type                      object
Air temperature [K]      float64
Process temperature [K]  float64
Rotational speed [rpm]     int64
Torque [Nm]              float64
Tool wear [min]            int64
Machine failure            int64
TWF                        int64
HDF                        int64
PWF                        int64
```

```
OSF                        int64
RNF                        int64
dtype: object
```

`dtypes` provides the data types of each feature in the dataset.

`object` types are typically storing `text data` in `pandas 2.x`. Here are no additional surprises, as we already noticed the oddities previously.

```
df.nunique()
```

```
UDI                        10000
Product ID                 10000
Type                           3
Air temperature [K]           93
Process temperature [K]       82
Rotational speed [rpm]       941
Torque [Nm]                  577
Tool wear [min]              246
Machine failure                2
TWF                            2
HDF                            2
PWF                            2
OSF                            2
RNF                            2
dtype: int64
```

`nunique` counts the number of unique values in each feature.

We can clearly see that `UDI` is a unique identifier indeed, as it has a count equal to the number of rows in the dataset. Obviously `Product ID` is another unique identifier, which most likely does not add anything useful for the analysis. We also observe that `Type` indeed only contains `L`, `M`, and `H`. And last, we observe that all the error types are indeed binary (as expected).

For digging deeper, we now define two global lists for accessing continuous and error variables more easily throughout the notebook.

```
CONTINUOUS_VARS = [
    "Air temperature [K]",
    "Process temperature [K]",
    "Rotational speed [rpm]",
    "Torque [Nm]",
    "Tool wear [min]",
]
ERROR_VARS = ["Machine failure", "TWF", "HDF", "PWF", "OSF", "RNF"]
```

```
df[ERROR_VARS].sum()
```

```
Machine failure    339
TWF                 46
HDF                115
PWF                 95
```

```
OSF                 98
RNF                 19
dtype: int64
```

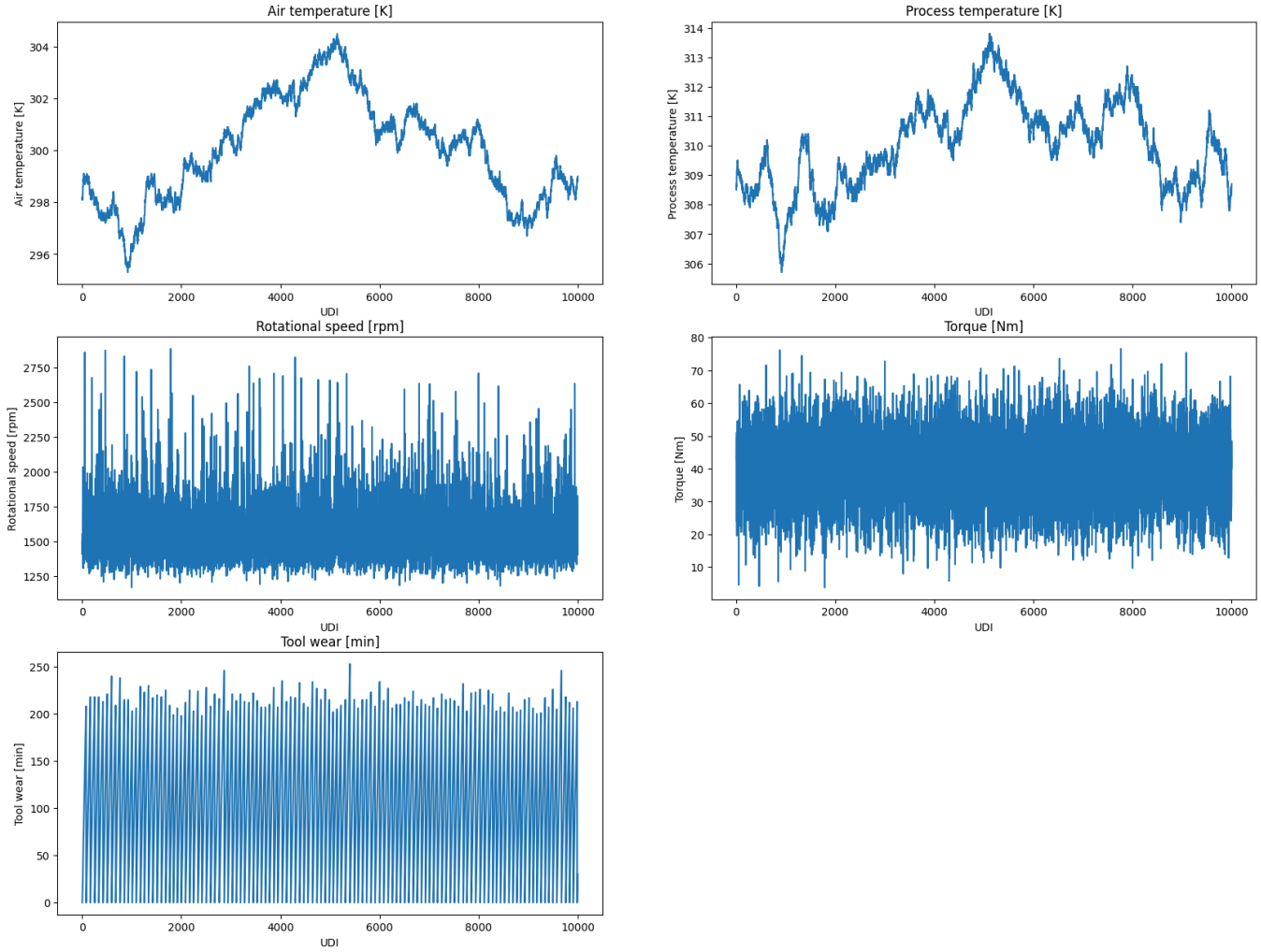
Out of 10.000 rows, we have 339 rows that correspond to machine failures. This corresponds to 3.39% of the dataset and is a clear indication of class imbalance.

Luckily we also have access to information of specific error types, which makes it easier to explore.

13.1.4 Exploring continuous variables by simple plots

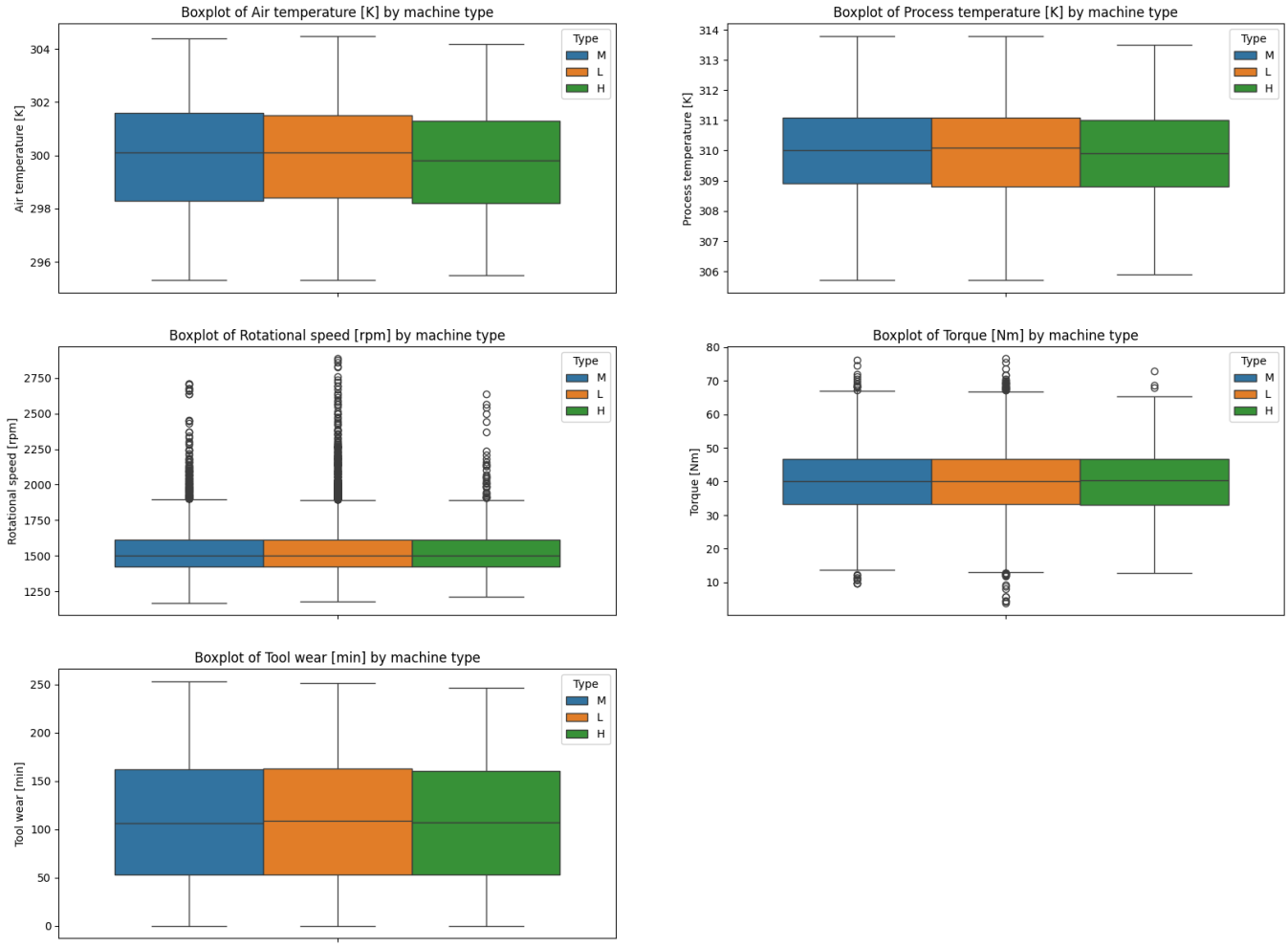
Here, we visualize the continuous variables (one by one) against the *Unique Device Identifier* (UDI) to get an idea of their temporal behavior.

```
plt.figure(figsize=(20, 15))

for i, col in enumerate(CONTINUOUS_VARS, 1):
    plt.subplot((len(CONTINUOUS_VARS) + 1) // 2, 2, i)
    sns.lineplot(data=df, x="UDI", y=col)
    plt.title(f"{col}")
```



Remember that we are working with a dataset that contains three process types? Let us explore, whether the distributions of the continuous variables differ across these process types.

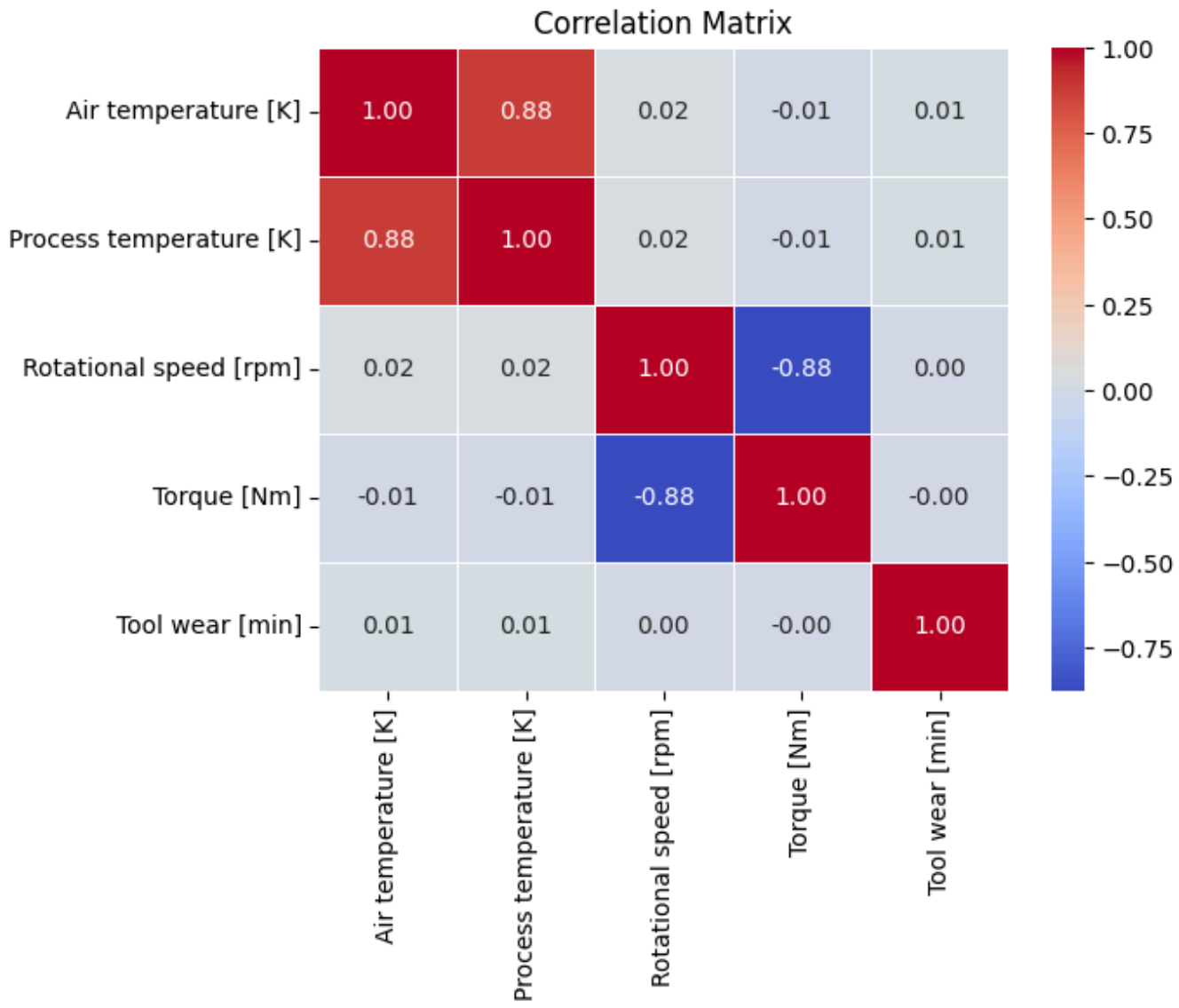
```
plt.figure(figsize=(20, 15))

for i, col in enumerate(CONTINUOUS_VARS, 1):
    plt.subplot((len(CONTINUOUS_VARS) + 1) // 2, 2, i)
    sns.boxplot(data=df, y=col, hue="Type")
    plt.title(f"Boxplot of {col} by machine type")
```



Apparently, there are only very little differences between the three process types.

Last, we will check for correlations between the variables.

```
correlation_matrix = df[CONTINUOUS_VARS].corr()
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Matrix");
```



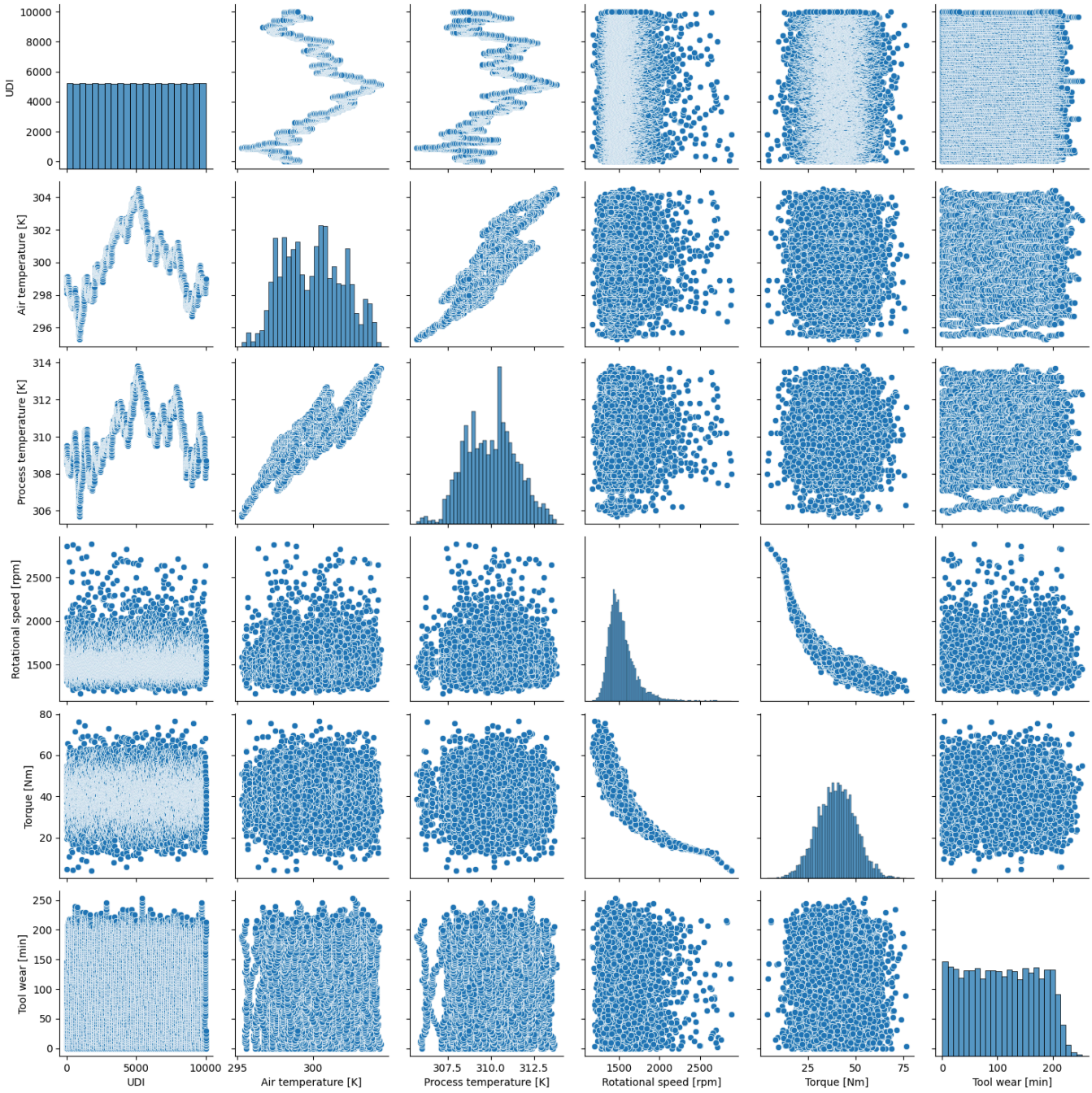
The correlation matrix reveals:

- Air temperature and process temperature are highly positively correlated, indicating that as one increases, the other tends to increase as well.
- Rotational speed and torque are negatively correlated, meaning that as rotational speed increases, torque tends to decrease and vice versa.

`seaborn` provides a convenient way to visualize important characteristics of the dataset using the `pairplot` function. The diagonal subplots show the distribution of each variable, while the off-diagonal subplots show the relationships between variables.

We included the `UDI` to have a temporal reference, allowing us to observe how the variables change over time (compare to the previous time series plots). We can also clearly see the correlating variables in this pairplots.

```
sns.pairplot(df, vars=["UDI"] + CONTINUOUS_VARS)
```



13.1.5 Feature Engineering

Sometimes, it is easy to create new, meaningful features from the existing ones. Often, process engineers have a deep understanding of the domain and can identify potential new features that may improve model performance. So talking is key!

We will engineer two new features:

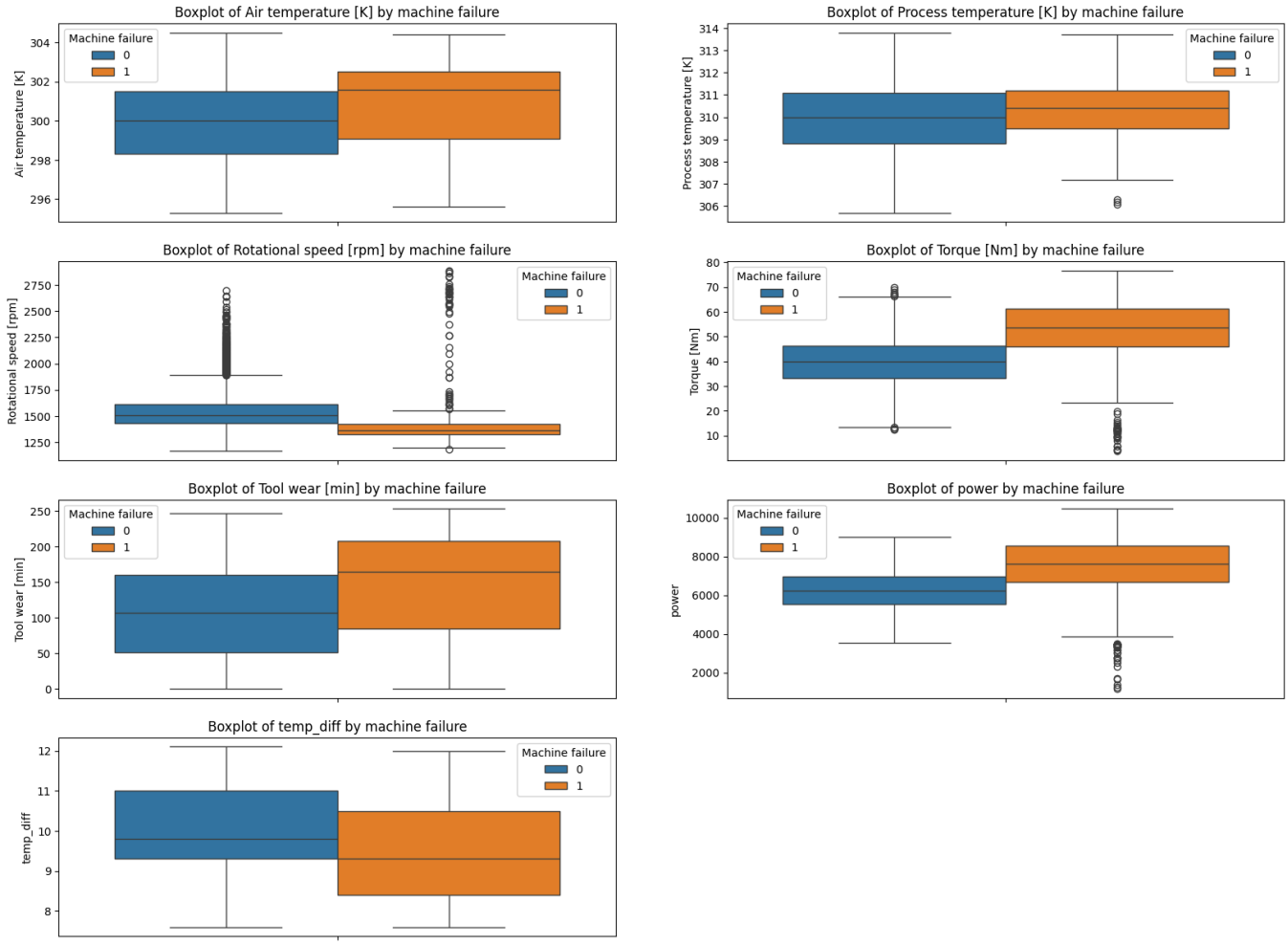
- **Temperature difference** between process and ambient temperature.
- **Power** is calculated as `Torque * Rotational Speed * 2 * pi`, where torque comes in Nm and rotational speed should be in revolutions per second.

```
df.loc[:, "temp_diff"] = df["Process temperature [K]"] - df["Air temperature [K]"]
df.loc[:, "power"] = df["Torque [Nm]"] * 2 * np.pi * (df["Rotational speed [rpm]"] / 60)
```

13.1.6 Investigating machine failures

It is easily possible to add another dimension to the visualizations. For example, let us compare the distribution of the continuous variables by machine failures.

```
CONTINUOUS_VARS_ENG = CONTINUOUS_VARS + ["power", "temp_diff"]

plt.figure(figsize=(20, 15))

for i, col in enumerate(CONTINUOUS_VARS_ENG, 1):
    plt.subplot((len(CONTINUOUS_VARS_ENG) + 1) // 2, 2, i)
    sns.boxplot(data=df, y=col, hue="Machine failure")
    plt.title(f"Boxplot of {col} by machine failure")
```



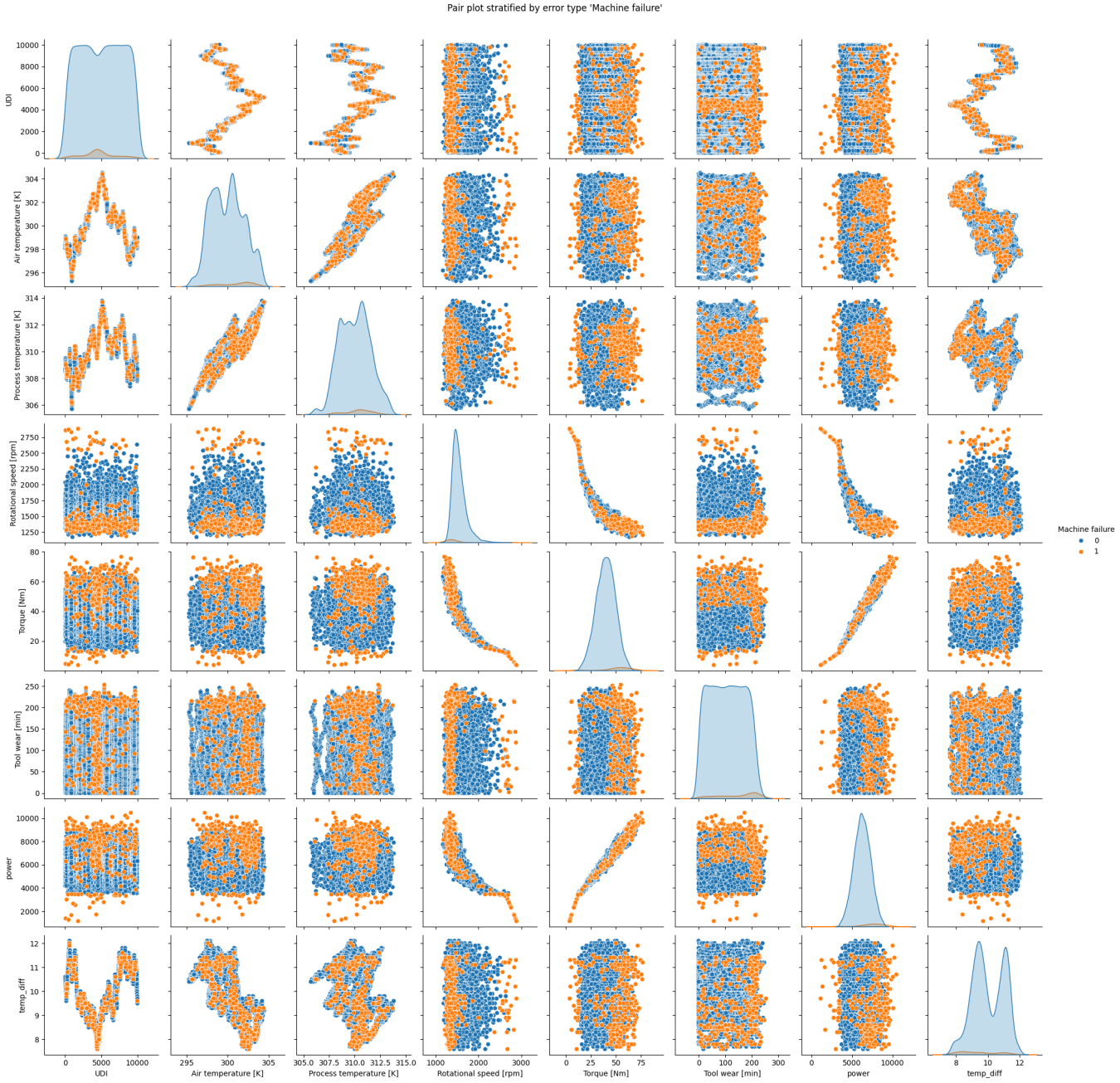
The boxplots reveal:

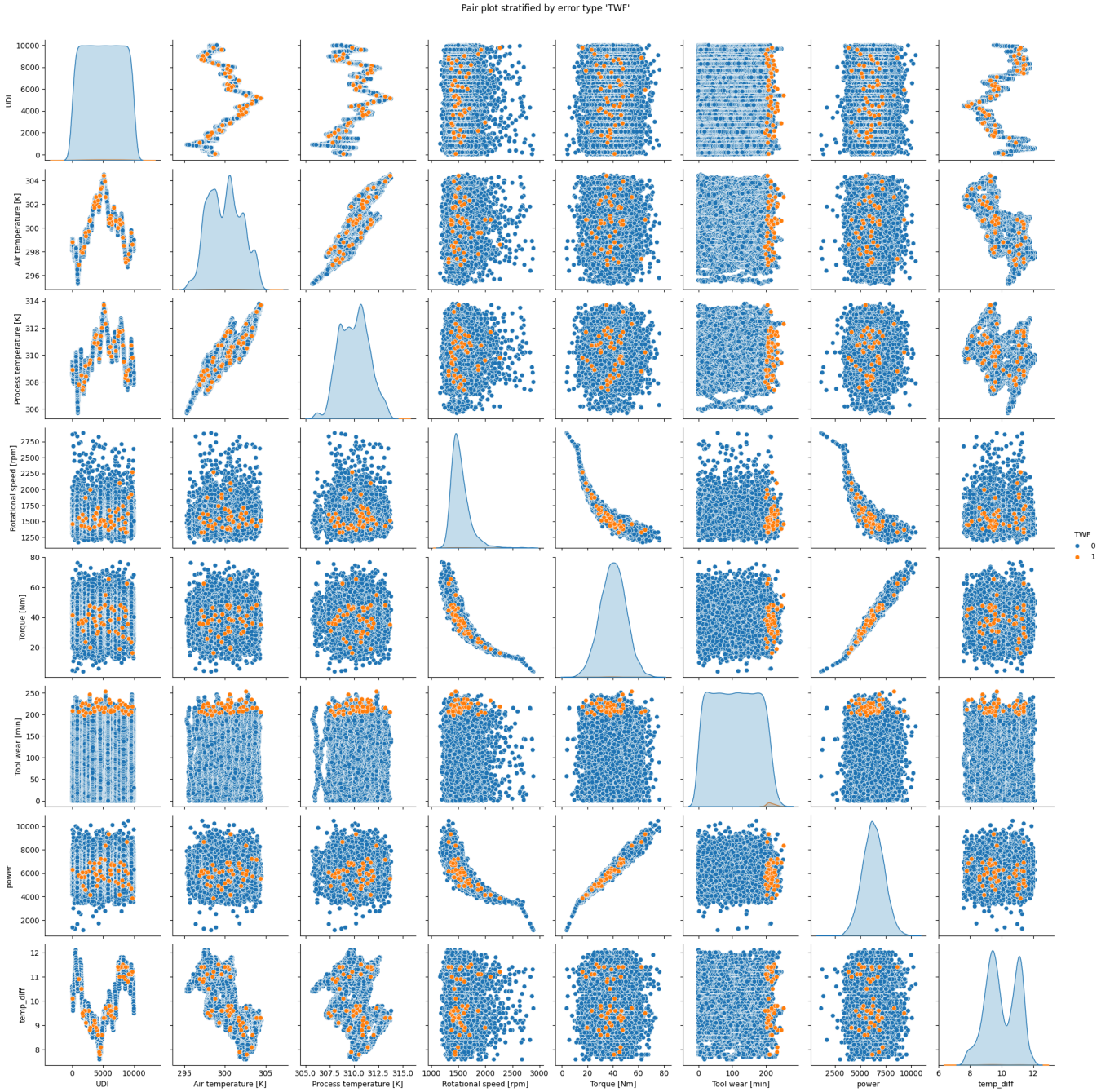
- As air temperature increases, the likelihood of machine failure also increases.
- High torque, high power and low rotational speed indicate a higher probability of machine failure.
- Higher tool wear comes with increased risk of machine failure.
- Smaller temperature difference between machine and ambient temperature increases the likelihood of failure.

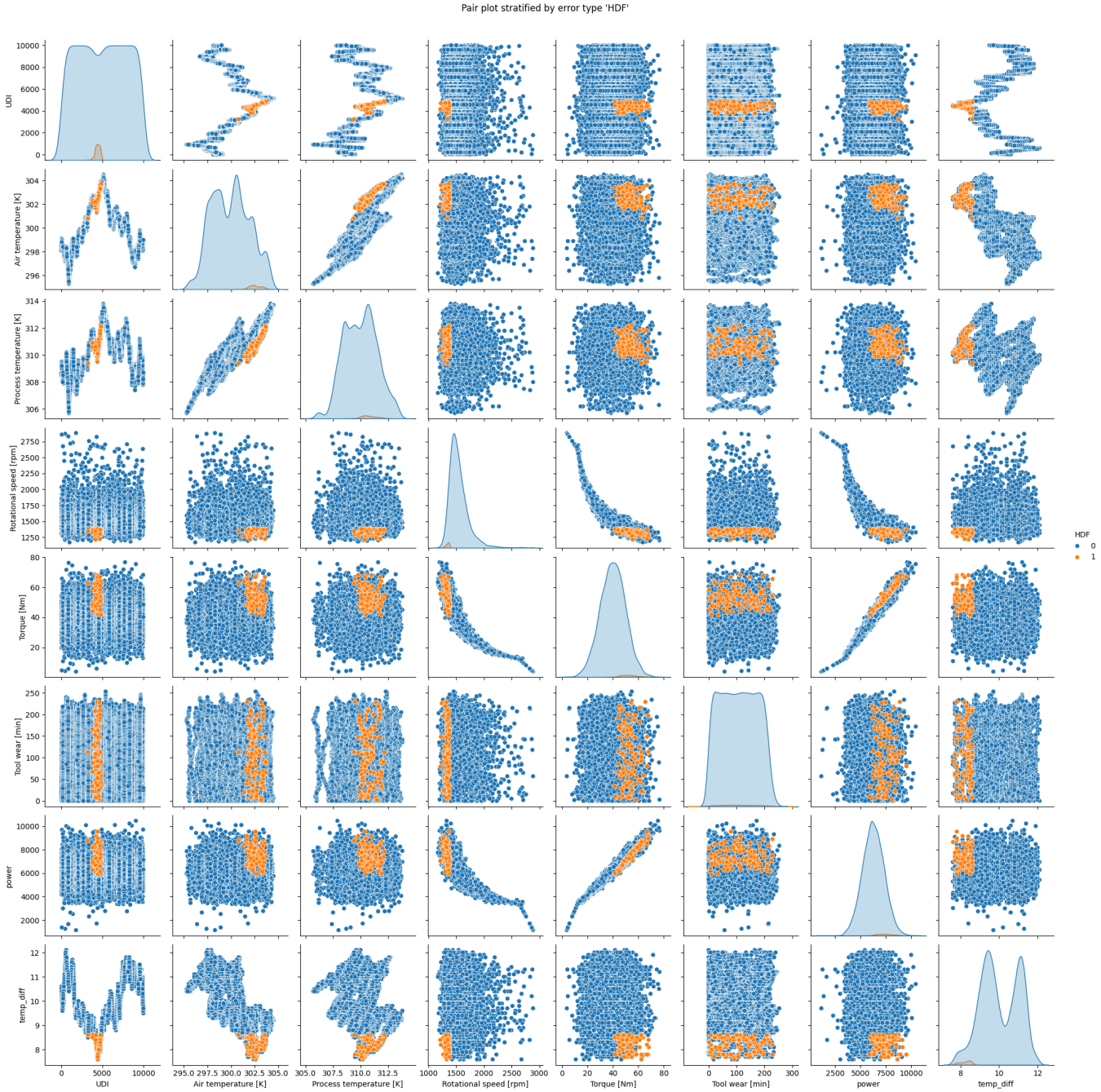
Luckily, we have more detailed failure information available. Next, we will look at every failure type separately and plot pairplots for each. Check the title of each plot and try to find patterns. Sorry for the plot overload and discussion follows below.

13.1.6.1 Pairplots

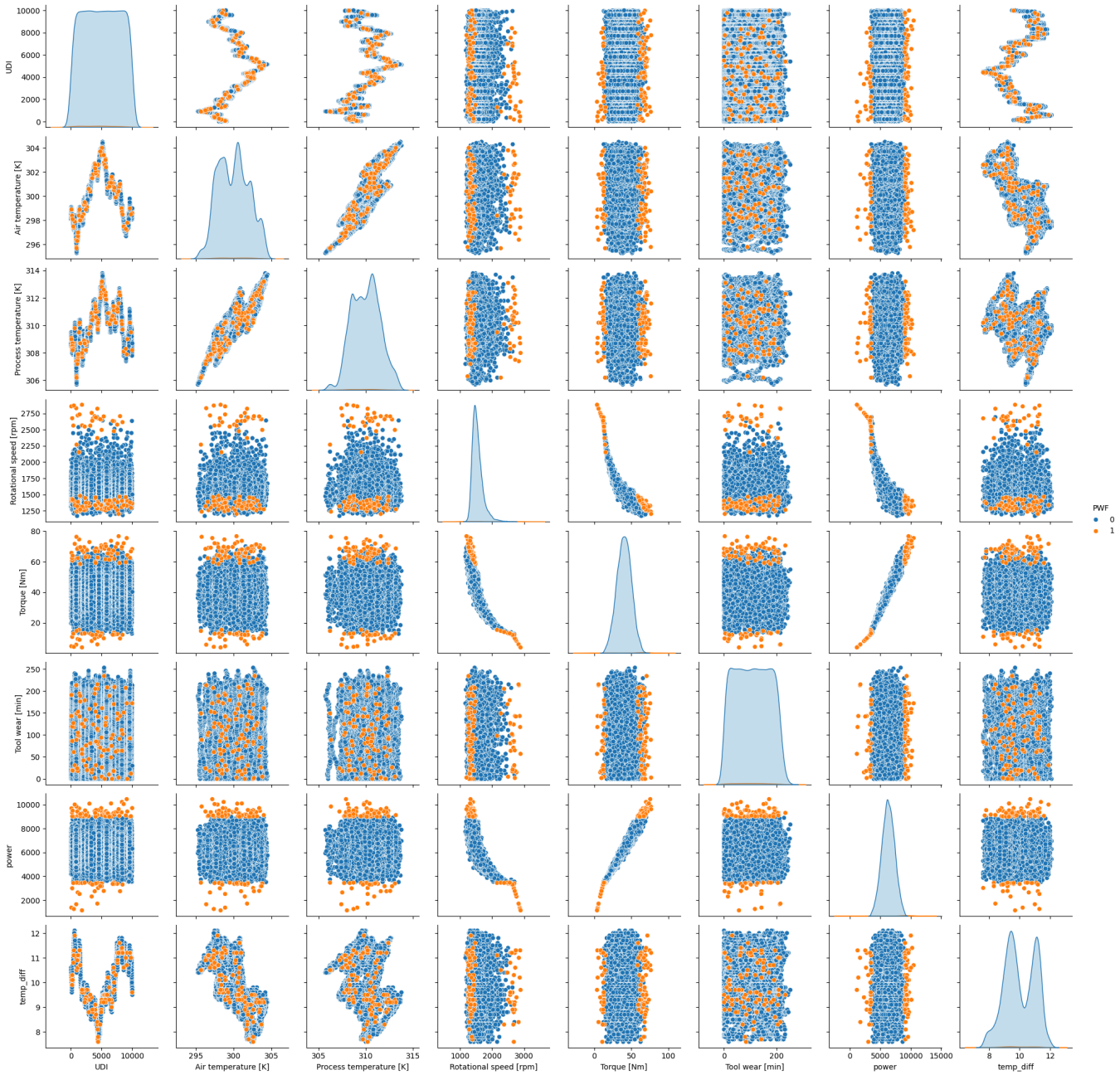
```
for col in ERROR_VARS:
    pp = sns.pairplot(df.sort_values(col), vars=["UDI"] + CONTINUOUS_VARS_ENG, hue=col)
    pp.figure.suptitle(f"Pair plot stratified by error type '{col}'", y=1.02)
    plt.show()
```

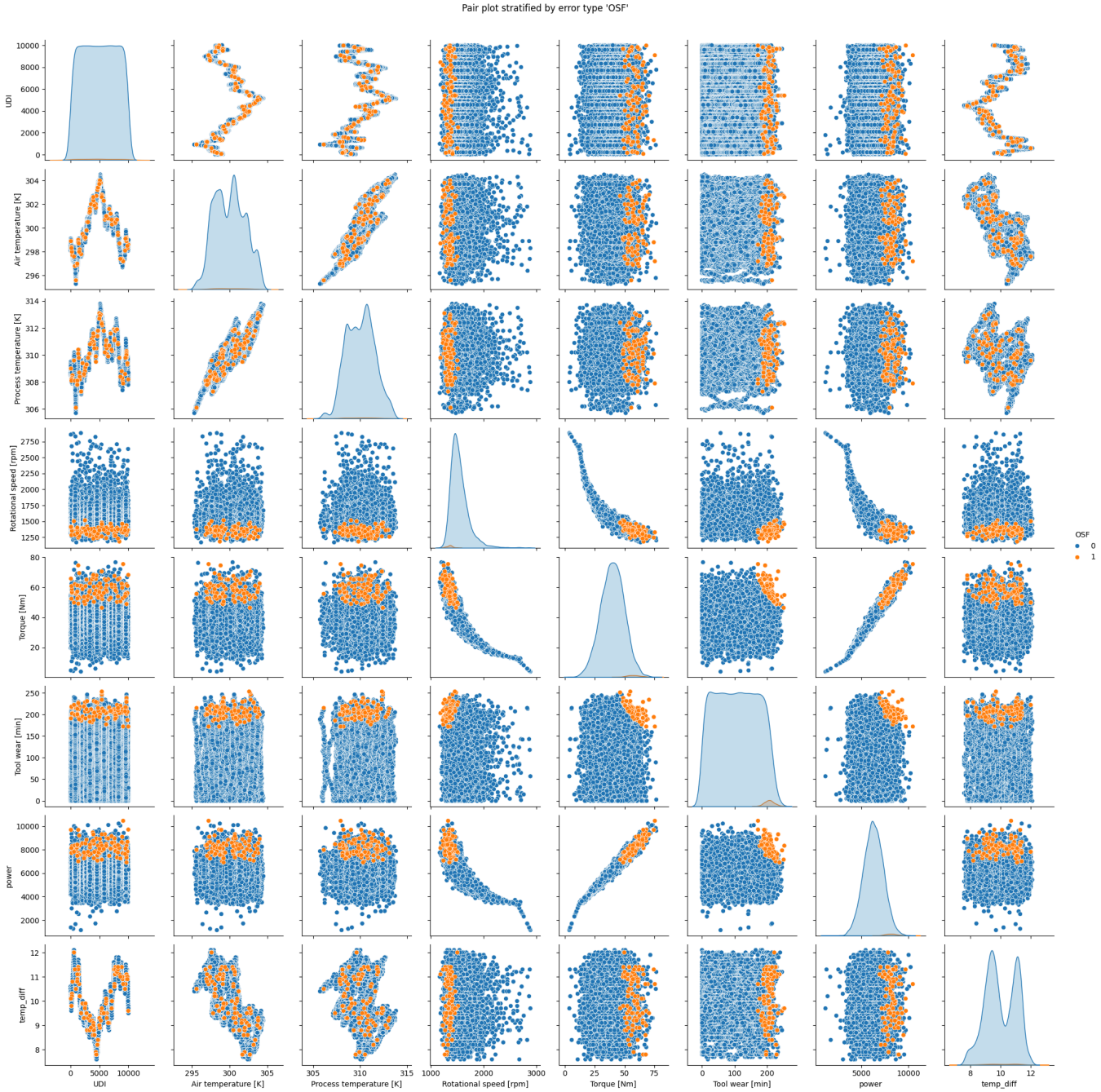


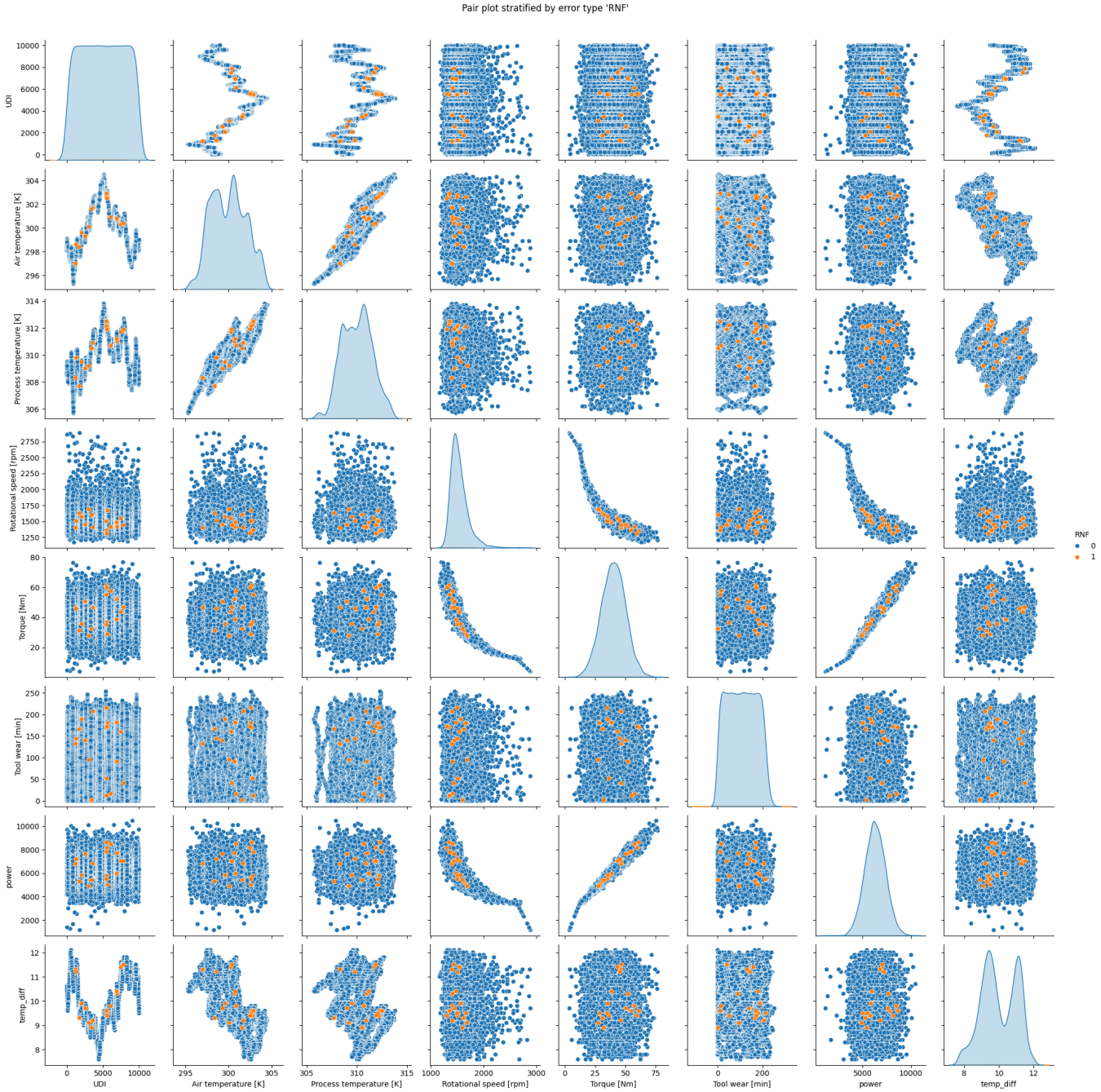

Pair plot stratified by error type 'TWF'
UDI
Air temperature [K]
Process temperature [K]
Rotational speed [rpm]
Torque [Nm]
Tool wear [min]
power
temp_diff
TWF
0
1


Pair plot stratified by error type 'HDF'
UDI
Air temperature [K]
Process temperature [K]
Rotational speed [rpm]
Torque [Nm]
Tool wear [min]
power
temp_diff
HDF
0
1

Pair plot stratified by error type 'PWF'




Pair plot stratified by error type 'OSF'
UDI
Air temperature [K]
Process temperature [K]
Rotational speed [rpm]
Torque [Nm]
Tool wear [min]
power
temp_diff
OSF
0
1



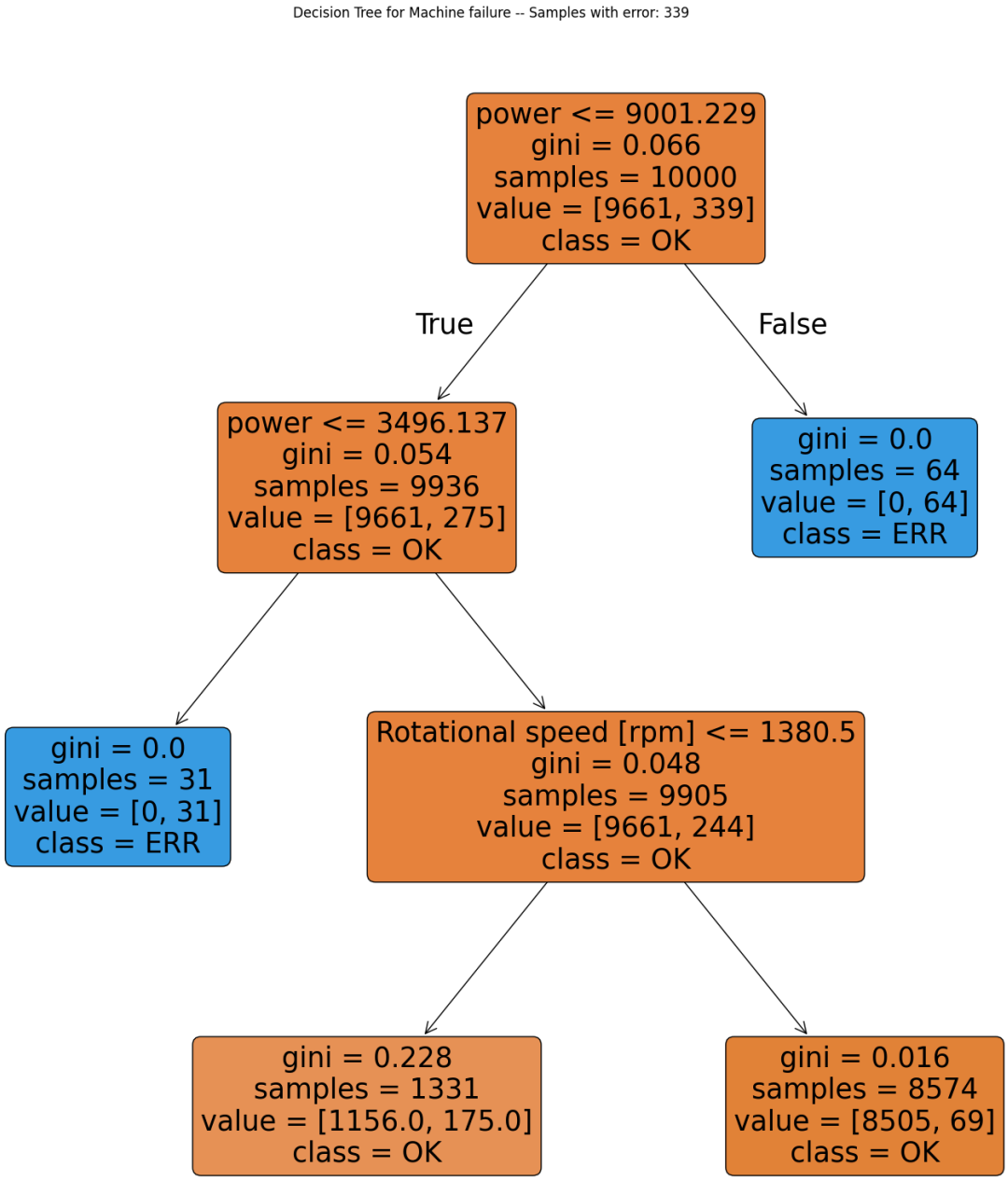
Interpretation

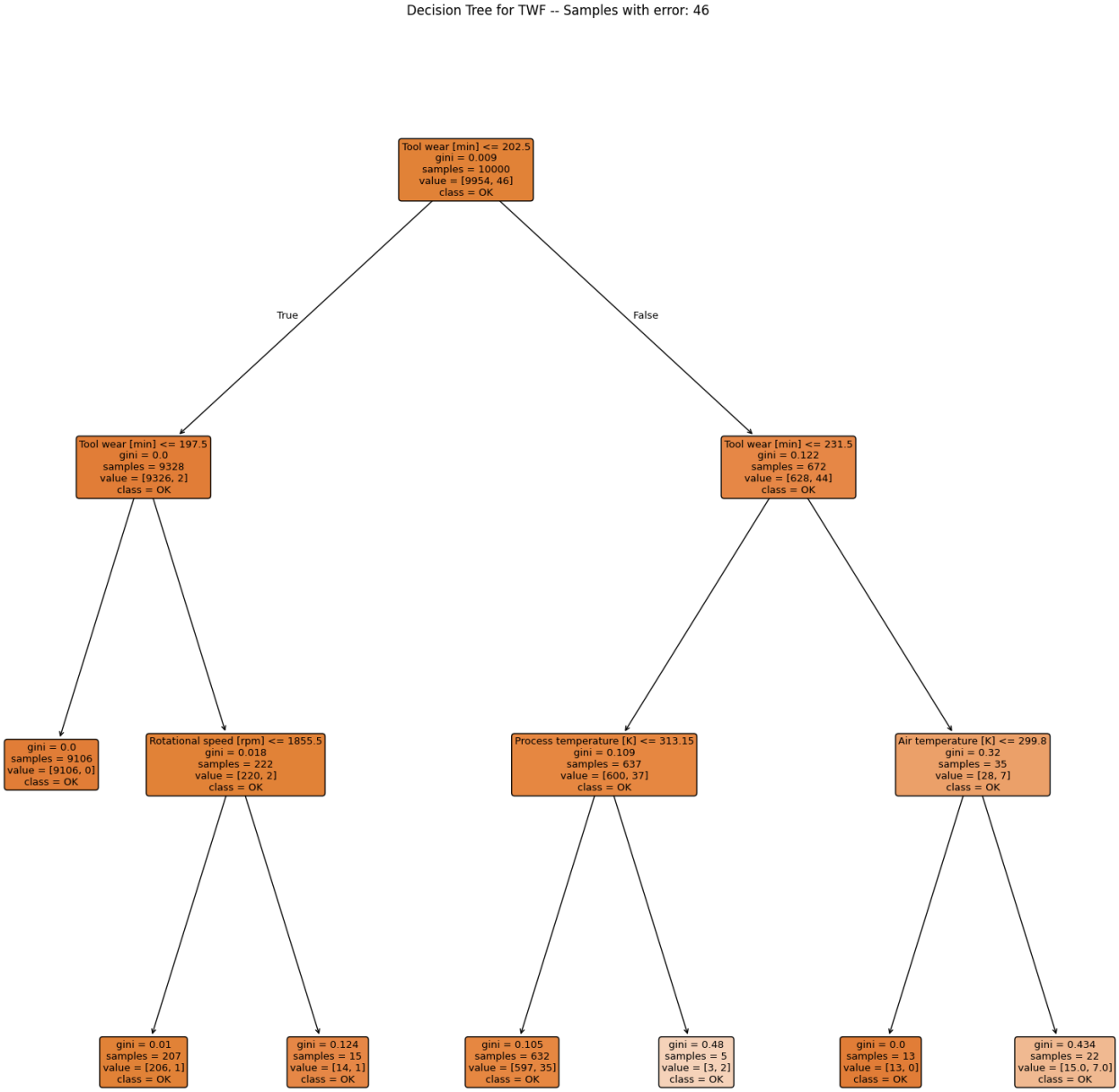
- The plot for `TWF` clearly indicates that the failure only appears for tool wear times exceeding 200 minutes.
- `HDF` occurs when rotational speed is small and difference between air temperature and machine temperature is low.
- `PWF` clearly occurs when power is smaller than ca. 4000 or higher than 9000.
- `OSF` is likely when torque and tool wear are elevated.
- `RNF` shows no clear pattern.

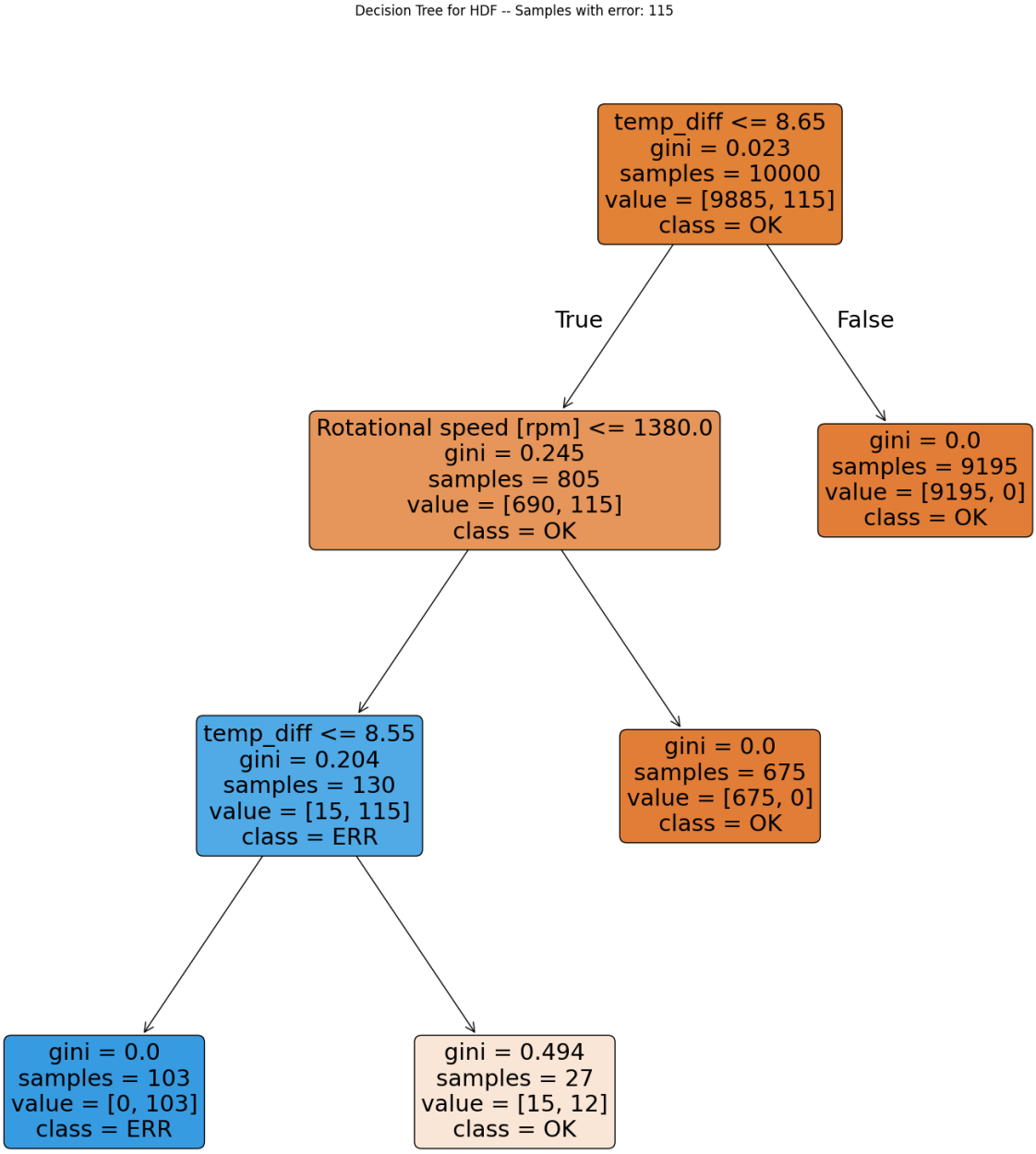
Alternatively, we could also use a simple explainable AI model to identify the key factors contributing to each failure type. Let us explore this approach with decision trees. Check the titles of the plots and interpret the results.

13.1.6.2 Decision Trees

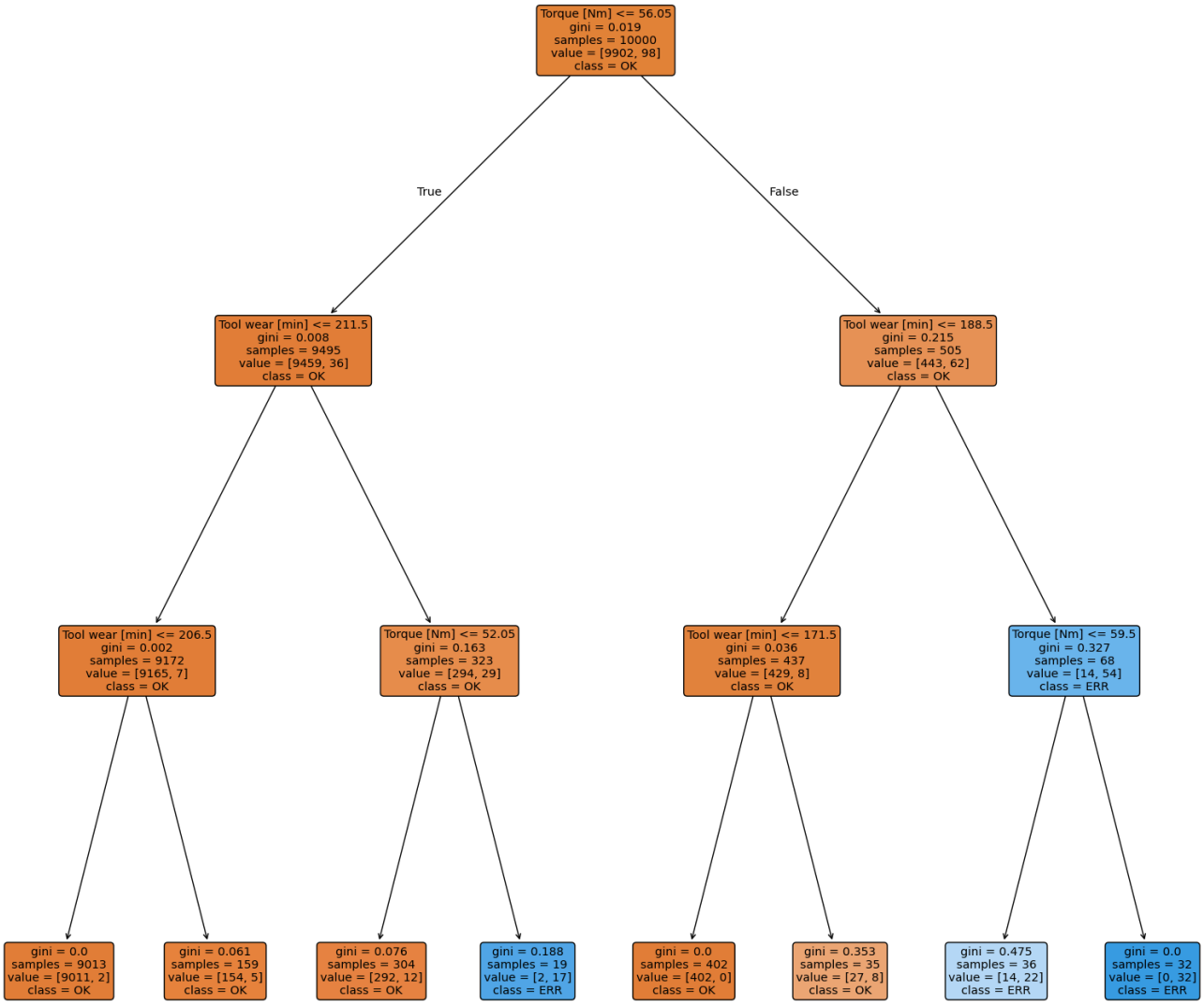
```
def plot_decision_tree(    df: pd.DataFrame,    target_col: str,    feature_cols: list[str] |
```

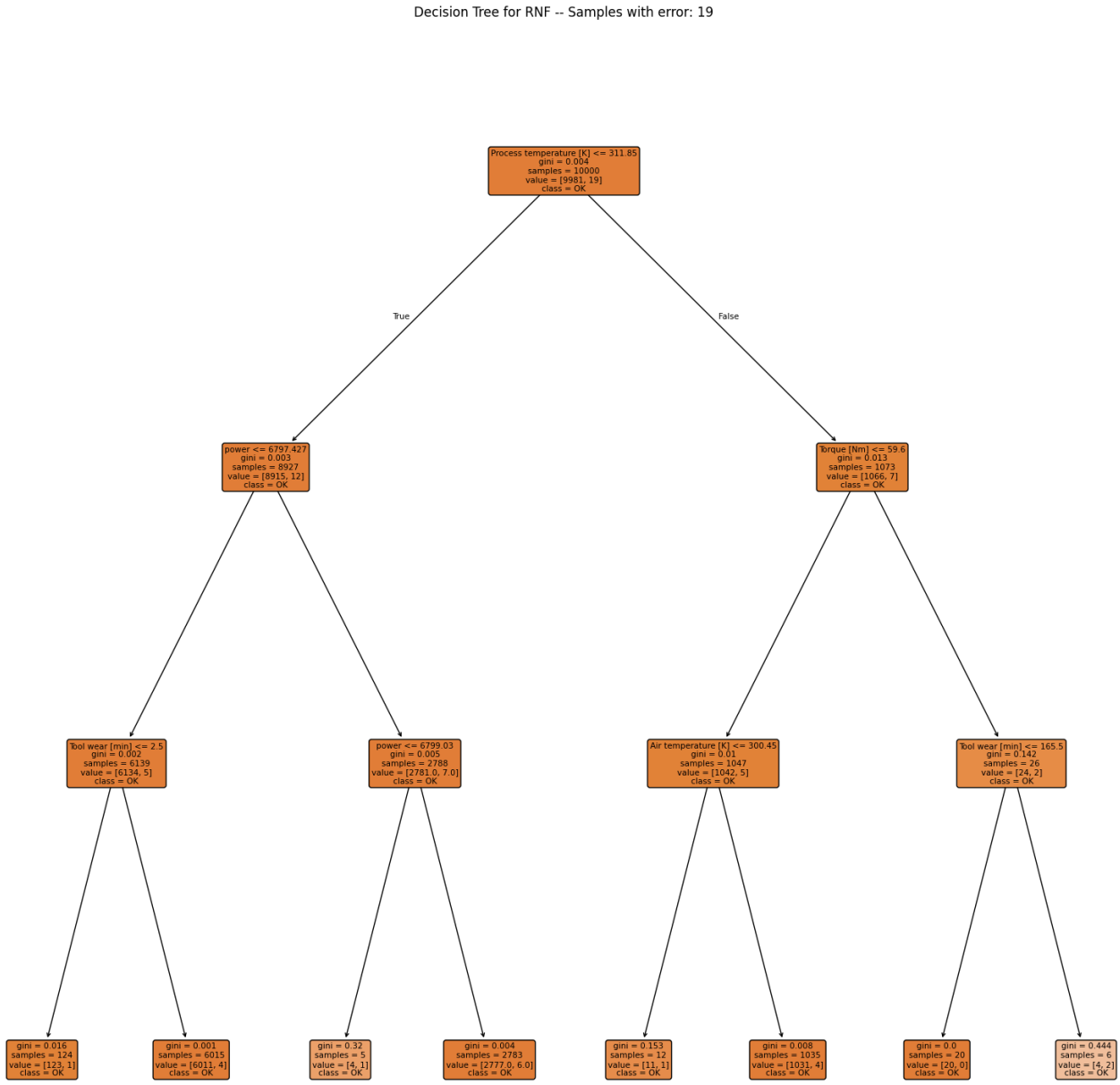



Decision Tree for TWF -- Samples with error: 46
Tool wear [min] <= 202.5
gini = 0.009
samples = 10000
value = [9954, 46]
class = OK
True
False
Tool wear [min] <= 197.5
gini = 0.0
samples = 9328
value = [9326, 2]
class = OK
Tool wear [min] <= 231.5
gini = 0.122
samples = 672
value = [628, 44]
class = OK
gini = 0.0
samples = 9106
value = [9106, 0]
class = OK
Rotational speed [rpm] <= 1855.5
gini = 0.018
samples = 222
value = [220, 2]
class = OK
Process temperature [K] <= 313.15
gini = 0.109
samples = 637
value = [600, 37]
class = OK
Air temperature [K] <= 299.8
gini = 0.32
samples = 35
value = [28, 7]
class = OK
gini = 0.01
samples = 207
value = [206, 1]
class = OK
gini = 0.124
samples = 15
value = [14, 1]
class = OK
gini = 0.105
samples = 632
value = [597, 35]
class = OK
gini = 0.48
samples = 5
value = [3, 2]
class = OK
gini = 0.0
samples = 13
value = [13, 0]
class = OK
gini = 0.434
samples = 22
value = [15.0, 7.0]
class = OK


Decision Tree for HDF -- Samples with error: 115
temp_diff <= 8.65
gini = 0.023
samples = 10000
value = [9885, 115]
class = OK
True
False
Rotational speed [rpm] <= 1380.0
gini = 0.245
samples = 805
value = [690, 115]
class = OK
gini = 0.0
samples = 9195
value = [9195, 0]
class = OK
temp_diff <= 8.55
gini = 0.204
samples = 130
value = [15, 115]
class = ERR
gini = 0.0
samples = 675
value = [675, 0]
class = OK
gini = 0.0
samples = 103
value = [0, 103]
class = ERR
gini = 0.494
samples = 27
value = [15, 12]
class = OK

Decision Tree for OSF -- Samples with error: 98





Interpretation

- `TWF` : Although the classification performs poorly, the first split is made along the tool wear time (value 202.5). For smaller values the process only fails in 2 out of 9328 cases, so almost all of the failures occur with tool wear time above 202.5 min.
- `HDF` : Clearly shows that the failure usually occurs when temperature difference is below 8.65 and rotational speed is slower than 1380 rpm.
- `PWF` : Occurs when power exceeds 9001 W or is lower than 3496 W.
- `OSF` : The decision tree alternately looks at tool wear and torque but is not able to find a good model with a depth of 3. Note that the decision tree is only able to make horizontal or vertical cuts and is therefor sensitive to rotations of the given data. This indicates, that tool wear and torque are the main drivers of the `OSF` machine failure type but a simple decision tree is not able to model this relationship.
- `RNF` : The model completely fails and shows no pattern.

Please note that the conclusions are quite similar to the ones we made on base of the pairplots.

13.1.7 The great reveal

The documentation of the dataset also describes the error types as follows:

- Tool wear failure (`TWF`): The tool will be replaced of fail at a randomly selected tool wear time between 200 and 240 mins (120 times in our dataset). At this point in time, the tool is replaced 69 times, and fails 51 times (randomly assigned).
- Heat dissipation failure (`HDF`): Heat dissipation causes a process failure, if the difference between air- and process temperature is below 8.6 K and the tool's rotational speed is below 1380 rpm. This is the case for 115 data points.
- Power failure (`PWF`): The product of torque and rotational speed (in rad/s) equals the power required for the process. If this power is below 3500 W or above 9000 W, the process fails, which is the case 95 times in our dataset.
- Overstrain failure (`OSF`): If the product of tool wear and torque exceeds 11.000 minNm for the L product variant (12.000 for M, 13.000 for H), the process fails due to overstrain. This is true for 98 datapoints.
- Random failures (`RNF`): Each process has a chance of 0.1 % to fail regardless of its process parameters. This is the case for only 5 datapoints, less than could be expected for 10,000 datapoints in our dataset.

13.1.8 Conclusion

Many roads lead to Rome. We successfully recovered most of the key factors for failure from the dataset, using clever visualizations and making use of an interpretable machine learning algorithm.

These insights can help improving the reliability of the processes and reduce the likelihood of failures in the future:

- The tool wear failure could be mitigated by implementing more frequent tool replacements.
- The heat dissipation failure could be addressed by not allowing rotational speeds below 1380rpm.
- The power failure could be prevented by ensuring that the product of torque and rotational speed stays within the specified limits.
- The overstrain failure could be avoided by monitoring the tool wear and torque closely.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

14 Self study - Session 1

> **Still under construction**
>
> This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

In this session, we will explore the dataset in Chapter 13 further.

When comparing our findings in Section 13.1.6.1 and Section 13.1.6.2 with the failure description in Section 13.1.7, we can see that we did a pretty good job in identifying failure reasons. However, we could have done better for `OSF`.

> **Exercise 14.1 (Overstrain failure)**
>
> According to Section 13.1.7, the failure type *overstrain failure* (`OSF`) occurs when the machine is subjected to excessive torque when a worn-out tool is used. Also the product variant is associated to the failure rate: When the product of torque and tool wear exceeds 11.000 minNm for the L product variant (12.000 for M, 13.000 for H), the process fails due to overstrain.
>
> Are you able to verify this relationship by visualizations?

Also, it seems that the description of `TWF` in Section 13.1.7 is contradicting our observations.

> **Exercise 14.2 (Tool wear failure)**
>
> Identify where the description of `TWF` in Section 13.1.7 does not align with your findings.

So far we have used `seaborn` to visualize the data. `seaborn` is based on `matplotlib` and provides a high-level interface for drawing statistical graphics. While `matplotlib` supports interactive figures that can zoom, pan and update (see here), for richer interactive visualizations we can use `plotly`. `plotly` is open-source and built on top of the Plotly JavaScript library `plotly.js`. It enables users to create interactive web-based visualizations that can be displayed in `Jupyter` notebooks as well as in standalone HTML files. Especially for explorative data analysis, `plotly` offers a more user-friendly and flexible approach to create and explore complex visualizations.

> **Exercise 14.3 (Introducing Plotly)**
>
> Consult the Plotly documentation for more information on how to use the library. Start with the notebook from Chapter 10, recreate the visualizations from Chapter 13 using `plotly`, and explore the data interactively.

In Section 13.1.6, we used pair plots, boxplots, and decision trees to investigate the relationship between different features and the machine failure types. Another approach could be to use parallel coordinates plots.

Exercise 14.4 (Parallel coordinates plots)

Use parallel coordinates plots to visualize the relationships between multiple features and the machine failure types:

1. Display every available continuous variable along with one selected failure type.
2. Also add `Type` as a categorical variable to the plot.
3. Make the failure type categorical.
4. Color the lines according to the selected failure type.
5. Allow for choosing the failure type interactively using a Dropdown widget (see Jupyter widgets) with the available failure types.
6. The parallel coordinate plot allows for dragging columns (just click and drag the title of a column) and reordering them. It also allows for selecting and highlighting specific lines, making it easier to focus on particular data points. Use these two features to get insights into the relationships between different features and the selected failure type. Can you identify the patterns which we have exploited in our previous analyses?

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

15 Self study - Session 2

Still under construction

This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

In this session, we will revisit the dataset from Chapter 13, the **Explainable AI (XAI) Drilling Dataset** by Wallsberger et al. (2023). We will explore it in depth, splitting our analysis into four milestones, each building on the previous one.

15.1 Milestone 1: Data Acquisition and Understanding

Exercise 15.1 (Register and download the dataset)

1. Register on Kaggle.
2. Read the description of the **Explainable AI (XAI) Drilling Dataset**.
3. Download the dataset.
4. Load the dataset into your environment and inspect its structure: number of rows, columns, data types, and any obvious issues.
5. Identify the target variable(s) and any class imbalances.

15.2 Milestone 2: Exploratory Data Analysis

Exercise 15.2 (Explore the dataset)

1. Pretend that you did not read the *Subgroup failures* paragraph in the description. Explore the dataset and find insights related to the different failure types.
2. Do your findings align with the description given in the *Subgroup failures* paragraph?
3. Compute summary statistics for each feature grouped by failure type.
4. Create at least two visualizations (e.g., histograms, box plots) that reveal differences between failure types.

15.3 Milestone 3: Deep Dive Visualization

Exercise 15.3 (Visualize failure patterns with plotly)

1. Recreate the key visualizations from milestone 2 using `plotly` for interactivity.
2. Create a parallel coordinates plot to explore relationships between multiple features and failure types.
3. Use dropdown widgets to filter the plot by failure type and observe how feature relationships change.
4. Document any new patterns or anomalies discovered through the interactive exploration.

15.4 Milestone 4: Reporting Findings

Exercise 15.4 (Generate a polished report with Quarto)

1. Polish your document: keep relevant insights, remove unnecessary details, focus on key findings.
2. Document with markdown cells and export the result as HTML or PDF (since your management most likely will not have JupyterLab installed).
   - PDF: The document will be a static version of your notebook, so make sure to include all relevant information and insights directly in the document.
   - HTML: The document will also be a static version of your notebook (the Dropdown would not work e.g.), but the interactivity of the `plotly` plots will still be available, since the HTML export retains the JavaScript functionality.
3. Use Quarto to generate a more polished report. Export your report as a PDF and HTML document. Use this documentation as a starting point.
4. Reflect on the entire analysis: What were the key findings? What surprised you? What would you investigate next?

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

16 Self study - Session 3

> **Still under construction**
>
> This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

In this session, we will have a look at clustering and anomaly detection applied to the AI4I predictive maintenance dataset from Chapter 13.

So far we have focused on supervised learning (classification and regression) for predicting failures. However, in many real-world industrial scenarios, labeled failure data is scarce or expensive to obtain. Unsupervised methods such as clustering and anomaly detection can help discover hidden patterns and identify unusual behavior without prior labels.

> **Note**
>
> You can load the AI4I dataset using the link provided in Chapter 13. The dataset contains features such as air temperature, process temperature, rotational speed, torque, and tool wear, along with multiple failure type labels.

16.1 Exercise A: Clustering the AI4I dataset

> **Exercise 16.1 (k-Means clustering on machine data)**
>
> 1. Load the AI4I dataset and select the continuous sensor features (`Air temperature`, `Process temperature`, `Rotational speed`, `Torque`, `Tool wear`).
> 2. Standardize the features using `StandardScaler` (important for distance-based methods).
> 3. Apply k-Means clustering with different values of k (e.g., 2 to 8).
> 4. Use the elbow method and silhouette scores to determine an appropriate k.
> 5. Visualize the clusters using pair plots or a scatter plot of the first two principal components (PCA).
> 6. Compare the resulting clusters with the known failure types. Do the clusters align with any specific failure?

16.2 Exercise B: DBSCAN and density-based clustering

> **Exercise 16.2 (Finding dense regions with DBSCAN)**
>
> 1. Apply DBSCAN to the standardized AI4I features. Experiment with different `eps` and `min_samples` values.
> 2. DBSCAN can identify outliers as noise points. How many data points are classified as noise?
> 3. Visualize the DBSCAN results alongside the k-Means results from the previous exercise.
> 4. Discuss: Which algorithm better separates the different failure types? What are the limitations of each approach?

16.3 Exercise C: Anomaly detection

Exercise 16.3 (Detecting anomalous machine behavior)

1. Train an isolation forest or one-class SVM on the AI4I data (you can use only the “normal” or non-failure samples as training data).
2. Predict anomalies on the full dataset and visualize the results.
3. Compare the detected anomalies with the known failure labels. Do anomalies correspond to specific failure types?
4. What threshold would you set for an early warning system in a real manufacturing environment?

16.4 Exercise D: Reflection and reporting

Exercise 16.4 (Synthesizing unsupervised insights)

1. Summarize your findings from all three exercises. Which clustering or anomaly detection approach performed best?
2. How could unsupervised methods complement the supervised approaches we studied earlier in Chapter 13?
3. In a real-world predictive maintenance scenario, what would be the practical value of these unsupervised techniques?
4. Document your reflections in a short report (using Quarto or a Jupyter notebook export).

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

17 Challenges

> **Still under construction**
>
> This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

Real machine data is often messy and requires substantial preprocessing before it can be used for analysis. Challenges from a data engineering perspective that often arise include:

- (Temporary) sensor failures
- Data gaps due to communication issues or buffer overflows
- Incomplete data
- Datatype inconsistencies
- Lack of documentation
- High data volume

Once data is collected and transformed, additional challenges may arise during analysis:

- Noisy or inconsistent measurements
- Outliers or anomalies in the data
- Variability in operating conditions
- There may be variables that are not captured in the data but still relevant for the process
- Complex relationships between variables
- Lack of labeled data for supervised learning
- Highly imbalanced datasets
- Data privacy concerns

Until now, we have primarily focused on static analysis of historical data.

However, many industrial applications require real-time data processing and analysis to enable timely decision-making and process optimization. Complexity increases significantly in real-time scenarios due to the need for low-latency processing, the handling of streaming data, and possibly the integration of data from multiple sources.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0

Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

18 Levels of system integration

> **Still under construction**
>
> This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

Level	Access & Action	Description
1. Historical Analysis (Passive Insight)	Data only – no machine or data stream access	Work from static CSVs or logs, providing insights, trends, or recommendations based solely on historical data.
2. Monitoring & Warning (Observational)	Real-time data stream, no control	Ingest live sensor or data feeds; support threshold alerts and anomaly detection but cannot affect machine behavior.
3. Feedback Loop (Latent Influence)	Network access; non-real-time control	Read from and write to systems remotely; can propose settings or adjustments (e.g., batch recalibration) but not apply instantly.
4. Real-Time Autonomous Control (Fully Integrated)	Full local access and control	Compute and apply decisions in real time. Includes online feature processing, low-latency inference, even continual learning.

18.1 Historical Analysis

The most basic level of system integration involves working with historical data that has been collected and stored over time. It may provide insights, trends, or recommendations based solely on this data, but it lacks the ability to interact with live systems or influence real-time operations.

E.g. analyzing historical production data to identify trends in equipment failures and recommending maintenance schedules.

18.2 Monitoring & Warning

The second level of system integration focuses on monitoring live data streams. Data science products at this level can analyze real-time data to identify potential issues or deviations from expected behavior, supporting operators in maintaining system performance and reliability.

E.g. detecting anomalies in the machine's behaviour as it runs and alerting operators to investigate potential system failures.

18.3 Feedback Loop

The third level of system integration introduces a feedback loop, allowing for more dynamic interactions with the system. It is not possible to achieve real-time control or immediate adjustments, but it is possible to adjust machine settings or parameters based on insights gained from the data.

E.g. adjust machine parameters such as pressure, temperature, flow-rate for the next production batch based on an analysis of failures in post-inspection of previous batches.

18.4 Real-Time Autonomous Control

The fourth level of system integration enables real-time autonomous control of machines and processes. At this level, data science products can make decisions and take actions in real time, allowing for immediate adjustments and optimizations.

E.g. automatically adjusting machine settings in real time based on live data inputs to optimize production efficiency.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

19 Batch vs Stream Processing

Still under construction

This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

	Batch Processing	Stream Processing
Definition	Processes datasets in periodic chunks	Continuously processes data arriving in a stream
Latency	High	Low
Context	Full data available; supports complex operations	Limited view (single event or window)
Typical Use	Historical analyses, model training, reporting, ETL	Real-time monitoring, anomaly detection, decision-making
ETL	Collects data over a given period, performs transformations on the entire dataset, and loads it into a target system, such as a data warehouse, all at once.	Continuously ingests and processes data as it arrives, applies transformations on the fly, and loads the processed data into a target system incrementally.

The time to insight is the most critical differentiator between batch and stream processing. Batch processing is best suited for less time-sensitive tasks, such as end-of-day reports, historical data analysis, or model training. In contrast, stream processing is designed for scenarios where immediate insights and actions are crucial, such as real-time optimization of production processes.

Batch processing tends to be less complex to manage because it operates on static datasets and follows a defined schedule, making it easier to plan and allocate resources. It is well-suited for scenarios where data consistency and completeness are more important than immediacy.

Stream processing, on the other hand, requires handling continuous data flows, which can introduce additional complexity in terms of system architecture, data consistency, and fault tolerance. Addressing these challenges necessitates a higher level of skill and expertise and might need specialized IT infrastructure.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0

Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

20 Process optimization on sensor data via EDA

> **Still under construction**
>
> This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

In this section, we will have a look at a more realistic example of industrial data.

20.1 Setting the stage

Imagine a manufacturing chain which is dedicated to producing knives with wooden handles. Various machines are interconnected and the process flow is as follows:

- Machine A prepares the steel blade and shaft.
- Machine B applies epoxy to the grip area.
- Machine C inserts the wooden handle material.
- Machine D is a curing oven, hardening the adhesive.
- Machine E coats the knife with protective finish and completes the manufacturing process.

Operators notice that the process is not running as smoothly as expected. The handle is not always properly bonded to the metal piece and sometimes the wood is cracked. They find that the root cause of the issue lies in the epoxy application step of Machine B. Common issues include excessive or insufficient epoxy application, uneven distribution, and occasional poor adhesion to the shaft. One clear economic impact is that these defective products cannot be sold. But there is more to it:

- Waste of material is not sustainable.
- The defect actually already happens in Machine B, but Machines C to E still process the faulty products, leading to further waste and costs, especially since the wooden handle is the most expensive part of the product and the curing oven is very energy-intensive.

Since there is a new department focused on data science, they decide to manually inspect the epoxy application right after Machine B and label the data accordingly. Additionally the machine collects some sensor data in a CSV file. This dataset is given to the data science team and in tight collaboration with the operators the ambitious goal is to:

- Optimize the epoxy application process to reduce defects and waste.
- Human inspection is time-consuming and expensive. Find a way to predict defects occuring in Machine B before the product is handed over to machine C.

Of course, this data science department is us.

20.2 Dataset

The dataset consists of sensor readings of a simulated industrial process and consists of two files:

- `simulated_machine_data.csv` stores the sensor data
- `simulated_inspection_data.csv` stores the human inspection results

The sensor data includes the following features in a 200ms resolution:

- `air_temperature_C` : Ambient air temperature in degrees Celsius.
- `machine_temperature_C` : Temperature of the machine in degrees Celsius.
- `pressure_measured_bar` : Measured pressure in bar.
- `pressure_setting_bar` : Pressure setting in bar.
- `process_speed_mm_s` : Set speed of the process in millimeters per second.
- `process_step_index` : Index indicating the current step within the process.
- `product_id` : Unique identifier for each product. The machine performs three processing steps: moving the nozzle to the shaft, applying epoxy, and blowing out residual epoxy from the nozzle while moving away.
- `timestamp` : Timestamp of the sensor reading.

The inspection data includes the `product_id` and the `error_code` :

- `error_code` : Code indicating the type of error detected during inspection.
- `product_id` : Unique identifier for each product.

They used `OK` when there was no defect, `NOK_1` (excessive epoxy), `NOK_2` (insufficient epoxy), `NOK_3` (poor adhesion), and `NOK_4` (uneven distribution).

20.3 Additional information

The operators state that the epoxy application process is highly sensitive to variations in environmental conditions, such as temperature and humidity. They believe that incorporating additional sensor data, such as humidity levels, could further enhance understanding of the process. Also, the pressure settings are critical, since they directly influence the amount of epoxy applied to the product. The first processing step ensures the correct positioning of the nozzle and the final processing step is supposed to clean the nozzle, preparing it for the next application.

20.4 Exploratory Data Analysis

20.4.1 Loading packages

```
import pandas as pd  # used for data handling
import plotly.express as px  # used for performant plotting
import plotly.io as pio  # used to set the default plotly renderer
import seaborn as sns  # used for statistical data visualization

pio.renderers.default = "notebook"  # set the default plotly renderer to "notebook" (necessary
```

20.4.2 Loading the dataset

The dataset is available online in the form of two `csv` files.

It was released at Ehrensperger (2026), but you can also clone the source repository from GitHub.

```
df_processes = pd.read_csv(
    "https://github.com/noxthot/industrial_datascience_sim_machine_knifes/raw/refs/tags/v1.0.2
)
df_inspections = pd.read_csv(
    "https://github.com/noxthot/industrial_datascience_sim_machine_knifes/raw/refs/tags/v1.0.2
)
```

20.4.3 Quick check data integrity

```
df_processes
```

	nr_iteration	process_step_index	timestamp	air_temperature_C	process_speed_mm_s	pressure_measured
0	0	0	2024-06-01 21:00:00.000	17.411810	10	14.221200
1	0	0	2024-06-01 21:00:00.200	17.411810	10	10.564433
2	0	0	2024-06-01 21:00:00.400	17.411810	10	10.054016
3	0	0	2024-06-01 21:00:00.600	17.411810	10	10.103557
4	0	0	2024-06-01 21:00:00.800	17.411810	10	11.594134
...	...	...	...	...	...	...
199995	4999	2	2024-06-02 08:06:39.000	20.261769	20	24.946507
199996	4999	2	2024-06-02 08:06:39.200	20.261769	20	26.045240
199997	4999	2	2024-06-02 08:06:39.400	20.261769	20	30.247436
199998	4999	2	2024-06-02 08:06:39.600	20.261769	20	30.317732
199999	4999	2	2024-06-02 08:06:39.800	20.261769	20	27.688261

200000 rows × 8 columns

```
df_inspections
```

	timestamp	error_code
0	2024-06-01 21:00:10.068501	OK
1	2024-06-01 21:00:20.934694	OK

	timestamp	error_code
2	2024-06-01 21:00:25.997543	OK
3	2024-06-01 21:00:36.090388	OK
4	2024-06-01 21:00:44.213738	OK
...	...	...
4995	2024-06-02 08:06:10.582952	OK
4996	2024-06-02 08:06:19.236113	NOK_4
4997	2024-06-02 08:06:25.214726	OK
4998	2024-06-02 08:06:36.041731	OK
4999	2024-06-02 08:06:44.301083	OK

5000 rows × 2 columns

```
if (
    df_processes.isna().sum().sum() > 0
    or df_inspections.isna().sum().sum() > 0
    or df_processes.duplicated().sum().sum() > 0
    or df_inspections.duplicated().sum().sum() > 0
):
    raise ValueError("Missing values or duplicates found in the dataframes.")
else:
    print("No missing values or duplicates found in the dataframes.")
```

```
No missing values or duplicates found in the dataframes.
```

```
df_processes.describe().T
```

	count	mean	std	min	25%	50%	75%	max
nr_iteration	200000.0	2499.500000	1443.379253	0.000000	1249.750000	2499.500000	3749.250000	4999.0
process_step_index	200000.0	0.625000	0.856959	0.000000	0.000000	0.000000	1.250000	2.0000
air_temperature_C	200000.0	13.240958	2.849175	10.000000	10.648648	12.539426	15.382514	20.261
process_speed_mm_s	200000.0	11.375000	5.764941	1.000000	10.000000	10.000000	12.500000	20.000
pressure_measured_bar	200000.0	27.621157	30.727644	2.112853	11.493736	13.658634	27.004350	114.61
pressure_setting_bar	200000.0	20.625000	30.663318	5.000000	5.000000	5.000000	20.000000	100.00
machine_temperature_C	200000.0	37.053597	4.185691	17.412700	36.009991	36.908348	38.663800	45.538

```
df_processes.dtypes
```

```
nr_iteration              int64
process_step_index        int64
timestamp                object
air_temperature_C       float64
process_speed_mm_s        int64
```

```
pressure_measured_bar    float64
pressure_setting_bar       int64
machine_temperature_C    float64
dtype: object
```

```
df_inspections.dtypes
```

```
timestamp     object
error_code    object
dtype: object
```

```
df_processes.nunique()
```

```
nr_iteration               5000
process_step_index            3
timestamp                200000
air_temperature_C           501
process_speed_mm_s            3
pressure_measured_bar    200000
pressure_setting_bar          3
machine_temperature_C    200000
dtype: int64
```

```
df_inspections.nunique()
```

```
timestamp     5000
error_code       5
dtype: int64
```

```
df_inspections["error_code"].value_counts()
```

```
error_code
OK       4628
NOK_4     296
NOK_1      70
NOK_3       5
NOK_2       1
Name: count, dtype: int64
```

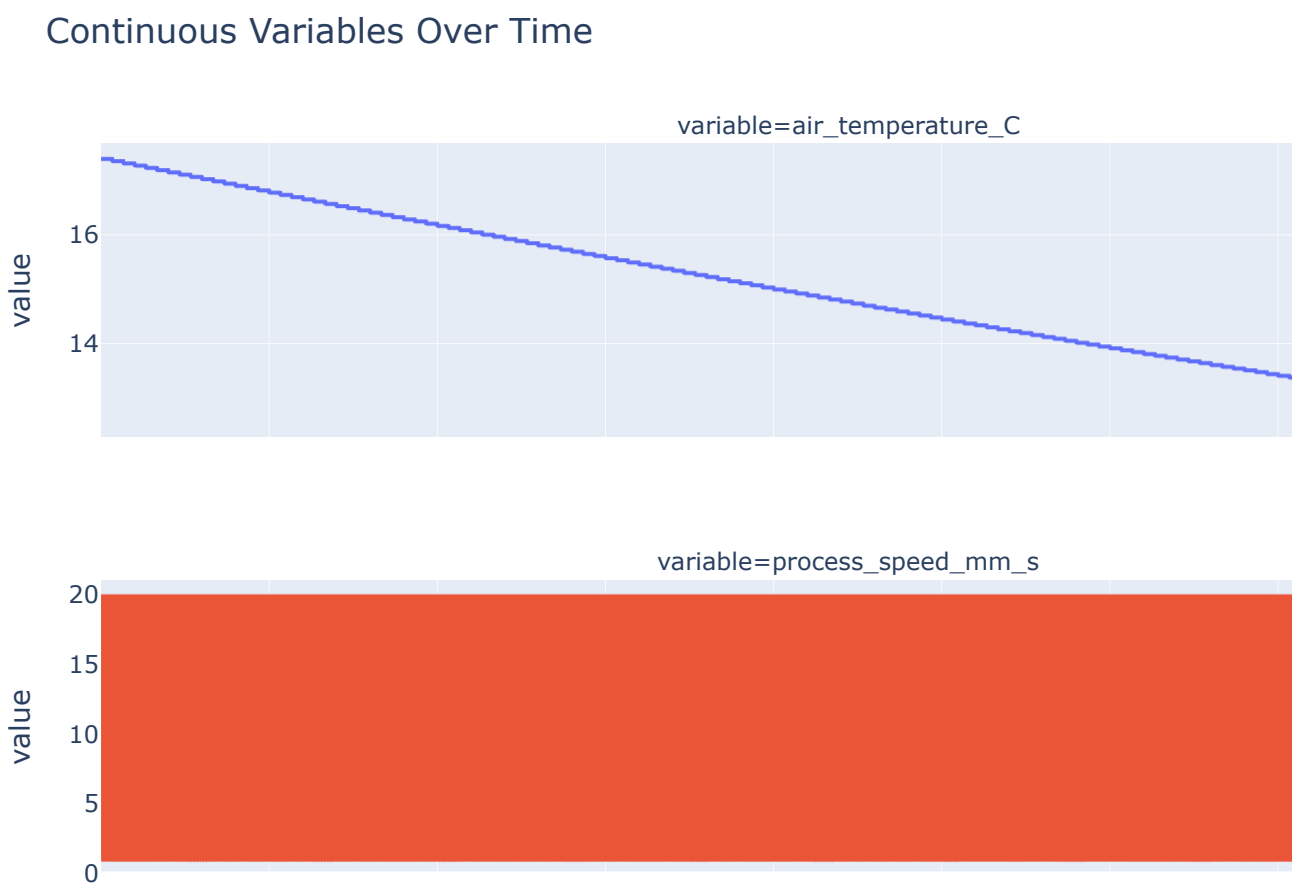
NOK_2 and NOK_3 seem to be hardly ever happening. Let us mentally drop these error codes for this investigation, since <= 10 samples give a very weak statistical basis and tend to distort plots.

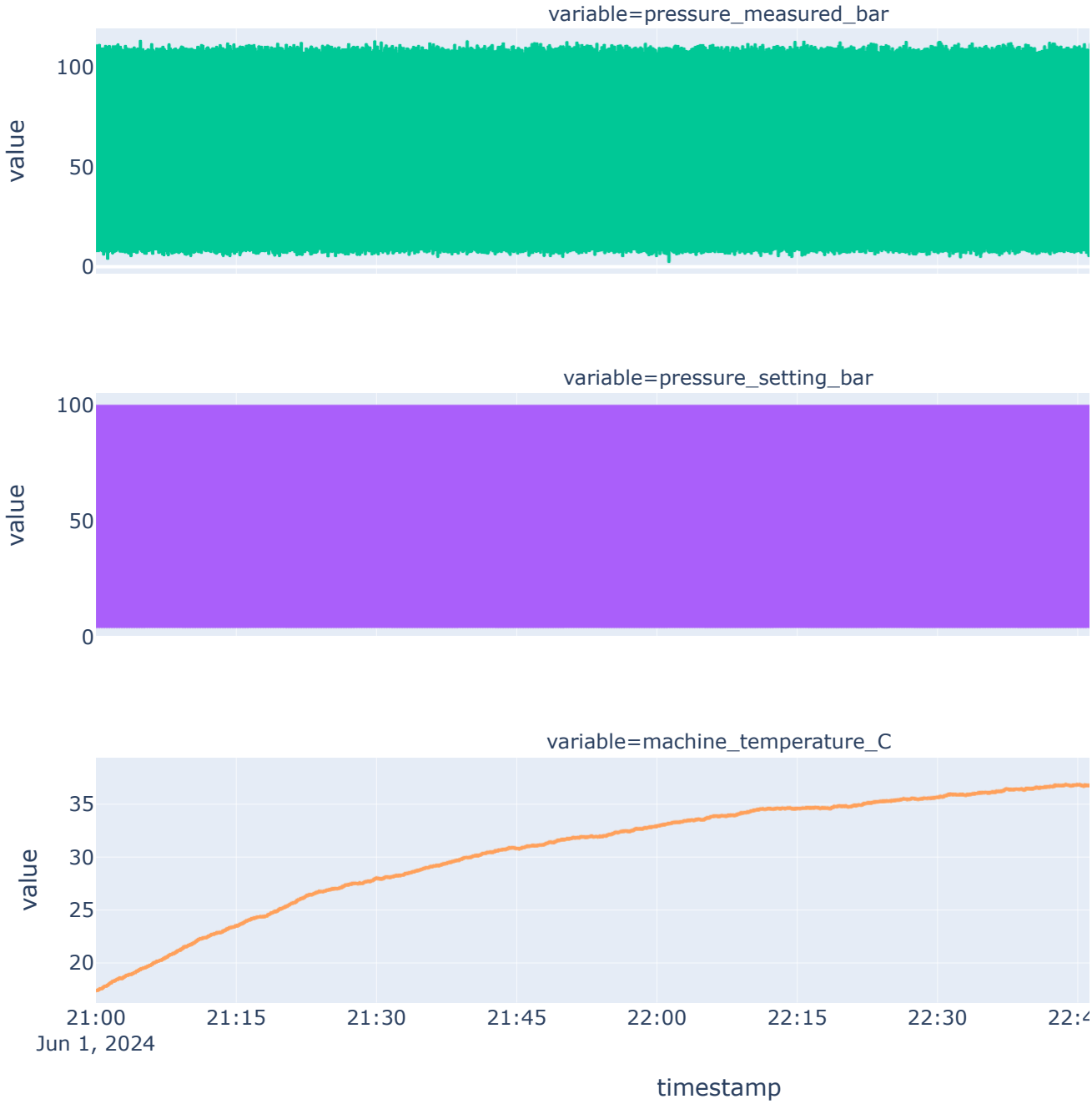
20.4.4 Exploring continuous variables by simple plots

Here, we visualize the continuous variables (one by one) against the timestamp to get an idea of their temporal behavior.

```
CONTINUOUS_VARS = [
    "air_temperature_C",
    "process_speed_mm_s",
    "pressure_measured_bar",
    "pressure_setting_bar",
    "machine_temperature_C",
]
```

```
df_filtered = df_processes.query("nr_iteration < 1000").sort_values(
    "timestamp"
)  # for performance reasons, we filter to the first 1000 cycles.

fig = px.line(
    df_filtered,
    x="timestamp",
    y=CONTINUOUS_VARS,
    facet_col="variable",
    facet_col_wrap=1,
    title="Continuous Variables Over Time",
)

fig.update_xaxes(matches="x")  # share x-axis zoom/pan
fig.update_yaxes(matches=None)  # do not share y-axis limits
fig.update_layout(height=1200)  # increase figure height (y axis size)

fig.show()
```



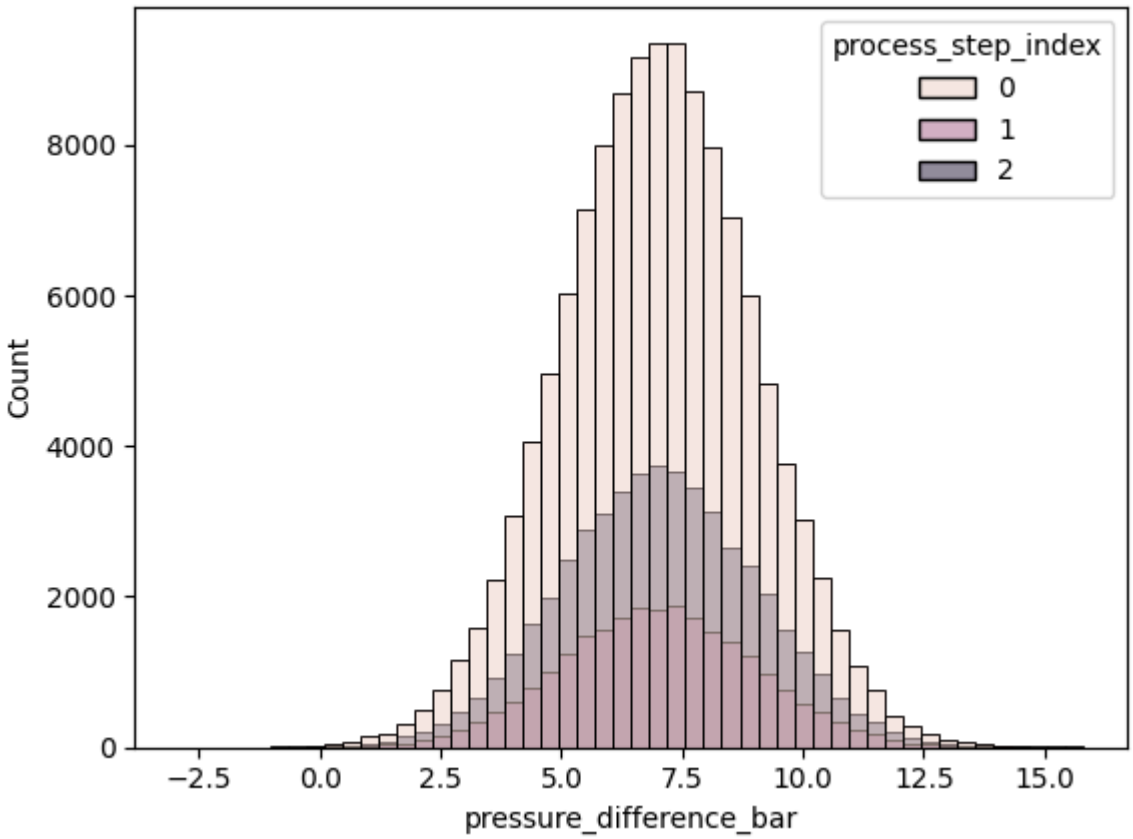


The plot is interactive. Zoom in to see more details. Observe that the set pressure is constant over time (within a processing step), while the measured pressure varies around the set pressure and also shows some systematic offset.

Let us investigate this difference between set and measured pressure in more detail.

```
df_processes["pressure_difference_bar"] = (
    df_processes["pressure_measured_bar"] - df_processes["pressure_setting_bar"]
)
```

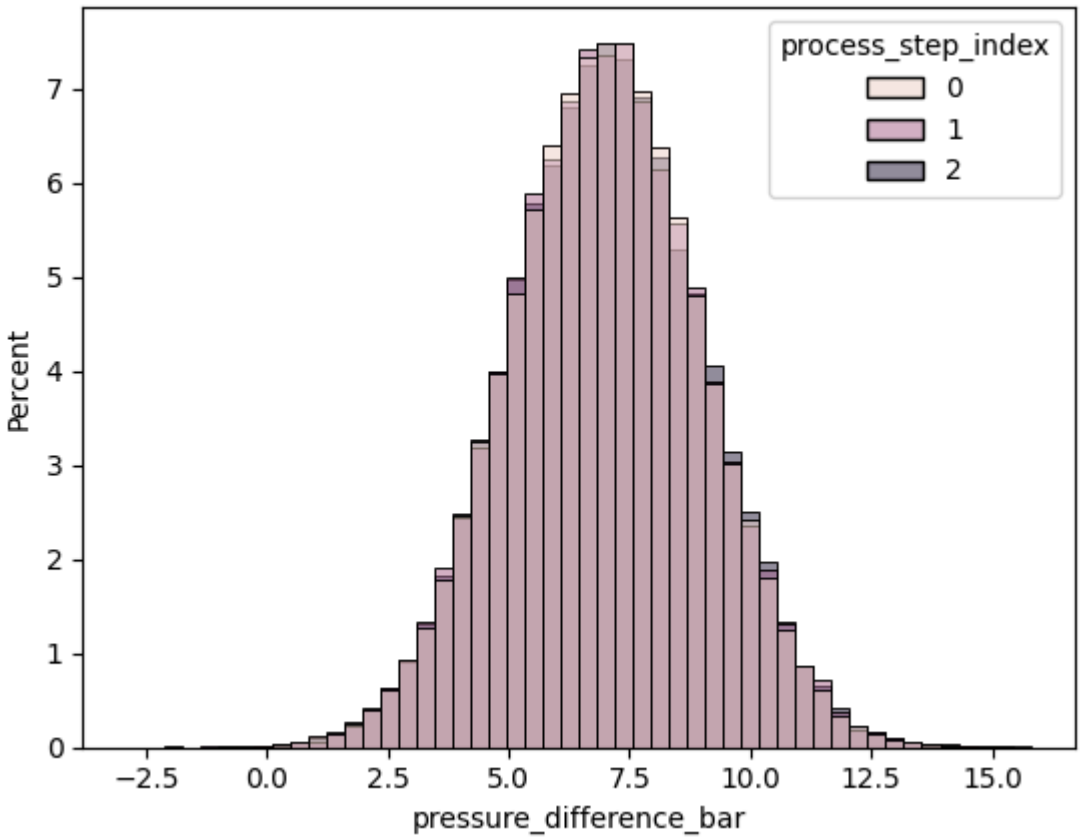
```
sns.histplot(
    data=df_processes, x="pressure_difference_bar", hue="process_step_index", bins=50
)
```



It appears like the pressure difference is roughly normally distributed and that the mean of the three processing steps is similar.

Absolute counts are visually harder to compare, so let us use relative frequencies instead and normalize each of the processing steps separately.

```
sns.histplot(
    data=df_processes,
    x="pressure_difference_bar",
    hue="process_step_index",
    bins=50,
    common_norm=False,
    stat="percent",
)  # common_norm is set to False to normalize each processing step separately, stat is set to
```



Apparently the pressure difference shares similar mean and standard deviation across the three processing steps. We can also show this by calculating the mean and standard deviation of the pressure difference for each processing step.

```
df_processes.groupby("process_step_index")["pressure_difference_bar"].mean()
```

```
process_step_index
0    6.992650
1    6.990845
2    7.007579
Name: pressure_difference_bar, dtype: float64
```

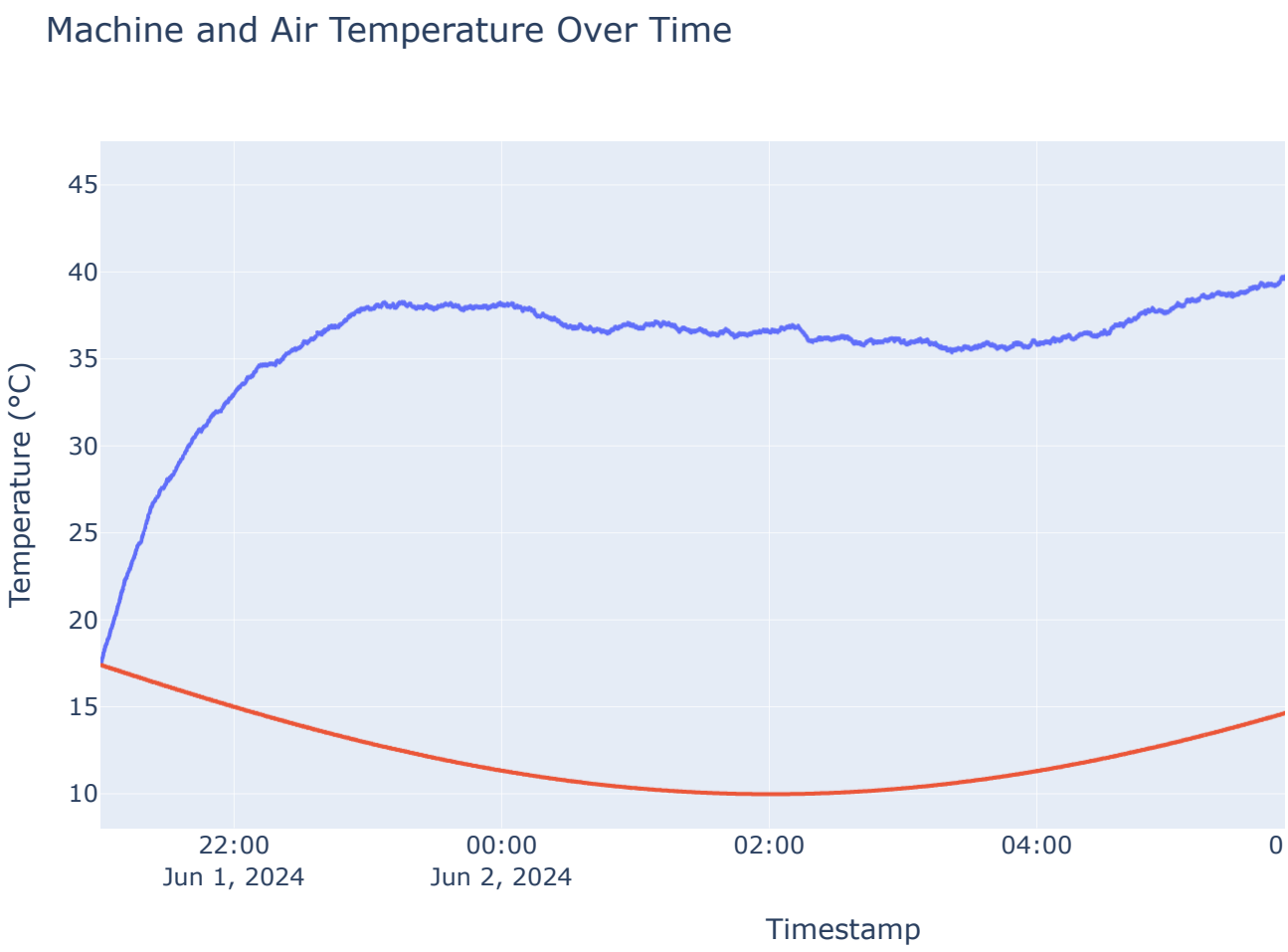
```
df_processes.groupby("process_step_index")["pressure_difference_bar"].std()
```

```
process_step_index
0    1.990051
1    1.997792
2    2.015344
Name: pressure_difference_bar, dtype: float64
```

We conclude that the pressure is not perfectly controlled (or measured) and have a systematic offset of 7 bar and a standard deviation of roughly 2 bar.

```
fig = px.line(
    df_processes.sample(10_000).sort_values(
        "timestamp"
    ),  # for performance reasons, we sample 10,000 rows. Always remember to sort by timestamp
    x="timestamp",
```

```
    y=["machine_temperature_C", "air_temperature_C"],
    labels={"value": "Temperature (°C)", "timestamp": "Timestamp", "variable": "Type"},
    title="Machine and Air Temperature Over Time",
)
fig.show()
```



We observe that the machine is in a cold state when the data collection starts and heats up to a more stable plateau, where it still seems to follow the shape of the air temperature.

20.4.5 Aggregating plots

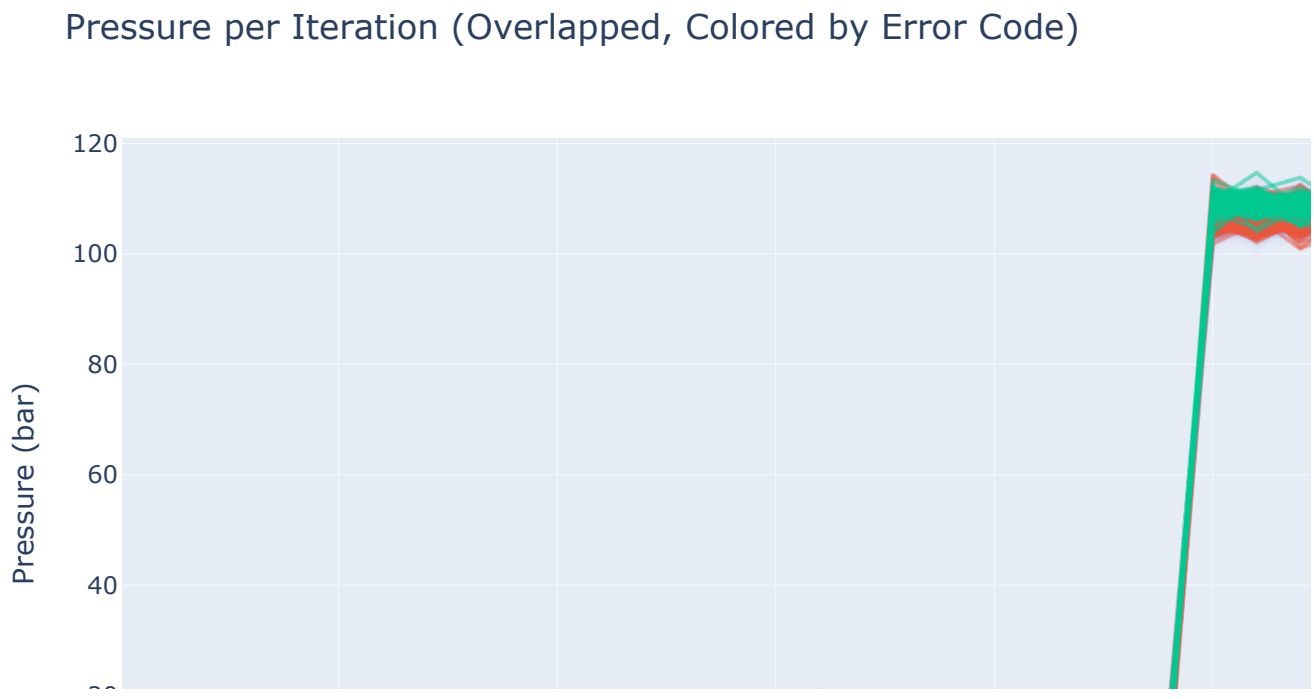
We are looking at iterations of the same, identical process. Let us look at a plot where the processes overlap each other, colored by error code.

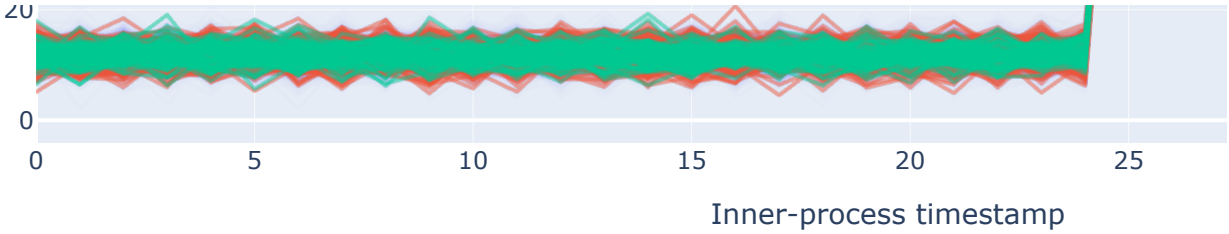
First, we create a new index which starts at 0 for each piece that is manufactured and counts each timestamp within that process step. This index will be useful for plotting the different process iterations on top of each other.

```
df_processes["process_inner_index"] = (
    df_processes.sort_values("timestamp").groupby("nr_iteration").cumcount()
)
```

```
# Merge error_code from df_inspections into df_processes based on 'nr_iteration'. This assigns
df_plot = df_processes.merge(
```

```
    df_inspections, left_on="nr_iteration", right_index=True, how="left"
)

fig = px.line(
    df_plot,
    x="process_inner_index",
    y="pressure_measured_bar",
    line_group="nr_iteration",
    color="error_code",
    title="Pressure per Iteration (Overlapped, Colored by Error Code)",
    labels={
        "process_inner_index": "Inner-process timestamp",
        "pressure_measured_bar": "Pressure (bar)",
        "error_code": "Error Code",
    },
)

# Set opacity: 0.01 for 'OK', 0.5 for others
# Setting opacity is always a bit tricky. Setting it too low makes the lines almost invisble,
# This settings were found in trial and error and work well for that case.
for trace in fig.data:
    if trace.name == "OK":
        trace.opacity = 0.01
    else:
        trace.opacity = 0.5

# Unselect NOK_2 and NOK_3 by default, since they are very rare and would clutter the plot.
for i, trace in enumerate(fig.data):
    if trace.name in ["NOK_2", "NOK_3"]:
        fig.data[i].visible = "legendonly"

fig.update_layout(showlegend=True)
fig.show()
```



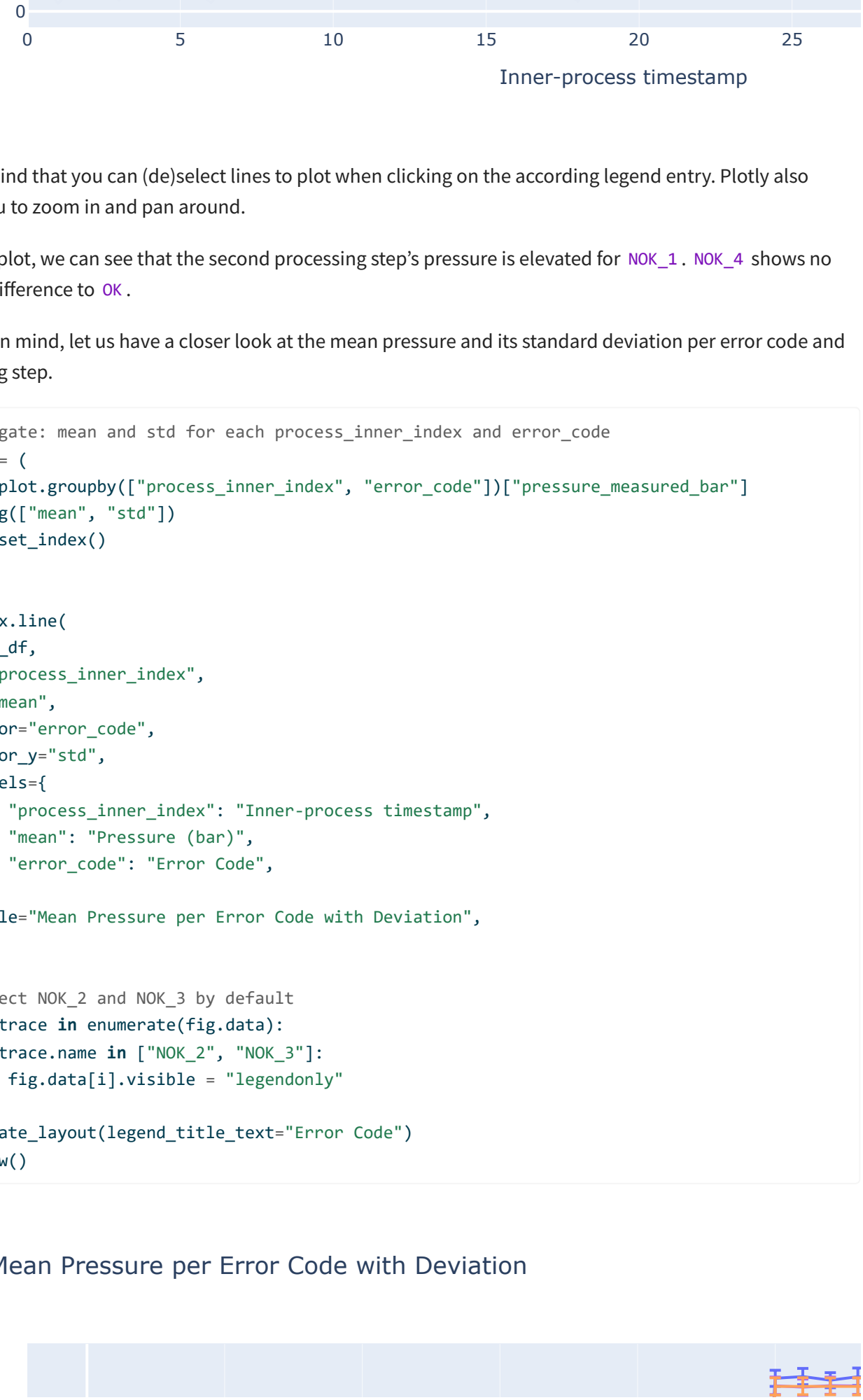


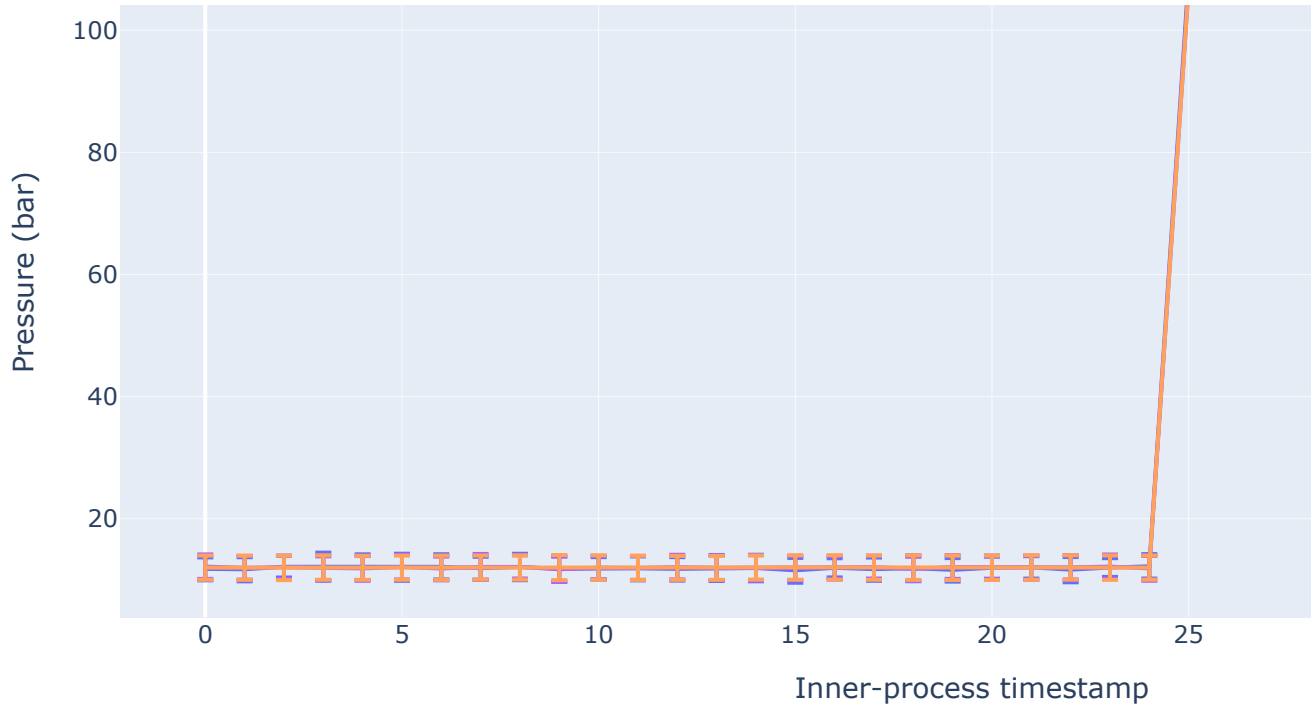
Keep in mind that you can (de)select lines to plot when clicking on the according legend entry. Plotly also allows you to zoom in and pan around.

From the plot, we can see that the second processing step's pressure is elevated for `NOK_1`. `NOK_4` shows no obvious difference to `OK`.

With this in mind, let us have a closer look at the mean pressure and its standard deviation per error code and processing step.

```
# Aggregate: mean and std for each process_inner_index and error_code
agg_df = (
    df_plot.groupby(["process_inner_index", "error_code"])["pressure_measured_bar"]
    .agg(["mean", "std"])
    .reset_index()
)

fig = px.line(
    agg_df,
    x="process_inner_index",
    y="mean",
    color="error_code",
    error_y="std",
    labels={
        "process_inner_index": "Inner-process timestamp",
        "mean": "Pressure (bar)",
        "error_code": "Error Code",
    },
    title="Mean Pressure per Error Code with Deviation",
)

# Unselect NOK_2 and NOK_3 by default
for i, trace in enumerate(fig.data):
    if trace.name in ["NOK_2", "NOK_3"]:
        fig.data[i].visible = "legendonly"

fig.update_layout(legend_title_text="Error Code")
fig.show()
```



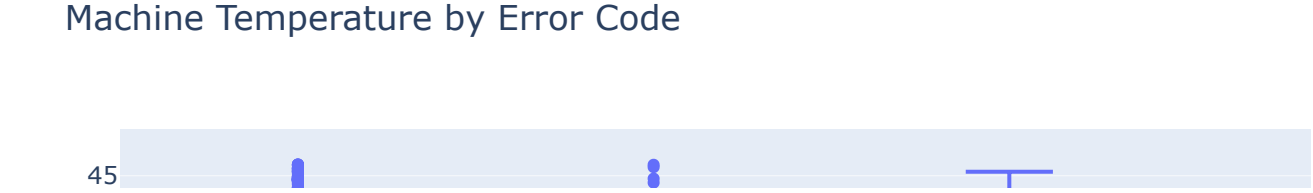


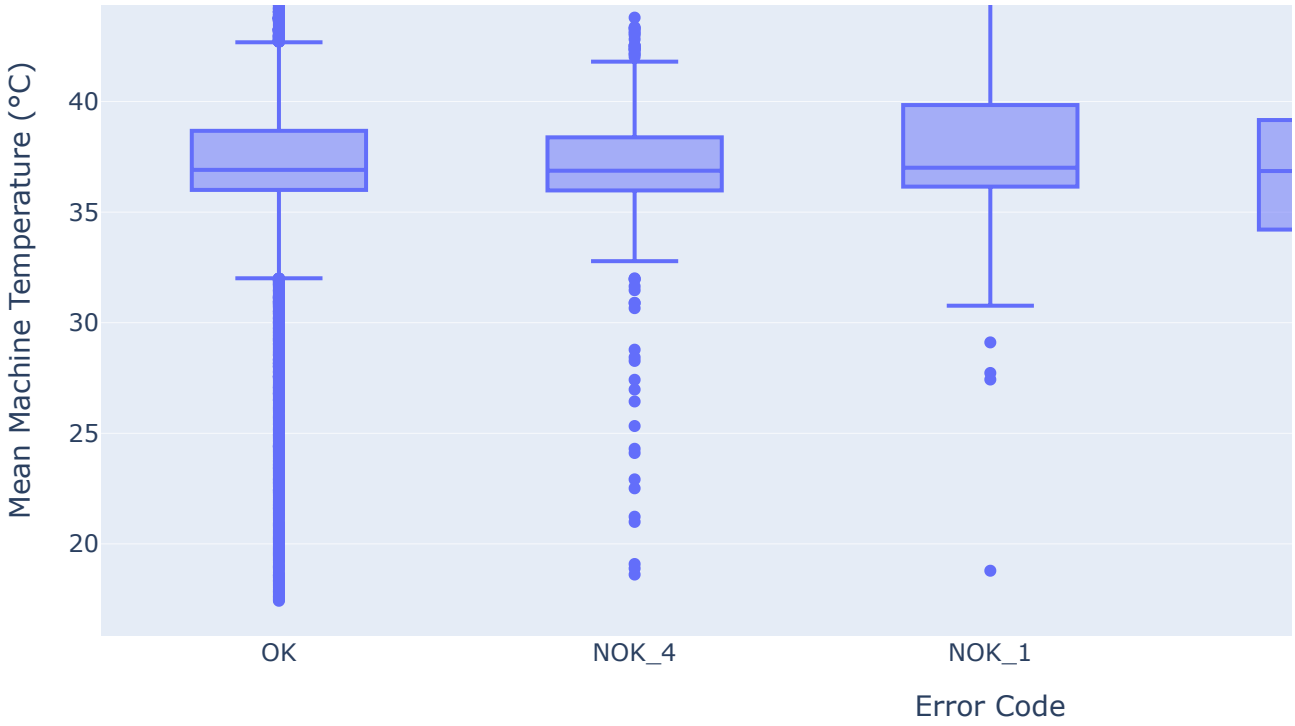
This plot confirms our previous observation that `NOK_1` has a systematic higher mean pressure during the second processing step and `NOK_4` does not differ notably from `OK`.

While the first line plot shows every individual process iteration as a singe line and therefor does not accidentally filter relevant information, it is a bit crowded and hard to read. The second plot aggregates the individual lines by calculating the mean and standard deviation, therefore losing some information, but making it easier to comprehend.

Now let us also check whether the machine temperature has an influence on failures.

```
df_agg = (
    df_plot.groupby(["nr_iteration", "error_code"])["machine_temperature_C"]
    .agg(["mean"])
    .reset_index()
)

fig = px.box(
    df_agg,
    x="error_code",
    y="mean",
    title="Machine Temperature by Error Code",
    labels={"mean": "Mean Machine Temperature (°C)", "error_code": "Error Code"},
)

fig.show()
```





Also here, `NOK_4` and `NOK_1` do not show a notable difference to `OK`, while `NOK_2` and `NOK_3` can not be judged well due to the low sample size.

20.4.6 Summary of findings

Let us summarize our findings so far in this dataset:

- The pressure during the second processing step is elevated for `NOK_1` compared to `OK`.
- The machine temperature does not show a notable difference between `OK` and `NOK_x`.
- The source of `NOK_4` failures was not identified in this analysis.
- The error codes `NOK_2` and `NOK_3` are too rare to draw any conclusions.

Chapter 22 will dive deeper into the data and guide you through the process of optimizing the manufacturing process based on these findings.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0

Manuscript designed with Quarto.

Gregor Ehrensperger

Edit this page Report an issue

21 Process optimization via Live Feedback

> **Still under construction**
>
> This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

In this section, we will have a look at a simulated example of a live feedback system in an industrial manufacturing context.

21.1 Setting the stage

Imagine a pick-and-place machine in a manufacturing line that is responsible for placing components onto a circuit board. The machine has two axes (X and Y) to position the placement head accurately over the board and a vacuum system to pick up and release components. Additionally it has a camera system to verify the placement accuracy of each component once it is placed. A couple of sensors provide real-time measurements and variables that can be used to monitor the process.

Unfortunately, the machine seems to be experiencing some issues with placement accuracy, leading to an increased number of defective boards. To address this, the engineers set up sandboxing mode where the machine performs four steps:

0. Move to pick up a component with random weight placed at a random location.
1. Pick up the component using the vacuum system.
2. Move to the target placement location on the circuit board with random speed. This is always at the center of the board at (150, 150).
3. Place the component and measure the placement accuracy using the camera system.

Until the end of step 2, it is possible to adjust the placement parameters by providing a correction vector that modifies the target placement location.

The goal is to optimize the placement parameters in real-time based on the feedback received from the camera system after each placement.

21.2 Accessible variables

The following variables are accessible during the process:

- `Air temperature [K]` : ambient temperature around the machine.
- `Attached Weight [mg]` : weight of the component picked up by the vacuum system.
- `Cycle` : unique identifier for each cycle.
- `Error x [mm]` : placement error in x direction (only available at the end of the cycle).
- `Error y [mm]` : placement error in y direction (only available at the end of the cycle).
- `Position x [mm]` : target position in x direction.
- `Position y [mm]` : target position in y direction.
- `Process step` : current step in the process (0 to 3).

- `Process temperature [K]` : temperature inside the machine.
- `Speed [mm/s]` : speed of the current movement.
- `Speed next [mm/s]` : speed for the next movement.

21.3 Data

One cycle usually takes 3-6 seconds to complete and the sensor data is available in realtime via *OPC Unified Architecture* (OPC UA).

> **Note**
>
> OPC UA is a cross-platform, open-source communication protocol for data exchange in the industrial automation space developed by the OPC Foundation. It is a client-server architecture that allows for secure and reliable data transfer between devices and systems from different manufacturers.

Luckily, the engineers already collected a dataset of 25.000 cycles coming in two files:

- `simulated_opcua_machinedata.csv` stores the sensor data.
- `simulated_opcua_postinspection.csv` stores the placement error data.

The two datasets can be merged via `cycle` column.

21.4 Loading packages

```
import pandas as pd  # used for data handling
import plotly.io as pio  # used to set the default plotly renderer
import seaborn as sns  # used for statistical data visualization

pio.renderers.default = "notebook"  # set the default plotly renderer to "notebook" (necessary
```

21.5 Loading the dataset

The dataset was released alongside the simulated machine at Ehrensperger (2026), but you can also clone the repository directly from GitHub. In this section, we will look at the provided `csv` files.

```
df_machinedata = pd.read_csv(
    "https://raw.githubusercontent.com/noxthot/industrial_datascience_live_machine/refs/tags/v
)
df_postinspection = pd.read_csv(
    "https://raw.githubusercontent.com/noxthot/industrial_datascience_live_machine/refs/tags/v
)
```

21.5.1 Data integrity check

```
df_machinedata
```

	Air temperature [K]	Attached Weight [mg]	Position x [mm]	Position y [mm]	Process step	Process temperature [K]	Speed [mm/s]	Speed next [mm/s]	timestamp
0	24.755379	0.000000	191.828040	7.503227	0	30.385614	100.000000	0.000000	2026-01-08T13:37:00
1	25.215286	223.987527	191.828040	7.503227	1	31.615630	0.000000	34.752639	2026-01-08T13:37:01.50
2	24.666009	223.987527	150.000000	150.000000	2	31.998852	34.752639	0.000000	2026-01-08T13:37:02.02
3	25.115265	0.000000	150.000000	150.000000	3	33.046526	0.000000	100.000000	2026-01-08T13:37:06.31
4	25.129309	0.000000	7.960791	59.651295	0	33.194048	100.000000	0.000000	2026-01-08T13:37:06.33
...	...	...	...	...	...	...	...	...	...
99995	8.068273	0.000000	150.000000	150.000000	3	17.642248	0.000000	100.000000	2026-01-09T21:57:12.23
99996	10.301384	0.000000	273.977920	126.801859	0	16.803874	100.000000	0.000000	2026-01-09T21:57:12.25
99997	9.924715	193.867063	273.977920	126.801859	1	16.295092	0.000000	27.812182	2026-01-09T21:57:13.53
99998	9.452700	193.867063	150.000000	150.000000	2	16.074561	27.812182	0.000000	2026-01-09T21:57:14.05
99999	10.116288	0.000000	150.000000	150.000000	3	17.222515	0.000000	100.000000	2026-01-09T21:57:18.60

100000 rows × 10 columns

```
df_postinspection
```

	cycle	error_x	error_y
0	1	-0.192236	0.654895
1	2	1.285577	0.817733
2	3	-0.456091	0.163004
3	4	-1.734704	-0.624146
4	5	-0.496285	0.494395
...	...	...	...
24995	24996	1.080388	2.691526
24996	24997	-0.566057	-0.794911
24997	24998	-0.580030	0.738004
24998	24999	0.773854	-1.404837
24999	25000	-0.842577	0.157659

25000 rows × 3 columns

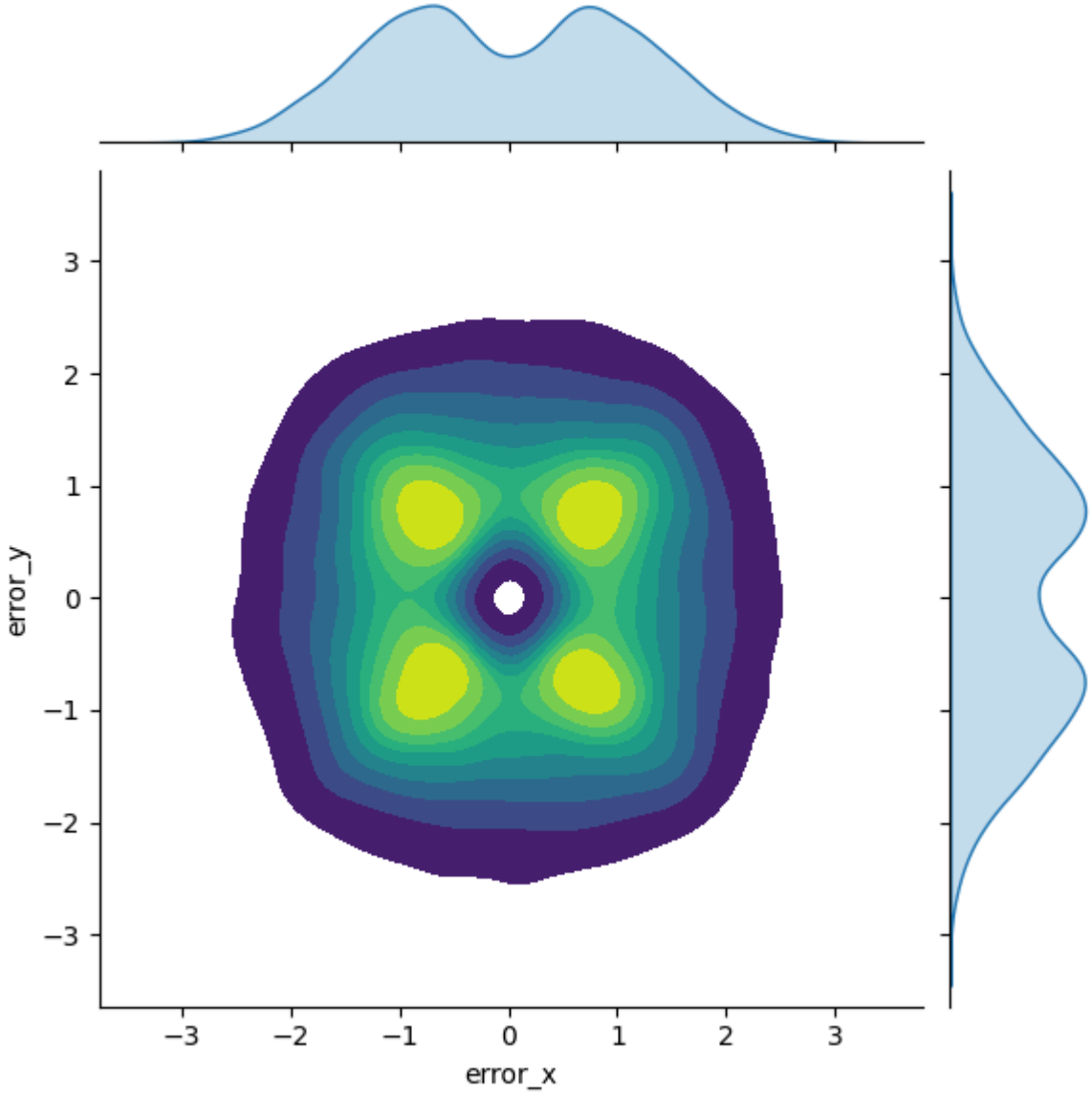
```
df = df_machinedata.merge(df_postinspection, on="cycle", how="left")
df
```

	Air temperature [K]	Attached Weight [mg]	Position x [mm]	Position y [mm]	Process step	Process temperature [K]	Speed [mm/s]	Speed next [mm/s]	timestamp
0	24.755379	0.000000	191.828040	7.503227	0	30.385614	100.000000	0.000000	2026-01-08T13:37:00
1	25.215286	223.987527	191.828040	7.503227	1	31.615630	0.000000	34.752639	2026-01-08T13:37:01.50
2	24.666009	223.987527	150.000000	150.000000	2	31.998852	34.752639	0.000000	2026-01-08T13:37:02.02
3	25.115265	0.000000	150.000000	150.000000	3	33.046526	0.000000	100.000000	2026-01-08T13:37:06.31
4	25.129309	0.000000	7.960791	59.651295	0	33.194048	100.000000	0.000000	2026-01-08T13:37:06.33
...	...	...	...	...	...	...	...	...	...
99995	8.068273	0.000000	150.000000	150.000000	3	17.642248	0.000000	100.000000	2026-01-09T21:57:12.23
99996	10.301384	0.000000	273.977920	126.801859	0	16.803874	100.000000	0.000000	2026-01-09T21:57:12.25
99997	9.924715	193.867063	273.977920	126.801859	1	16.295092	0.000000	27.812182	2026-01-09T21:57:13.53
99998	9.452700	193.867063	150.000000	150.000000	2	16.074561	27.812182	0.000000	2026-01-09T21:57:14.05
99999	10.116288	0.000000	150.000000	150.000000	3	17.222515	0.000000	100.000000	2026-01-09T21:57:18.60

100000 rows × 12 columns

21.6 Error Visualization

```
sns.jointplot(
    data=df_postinspection,
    x="error_x",
    y="error_y",
    kind="kde",
    fill=True,
    cmap="viridis",
)
```



This is a rather interesting error distribution.

21.7 Next Steps

Chapter 23 will guide you through the optimization process aimed at reducing these errors and improving overall process accuracy.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

22 Self study - Session 1

Still under construction

This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

In this session, we will revisit Chapter 20 and continue where we left off, trying to improve the process.

The findings in Section 20.4.6 were not sufficient to explain all defects, but we found a correlation between elevated pressure during the epoxy application step and excessive epoxy application (`NOK_1`). Regarding the uneven distribution of applied epoxy (`NOK_4`), the operators provided the information that they suspect that this is caused by an insufficiently cleaned nozzle. The cleaning process happens at the end of the previous processing step, where the nozzle moves away from the product and blows out residual epoxy.

Exercise 22.1 (Investigating the blow-out step)

Investigate the pressure during the blow-out step (process step index 3 of the previous iteration) and see if you can find a correlation to `NOK_4` defects.

Having more information about `NOK_1` and `NOK_4` , we can now try to fine-tune the machine's parameters to reduce the number of defects.

Exercise 22.2 (Fine-tuning the machine parameters)

The accompanying `readme.md` file shipped with the `industrial_datascience_sim_machine_knifes` repository contains information on how to run the machine again with different parameters. Try to improve the parameters to reduce the number of defects. Keep the `night` shift and `NUM_PIECES` at the default values.

You experience new issues or want to further fine-tune the parameters? Feel free to have a look at the new `csv` files that the machine is producing.

Hint: The process is stochastic, so you can not expect to get rid of every single defect.

How do the results of the new parameters compare to the original ones?

The original dataset had 5000 pieces with the following distribution:

Error Code	Count	Percentage
OK	4628	92.56%
NOK_1	70	1.40%
NOK_2	1	0.02%
NOK_3	5	0.10%
NOK_4	296	5.92%

Great job? Everything looks good on the night shift?

Exercise 22.3 (Try the machine on day shift)

Run the machine again with your parameters, but this time during `day` shift.

How do the results of the new parameters compare to the ones from the night shift?

Investigate the root cause of this difference by looking at the produced `csv` files.

Exercise 22.4 (Fine-tune the parameters to work well for both shifts)

The machine is not only telling you a defect summary, but also the duration it took to produce the parts. Try to fine-tune the parameters to achieve a good compromise between quality (few defects) and speed (short duration).

Hints:

- *The machine's temperature depends on the ambient temperature, the processing speed and the pressure settings.*
- *The duration it takes to produce knifes is determined by the processing speed.*

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

23 Self study - Session 2

Still under construction

This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

In this session, we will revisit Chapter 21 and try to optimize the placement accuracy of the pick-and-place machine.

Before we are touching the live machine, we will first try to find a suitable model based on the provided dataset.

Note

Since we are working with a live feedback system, we will need multiple terminals/consoles at the same time:

1. One terminal to run the simulated pick and place machine (the OPC UA server).
2. Optionally, one terminal to run the OPC UA client that connects to the machine and provides real-time.
3. Optionally, one terminal to run the `opcua-client` GUI to visualize the live data.

Exercise 23.1 (Train a model to predict placement error)

We are told that the placement can be adjusted until the end of step 2 by providing a correction vector that modifies the target placement location. This means that we need to predict the placement error (`Error x [mm]` and `Error y [mm]`) and provide the correction vector based on the available variables until the end of step 2.

Furthermore, engineers tell us that the rows where `Process step` is 1 should contain the necessary information to predict the placement error.

Your task:

1. Split the provided dataset into a training and test set (depending on the chosen model, you will also need a validation set).
2. Train a model that predicts `Error x [mm]` and `Error y [mm]` .
3. Take a look at the model's performance on the test set by looking at certain metrics, e.g., by calculating the Mean Absolute Error (MAE).
4. Visualize the predictive capabilities by e.g. plotting a histogram showing the errors and residuals (predicted - true values).
5. Persist the model to disk so that we can later load it in the live feedback system. In case you use a scaler, you will need to persist it as well.

Once you have a model that performs well on the test set, we can work on integrating it in the live feedback system.

But first, let us understand how to interact with the simulator. The accompanying `readme.md` file shipped with the `industrial_datascience_live_machine` repository contains all necessary information.

Exercise 23.2 (First encounter with the live machine simulator)

Running the `opc-ua-server.py` or the provided docker-container will start the simulated pick and place machine that exposes the sensor data via OPC UA. The machine runs in real-time, meaning that one cycle takes 3-6 seconds to complete. You will see the process steps advancing in the console output. Simultaneously, the sensor data is streamed via the OPC UA protocol.

Your task:

1. Start the machine and copy/paste the console output for the first 5 cycles.

2. Use opcua-client or any other OPC UA client of your choice to connect to the machine:

   - Usually it should be enough to run `uv add opcua-client`, followed by `uv run opcua-client` (Windows might need an extra step to install the GUI dependencies; use Python `3.13` to avoid issues in the following steps).
   - The server URL should be `opc.tcp://localhost:4840`, which is prefilled in the client. Simply click on `Connect` (the machine must be actively running in another terminal before connecting).
   - Once connected, you should see a tree structure on the left side. Navigate to `0:Root,0:Objects,2:plcSimulator`.
   - Provide a screenshot of the variable tree below `2:plcSimulator`.

3. Look at the attributes of a variable:

   - Click through the variables and take a close look at the Attributes tab on the right side.
   - The server is constantly streaming new values for each variable, but you need to hit the `Refresh` button to see the updated value.
   - Provide a screenshot of the attributes of `position_x`.

4. Subscribe to data changes:

   - Right-click on one of the variables (e.g. `position_x`) and select `Subscribe to data changes`.
   - Subscribed variables will appear in the `Subscriptions` tab at the bottom.
   - Observe that the values are changing in real-time as the machine is running.
   - Also observe that they are not updated at the very same time, but rather whenever there is a new measurement available.
   - Provide a screenshot of the `Subscriptions` tab showing the data changes of `position_x`, `position_y`, and `cycle`.

5. Look at the graph of subscribed variables:

   - This requires two more packages to be installed. If you are using `uv`, run `uv add numpy pyqtgraph` to add the necessary dependencies.
   - Switch to the `Graphs` tab at the bottom.
   - Change update interval to `1` s (unfortunately it appears that it is not possible to go below 1 second in this client).
   - Contrary to subscribing to data changes, this graph will poll the server for new values at the specified interval (meaning that we might miss some updates if the frequency is higher).
   - Add the variables you want to visualize by right-clicking on them and selecting `Add to Graph`.
   - Provide a screenshot of the graph showing `error_x` and `error_y` over time.

6. Interact with the machine by writing values:

- While looking at the graph, click on `correction_x` in the variable tree on the left side.
- In the Attributes tab, double click on the `Value` field to edit it.
- Provide a correction value (e.g. `10.0`) and hit `ENTER` to write the new value to the server.
- Observe how the machine reacts to the provided correction in the graph (note that the server always resets the correction values to `0.0` after applying them).
- Provide a screenshot of the graph showing the effect of your correction on `error_x`.
- Have a look at the console output of the running machine simulator to see how the correction was applied.
- Provide a screenshot of the console output showing the application of your correction.

Now that we know how to interact with the live machine simulator, in a first step we will set up a Python script `opc-ua-client.py` that connects to the machine and subscribes to the data changes of the necessary variables.

Exercise 23.3 (Set up an OPC UA client in Python)

Have a look at the provided `opc-ua-client.py` script, understand how it works and run it while the machine simulator is active.

Your task:

1. For understanding, it is likely best to start by studying the `main()` function at the bottom of the script and then look at the `SubHandler` class that handles the data changes.
2. Explain in your own words what the script is doing (a brief summary is sufficient).
3. Run the script while the machine simulator is active and observe the printed output.
4. Provide a screenshot of the console output showing the data changes received from the machine.

Great! It is now time to integrate our trained model into the live feedback system and hope that the chosen model generalizes well to the live data and is fast enough to provide corrections in real-time.

Exercise 23.4 (Integrate the trained model into the live feedback system)

The `opc-ua-client.py` script contains various placeholders (marked with `# >>> Exercise`) where you need to integrate your trained model to provide real-time corrections to the pick and place machine.

Your task:

Load model:

1. Part 1: Load your model (and optionally scaler) from the first exercise.
2. Part 2: Select and order the features that your model needs to make predictions (they must be in the very same order as during training).

Use the model to compute correction values:

3. Part 3 - Optional: Scale your data, if you used a scaler during training.
4. Part 4: Predict the placement error using your model.
5. Part 5: Based on your prediction, compute the necessary correction values.

Integrate the model into the live feedback loop:

6. Part 6: Print the collected variables (this is for debugging purposes in case you need it).
7. Part 7: Use the previously implemented function `predict_error` to get correction values for `x` and `y`.
8. Part 8: Write the computed correction values back to the OPC UA server to adjust the placement in real-time.
9. Part 9: Now remove the `pass` statement to enable your implementation.

Test your implementation:

10. Run the modified `opc-ua-client.py` script while the machine simulator is active.
11. In case it is working fine: Well done. Otherwise, debug your implementation until it is working as expected.
12. Part 10: Remove datachange notification to reduce console output clutter.

Proof of work:

13. Provide a screenshot of the client's console output showing the computed correction values being sent to the server.
14. Provide a screenshot of the machine simulator's console output showing the effect of your corrections on the placement error.

This is it! Almost.

Visualizations are almost always nicer to look at than raw console output.

Exercise 23.5 (Visualize the live placement error and corrections)

Your task:

1. Ensure that the machine simulator is running.
2. Ensure that your live feedback client is *not* running.
3. Start the GUI client `opcua-client` once more and connect it to your machine.
4. Add `error_x`, `error_y` to the graph and set the update interval to `1` s.
5. Wait for a couple of cycles to see how the placement error evolves over time.
6. Now, start your live feedback client.
7. Observe how the placement error changes as your model provides corrections in real-time.
8. Provide a screenshot of the graph showing the live placement error and the effect of your corrections: It should show both, the error before the feedback loop is started as well as the reduced error once the feedback loop is active.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

24 Outlook

> **Still under construction**
>
> This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

The remaining chapters should serve as an outlook into further topics in industrial data science, which are not covered in detail in this course.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0

Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

25 Resampling

> **Still under construction**
>
> This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

In many real scenarios, data is acquired by manual input or when a certain event occurred, leading to irregular time steps.

Finding patterns in time series data is usually easier when time series have constant time steps. Some common time series methods, such as ARIMA or Recurrent Neural Networks, even require constant time steps.

When time series have irregular time steps, we can resample them to a regular frequency. To illustrate this, we generate a synthetic time series with irregular time steps and resample it to a regular frequency.

25.1 Example

25.1.1 Load packages

▶ Code

25.1.2 Data Generation

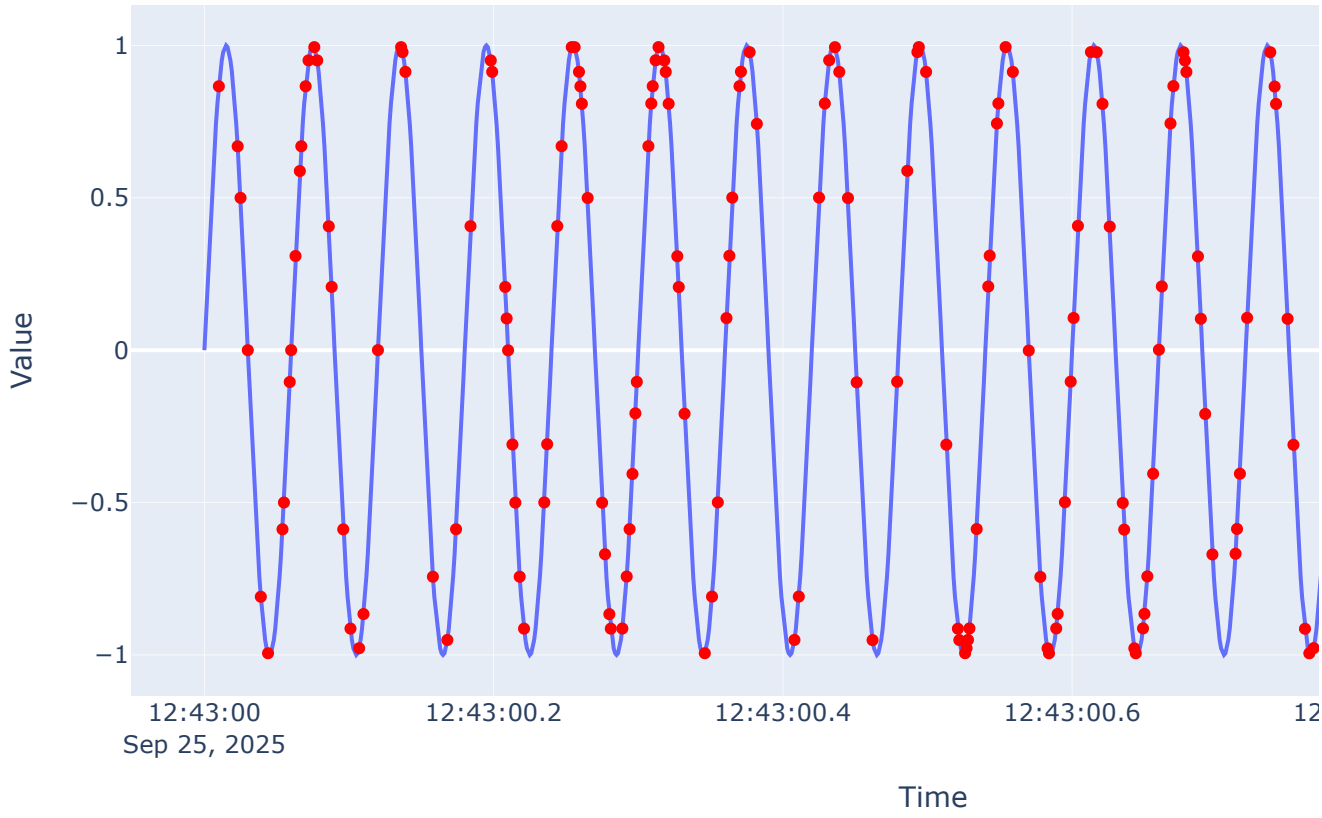
Generate synthetic time series with irregular time steps.

▶ Code

25.1.3 Data Visualization

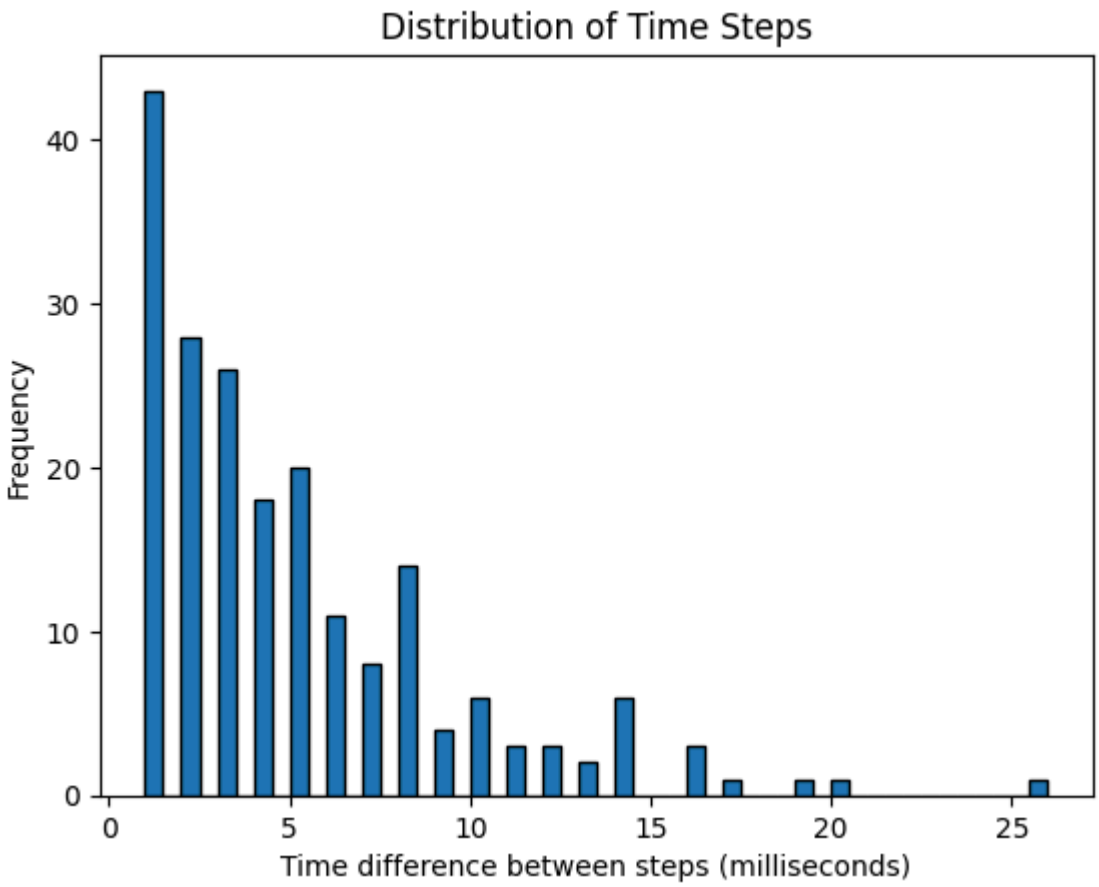
```
fig = px.line(
    x=df_sinus.index,
    y=df_sinus.values,
    labels={"x": "Time", "y": "Value"},
    title="Original sinus curve with sampled points at irregular intervals",
)
fig.add_scatter(
    x=df_sinus_sampled.index,
    y=df_sinus_sampled.values,
    mode="markers",
    name="Sampled Points",
    marker={"color": "red", "size": 6},
)
fig.show()
```

Original sinus curve with sampled points at irregular intervals



Using a histogram, visualizing the difference between consecutive time steps, we can see that time steps are not constant.

```
# Calculate time differences in seconds between consecutive time steps
time_diffs = df_sinus_sampled.index[1:] - df_sinus_sampled.index[:-1]
time_diffs_seconds = time_diffs.total_seconds() * 1000

plt.hist(time_diffs_seconds, bins=50, edgecolor="k")
plt.xlabel("Time difference between steps (milliseconds)")
plt.ylabel("Frequency")
plt.title("Distribution of Time Steps")
plt.show()
```



25.1.4 Choosing a sampling frequency

Resampling ensures constant time steps, but usually requires some sort of alignment.

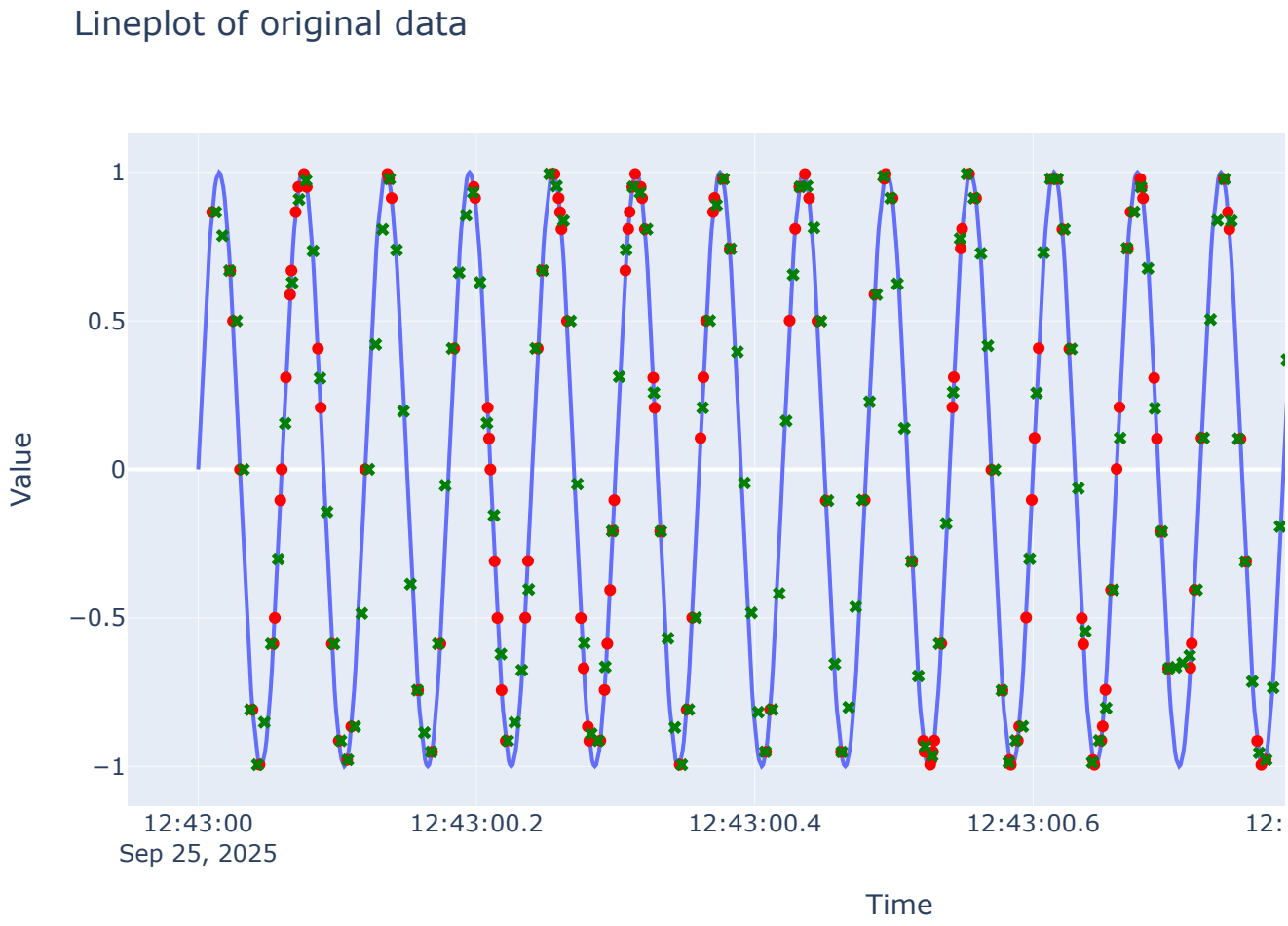
This can be done by oversampling (increasing the frequency) or undersampling (decreasing the frequency). Using a fixed frequency, the question arises, which value should be used for values at the newly introduced time step.

For undersampling, we usually have multiple values between two consecutive time steps of the resampled time series. This can be handled by aggregation functions, such as `mean`, `median`, `min`, `max`, `sum`, last value, next value, etc.

For oversampling, we usually have no value at the newly introduced time steps, which requires some form of interpolation.

```
resample_freq = 5  # milliseconds

df_sinus_resampled = (
    df_sinus_sampled.resample(f"{resample_freq}ms").median().interpolate(method="pchip")
)
df_sinus_resampled.index += +datetime.timedelta(
    milliseconds=(resample_freq / 2)
)  # pd.resample is not center-aligned; adjust for that

fig = px.line(
    x=df_sinus.index,
    y=df_sinus.values,
    labels={"x": "Time", "y": "Value"},
```

```
    title="Lineplot of original data",
)
fig.add_scatter(
    x=df_sinus_sampled.index,
    y=df_sinus_sampled.values,
    mode="markers",
    name="sampled Points",
    marker={"color": "red", "size": 6},
)
fig.add_scatter(
    x=df_sinus_resampled.index,
    y=df_sinus_resampled.values,
    mode="markers",
    name="resampled Points",
    marker={"color": "green", "symbol": "x", "size": 6},
)
fig.show()
```



Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

26 Multimodal Machine Learning

> **Still under construction**
>
> This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

In industrial settings, data often comes from multiple sources and modalities. A production line might generate time series data from sensors (temperature, vibration, pressure) alongside visual data from cameras inspecting products, or thermal imaging systems monitoring equipment health. Multimodal machine learning combines these different data types to make more informed predictions than would be possible using a single modality alone.

26.1 Why Multimodal Learning Matters in Industry

Traditional machine learning typically focuses on a single data type. However, industrial scenarios often naturally produce multiple data streams, such as:

- sensor readings
- thermal images
- acoustic signals
- process parameters
- product images
- specifications
- equipment telemetry
- environmental conditions
- operator logs

By combining these modalities, models can capture complementary information. For instance, a sensor might indicate elevated temperature, while an image shows visible wear patterns - together providing stronger evidence of impending failure than either signal alone.

26.2 Fusion Strategies

There are three main approaches to combining multimodal data:

1. **Early fusion**: Concatenate raw features from all modalities before feeding into a single model
2. **Late fusion**: Train separate models for each modality and combine their predictions
3. **Intermediate fusion**: Process each modality to extract features, then combine these features for joint modeling.

26.3 Example: Equipment Health Monitoring with Time Series and Images

In this example, we will simulate an industrial scenario where we monitor machine health using both vibration sensor data (time series) and thermal camera images.

In this example, we simulate a scenario where we monitor equipment health using: - **Time series data**: Vibration sensor readings over time - **Image data**: Thermal camera images of the equipment

We'll classify equipment into three states: **Normal**, **Warning**, and **Critical**.

26.3.1 Loading Packages

▶ Code

```
<torch._C.Generator at 0x7bfd2df30c70>
```

26.3.2 Data Generation

We will simulate 500 equipment monitoring samples to demonstrate the value of multimodal fusion:

- A vibration time series (80 time steps)
- A thermal image (32x32 pixels)
- A health status label (0=Normal, 1=Warning, 2=Critical)

Key challenge: Some warning cases have ambiguous single-modality signals:

- **Type A**: High vibration (looks critical) but moderate temperature → actually warning
- **Type B**: Low vibration (looks normal) but elevated temperature → actually warning

▶ Code

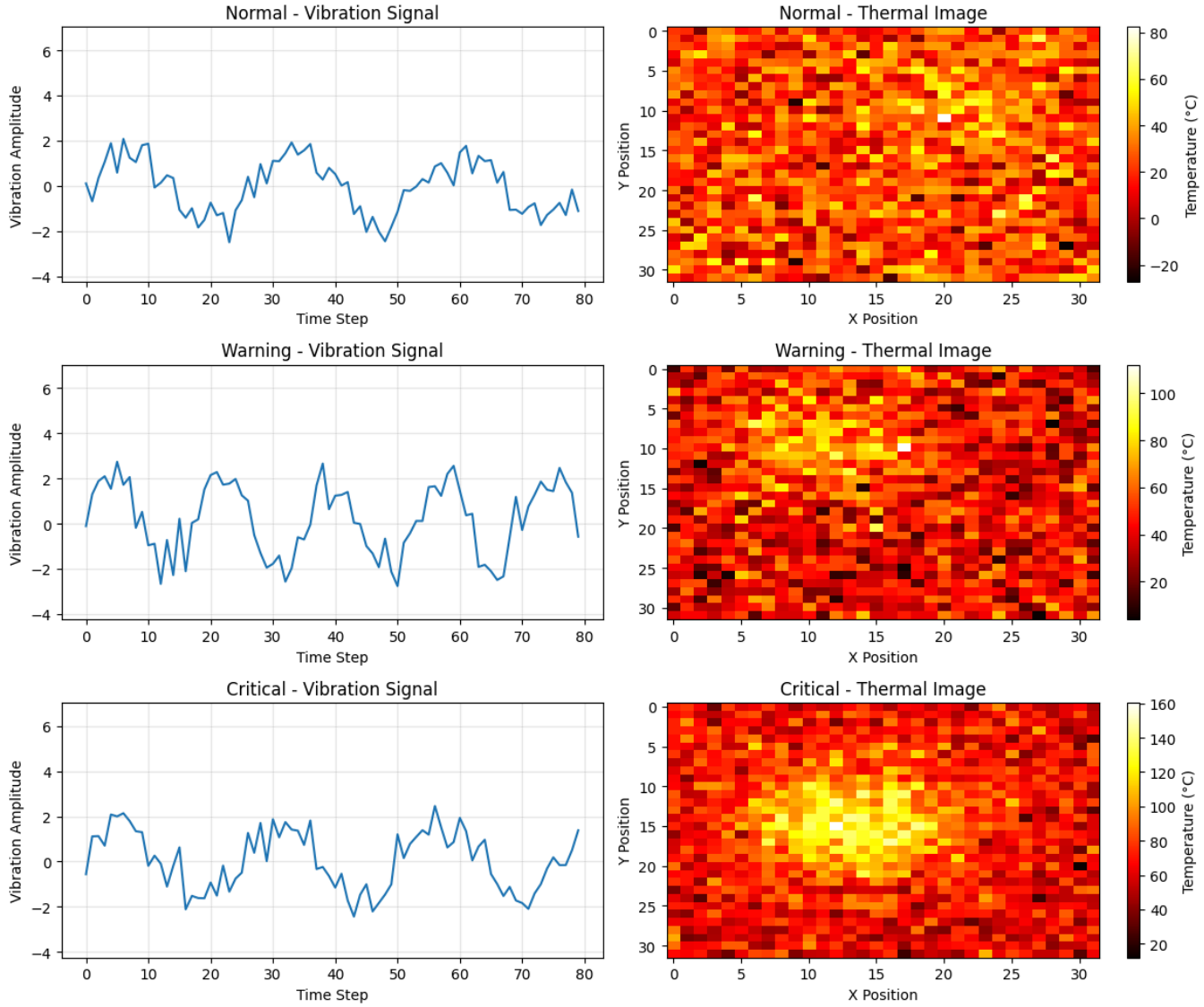
```
Time series shape: (500, 80)
Images shape: (500, 32, 32)
Labels shape: (500,)

Class distribution:
  Normal: 222 (44.4%)
  Warning: 204 (40.8%)
  Critical: 74 (14.8%)
```

26.3.3 Visualization

Let us visualize examples from each health status class:

```
fig, axes = plt.subplots(3, 2, figsize=(12, 10))
status_names = ["Normal", "Warning", "Critical"]

for status in range(3):
    # Find first sample of this status
    idx = np.where(labels == status)[0][0]

    # Plot time series
    axes[status, 0].plot(ts_data[idx])
    axes[status, 0].set_title(f"{status_names[status]} - Vibration Signal")
    axes[status, 0].set_ylim(ts_data.min(), ts_data.max())
```

```
    axes[status, 0].set_xlabel("Time Step")
    axes[status, 0].set_ylabel("Vibration Amplitude")
    axes[status, 0].grid(True, alpha=0.3)

    # Plot thermal image
    im = axes[status, 1].imshow(img_data[idx], cmap="hot", aspect="auto")
    axes[status, 1].set_title(f"{status_names[status]} - Thermal Image")
    axes[status, 1].set_xlabel("X Position")
    axes[status, 1].set_ylabel("Y Position")
    plt.colorbar(im, ax=axes[status, 1], label="Temperature (°C)")

plt.tight_layout()
plt.show()
```



26.3.4 Train-Test Split

We split the data into training (80%) and testing (20%) sets:

```
# Split data
ts_train, ts_test, img_train, img_test, y_train, y_test = train_test_split(
    ts_data, img_data, labels, test_size=0.2, random_state=42, stratify=labels
)
```

```
print(f"Training set: {len(ts_train)} samples")
print(f"Test set: {len(ts_test)} samples")
```

```
Training set: 400 samples
Test set: 100 samples
```

26.3.5 Model 1: Time Series Classifier

We will use a Random Forest on extracted time series features (mean, std, min, max, etc.):

```
# Reshape time series for sktime format (n_samples, n_features, n_timesteps)
# sktime expects 3D input where the middle dimension is the number of features (variables)
# Since we have univariate time series, n_features = 1
X_train_ts = ts_train.reshape(ts_train.shape[0], 1, ts_train.shape[1])
X_test_ts = ts_test.reshape(ts_test.shape[0], 1, ts_test.shape[1])

# Train Time Series Forest Classifier
print("Training Time Series Model...")
tsf = TimeSeriesForestClassifier(n_estimators=50, random_state=42)
tsf.fit(X_train_ts, y_train)

# Predict probabilities
ts_train_probs = tsf.predict_proba(X_train_ts)
ts_test_probs = tsf.predict_proba(X_test_ts)

print("Time Series Model - Test Set Performance:")
ts_pred = tsf.predict(X_test_ts)
print(
    classification_report(
        y_test, ts_pred, target_names=["Normal", "Warning", "Critical"]
    )
)
```

```
Training Time Series Model...
Time Series Model - Test Set Performance:
              precision    recall  f1-score   support

      Normal       0.70      0.89      0.78        44
     Warning       0.65      0.49      0.56        41
    Critical       0.69      0.60      0.64        15

    accuracy                           0.68       100
   macro avg       0.68      0.66      0.66       100
weighted avg       0.67      0.68      0.67       100
```

26.3.6 Model 2: Image Classifier

We will use a Convolutional Neural Network (CNN) for the thermal images:

```
# Normalize images to [0, 1]
img_train_norm = (img_train - img_train.min()) / (img_train.max() - img_train.min())
img_test_norm = (img_test - img_train.min()) / (img_train.max() - img_train.min())
```

```python
# Reshape for CNN - PyTorch uses (batch, channels, height, width)
img_train_cnn = img_train_norm.reshape(-1, 1, 32, 32)
img_test_cnn = img_test_norm.reshape(-1, 1, 32, 32)


# Define simpler CNN model in PyTorch
class ThermalCNN(nn.Module):
    """2D CNN for thermal image classification.

    Applies two convolutional layers with max pooling and global average
    pooling to classify thermal camera images into three health states.
    """

    def __init__(self) -> None:
        """Initialize the ThermalCNN architecture."""
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.global_pool = nn.AdaptiveAvgPool2d(1)
        self.fc1 = nn.Linear(32, 32)
        self.fc2 = nn.Linear(32, 3)
        self.dropout = nn.Dropout(0.4)
        self.relu = nn.ReLU()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """Forward pass through the network.

        Args:
            x: Input tensor of shape (batch, 1, 32, 32).

        Returns:
            Output logits of shape (batch, 3).
        """
        x = self.relu(self.conv1(x))
        x = self.pool(x)
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = self.global_pool(x)
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x


# Initialize model, loss, and optimizer
device = torch.device("cpu")
model_img = ThermalCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_img.parameters(), lr=0.001)

# Prepare data loaders
```

```
train_dataset = TensorDataset(
    torch.FloatTensor(img_train_cnn), torch.LongTensor(y_train)
)
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)

test_dataset = TensorDataset(torch.FloatTensor(img_test_cnn), torch.LongTensor(y_test))
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

# Training loop with fewer epochs
print("Training Image Model...")
model_img.train()
for epoch in range(20):  # Reduced from 30 to 20 epochs
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model_img(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

# Get predictions
model_img.eval()
with torch.no_grad():
    # Training set predictions
    img_train_probs = []
    for images, _ in DataLoader(train_dataset, batch_size=16, shuffle=False):
        images = images.to(device)
        outputs = model_img(images)
        probs = torch.softmax(outputs, dim=1)
        img_train_probs.append(probs.cpu().numpy())
    img_train_probs = np.vstack(img_train_probs)

    # Test set predictions
    img_test_probs = []
    img_preds = []
    for images, _ in test_loader:
        images = images.to(device)
        outputs = model_img(images)
        probs = torch.softmax(outputs, dim=1)
        img_test_probs.append(probs.cpu().numpy())
        img_preds.append(torch.argmax(outputs, dim=1).cpu().numpy())
    img_test_probs = np.vstack(img_test_probs)
    img_pred = np.concatenate(img_preds)

print("\nImage Model - Test Set Performance:")
print(
    classification_report(
        y_test, img_pred, target_names=["Normal", "Warning", "Critical"]
    )
)
```

```
Training Image Model...

Image Model - Test Set Performance:
              precision    recall  f1-score   support

      Normal       0.89      0.89      0.89        44
     Warning       0.78      0.76      0.77        41
    Critical       0.69      0.73      0.71        15

    accuracy                           0.81       100
   macro avg       0.78      0.79      0.79       100
weighted avg       0.81      0.81      0.81       100
```

26.3.7 Early Fusion: Combining Both Modalities

We build a model that combines both modalities at the input level.

```python
# Define a CNN for images (2D CNN)
class ThermalCNN(nn.Module):
    """2D CNN for thermal image feature extraction and classification.

    Optionally returns intermediate features for multimodal fusion
    instead of final class logits.
    """

    def __init__(self) -> None:
        """Initialize the ThermalCNN architecture."""
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.global_pool = nn.AdaptiveAvgPool2d(1)
        self.fc1 = nn.Linear(32, 32)
        self.fc2 = nn.Linear(32, 3)
        self.dropout = nn.Dropout(0.4)
        self.relu = nn.ReLU()

    def forward(self, x: torch.Tensor, features_only: bool = False) -> torch.Tensor:
        """Forward pass through the network.

        Args:
            x: Input tensor of shape (batch, 1, 32, 32).
            features_only: If True, return features before the final classifier.

        Returns:
            Output logits (batch, 3) or features (batch, 32).
        """
        x = self.relu(self.conv1(x))
        x = self.pool(x)
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = self.global_pool(x)
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
```

```
        x = self.dropout(x)
        if features_only:
            return x
        x = self.fc2(x)
        return x


# Define a CNN for time series (1D CNN)
class TimeSeriesCNN(nn.Module):
    """1D CNN for time series classification.

    Applies two convolutional layers with max pooling and global average
    pooling to classify time series data into three health states.
    """

    def __init__(self, ts_length: int = 80) -> None:
        """Initialize the TimeSeriesCNN architecture.

        Args:
            ts_length: Length of the input time series.
        """
        super().__init__()
        self.conv1 = nn.Conv1d(1, 16, kernel_size=5, padding=2)
        self.conv2 = nn.Conv1d(16, 32, kernel_size=5, padding=2)
        self.pool = nn.MaxPool1d(2)
        self.global_pool = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(32, 32)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.4)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """Forward pass through the network.

        Args:
            x: Input tensor of shape (batch, 1, ts_length).

        Returns:
            Output features of shape (batch, 32).
        """
        x = self.relu(self.conv1(x))
        x = self.pool(x)
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = self.global_pool(x)
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc(x))
        x = self.dropout(x)
        return x


# Combined multimodal network
class MultimodalNet(nn.Module):
    """Multimodal neural network combining thermal images and time series.
```

```
    Fuses features from a 2D CNN (thermal images) and a 1D CNN
    (time series) for equipment health classification.
    """

    def __init__(self) -> None:
        """Initialize the MultimodalNet architecture."""
        super().__init__()
        self.img_net = ThermalCNN()
        self.ts_net = TimeSeriesCNN()
        self.fc_combined = nn.Linear(32 + 32, 32)
        self.fc_out = nn.Linear(32, 3)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.4)

    def forward(self, img: torch.Tensor, ts: torch.Tensor) -> torch.Tensor:
        """Forward pass through the multimodal network.

        Args:
            img: Thermal image tensor of shape (batch, 1, 32, 32).
            ts: Time series tensor of shape (batch, 1, ts_length).

        Returns:
            Output logits of shape (batch, 3).
        """
        img_feat = self.img_net(img, features_only=True)
        ts_feat = self.ts_net(ts)
        x = torch.cat([img_feat, ts_feat], dim=1)
        x = self.relu(self.fc_combined(x))
        x = self.dropout(x)
        x = self.fc_out(x)
        return x


# Prepare data for multimodal network
# Normalize and reshape time series for 1D CNN: (batch, channels=1, length)
ts_train_norm = (ts_train - ts_train.min()) / (ts_train.max() - ts_train.min())
ts_test_norm = (ts_test - ts_train.min()) / (ts_train.max() - ts_train.min())

ts_train_cnn = ts_train_norm.reshape(-1, 1, ts_train_norm.shape[1]).astype(np.float32)
ts_test_cnn = ts_test_norm.reshape(-1, 1, ts_test_norm.shape[1]).astype(np.float32)

# Normalize and reshape images for 2D CNN: (batch, channels=1, height, width)
img_train_cnn = img_train_norm.reshape(
    -1, 1, img_train_norm.shape[1], img_train_norm.shape[2]
).astype(np.float32)
img_test_cnn = img_test_norm.reshape(
    -1, 1, img_test_norm.shape[1], img_test_norm.shape[2]
).astype(np.float32)

# Convert to torch tensors
ts_train_tensor = torch.FloatTensor(ts_train_cnn)
ts_test_tensor = torch.FloatTensor(ts_test_cnn)
img_train_tensor = torch.FloatTensor(img_train_cnn)
img_test_tensor = torch.FloatTensor(img_test_cnn)
```

```
y_train_tensor = torch.LongTensor(y_train)
y_test_tensor = torch.LongTensor(y_test)

# DataLoader for multimodal training
multimodal_train_dataset = TensorDataset(
    img_train_tensor, ts_train_tensor, y_train_tensor
)
multimodal_test_dataset = TensorDataset(img_test_tensor, ts_test_tensor, y_test_tensor)
multimodal_train_loader = DataLoader(
    multimodal_train_dataset, batch_size=16, shuffle=True
)
multimodal_test_loader = DataLoader(
    multimodal_test_dataset, batch_size=16, shuffle=False
)

# Initialize and train the multimodal model
device = torch.device("cpu")
multimodal_model = MultimodalNet().to(device)
optimizer_mm = optim.Adam(multimodal_model.parameters(), lr=0.001)
criterion_mm = nn.CrossEntropyLoss()

print("Training Multimodal Model...")
multimodal_model.train()
for epoch in range(30):
    running_loss = 0.0
    for img_batch, ts_batch, labels_batch in multimodal_train_loader:
        img_batch, ts_batch, labels_batch = (
            img_batch.to(device),
            ts_batch.to(device),
            labels_batch.to(device),
        )
        optimizer_mm.zero_grad()
        outputs = multimodal_model(img_batch, ts_batch)
        loss = criterion_mm(outputs, labels_batch)
        loss.backward()
        optimizer_mm.step()
        running_loss += loss.item()

# Evaluate on test set
multimodal_model.eval()
mm_test_probs = []
mm_test_preds = []
with torch.no_grad():
    for img_batch, ts_batch, _ in multimodal_test_loader:
        img_batch, ts_batch = img_batch.to(device), ts_batch.to(device)
        outputs = multimodal_model(img_batch, ts_batch)
        probs = F.softmax(outputs, dim=1)
        mm_test_probs.append(probs.cpu().numpy())
        mm_test_preds.append(torch.argmax(outputs, dim=1).cpu().numpy())
mm_test_probs = np.vstack(mm_test_probs)
early_fusion_pred = np.concatenate(mm_test_preds)

print("\nMULTIMODAL STACKED MODEL - Test Set Performance:")
print(
```

```
    classification_report(
        y_test, early_fusion_pred, target_names=["Normal", "Warning", "Critical"]
    )
)

ts_accuracy = accuracy_score(y_test, ts_pred)
img_accuracy = accuracy_score(y_test, img_pred)
combined_accuracy = accuracy_score(y_test, early_fusion_pred)

print(f"Time Series Only:  {ts_accuracy:.3f}")
print(f"Image Only:        {img_accuracy:.3f}")
print(f"Multimodal Fusion: {combined_accuracy:.3f}")
print(
    f"\nImprovement: {(combined_accuracy - max(ts_accuracy, img_accuracy)) * 100:.1f}% over be
)
```

```
Training Multimodal Model...

MULTIMODAL STACKED MODEL - Test Set Performance:
              precision    recall  f1-score   support

      Normal       0.93      0.98      0.96        44
     Warning       0.92      0.88      0.90        41
    Critical       0.87      0.87      0.87        15

    accuracy                           0.92       100
   macro avg       0.91      0.91      0.91       100
weighted avg       0.92      0.92      0.92       100

Time Series Only:  0.680
Image Only:        0.810
Multimodal Fusion: 0.920

Improvement: 11.0% over best single modality
```

26.3.8 Late Fusion: Combining Both Modalities

Now we implement a simple late fusion approach that combines the predictions from both unimodal models by averaging their probability outputs:

```
# Simple late fusion by averaging probabilities from both models
late_fusion_probs = (ts_test_probs + img_test_probs) / 2
late_fusion_pred = np.argmax(late_fusion_probs, axis=1)

print("LATE FUSION MODEL - Test Set Performance:")
print(
    classification_report(
        y_test, late_fusion_pred, target_names=["Normal", "Warning", "Critical"]
    )
)
```

```
late_fusion_accuracy = accuracy_score(y_test, late_fusion_pred)
print(f"\nLate Fusion:       {late_fusion_accuracy:.3f}")
```

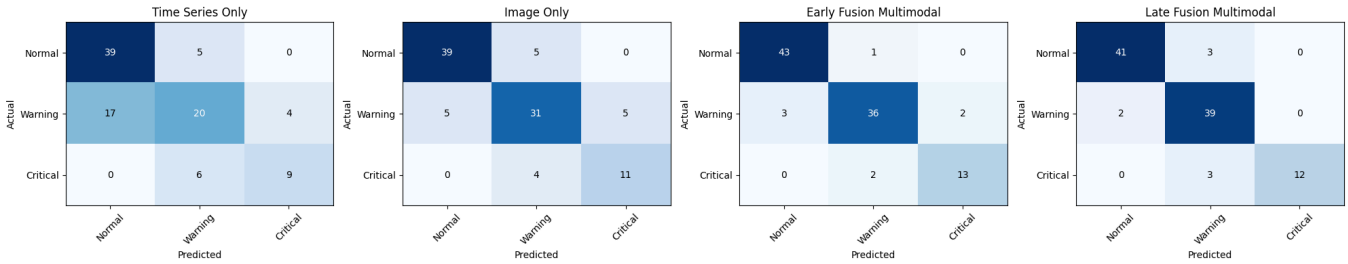
```
LATE FUSION MODEL - Test Set Performance:
              precision    recall  f1-score   support

      Normal       0.95      0.93      0.94        44
     Warning       0.87      0.95      0.91        41
    Critical       1.00      0.80      0.89        15

    accuracy                           0.92       100
   macro avg       0.94      0.89      0.91       100
weighted avg       0.92      0.92      0.92       100


Late Fusion:       0.920
```

26.3.9 Confusion Matrices

Let us visualize the confusion matrices for all approaches:

```
fig, axes = plt.subplots(1, 4, figsize=(20, 4))

models = [
    ("Time Series Only", ts_pred),
    ("Image Only", img_pred),
    ("Early Fusion Multimodal", early_fusion_pred),
    ("Late Fusion Multimodal", late_fusion_pred),
]

for idx, (title, pred) in enumerate(models):
    cm = confusion_matrix(y_test, pred)
    im = axes[idx].imshow(cm, cmap="Blues", aspect="auto")
    axes[idx].set_title(title)
    axes[idx].set_xlabel("Predicted")
    axes[idx].set_ylabel("Actual")
    axes[idx].set_xticks([0, 1, 2])
    axes[idx].set_yticks([0, 1, 2])
    axes[idx].set_xticklabels(["Normal", "Warning", "Critical"], rotation=45)
    axes[idx].set_yticklabels(["Normal", "Warning", "Critical"])

    # Add text annotations
    for i in range(3):
        for j in range(3):
            text = axes[idx].text(
                j,
                i,
                cm[i, j],
                ha="center",
                va="center",
                color="black" if cm[i, j] < cm.max() / 2 else "white",
            )

plt.tight_layout()
```

```
plt.show()

# Print summary
print("\nModel Performance Summary:")
print(f"Time Series Only:     {accuracy_score(y_test, ts_pred):.3f}")
print(f"Image Only:           {accuracy_score(y_test, img_pred):.3f}")
print(f"Late Fusion:          {late_fusion_accuracy:.3f}")
print(f"Multimodal Network:   {accuracy_score(y_test, early_fusion_pred):.3f}")
```




```
Model Performance Summary:
Time Series Only:     0.680
Image Only:           0.810
Late Fusion:          0.920
Multimodal Network:   0.920
```

The multimodal approaches typically outperform single-modality models, especially when modalities provide complementary information.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

27 Reinforcement Learning

> **Still under construction**
>
> This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

Reinforcement Learning (RL) is a machine learning paradigm where an **agent** learns to make decisions by interacting with an **environment**. Unlike supervised learning, which relies on labeled datasets, RL agents learn through trial and error, receiving **rewards** or penalties based on their actions. This makes RL particularly suitable for industrial applications where optimal control strategies need to be discovered through experimentation.

27.1 Key Concepts

The core components of reinforcement learning are:

- **Agent**: The learner or decision maker (e.g., a control system)
- **Environment**: The system the agent interacts with (e.g., a machine or process)
- **State** (s): The current situation or configuration of the environment
- **Action** (a): Choices the agent can make to influence the environment
- **Policy** (π): The agent's way of selecting actions based on states
- **Reward** (r_t): Feedback signal indicating how good an action was at time t
- **Value Function** (v_π): Expected cumulative reward from a state under a policy
- **Model**: The agent's representation of the environment dynamics (optional)

The agent's goal is to learn a policy that maximizes the **cumulative reward** G_t following time t. This is formalized as:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \cdots + r_T,$$

where T is the final time step for episodic tasks, or more generally for continuing tasks:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1},$$

where $\gamma \in [0, 1]$ is called the *discount rate* that determines how much future rewards are valued.

The rewards that the agent can expect to receive when starting from state s and following policy π are captured by the **value function** of a state s under policy π:

$$v_\pi(s) = \mathbb{E}_\pi[G_t | s_t = s].$$

Solving a reinforcement learning problem typically involves finding an optimal policy π^* that maximizes the expected cumulative reward from any state. Due to the complexity of real-world environments, RL algorithms usually rely on approximations and heuristics.

27.2 Classic Agent-Environment Loop

The interaction between the agent and the environment can be summarized in the following loop:

1. The agent observes the current state s_t of the environment.
2. Based on its policy π, the agent selects an action a_t.
3. The environment transitions to a new state s_{t+1} and provides a reward r_{t+1}.
4. The agent updates its policy based on the received reward and the new state.
5. This process repeats until a terminal state is reached or a predefined number of steps is completed.

Step 4 is where learning occurs, and different RL algorithms implement various strategies for updating the policy. This is a very broad field, which would be subject to a separate dedicated course, so we refer to Section 27.4 for a detailed treatment of specific algorithms and theoretical foundations.

27.3 Industrial Applications

27.3.1 Advantages and Challenges

Reinforcement learning offers several advantages for industrial applications:

- **Adaptability**: RL agents can adapt to changing environments and learn optimal strategies over time.
- **Exploration**: RL encourages exploration of different strategies, which can lead to innovative solutions.
- **Automation**: RL can automate complex decision-making processes that are difficult to model explicitly.

27.3.2 Applications

- **Energy management**: Optimizing energy consumption in production facilities
- **Predictive maintenance**: Learning optimal maintenance schedules based on equipment state
- **Process optimization**: Adjusting manufacturing parameters to maximize yield or quality
- **Robotic control**: Training robots to perform complex assembly or manipulation tasks
- **Supply chain**: Dynamic inventory management and routing decisions

27.4 Literature

For a comprehensive introduction to reinforcement learning, consider Sutton and Barto (2018) or the more application-focused documentation of the `Gymnasium` library, which is a widely used open-source library for developing and comparing RL algorithms. Dogru et al. (2024) provides a recent survey of industrial applications of reinforcement learning, discussing various case studies and practical considerations.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

28 Optional Self study Session

Still under construction

This section is still under construction and will be completed in the near future. Please do not go beyond this point for now.

Real-world applications on industrial data face many challenges. One of them being that the data from machines is usually retrieved via communication protocols from *programmable logic controllers* (PLC). These communication protocols often lack standardization and can vary significantly between different manufacturers and even different models of PLCs. This heterogeneity makes it difficult to integrate data from (multiple) sources and can lead to massive complexity in data processing and analysis.

In the following, you will read a review paper discussing challenges in introducing industrial data science to existing (brownfield) environments.

Predatory Journals

Predatory journals exploit the academic publishing model for profit, often lacking rigorous peer review and editorial oversight. They may present themselves as legitimate outlets but prioritize financial gain over scholarly integrity. Researchers should be cautious when selecting journals for publication, ensuring they choose reputable venues that uphold high academic standards.

Wikipedia (accessed: 09 09 2025) states: *MDPI's business model is based on establishing entirely open access broad-discipline journals, with fast processing times from submission to publication and article processing charges paid by the author, their institutions or funders. MDPI's business practices have attracted controversy, with critics suggesting it sacrifices editorial and academic rigor in favor of operational speed and business interests. MDPI was included on Jeffrey Beall's list of predatory open access publishing companies in 2014; it was removed in 2015 following a successful appeal, while applying pressure on Beall's employer. Some journals published by MDPI have also been noted by the Chinese Academy of Sciences (CAS) and Norwegian Scientific Index for lack of rigor and possible predatory practices, as of 2025, CAS no longer lists any MDPI journals on its Early Warning List. In 2024, Finland's Public Forum, which classifies publication channels for academic research, downgraded 193 MDPI journals to its lowest, level 0 rating.*

PredatoryJournals.org published a blog post discussing MDPI (accessed: 10 09 2026) in this regard.

Exercise 28.1 (Data Science on Industrial Data – Today's Challenges in Brown Field Applications)

Read Klaeger et al. (2021), published in MDPI. You should be able to discuss the following questions:

- What is the typical data flow from a machine to a data store?
- Why is data acquisition harder in brownfield environments than in labs or new machines?
- Why is ground truth data difficult to obtain in industrial settings and what are possible implications?
- How does the long lifecycle of industrial machines complicate the adoption of AI technologies?
- Why does data preparation in industry take even more effort than the often proclaimed *80% of the time needed for data preparation rule*?

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue

References

AI4I 2020 Predictive Maintenance Dataset. 2020. UCI Machine Learning Repository. https://doi.org/10.24432/C5HS5C.

Cuturi, Marco, and Mathieu Blondel. 2017. “Soft-Dtw: A Differentiable Loss Function for Time-Series.” *International Conference on Machine Learning*, 894–903.

Dogru, Oguzhan, Junyao Xie, Om Prakash, et al. 2024. “Reinforcement Learning in Process Industries: Review and Perspective.” *IEEE/CAA Journal of Automatica Sinica* 11 (2): 283–300. https://doi.org/10.1109/JAS.2024.124227.

Ehrensperger, Gregor. 2026a. “Industrial_datascience_live_machine: V1.0.0.” Zenodo, January. https://doi.org/10.5281/zenodo.18183373.

Ehrensperger, Gregor. 2026b. “Industrial_datascience_sim_machine_knifes: V1.0.2.” Zenodo, January. https://doi.org/10.5281/zenodo.18148106.

Faouzi, Johann. 2024. “Time Series Classification: A review of Algorithms and Implementations.” In *Time Series Analysis - Recent Advances, New Perspectives and Applications*, edited by Jorge Rocha, Sandra Oliveira, and Cláudia Viana. https://doi.org/10.5772/intechopen.1004810.

Holder, Christopher, and Anthony Bagnall. 2026. “Rock the KASBA: Blazingly Fast and Accurate Time Series Clustering.” *Data Mining and Knowledge Discovery* 40 (2): 21. https://doi.org/10.1007/s10618-026-01189-9.

Holder, Christopher, Anthony Bagnall, and Jason Lines. 2024. “On Time Series Clustering with k-Means.” *arXiv Preprint arXiv:2410.14269*.

Klaeger, Tilman, Sebastian Gottschall, and Lukas Oehm. 2021. “Data Science on Industrial Data—Today’s Challenges in Brown Field Applications.” *Challenges* 12 (1). https://doi.org/10.3390/challe12010002.

Lahreche, Abdelmadjid, and Bachir Boucheham. 2021. “A Comparison Study of Dynamic Time Warping’s Variants for Time Series Classification.” *International Journal of Informatics and Applied Mathematics* 4 (1): 56–71. https://dergipark.org.tr/en/pub/ijiam/issue/59559/819482.

Mathur, Aeshita, Ameesha Dabas, and Nikhil Sharma. 2022. “Evolution from Industry 1.0 to Industry 5.0.” *2022 4th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)*, 1390–94. https://doi.org/10.1109/ICAC3N56670.2022.10074274.

Paparrizos, John, Fan Yang, and Haojun Li. 2024. “Bridging the Gap: A Decade Review of Time-Series Clustering Methods.” *arXiv Preprint arXiv:2412.20582*.

Peres, Ricardo Silva, Xiaodong Jia, Jay Lee, Keyi Sun, Armando Walter Colombo, and Jose Barata. 2020. “Industrial Artificial Intelligence in Industry 4.0 - Systematic Review, Challenges and Outlook.” *IEEE Access* 8: 220121–39. https://doi.org/10.1109/ACCESS.2020.3042874.

Roverso, Davide. 2002. “Plant Diagnostics by Transient Classification: The Aladdin Approach.” *International Journal of Intelligent Systems* 17 (8): 767–90.

Sutton, R. S., and A. G. Barto. 2018. *Reinforcement Learning, Second Edition: An Introduction*. Adaptive Computation and Machine Learning Series. MIT Press. https://books.google.at/books?id=sWV0DwAAQBAJ.

Tan, Chang Wei, Christoph Bergmeir, Francois Petitjean, and Geoffrey I Webb. 2020. “IEEEPPG Dataset.” Zenodo, June. https://doi.org/10.5281/zenodo.3902710.

Tan, Chang Wei, Christoph Bergmeir, François Petitjean, and Geoffrey I Webb. 2021. “Time Series Extrinsic Regression: Predicting Numeric Values from Time Series Data.” *Data Mining and Knowledge Discovery* 35 (3): 1032–60.

Wallsberger, Raphael, Ricardo Knauer, and Stephan Matzka. 2023. “Explainable Artificial Intelligence in Mechanical Engineering: A Synthetic Dataset for Comprehensive Failure Mode Analysis.” *2023 Fifth International Conference on Transdisciplinary AI (TransAI)*, 249–52. https://doi.org/10.1109/TransAI60598.2023.00032.

Zhao, Bendong, Huanzhang Lu, Shangfeng Chen, Junliang Liu, and Dongya Wu. 2017. “Convolutional Neural Networks for Time Series Classification.” *Journal of Systems Engineering and Electronics* 28 (1): 162–69. https://doi.org/10.21629/JSEE.2017.01.18.

Industrial Data Science - 2026 - CC BY-NC-SA 4.0 Manuscript designed with Quart

Gregor Ehrensperger

Edit this page Report an issue